

Tennis Court Tracking System

Complete Code Documentation

Generated on: September 11, 2025 at 12:38

Project Overview

This document contains all Python code files from the Tennis Court Tracking project. The project implements computer vision techniques for tennis court detection, player tracking, and distance measurement using YOLO object detection and DeepSORT tracking algorithms. The code is organized into the following main components:

- Detection and Tracking modules for player and ball tracking
- Distance Measurement modules for court mapping and coordinate transformation
- Supporting utilities and helper functions

Table of Contents

Folder/File	Page
[FOLDER] Debugging	3
tracking_app_v4.py	4
[FOLDER] Detect_and_Track	35
__init__.py	36
court_detection.py	37
create_tracking_boxes_video.py	44
create_tracking_video_and_init_data.py	45
demo_detection.py	46
demo_initial_df.py	47
demo_tracking.py	48
get_init_data.py	49
get_tracks.py	50
roi_main.py	51
[FOLDER] Detect_and_Trackdeep_sort	62
__init__.py	63
detection.py	64
generate_detections.py	65
iou_matching.py	69
kalman_filter.py	70
linear_assignment.py	74
nn_matching.py	77
roi_tracking.py	80
standard_one_player.py	91
track.py	95
tracker.py	98
tracker_implementation.py	100
tracking_video.py	103
tracks_cleaning.py	104
utils.py	105
[FOLDER] Detect_and_Trackinit_dataframe	106
__init__.py	107
create_df.py	108
players_positions.py	109
teams_classification.py	110

[FOLDER] Detect_and_TrackyoloV5	111
__init__.py	112
detector.py	113
load_models.py	114
utils.py	115
[FOLDER] Distance_measurement	116
__init__.py	117
advanced_tennis_tracker.py	118
ball_distance_calculator.py	126
coordinate_mapper.py	132
corner_detection.py	135
court_corners_average.py	144
distance_calculation.py	145
enhanced_tennis_court_tracker.py	146
homography_manager.py	152
improved_coordinate_mapper.py	154
intelligent_court_mapper.py	160
main_court_tracker.py	170
pixel_to_court_mapping.py	177
player_distance_calculator.py	178
tennis_court_diagnostics.py	185
tennis_court_tracker.py	193
tracker_custom_frame.py	218
[FOLDER] Root Directory	219
generate_code_pdf.py	220
try_optimization.py	225
try_roi.py	226
try_tracking.py	227
[FOLDER] Time_Optimization	228
__init__.py	229
resolution_optimizer.py	230
[FOLDER] User_Interface	255
tracking_app_v4.py	256

[FOLDER] Debugging

Location: Debugging

[FILE] tracking_app_v4.py

Path: Debugging\tracking_app_v4.py

```
import tkinter as tk
from tkinter import ttk, filedialog, messagebox, scrolledtext
import threading
import os
import sys
from pathlib import Path
import cv2
from PIL import Image, ImageTk
import pandas as pd
import json
from datetime import datetime
import subprocess
import webbrowser

# Add parent directory to path for project imports
sys.path.append(os.path.dirname(os.path.dirname(os.path.abspath(__file__))))

# Add your project imports here
try:
    from Distance_measurement.main_court_tracker import *
    from Distance_measurement.corner_detection import *
    from Detect_and_Track.create_tracking_video_and_init_data import
    create_tracking_video_and_init_
    data
except ImportError as e:
    print(f"Warning: Could not import project modules: {e}")

class VideoPlayer:
    """Simple video player widget"""

    def __init__(self, parent, width=640, height=360):
        self.parent = parent
        self.width = width
        self.height = height

        # Video state
        self.video_path = None
        self.cap = None
        self.is_playing = False
        self.current_frame = 0
        self.total_frames = 0
        self.fps = 30
        self.frame_delay = int(1000 / self.fps)

        self.create_widgets()

    def create_widgets(self):
```

```

        """Create video player widgets"""
        # Main frame
        self.frame = ttk.Frame(self.parent)

        # Video display
        self.video_label = ttk.Label(self.frame, text="No video loaded",
                                     anchor='center', background='black',
foreground='white')
        self.video_label.grid(row=0, column=0, columnspan=4, pady=(0, 10),
sticky=(tk.W, tk.E))

        # Controls frame
        controls_frame = ttk.Frame(self.frame)
        controls_frame.grid(row=1, column=0, columnspan=4, sticky=(tk.W, tk.E))

        # Control buttons
        self.play_button = ttk.Button(controls_frame, text="■ Play",
command=self.toggle_play)
        self.play_button.grid(row=0, column=0, padx=2)

        ttk.Button(controls_frame, text="■ Stop", command=self.stop_video).grid(row=0,
column=1,
        padx=2)
        ttk.Button(controls_frame, text="■ Start", command=self.goto_start).grid(row=0,
column=2,
        padx=2)
        ttk.Button(controls_frame, text="■ End", command=self.goto_end).grid(row=0,
column=3,
        padx=2)

        # Progress bar
        self.progress_var = tk.DoubleVar()
        self.progress_bar = ttk.Scale(controls_frame, from_=0, to=100,
orient=tk.HORIZONTAL,
        variable=self.progress_var,
command=self.seek_video)
        self.progress_bar.grid(row=0, column=4, sticky=(tk.W, tk.E), padx=(10,
10))

        # Time labels
        self.time_label = ttk.Label(controls_frame, text="00:00 / 00:00")
        self.time_label.grid(row=0, column=5, padx=2)

        # Configure column weights
        controls_frame.columnconfigure(4, weight=1)

    def load_video(self, video_path):
        """Load a video file"""
        try:
            if self.cap:
                self.cap.release()

            self.video_path = video_path
            self.cap = cv2.VideoCapture(video_path)

            if not self.cap.isOpened():
                raise ValueError("Could not open video file")

            # Get video properties
            self.total_frames = int(self.cap.get(cv2.CAP_PROP_FRAME_COUNT))
            self.fps = self.cap.get(cv2.CAP_PROP_FPS) or 30

```

```

        self.frame_delay = int(1000 / self.fps)
        self.current_frame = 0

        # Load first frame
        ret, frame = self.cap.read()
        if ret:
            self.display_frame(frame)
            self.update_time_display()

        return True

    except Exception as e:
        messagebox.showerror("Error", f"Error loading video: {e}")
        return False

def display_frame(self, frame):
    """Display a frame in the video label"""
    try:
        # Resize frame to fit display
        height, width = frame.shape[:2]
        if width > self.width or height > self.height:
            scale = min(self.width/width, self.height/height)
            new_width = int(width * scale)
            new_height = int(height * scale)
            frame = cv2.resize(frame, (new_width, new_height))

        # Convert to RGB and then to PhotoImage
        frame_rgb = cv2.cvtColor(frame, cv2.COLOR_BGR2RGB)
        image = Image.fromarray(frame_rgb)
        photo = ImageTk.PhotoImage(image)

        self.video_label.configure(image=photo, text="")
        self.video_label.image = photo

    except Exception as e:
        print(f"Error displaying frame: {e}")

def toggle_play(self):
    """Toggle play/pause"""
    if not self.cap:
        return

    self.is_playing = not self.is_playing
    if self.is_playing:
        self.play_button.config(text="■ Pause")
        self.play_video()
    else:
        self.play_button.config(text="■ Play")

def play_video(self):
    """Play video frames"""
    if not self.is_playing or not self.cap:
        return

    ret, frame = self.cap.read()
    if ret:
        self.current_frame += 1
        self.display_frame(frame)
        self.update_progress()
        self.update_time_display()

```

```

        # Schedule next frame
        self.parent.after(self.frame_delay, self.play_video)
    else:
        # End of video
        self.is_playing = False
        self.play_button.config(text="■ Play")

def stop_video(self):
    """Stop video and go to beginning"""
    self.is_playing = False
    self.play_button.config(text="■ Play")
    self.goto_start()

def goto_start(self):
    """Go to start of video"""
    if not self.cap:
        return

    self.cap.set(cv2.CAP_PROP_POS_FRAMES, 0)
    self.current_frame = 0
    ret, frame = self.cap.read()
    if ret:
        self.display_frame(frame)
        self.cap.set(cv2.CAP_PROP_POS_FRAMES, 0) # Reset position
    self.update_progress()
    self.update_time_display()

def goto_end(self):
    """Go to end of video"""
    if not self.cap:
        return

    self.cap.set(cv2.CAP_PROP_POS_FRAMES, self.total_frames - 1)
    self.current_frame = self.total_frames - 1
    ret, frame = self.cap.read()
    if ret:
        self.display_frame(frame)
    self.update_progress()
    self.update_time_display()

def seek_video(self, value):
    """Seek to specific position"""
    if not self.cap:
        return

    # Calculate frame number from percentage
    frame_number = int((float(value) / 100) * self.total_frames)
    self.cap.set(cv2.CAP_PROP_POS_FRAMES, frame_number)
    self.current_frame = frame_number

    ret, frame = self.cap.read()
    if ret:
        self.display_frame(frame)
        self.cap.set(cv2.CAP_PROP_POS_FRAMES, frame_number) # Reset position
    self.update_time_display()

def update_progress(self):
    """Update progress bar"""
    if self.total_frames > 0:
        progress = (self.current_frame / self.total_frames) * 100

```

```

        self.progress_var.set(progress)

    def update_time_display(self):
        """Update time display"""
        if self.total_frames > 0 and self.fps > 0:
            current_seconds = self.current_frame / self.fps
            total_seconds = self.total_frames / self.fps

            current_time = f"{int(current_seconds//60):02d}:{int(current_seconds%60):02d}"
            total_time = f"{int(total_seconds//60):02d}:{int(total_seconds%60):02d}"

            self.time_label.config(text=f"{current_time} / {total_time}")

    def get_frame(self):
        """Get the video player frame widget"""
        return self.frame

    def destroy(self):
        """Clean up video player"""
        if self.cap:
            self.cap.release()

class FileViewerWindow:
    """Window for viewing files in-app"""

    def __init__(self, parent, file_path):
        self.window = tk.Toplevel(parent)
        self.window.title(f"File Viewer - {os.path.basename(file_path)}")
        self.window.geometry("900x600")
        self.window.transient(parent)

        self.file_path = file_path
        self.create_widgets()
        self.load_file()

    def create_widgets(self):
        """Create file viewer widgets"""
        # Toolbar
        toolbar = ttk.Frame(self.window)
        toolbar.grid(row=0, column=0, sticky=(tk.W, tk.E), padx=5, pady=5)

        ttk.Label(toolbar, text=f"File: {os.path.basename(self.file_path)}").grid(row=0,
column=0,
        padx=5)
        ttk.Button(toolbar, text="Open in External App",
command=self.open_external).grid(row=0,
        column=1, padx=5)
        ttk.Button(toolbar, text="Open Folder", command=self.open_folder).grid(row=0,
column=2,
        padx=5)

        # Content area
        self.content_frame = ttk.Frame(self.window, padding="10")
        self.content_frame.grid(row=1, column=0, sticky=(tk.W, tk.E, tk.N, tk.S))

        # Configure grid weights
        self.window.columnconfigure(0, weight=1)
        self.window.rowconfigure(1, weight=1)

```



```

self.content_frame.columnconfigure(0, weight=1)
self.content_frame.rowconfigure(0, weight=1)

def load_file(self):
    """Load and display file content"""
    file_ext = os.path.splitext(self.file_path)[1].lower()

    try:
        if file_ext == '.csv':
            self.load_csv()
        elif file_ext == '.json':
            self.load_json()
        elif file_ext in ['.txt', '.log']:
            self.load_text()
        elif file_ext in ['.mp4', '.avi', '.mov']:
            self.load_video()
        elif file_ext in ['.png', '.jpg', '.jpeg']:
            self.load_image()
        else:
            ttk.Label(self.content_frame, text=f"File type {file_ext} not supported for
                preview").pack(pady=20)

    except Exception as e:
        ttk.Label(self.content_frame, text=f"Error loading file:
            {e}").pack(pady=20)

def load_csv(self):
    """Load CSV file"""
    df = pd.read_csv(self.file_path)

    # Create treeview
    columns = list(df.columns)
    tree = ttk.Treeview(self.content_frame, columns=columns, show='headings',
        height=20)

    # Configure columns
    for col in columns:
        tree.heading(col, text=col)
        tree.column(col, width=120)

    # Insert data (first 1000 rows)
    display_df = df.head(1000)
    for _, row in display_df.iterrows():
        values = [f"{val:.3f}" if isinstance(val, float) else str(val) for
            val in row]
        tree.insert('', tk.END, values=values)

    # Add scrollbars
    v_scrollbar = ttk.Scrollbar(self.content_frame, orient=tk.VERTICAL,
        command=tree.yview)
    h_scrollbar = ttk.Scrollbar(self.content_frame, orient=tk.HORIZONTAL,
        command=tree.xview)
    tree.configure(yscrollcommand=v_scrollbar.set,
        xscrollcommand=h_scrollbar.set)

    # Info label
    info_label = ttk.Label(self.content_frame,
        text=f"Showing first 1000 of {len(df)} rows, {len(df.columns)}
            columns")
    info_label.grid(row=0, column=0, columnspan=2, pady=5)

```

```

# Grid layout
tree.grid(row=1, column=0, sticky=(tk.W, tk.E, tk.N, tk.S))
v_scrollbar.grid(row=1, column=1, sticky=(tk.N, tk.S))
h_scrollbar.grid(row=2, column=0, sticky=(tk.W, tk.E))

self.content_frame.columnconfigure(0, weight=1)
self.content_frame.rowconfigure(1, weight=1)

def load_json(self):
    """Load JSON file"""
    with open(self.file_path, 'r') as f:
        data = json.load(f)

    text_widget = scrolledtext.ScrolledText(self.content_frame, wrap=tk.WORD)
    text_widget.grid(row=0, column=0, sticky=(tk.W, tk.E, tk.N, tk.S))

    formatted_json = json.dumps(data, indent=2, ensure_ascii=False)
    text_widget.insert(1.0, formatted_json)
    text_widget.config(state='disabled')

def load_text(self):
    """Load text file"""
    with open(self.file_path, 'r', encoding='utf-8', errors='ignore') as f:
        content = f.read()

    text_widget = scrolledtext.ScrolledText(self.content_frame, wrap=tk.WORD)
    text_widget.grid(row=0, column=0, sticky=(tk.W, tk.E, tk.N, tk.S))

    text_widget.insert(1.0, content)
    text_widget.config(state='disabled')

def load_video(self):
    """Load video file"""
    # Create video player
    self.video_player = VideoPlayer(self.content_frame, width=800, height=450)
    self.video_player.get_frame().grid(row=0, column=0, sticky=(tk.W, tk.E,
tk.N, tk.S))

    # Load video
    self.video_player.load_video(self.file_path)

def load_image(self):
    """Load image file"""
    image = cv2.imread(self.file_path)
    if image is not None:
        # Convert to RGB
        image_rgb = cv2.cvtColor(image, cv2.COLOR_BGR2RGB)

        # Resize if too large
        height, width = image_rgb.shape[:2]
        max_width, max_height = 800, 600

        if width > max_width or height > max_height:
            scale = min(max_width/width, max_height/height)
            new_width = int(width * scale)
            new_height = int(height * scale)
            image_rgb = cv2.resize(image_rgb, (new_width, new_height))

        # Convert to PhotoImage
        pil_image = Image.fromarray(image_rgb)
        photo = ImageTk.PhotoImage(pil_image)

```

```

        image_label = ttk.Label(self.content_frame, image=photo)
        image_label.image = photo # Keep reference
        image_label.grid(row=0, column=0, sticky=(tk.W, tk.E, tk.N, tk.S))

    def open_external(self):
        """Open file in external application"""
        try:
            if sys.platform == "win32":
                os.startfile(self.file_path)

            elif sys.platform == "darwin":
                subprocess.run(["open", self.file_path])
            else:
                subprocess.run(["xdg-open", self.file_path])
        except Exception as e:
            messagebox.showerror("Error", f"Could not open file: {e}")

    def open_folder(self):
        """Open folder containing the file"""
        try:
            folder_path = os.path.dirname(os.path.abspath(self.file_path))
            if sys.platform == "win32":
                subprocess.run(f'explorer /select,"{os.path.abspath(self.file_path)}', shell=True)
            elif sys.platform == "darwin":
                subprocess.run(["open", "-R", os.path.abspath(self.file_path)])
            else:
                subprocess.run(["xdg-open", folder_path])
        except Exception as e:
            messagebox.showerror("Error", f"Could not open folder: {e}")

class TennisTrackingApp:
    def __init__(self, root):
        self.root = root
        self.root.title("Tennis Court Tracking System v1.0")
        self.root.geometry("1200x540")
        self.root.configure(bg='#f0f0f0')

        # Application state
        self.video_path = None
        self.output_name = ""
        self.init_df_path = ""
        self.init_df = None
        self.ziframes = None
        self.zitboxes = None
        self.results = None
        self.processing = False

        # Create main layout
        self.setup_styles()
        self.create_menu()
        self.create_main_layout()

    def setup_styles(self):
        """Configure custom styles for the application"""
        style = ttk.Style()
        style.theme_use('clam')

        # Configure custom styles
        style.configure('Title.TLabel', font=('Arial', 16, 'bold'),

```

```

foreground='#2c3e50')

        style.configure('Section.TLabel', font=('Arial', 12, 'bold'),
foreground='#34495e')
        style.configure('Success.TLabel', font=('Arial', 10),
foreground='#27ae60')
        style.configure('Error.TLabel', font=('Arial', 10), foreground='#e74c3c')
        style.configure('Process.TButton', font=('Arial', 10, 'bold'))

def create_menu(self):
    """Create application menu bar"""
    menubar = tk.Menu(self.root)
    self.root.config(menu=menubar)

    # File menu
    file_menu = tk.Menu(menubar, tearoff=0)
    menubar.add_cascade(label="File", menu=file_menu)
    file_menu.add_command(label="Open Video", command=self.select_video)
    file_menu.add_separator()
    file_menu.add_command(label="Exit", command=self.root.quit)

    # View menu
    view_menu = tk.Menu(menubar, tearoff=0)
    menubar.add_cascade(label="View", menu=view_menu)
    view_menu.add_command(label="Play Original Video",
command=self.play_original_video)
    view_menu.add_command(label="Play Clean Video",
command=self.play_clean_video)
    view_menu.add_command(label="Play Tracking Video",
command=self.play_tracking_video)
    view_menu.add_separator()
    view_menu.add_command(label="View Results", command=self.view_results)
    view_menu.add_command(label="Open Output Folder",
command=self.open_output_folder)

    # Tools menu
    tools_menu = tk.Menu(menubar, tearoff=0)
    menubar.add_cascade(label="Tools", menu=tools_menu)
    tools_menu.add_command(label="Corner Detection Test",
command=self.open_corner_test)

    # Help menu
    help_menu = tk.Menu(menubar, tearoff=0)
    menubar.add_cascade(label="Help", menu=help_menu)
    help_menu.add_command(label="About", command=self.show_about)

def create_main_layout(self):
    """Create the main application layout"""
    # Create main container with padding
    main_container = ttk.Frame(self.root, padding="3")
    main_container.grid(row=0, column=0, sticky=(tk.W, tk.E, tk.N, tk.S))

    # Configure grid weights
    self.root.columnconfigure(0, weight=1)
    self.root.rowconfigure(0, weight=1)
    main_container.columnconfigure(1, weight=1)
    main_container.rowconfigure(1, weight=2) # Give more weight to main
content
    main_container.rowconfigure(2, weight=0) # Fixed height for progress panel

```

```

        # Title
title_label = ttk.Label(main_container, text="Tennis Court Tracking System",
                        style='Title.TLabel')
title_label.grid(row=0, column=0, columnspan=2, pady=(0, 5))

        # Left panel - Controls
self.create_control_panel(main_container)

        # Right panel - Video Preview and Results
self.create_preview_panel(main_container)

        # Bottom panel - Progress and Logs
self.create_progress_panel(main_container)

    def create_control_panel(self, parent):
        """Create the left control panel"""
        control_frame = ttk.LabelFrame(parent, text="Control Panel", padding="5")
        control_frame.grid(row=1, column=0, sticky=(tk.W, tk.E, tk.N, tk.S),
padx=(0, 5))
        control_frame.configure(width=350)

        # Video Input Section
        ttk.Label(control_frame, text="Video Input", style='Section.TLabel').grid(row=0,
column=0,
            columnspan=2, sticky=tk.W, pady=(0, 10))

        # Video selection
        ttk.Button(control_frame, text="■ Select Video", command=self.select_video,
            width=20).grid(row=1, column=0, sticky=tk.W, pady=2)
        self.video_label = ttk.Label(control_frame, text="No video selected",
foreground='gray')
        self.video_label.grid(row=1, column=1, sticky=tk.W, padx=(10, 0))

        # Video playback buttons
        video_controls = ttk.Frame(control_frame)
        video_controls.grid(row=2, column=0, columnspan=2, sticky=tk.W, pady=5)
        ttk.Button(video_controls, text="■ Original",
command=self.play_original_video).grid(row=0,
            column=0, padx=(0, 3))
        ttk.Button(video_controls, text="■ Clean",
command=self.play_clean_video).grid(row=0,
            column=1, padx=3)
        ttk.Button(video_controls, text="■ Tracking",
command=self.play_tracking_video).grid(row=0,
            column=2, padx=3)

        # Output name
        ttk.Label(control_frame, text="Output Name:").grid(row=3, column=0, sticky=tk.W,
pady=(10,
            5))
        self.output_name_var = tk.StringVar(value="tennis_analysis")
        ttk.Entry(control_frame, textvariable=self.output_name_var,
width=25).grid(row=3, column=1,
            sticky=tk.W, padx=(10, 0))

        # Team Colors Section
        ttk.Separator(control_frame, orient='horizontal').grid(row=4, column=0,
columnspan=2,
            sticky=(tk.W, tk.E), pady=20)

        ttk.Label(control_frame, text="Team Configuration",
style='Section.TLabel').grid(row=5,

```

```

        column=0, columnspan=2, sticky=tk.W, pady=(0, 10))

        # Team colors
        ttk.Label(control_frame, text="Team 1 Color:").grid(row=6, column=0,
sticky=tk.W, pady=2)
        self.team1_color = ttk.Combobox(control_frame, values=["red", "blue", "white", "
black",
            "green", "yellow"], width=12)
        self.team1_color.set("red")
        self.team1_color.grid(row=6, column=1, sticky=tk.W, padx=(10, 0))

        ttk.Label(control_frame, text="Team 2 Color:").grid(row=7, column=0,
sticky=tk.W, pady=2)
        self.team2_color = ttk.Combobox(control_frame, values=["red", "blue", "white", "
black",
            "green", "yellow"], width=12)
        self.team2_color.set("blue")
        self.team2_color.grid(row=7, column=1, sticky=tk.W, padx=(10, 0))

        # Ball only option
        self.ball_only_var = tk.BooleanVar(value=False)
        ttk.Checkbutton(control_frame, text="Track ball only",
variable=self.ball_only_var).grid(
            row=8, column=0, columnspan=2, sticky=tk.W, pady=10)

        # Processing Section
        ttk.Separator(control_frame, orient='horizontal').grid(row=9, column=0,
columnspan=2,
            sticky=(tk.W, tk.E), pady=20)
        ttk.Label(control_frame, text="Processing Pipeline",
style='Section.TLabel').grid(row=10,
            column=0, columnspan=2, sticky=tk.W, pady=(0, 10))

        # Step 1: Object Detection & Tracking (Combined)
        step1_frame = ttk.Frame(control_frame)
        step1_frame.grid(row=11, column=0, columnspan=2, sticky=(tk.W, tk.E),
pady=5)
        ttk.Label(step1_frame, text="1. Complete Object Tracking").grid(row=0, column=0,
            sticky=tk.W)
        self.tracking_button = ttk.Button(step1_frame, text="Start Processing",
            command=self.run_complete_tracking, style='Process.TButton')
        self.tracking_button.grid(row=0, column=1, padx=(10, 0))
        self.tracking_status = ttk.Label(step1_frame, text="■ Ready",
foreground='gray')
        self.tracking_status.grid(row=0, column=2, padx=(10, 0))

        # Step 2: Court Analysis
        step2_frame = ttk.Frame(control_frame)
        step2_frame.grid(row=12, column=0, columnspan=2, sticky=(tk.W, tk.E),
pady=5)
        ttk.Label(step2_frame, text="2. Court Analysis").grid(row=0, column=0,
sticky=tk.W)

        # Pipeline selection
        self.pipeline_var = tk.StringVar(value="standard")
        ttk.Radiobutton(step2_frame, text="Standard", variable=self.pipeline_var,
            value="standard").grid(row=1, column=0, sticky=tk.W, padx=(20, 0))
        ttk.Radiobutton(step2_frame, text="Dynamic", variable=self.pipeline_var,
            value="dynamic").grid(row=1, column=1, sticky=tk.W)

```

```

self.analysis_button = ttk.Button(step2_frame, text="Start Analysis",
                                  command=self.run_analysis, style='Process.TButton')
self.analysis_button.grid(row=0, column=1, padx=(10, 0))
self.analysis_status = ttk.Label(step2_frame, text="■ Ready",
foreground='gray')
self.analysis_status.grid(row=0, column=2, padx=(10, 0))

def create_preview_panel(self, parent):
    """Create the right preview panel"""
    preview_frame = ttk.LabelFrame(parent, text="Video Preview & Results",
padding="5")
    preview_frame.grid(row=1, column=1, sticky=(tk.W, tk.E, tk.N, tk.S))
    preview_frame.columnconfigure(0, weight=1)
    preview_frame.rowconfigure(1, weight=1) # Results area gets remaining
space

    # Video player
    self.video_player = VideoPlayer(preview_frame, width=480, height=180)
    self.video_player.get_frame().grid(row=0, column=0, sticky=(tk.W, tk.E),
pady=(0, 3))

    # Results notebook
    self.results_notebook = ttk.Notebook(preview_frame)
    self.results_notebook.grid(row=1, column=0, sticky=(tk.W, tk.E, tk.N,
tk.S))

    # Statistics tab
    self.stats_frame = ttk.Frame(self.results_notebook)
    self.results_notebook.add(self.stats_frame, text="Statistics")

    self.stats_text = scrolledtext.ScrolledText(self.stats_frame,
wrap=tk.WORD, height=3)
    self.stats_text.grid(row=0, column=0, sticky=(tk.W, tk.E, tk.N, tk.S),
padx=3, pady=3)
    self.stats_frame.columnconfigure(0, weight=1)
    self.stats_frame.rowconfigure(0, weight=1)

    # Output Files tab
    self.files_frame = ttk.Frame(self.results_notebook)
    self.results_notebook.add(self.files_frame, text="Output Files")

    # Files controls
    files_controls = ttk.Frame(self.files_frame)
    files_controls.grid(row=0, column=0, columnspan=2, sticky=(tk.W, tk.E),
padx=5, pady=5)

    ttk.Label(files_controls, text="Output folder contents:").grid(row=0,
column=0, sticky=tk.W)
    ttk.Button(files_controls, text="■ Refresh",
command=self.refresh_output_files).grid(row=0,
column=1, padx=(10, 0))
    ttk.Button(files_controls, text="■ Open Folder",
command=self.open_output_folder).grid(
row=0, column=2, padx=(5, 0))

    # Files tree
    self.files_tree = ttk.Treeview(self.files_frame, columns=('Size', 'Modified', '
Type'),
show='tree headings')
    self.files_tree.heading('#0', text='File Name')
    self.files_tree.heading('Size', text='Size')
    self.files_tree.heading('Modified', text='Modified')

```

```

        self.files_tree.heading('Type', text='Type')
        self.files_tree.grid(row=1, column=0, sticky=(tk.W, tk.E, tk.N, tk.S),
padx=3, pady=3)

        # Files scrollbar
files_scrollbar = ttk.Scrollbar(self.files_frame, orient=tk.VERTICAL,
        command=self.files_tree.yview)
        files_scrollbar.grid(row=1, column=1, sticky=(tk.N, tk.S))
        self.files_tree.configure(yscrollcommand=files_scrollbar.set)

        # Double-click to open file
        self.files_tree.bind('<Double-1>', self.on_file_double_click)

        # Right-click context menu
        self.create_file_context_menu()

        self.files_frame.columnconfigure(0, weight=1)
        self.files_frame.rowconfigure(1, weight=1)

        # Load initial file list
        self.refresh_output_files()

    def create_file_context_menu(self):
        """Create context menu for files tree"""
        self.file_context_menu = tk.Menu(self.root, tearoff=0)
        self.file_context_menu.add_command(label="Open in App",
command=self.open_file_in_app)
        self.file_context_menu.add_command(label="Open in External App",
        command=self.open_file_external)
        self.file_context_menu.add_separator()
        self.file_context_menu.add_command(label="Show in Folder",
command=self.show_file_in_folder)
        self.file_context_menu.add_separator()
        self.file_context_menu.add_command(label="Play Video (if video)",
        command=self.play_selected_video)

        self.files_tree.bind('<Button-3>', self.show_file_context_menu) #
Right-click

    def show_file_context_menu(self, event):
        """Show context menu for files"""
        item = self.files_tree.selection()[0] if self.files_tree.selection() else
None
        if item:
            self.file_context_menu.post(event.x_root, event.y_root)

    def create_progress_panel(self, parent):
        """Create the bottom progress panel"""
        progress_frame = ttk.LabelFrame(parent, text="Progress & Logs",
padding="3")
        progress_frame.grid(row=2, column=0, columnspan=2, sticky=(tk.W, tk.E),
pady=(3, 0))
        progress_frame.columnconfigure(0, weight=1)

        # Progress bar
        self.progress_var = tk.DoubleVar()
        self.progress_bar = ttk.Progressbar(progress_frame, variable=self.progress_var,

        mode='determinate')
        self.progress_bar.grid(row=0, column=0, sticky=(tk.W, tk.E), pady=(0, 5))

        self.progress_label = ttk.Label(progress_frame, text="Ready to start

```



```

processing...)
    self.progress_label.grid(row=0, column=1, padx=(10, 0))

    # Log output
    self.log_text = scrolledtext.ScrolledText(progress_frame, wrap=tk.WORD,
height=2)
    self.log_text.grid(row=1, column=0, columnspan=2, sticky=(tk.W, tk.E,
tk.N, tk.S))

    def select_video(self):
        """Handle video file selection"""
        file_path = filedialog.askopenfilename(
            title="Select Tennis Video",
            filetypes=[
                ("Video files", "*.mp4 *.avi *.mov *.mkv *.wmv"),
                ("All files", "*.*")
            ]
        )

        if file_path:
            self.video_path = file_path
            filename = os.path.basename(file_path)
            self.video_label.config(text=f"■ {filename}", foreground='black')

            # Auto-generate output name from filename
            base_name = os.path.splitext(filename)[0]
            self.output_name_var.set(base_name)

            # Load video in player
            self.video_player.load_video(file_path)

            self.log_message(f"Video selected: {filename}")

            # Check if there are existing analysis files for this video
            self.check_existing_analysis_files()

    def check_existing_analysis_files(self):
        """Check if analysis files exist for current video and enable statistics
if available"""
        if not self.video_path:
            return

        output_name = self.output_name_var.get()
        if not output_name:
            return

        # Check for key analysis files
        init_df_file = f"./Out/{output_name}_init_df.csv"
        analysis_files = [
            f"./Out/{output_name}_with_video_court_coords.csv",

            f"./Out/{output_name}_with_dynamic_court_coords.csv"
        ]

        # Update init_df_path if file exists
        if os.path.exists(init_df_file):
            self.init_df_path = init_df_file
            self.tracking_status.config(text="■ Complete", foreground='green')
            self.log_message(f"Found existing tracking data:
{os.path.basename(init_df_file)}")

            # Check for analysis results

```

```

        for analysis_file in analysis_files:
            if os.path.exists(analysis_file):
                self.analysis_status.config(text="■ Complete", foreground='green')
                self.log_message(f"Found existing analysis:
{os.path.basename(analysis_file)}")

                # Try to load and display existing results
                try:
                    self.load_existing_analysis_results(analysis_file)
                except Exception as e:
                    self.log_message(f"Could not load existing analysis: {e}", "
WARNING")
                    break

            # Refresh file list to show all available files
            self.refresh_output_files()

def load_existing_analysis_results(self, analysis_file):
    """Load existing analysis results for display"""
    try:
        # Load the analysis CSV
        df_mapped = pd.read_csv(analysis_file)

        # Try to generate basic statistics from the loaded data
        player_ids = df_mapped['ID'].unique()
        player_distances = pd.DataFrame()

        # Calculate basic player statistics
        for player_id in player_ids:
            if player_id == 'ball': # Skip ball for player stats
                continue

            player_data = df_mapped[df_mapped['ID'] == player_id]
            if len(player_data) < 2:
                continue

            # Calculate total distance (simplified)
            coords = player_data[['court_x', 'court_y']].dropna()
            if len(coords) > 1:
                distances = []
                for i in range(1, len(coords)):
                    prev_x, prev_y = coords.iloc[i-1]

                    curr_x, curr_y = coords.iloc[i]
                    dist = ((curr_x - prev_x)**2 + (curr_y - prev_y)**2)**0.5
                    distances.append(dist)

                total_distance = sum(distances)
                player_distances = pd.concat([player_distances, pd.DataFrame({
                    'ID': [player_id],
                    'TotalDistance': [total_distance],
                    'DataPoints': [len(player_data)]
                })], ignore_index=True)

            # Create simplified results tuple
            self.results = (df_mapped, player_distances, {}, None, None)
            self.display_results()

    except Exception as e:
        self.log_message(f"Error loading existing results: {e}", "ERROR")

def play_original_video(self):

```

```

        """Play original video in video player"""
        if self.video_path:
            self.video_player.load_video(self.video_path)
            self.log_message("Playing original video")
        else:
            messagebox.showwarning("Warning", "Please select a video file first.")

    def play_clean_video(self):
        """Play clean video if available"""
        output_name = self.output_name_var.get()
        if output_name:
            clean_video_path = f"./Out/{output_name}_out.mp4"

            if os.path.exists(clean_video_path):
                self.video_player.load_video(clean_video_path)
                self.log_message(f"Playing clean video: {os.path.basename(clean_video_path)}")
                return

            messagebox.showinfo("Info", "Clean video not available. Please run tracking first.")

    def play_tracking_video(self):
        """Play tracking video if available"""
        output_name = self.output_name_var.get()
        if output_name:
            tracking_video_path = f"./Out/{output_name}_out_tracked.mp4"

            if os.path.exists(tracking_video_path):
                self.video_player.load_video(tracking_video_path)
                self.log_message(f"Playing tracking video: {os.path.basename(tracking_video_path)}")
                return

            messagebox.showinfo("Info", "Tracking video not available. Please run tracking first.")

    def play_selected_video(self):
        """Play selected video from files tree"""
        selected_item = self.files_tree.selection()
        if selected_item:
            filename = self.files_tree.item(selected_item[0])['text']
            file_path = os.path.join("./Out", filename)

            # Check if it's a video file
            video_extensions = ['.mp4', '.avi', '.mov', '.mkv', '.wmv']
            if any(file_path.lower().endswith(ext) for ext in video_extensions):
                self.video_player.load_video(file_path)
                self.log_message(f"Playing video: {filename}")
            else:
                messagebox.showinfo("Info", "Selected file is not a video.")

    def on_file_double_click(self, event):
        """Handle double-click on file"""
        selected_item = self.files_tree.selection()
        if selected_item:
            filename = self.files_tree.item(selected_item[0])['text']
            file_path = os.path.join("./Out", filename)
            self.open_file_viewer(file_path)

    def open_file_in_app(self):

```

```

        """Open selected file in app viewer"""
        selected_item = self.files_tree.selection()
        if selected_item:
            filename = self.files_tree.item(selected_item[0])['text']
            file_path = os.path.join("./Out", filename)
            self.open_file_viewer(file_path)

def open_file_external(self):
    """Open selected file in external application"""
    selected_item = self.files_tree.selection()
    if selected_item:
        filename = self.files_tree.item(selected_item[0])['text']
        file_path = os.path.join("./Out", filename)
        try:
            if sys.platform == "win32":
                os.startfile(file_path)
            elif sys.platform == "darwin":
                subprocess.run(["open", file_path])
            else:
                subprocess.run(["xdg-open", file_path])
        except Exception as e:
            messagebox.showerror("Error", f"Could not open file: {e}")

def show_file_in_folder(self):

    """Show selected file in folder"""
    selected_item = self.files_tree.selection()
    if selected_item:
        filename = self.files_tree.item(selected_item[0])['text']
        file_path = os.path.abspath(os.path.join("./Out", filename))
        try:
            if sys.platform == "win32":
                subprocess.run(f'explorer /select,"{file_path}"', shell=True)
            elif sys.platform == "darwin":
                subprocess.run(["open", "-R", file_path])
            else:
                folder_path = os.path.dirname(file_path)
                subprocess.run(["xdg-open", folder_path])
        except Exception as e:
            messagebox.showerror("Error", f"Could not show file in folder:
{e}")

def open_file_viewer(self, file_path):
    """Open file in internal viewer"""
    if os.path.exists(file_path):
        FileViewerWindow(self.root, file_path)
    else:
        messagebox.showerror("Error", "File not found.")

def refresh_output_files(self):
    """Refresh the output files list"""
    try:
        # Clear existing items
        for item in self.files_tree.get_children():
            self.files_tree.delete(item)

        # Check output directory
        output_dir = Path("./Out")
        if not output_dir.exists():
            self.log_message("Output directory does not exist yet")
            return

```

```

        file_count = 0
        # Add all files from output directory
        for file_path in output_dir.iterdir():
            if file_path.is_file():
                try:
                    stat = os.stat(file_path)
                    size = stat.st_size
                    modified = datetime.fromtimestamp(stat.st_mtime).strftime("%Y-%m-%d %H:%M")
                    size_str = f"{size:,} bytes" if size < 1024*1024 else f"{size/(1024*1024):.1f} MB"

                    # Determine file type
                    ext = file_path.suffix.lower()
                    file_type = {
                        '.csv': 'Data',
                        '.mp4': 'Video', '.avi': 'Video', '.mov': 'Video',
                        '.png': 'Image', '.jpg': 'Image', '.jpeg': 'Image',
                        '.json': 'Config',
                        '.txt': 'Text', '.log': 'Log'
                    }.get(ext, 'Other')

                    filename = file_path.name
                    self.files_tree.insert('', tk.END, text=filename,
                                           values=(size_str, modified,
file_type))

                    file_count += 1
                except Exception:
                    continue

        # Log the number of files found
        self.log_message(f"Output files refreshed: {file_count} files found")

    except Exception as e:
        self.log_message(f"Error refreshing file list: {e}", "ERROR")

    def run_complete_tracking(self):
        """Run complete object detection, tracking, and video creation in a
        separate thread"""
        if not self.video_path:
            messagebox.showwarning("Warning", "Please select a video file first.")
            return

        if self.processing:
            messagebox.showinfo("Info", "Processing is already running.")
            return

        # Start tracking in a separate thread
        thread = threading.Thread(target=self._complete_tracking_worker)
        thread.daemon = True
        thread.start()

    def _complete_tracking_worker(self):
        """Worker thread for complete tracking process"""
        try:
            self.processing = True
            self.update_ui_state()

            self.output_name = self.output_name_var.get() or "tennis_analysis"
            teams_colors = [self.team1_color.get(), self.team2_color.get()]
            ball_only = self.ball_only_var.get()

```

```

        self.progress_var.set(0)
        self.update_progress("Starting complete tracking process...", 10)

        # Run the combined function
        self.log_message("Starting complete object detection, tracking, and
video creation...")

        self.init_df, self.ziframes, self.zitboxes =
create_tracking_video_and_init_data(
            self.video_path, self.output_name, teams_colors, ball_only
        )

        self.update_progress("Complete tracking process finished", 100)

        self.init_df_path = f"./Out/{self.output_name}_init_df.csv"

        self.log_message("Complete tracking process completed successfully!", "
SUCCESS")
        self.root.after(0, lambda: self.tracking_status.config(text="■ Complete",
            foreground='green'))
        self.root.after(0, self.refresh_output_files)

    except Exception as e:
        error_msg = f"Error in complete tracking process: {str(e)}"
        self.log_message(error_msg, "ERROR")
        self.root.after(0, lambda: self.tracking_status.config(text="■ Error",
            foreground='red'))
    finally:
        self.processing = False
        self.root.after(0, self.update_ui_state)

def run_analysis(self):
    """Run court analysis pipeline"""
    if not self.init_df_path or not os.path.exists(self.init_df_path):
        messagebox.showwarning("Warning", "Please run object tracking first.")
        return

    if self.processing:
        messagebox.showinfo("Info", "Processing is already running.")
        return

    # Start analysis in a separate thread
    thread = threading.Thread(target=self._analysis_worker)
    thread.daemon = True
    thread.start()

def _analysis_worker(self):
    """Worker thread for analysis process"""
    try:
        self.processing = True
        self.update_ui_state()

        self.progress_var.set(0)
        self.update_progress("Starting court analysis...", 10)

        if self.pipeline_var.get() == "dynamic":
            self.log_message("Running dynamic homography pipeline...")
            self.update_progress("Processing with dynamic homography...", 30)
            results = main_video_processing_pipeline_dynamic(

```

```

        video_path=self.video_path,
        data_csv_path=self.init_df_path
    )
else:
    self.log_message("Running standard homography pipeline...")
    self.update_progress("Processing with standard homography...", 30)
    results = main_video_processing_pipeline(
        video_path=self.video_path,
        data_csv_path=self.init_df_path
    )

self.update_progress("Analysis completed", 100)
self.results = results

self.log_message("Court analysis completed successfully!", "SUCCESS")
self.root.after(0, lambda: self.analysis_status.config(text="■ Complete",
    foreground='green'))
self.root.after(0, self.display_results)
self.root.after(0, self.refresh_output_files)

except Exception as e:
    error_msg = f"Error in analysis process: {str(e)}"
    self.log_message(error_msg, "ERROR")
self.root.after(0, lambda: self.analysis_status.config(text="■ Error",
    foreground='red'))
finally:
    self.processing = False
    self.root.after(0, self.update_ui_state)

def display_results(self):
    """Display analysis results in the statistics tab"""
    if not self.results:
        return

    try:
        df_mapped, player_distances, ball_results, average_corners,
frame_info = self.results

        stats_text = "=== TENNIS COURT ANALYSIS RESULTS ===\n\n"

        # Player Statistics
        stats_text += "PLAYER MOVEMENT ANALYSIS:\n"
        stats_text += "-" * 40 + "\n"

        if not player_distances.empty:
            for _, player in player_distances.iterrows():
                stats_text += f"\nPlayer {player['ID']}: \n"
                stats_text += f"Total Distance:
{player['TotalDistance']:.1f}m\n"
                if 'AverageSpeedKmh' in player:
                    stats_text += f"Average Speed:
{player['AverageSpeedKmh']:.1f} km/h\n"
                if 'MaxSpeedKmh' in player:

                    stats_text += f"Max Speed: {player['MaxSpeedKmh']:.1f}
km/h\n"

                if 'TimeActive' in player:
                    stats_text += f"Active Time:
{player['TimeActive']:.1f}s\n"
                if 'CourtCoverage' in player:
                    stats_text += f"Court Coverage:
{player['CourtCoverage']:.1f}m²\n"

```

```

        if 'DataPoints' in player:
            stats_text += f" Data Points: {player['DataPoints']}\n"
        else:
            stats_text += "No player movement data available.\n"

    # Ball Statistics
    stats_text += "\n\nBALL MOVEMENT ANALYSIS:\n"
    stats_text += "-" * 40 + "\n"
    if ball_results:
        stats_text += f"Total Distance:
{ball_results.get('total_distance', 0):.1f}m\n"
        stats_text += f"Average Speed: {ball_results.get('average_speed_kmh', 0):.1f}
km/h\n"
        stats_text += f"Max Speed: {ball_results.get('max_speed_kmh',
0):.1f} km/h\n"
        stats_text += f"Detection Gaps:
{ball_results.get('detection_gaps', 0)}\n"
        stats_text += f"Valid Points: {ball_results.get('valid_points',
0)}\n"
    else:
        stats_text += "No ball movement data available.\n"

    # Corner Detection Statistics
    if frame_info:
        stats_text += "\n\nCORNER DETECTION ANALYSIS:\n"
        stats_text += "-" * 40 + "\n"
        valid_frames = sum(1 for info in frame_info if info.get('valid',
False))

        total_frames = len(frame_info)
        stats_text += f"Frames Processed: {total_frames}\n"
        stats_text += f"Valid Corner Detections: {valid_frames}\n"
        stats_text += f"Success Rate:
{(valid_frames/total_frames*100):.1f}%\n"

        if average_corners:
            stats_text += f"\nAverage Corner Positions:\n"
            corner_labels = ["Bottom-Left", "Bottom-Right", "Top-Right", "
Top-Left"]
            for i, (corner, label) in enumerate(zip(average_corners,
corner_labels)):
                stats_text += f" {label}: ({corner[0]:.1f},
{corner[1]:.1f})\n"

    # Data Statistics
    stats_text += "\n\nDATA STATISTICS:\n"
    stats_text += "-" * 40 + "\n"
    stats_text += f"Total Tracking Points: {len(df_mapped)}\n"
    stats_text += f"Unique Objects: {df_mapped['ID'].nunique()}\n"
    stats_text += f"Frame Range: {df_mapped['frame'].min()} -
{df_mapped['frame'].max()}\n"

    # Update the text widget
    self.stats_text.delete(1.0, tk.END)
    self.stats_text.insert(1.0, stats_text)

except Exception as e:
    self.log_message(f"Error displaying results: {e}", "ERROR")

def open_output_folder(self):
    """Open the output folder in file explorer"""
    output_dir = os.path.abspath("./Out")

```



```

if os.path.exists(output_dir):
    try:
        if sys.platform == "win32":
            os.startfile(output_dir)
        elif sys.platform == "darwin":
            subprocess.run(["open", output_dir])
        else:
            subprocess.run(["xdg-open", output_dir])
    except Exception as e:
        self.log_message(f"Error opening output folder: {e}", "ERROR")
else:
    messagebox.showinfo("Info", "Output folder does not exist yet.")
    # Create the folder if it doesn't exist
    try:
        os.makedirs(output_dir, exist_ok=True)
        self.log_message("Created output folder: ./Out")
    except Exception as e:
        self.log_message(f"Error creating output folder: {e}", "ERROR")

def open_corner_test(self):
    """Open corner detection test window"""
    CornerTestWindow(self.root)

def view_results(self):
    """Open results viewer window"""
    if self.results:
        ResultsViewerWindow(self.root, self.results)
    else:
        messagebox.showinfo("Info", "No results available. Please run
analysis first.")

def show_about(self):
    """Show about dialog"""
    about_text = """Tennis Court Tracking System v1.0

A computer vision application for analyzing tennis gameplay using:
• YOLO object detection
• DeepSORT tracking algorithms
• Homography-based coordinate mapping
• Player and ball movement analysis

Features:
• Video playback with controls
• Output file management
• In-app file viewing

• Real-time progress tracking
• Combined processing pipeline

Developed for tennis performance analysis and statistics generation."""

    messagebox.showinfo("About", about_text)

def update_progress(self, message, value):
    """Update progress bar and label"""
    self.root.after(0, lambda: self.progress_var.set(value))
    self.root.after(0, lambda: self.progress_label.config(text=message))

def log_message(self, message, level="INFO"):
    """Add message to log output"""
    timestamp = datetime.now().strftime("%H:%M:%S")

```

```

def update_log():
    self.log_text.insert(tk.END, f"[{timestamp}] {level}: {message}\n")
    self.log_text.see(tk.END)

self.root.after(0, update_log)

def update_ui_state(self):
    """Update UI state based on processing status"""
    state = "disabled" if self.processing else "normal"
    self.tracking_button.config(state=state)
    self.analysis_button.config(state=state)

class CornerTestWindow:
    """Window for testing corner detection on single frames"""

    def __init__(self, parent):
        self.window = tk.Toplevel(parent)
        self.window.title("Corner Detection Test")
        self.window.geometry("800x600")
        self.window.transient(parent)

        self.image_path = None
        self.original_image = None
        self.processed_image = None

        self.create_widgets()

    def create_widgets(self):
        """Create widgets for corner test window"""
        # Control frame
        control_frame = ttk.Frame(self.window, padding="10")
        control_frame.grid(row=0, column=0, sticky=(tk.W, tk.E))

        ttk.Button(control_frame, text="Select Image",
command=self.select_image).grid(row=0,
        column=0, padx=5)
        ttk.Button(control_frame, text="Detect Corners",
command=self.detect_corners).grid(row=0,
        column=1, padx=5)
        ttk.Button(control_frame, text="Save Result",
command=self.save_result).grid(row=0,
        column=2, padx=5)

        self.status_label = ttk.Label(control_frame, text="Select an image to
start")
        self.status_label.grid(row=0, column=3, padx=(20, 0))

        # Image display frame
        image_frame = ttk.Frame(self.window, padding="10")
        image_frame.grid(row=1, column=0, sticky=(tk.W, tk.E, tk.N, tk.S))

        self.image_label = ttk.Label(image_frame, text="Image will appear here",
        anchor='center', background='#f0f0f0',
relief='sunken')
        self.image_label.grid(row=0, column=0, sticky=(tk.W, tk.E, tk.N, tk.S))

        # Configure grid weights
        self.window.columnconfigure(0, weight=1)
        self.window.rowconfigure(1, weight=1)
        image_frame.columnconfigure(0, weight=1)

```

```

        image_frame.rowconfigure(0, weight=1)

    def select_image(self):
        """Select image for corner detection"""
        file_path = filedialog.askopenfilename(
            title="Select Tennis Court Image",
            filetypes=[
                ("Image files", "*.png *.jpg *.jpeg *.bmp *.tiff"),
                ("All files", "*.*")
            ]
        )

        if file_path:
            self.image_path = file_path
            self.load_image()
            self.status_label.config(text="Image loaded - ready for corner
detection")

    def load_image(self):
        """Load and display the selected image"""
        try:
            # Load with OpenCV
            self.original_image = cv2.imread(self.image_path)

            # Convert to RGB for display
            image_rgb = cv2.cvtColor(self.original_image, cv2.COLOR_BGR2RGB)

            # Resize for display
            height, width = image_rgb.shape[:2]

            max_width, max_height = 700, 400

            if width > max_width or height > max_height:
                scale = min(max_width/width, max_height/height)
                new_width = int(width * scale)
                new_height = int(height * scale)
                image_rgb = cv2.resize(image_rgb, (new_width, new_height))

            # Convert to PhotoImage
            image = Image.fromarray(image_rgb)
            photo = ImageTk.PhotoImage(image)

            self.image_label.configure(image=photo, text="")
            self.image_label.image = photo

        except Exception as e:
            messagebox.showerror("Error", f"Error loading image: {e}")

    def detect_corners(self):
        """Detect and display corners on the image"""
        if self.original_image is None:
            messagebox.showwarning("Warning", "Please select an image first.")
            return

        try:
            # Import corner detector
            from Distance_measurement.corner_detection import CornerDetector

            cd = CornerDetector()
            corners = cd.detect_court_corners(self.original_image.copy(),
debug=False)

```

```

        # Draw corners on image
        result_image = self.original_image.copy()
        cd.draw_corners_on_frame(result_image, corners)

        # Convert to RGB for display
        image_rgb = cv2.cvtColor(result_image, cv2.COLOR_BGR2RGB)

        # Resize for display
        height, width = image_rgb.shape[:2]
        max_width, max_height = 700, 400

        if width > max_width or height > max_height:
            scale = min(max_width/width, max_height/height)
            new_width = int(width * scale)
            new_height = int(height * scale)
            image_rgb = cv2.resize(image_rgb, (new_width, new_height))

        # Convert to PhotoImage
        image = Image.fromarray(image_rgb)

        photo = ImageTk.PhotoImage(image)

        self.image_label.configure(image=photo)
        self.image_label.image = photo

        self.processed_image = result_image
        self.status_label.config(text=f"Found {len(corners)} corners")

    except Exception as e:
        messagebox.showerror("Error", f"Error detecting corners: {e}")

def save_result(self):
    """Save the processed image with corners"""
    if self.processed_image is None:
        messagebox.showwarning("Warning", "No processed image to save.")
        return

    file_path = filedialog.asksaveasfilename(
        title="Save Corner Detection Result",
        defaultextension=".png",
        filetypes=[
            ("PNG files", "*.png"),
            ("JPEG files", "*.jpg"),
            ("All files", "*.*")
        ]
    )

    if file_path:
        try:
            cv2.imwrite(file_path, self.processed_image)
            messagebox.showinfo("Success", "Image saved successfully!")
        except Exception as e:
            messagebox.showerror("Error", f"Error saving image: {e}")

class ResultsViewerWindow:
    """Window for viewing detailed analysis results"""

    def __init__(self, parent, results):
        self.window = tk.Toplevel(parent)
        self.window.title("Analysis Results Viewer")
        self.window.geometry("1000x700")

```

```

self.window.transient(parent)

self.results = results
self.create_widgets()
self.load_results()

def create_widgets(self):
    """Create widgets for results viewer"""

    # Create notebook for different result views
    self.notebook = ttk.Notebook(self.window, padding="10")
    self.notebook.grid(row=0, column=0, sticky=(tk.W, tk.E, tk.N, tk.S))

    # Player data tab
    self.player_frame = ttk.Frame(self.notebook)
    self.notebook.add(self.player_frame, text="Player Data")

    # Ball data tab
    self.ball_frame = ttk.Frame(self.notebook)
    self.notebook.add(self.ball_frame, text="Ball Data")

    # Corner data tab
    self.corner_frame = ttk.Frame(self.notebook)
    self.notebook.add(self.corner_frame, text="Corner Analysis")

    # Raw data tab
    self.raw_frame = ttk.Frame(self.notebook)
    self.notebook.add(self.raw_frame, text="Raw Data")

    # Configure grid weights
    self.window.columnconfigure(0, weight=1)
    self.window.rowconfigure(0, weight=1)

def load_results(self):
    """Load and display results in appropriate tabs"""
    try:
        df_mapped, player_distances, ball_results, average_corners,
frame_info = self.results

        # Player data
        self.create_player_view(player_distances)

        # Ball data
        self.create_ball_view(ball_results)

        # Corner data
        self.create_corner_view(average_corners, frame_info)

        # Raw data
        self.create_raw_view(df_mapped)

    except Exception as e:
        messagebox.showerror("Error", f"Error loading results: {e}")

def create_player_view(self, player_distances):
    """Create player data view"""
    if player_distances.empty:
        ttk.Label(self.player_frame, text="No player data
available").pack(pady=20)
    return

```

```

# Create treeview for player data
columns = list(player_distances.columns)
tree = ttk.Treeview(self.player_frame, columns=columns[1:], show='tree
headings')

# Configure columns
tree.heading('#0', text='Player ID')
tree.column('#0', width=100)

for col in columns[1:]:
    tree.heading(col, text=col)
    tree.column(col, width=120)

# Insert data
for _, row in player_distances.iterrows():
    values = [f"{val:.2f}" if isinstance(val, (int, float)) else str(val)
               for val in row.iloc[1:]]
    tree.insert('', tk.END, text=str(row.iloc[0]), values=values)

# Add scrollbars
v_scrollbar = ttk.Scrollbar(self.player_frame, orient=tk.VERTICAL,
command=tree.yview)
h_scrollbar = ttk.Scrollbar(self.player_frame, orient=tk.HORIZONTAL,
command=tree.xview)
tree.configure(yscrollcommand=v_scrollbar.set,
xscrollcommand=h_scrollbar.set)

# Grid layout
tree.grid(row=0, column=0, sticky=(tk.W, tk.E, tk.N, tk.S))
v_scrollbar.grid(row=0, column=1, sticky=(tk.N, tk.S))
h_scrollbar.grid(row=1, column=0, sticky=(tk.W, tk.E))

self.player_frame.columnconfigure(0, weight=1)
self.player_frame.rowconfigure(0, weight=1)

def create_ball_view(self, ball_results):
    """Create ball data view"""
    if not ball_results:
        ttk.Label(self.ball_frame, text="No ball data
available").pack(pady=20)
        return

    # Create text widget for ball data
    text_widget = scrolledtext.ScrolledText(self.ball_frame, wrap=tk.WORD)
    text_widget.grid(row=0, column=0, sticky=(tk.W, tk.E, tk.N, tk.S),
padx=10, pady=10)

    # Format ball data
    ball_text = "BALL MOVEMENT ANALYSIS\n" + "="*50 + "\n\n"
    for key, value in ball_results.items():
        if isinstance(value, (int, float)):
            ball_text += f"{key.replace('_', ' ').title():} {value:.2f}\n"
        else:
            ball_text += f"{key.replace('_', ' ').title():} {value}\n"

    text_widget.insert(1.0, ball_text)

    text_widget.config(state='disabled')

self.ball_frame.columnconfigure(0, weight=1)
self.ball_frame.rowconfigure(0, weight=1)

```

```

def create_corner_view(self, average_corners, frame_info):
    """Create corner analysis view"""
    # Create text widget for corner data
    text_widget = scrolledtext.ScrolledText(self.corner_frame, wrap=tk.WORD)
    text_widget.grid(row=0, column=0, sticky=(tk.W, tk.E, tk.N, tk.S),
padx=10, pady=10)

    corner_text = "CORNER DETECTION ANALYSIS\n" + "="*50 + "\n\n"

    # Average corners
    if average_corners:
        corner_text += "Average Corner Positions:\n"
        corner_labels = ["Bottom-Left", "Bottom-Right", "Top-Right", "
Top-Left"]
        for i, (corner, label) in enumerate(zip(average_corners,
corner_labels)):
            corner_text += f" {label}: ({corner[0]:.1f}, {corner[1]:.1f})\n"
            corner_text += "\n"

    # Frame analysis
    if frame_info:
        valid_frames = sum(1 for info in frame_info if info.get('valid',
False))
        total_frames = len(frame_info)
        corner_text += f"Frame Analysis:\n"
        corner_text += f" Total Frames Processed: {total_frames}\n"
        corner_text += f" Valid Corner Detections: {valid_frames}\n"
        corner_text += f" Success Rate:
{valid_frames/total_frames*100:.1f}%\n\n"

        corner_text += "Frame-by-Frame Results:\n"
        for info in frame_info[:10]: # Show first 10 frames
            status = "■" if info.get('valid', False) else "□"
            corner_text += f" Frame {info.get('frame_number', 'N/A')}:
{status} "
            corner_text += f"({info.get('corners_found', 0)} corners found)\n"

        if len(frame_info) > 10:
            corner_text += f" ... and {len(frame_info) - 10} more frames\n"

    text_widget.insert(1.0, corner_text)
    text_widget.config(state='disabled')

    self.corner_frame.columnconfigure(0, weight=1)
    self.corner_frame.rowconfigure(0, weight=1)

def create_raw_view(self, df_mapped):
    """Create raw data view"""
    if df_mapped.empty:
        ttk.Label(self.raw_frame, text="No tracking data
available").pack(pady=20)
        return

    # Create treeview for raw data (show first 1000 rows)
    display_df = df_mapped.head(1000)
    columns = list(display_df.columns)

    tree = ttk.Treeview(self.raw_frame, columns=columns, show='headings',
height=20)

    # Configure columns

```

```

        for col in columns:
            tree.heading(col, text=col)
            tree.column(col, width=100)

        # Insert data
        for _, row in display_df.iterrows():
            values = [f"{val:.3f}" if isinstance(val, float) else str(val) for
val in row]
            tree.insert('', tk.END, values=values)

        # Add scrollbars
        v_scrollbar = ttk.Scrollbar(self.raw_frame, orient=tk.VERTICAL,
command=tree.yview)
        h_scrollbar = ttk.Scrollbar(self.raw_frame, orient=tk.HORIZONTAL,
command=tree.xview)
        tree.configure(yscrollcommand=v_scrollbar.set,
xscrollcommand=h_scrollbar.set)

        # Info label
        info_label = ttk.Label(self.raw_frame,
                               text=f"Showing first 1000 of {len(df_mapped)} total
tracking points")
        info_label.grid(row=0, column=0, columnspan=2, pady=5)

        # Grid layout
        tree.grid(row=1, column=0, sticky=(tk.W, tk.E, tk.N, tk.S))
        v_scrollbar.grid(row=1, column=1, sticky=(tk.N, tk.S))
        h_scrollbar.grid(row=2, column=0, sticky=(tk.W, tk.E))

        self.raw_frame.columnconfigure(0, weight=1)
        self.raw_frame.rowconfigure(1, weight=1)

def main():
    """Main application entry point"""
    # Create output directory if it doesn't exist
    os.makedirs("./Out", exist_ok=True)

    # Create and run the application
    root = tk.Tk()
    app = TennisTrackingApp(root)

    # Center the window
    root.update_idletasks()
    x = (root.winfo_screenwidth() // 2) - (root.winfo_width() // 2)
    y = (root.winfo_screenheight() // 2) - (root.winfo_height() // 2)
    root.geometry(f"+{x}+{y}")

    root.mainloop()

if __name__ == "__main__":
    main()

```


[FOLDER] Detect_and_Track

Location: Detect_and_Track

[FILE] __init__.py

Path: Detect_and_Track__init__.py

File is empty or could not be read.

[FILE] court_detection.py

Path: Detect_and_Track\court_detection.py

```
import cv2
import numpy as np
from sklearn.cluster import DBSCAN
from sklearn.linear_model import LinearRegression

def detect_court_corners_simple(frame, debug=False):
    """
    Improved approach: Find the four corners by tracing the same line segments
    that contain the bottom corners to find the corresponding top corners.
    """
    print("[DEBUG] Converting to grayscale...")
    gray = cv2.cvtColor(frame, cv2.COLOR_BGR2GRAY)

    print("[DEBUG] Applying Gaussian blur...")
    blurred = cv2.GaussianBlur(gray, (5, 5), 0)

    print("[DEBUG] Performing Canny edge detection...")
    edges = cv2.Canny(blurred, 50, 150, apertureSize=3)

    print("[DEBUG] Running Hough Line Transform...")
    lines = cv2.HoughLinesP(edges, 1, np.pi / 180, threshold=80,
minLineLength=100, maxLineGap=50)

    debug_img = frame.copy()
    h, w = frame.shape[:2]

    if lines is None:
        print("[DEBUG] No lines detected.")
        return []

    print(f"[DEBUG] Number of lines detected: {len(lines)}")

    # Draw all detected lines in green
    for line in lines:
        x1, y1, x2, y2 = line[0]
```

```

        cv2.line(debug_img, (x1, y1), (x2, y2), (0, 255, 0), 1)

# Find potential sideline segments
sideline_segments = find_sideline_segments(lines, w, h)

if len(sideline_segments) < 2:
    print("[DEBUG] Not enough sideline segments found.")
    return []

# Group segments into left and right sidelines
left_sideline, right_sideline = group_sideline_segments(sideline_segments, w,
debug_img)

if not left_sideline or not right_sideline:
    print("[DEBUG] Could not identify both sidelines.")
    return []

# Find corners by tracing the same lines that contain bottom corners
corners = find_corners_by_line_tracing(left_sideline, right_sideline,
debug_img, h, w)

if debug:
    print("[DEBUG] Displaying image with detected sidelines and corners...")
    cv2.imshow('Improved Corner Detection', debug_img)
    cv2.waitKey(0)
    cv2.destroyAllWindows()

print(f"[DEBUG] Found {len(corners)} corners: {corners}")
return corners

def find_sideline_segments(lines, img_width, img_height):
    """
    Find line segments that could be part of tennis court sidelines.
    """
    sideline_segments = []

    for line in lines:
        x1, y1, x2, y2 = line[0]

        # Calculate line properties
        dx = x2 - x1
        dy = y2 - y1
        length = np.sqrt(dx**2 + dy**2)

        # Calculate angle from horizontal
        if abs(dx) < 1e-6:
            angle = 90.0
        else:
            angle = abs(np.arctan(dy / dx) * 180 / np.pi)

        # Calculate slope for line extension
        if abs(dx) < 1e-6:
            slope = float('inf')
        else:
            slope = dy / dx

        # Filter for sideline candidates:
        # 1. Reasonably long lines
        # 2. Not perfectly horizontal (angle > 30 degrees)
        # 3. Not perfectly vertical (angle < 85 degrees) to account for

```

```

perspective
    min_length = max(80, min(img_width, img_height) * 0.15)

    if length > min_length and 30 < angle < 85:
        # Calculate center point and average x-coordinate
        center_x = (x1 + x2) / 2
        center_y = (y1 + y2) / 2

        sideline_segments.append({
            'line': (x1, y1, x2, y2),
            'length': length,
            'angle': angle,
            'slope': slope,
            'center_x': center_x,
            'center_y': center_y,
            'top_y': min(y1, y2),
            'bottom_y': max(y1, y2),
            'top_point': (x1, y1) if y1 < y2 else (x2, y2),
            'bottom_point': (x1, y1) if y1 > y2 else (x2, y2)
        })

    print(f"[DEBUG] Found {len(sideline_segments)} potential sideline segments")
    return sideline_segments

def group_sideline_segments(segments, img_width, debug_img):
    """
    Group segments into left and right sidelines using clustering.
    """
    if len(segments) < 2:
        return [], []

    # Extract x-coordinates for clustering
    x_coords = np.array([[seg['center_x']] for seg in segments])

    # Use DBSCAN to cluster segments by x-coordinate
    clustering = DBSCAN(eps=img_width * 0.1, min_samples=1).fit(x_coords)
    labels = clustering.labels_

    # Group segments by cluster
    clusters = {}
    for i, label in enumerate(labels):
        if label not in clusters:
            clusters[label] = []
        clusters[label].append(segments[i])

    # Find the two main clusters (left and right sidelines)
    cluster_info = []
    for label, cluster_segments in clusters.items():
        if len(cluster_segments) >= 1: # At least 1 segment
            avg_x = np.mean([seg['center_x'] for seg in cluster_segments])
            total_length = sum([seg['length'] for seg in cluster_segments])
            cluster_info.append((label, avg_x, total_length, cluster_segments))

    # Sort by total length (descending) and take the two longest
    cluster_info.sort(key=lambda x: x[2], reverse=True)

    if len(cluster_info) < 2:
        print("[DEBUG] Could not find two distinct sideline clusters.")
        return [], []

```

```

# Assign left and right based on x-coordinate
cluster1 = cluster_info[0]
cluster2 = cluster_info[1]

if cluster1[1] < cluster2[1]: # cluster1 is more to the left
    left_sideline = cluster1[3]
    right_sideline = cluster2[3]
else:
    left_sideline = cluster2[3]
    right_sideline = cluster1[3]

# Draw clustered segments
colors = [(255, 0, 0), (0, 0, 255)] # Blue for left, Red for right
for i, sideline in enumerate([left_sideline, right_sideline]):
    color = colors[i]
    for seg in sideline:
        x1, y1, x2, y2 = seg['line']
        cv2.line(debug_img, (x1, y1), (x2, y2), color, 3)

print(f"[DEBUG] Left sideline: {len(left_sideline)} segments, Right sideline:
{len(right_sideline)} segments")
return left_sideline, right_sideline

def find_corners_by_line_tracing(left_sideline, right_sideline, debug_img,
img_height, img_width):
    """
    Find the four corners by:
    1. Finding the bottommost points for each sideline
    2. Identifying which line segment contains each bottom corner
    3. Extending that same line upward to find the corresponding top corner
    """
    corners = []

    # Process left sideline
    if left_sideline:
        left_bottom, left_top = find_corner_pair(left_sideline, img_height,
img_width, is_left=True)

        if left_bottom and left_top:
            corners.extend([left_top, left_bottom])

        # Draw corners and labels
        cv2.circle(debug_img, left_top, 12, (0, 255, 0), -1)
        cv2.circle(debug_img, left_bottom, 12, (0, 255, 0), -1)
        cv2.putText(debug_img, "TL", (left_top[0]-20, left_top[1]-10),
cv2.FONT_HERSHEY_SIMPLEX, 0.7, (0, 255, 0), 2)
        cv2.putText(debug_img, "BL", (left_bottom[0]-20, left_bottom[1]+25),

cv2.FONT_HERSHEY_SIMPLEX, 0.7, (0, 255, 0), 2)

        # Draw the extended line
        cv2.line(debug_img, left_top, left_bottom, (0, 255, 255), 2)

        print(f"[DEBUG] Left sideline corners: Top={left_top},
Bottom={left_bottom}")

    # Process right sideline
    if right_sideline:
        right_bottom, right_top = find_corner_pair(right_sideline, img_height, img_width,
is_left=False)

```

```

    if right_bottom and right_top:
        corners.extend([right_top, right_bottom])

        # Draw corners and labels
        cv2.circle(debug_img, right_top, 12, (0, 255, 0), -1)
        cv2.circle(debug_img, right_bottom, 12, (0, 255, 0), -1)
        cv2.putText(debug_img, "TR", (right_top[0]+15, right_top[1]-10),
                    cv2.FONT_HERSHEY_SIMPLEX, 0.7, (0, 255, 0), 2)
        cv2.putText(debug_img, "BR", (right_bottom[0]+15,
right_bottom[1]+25),
                    cv2.FONT_HERSHEY_SIMPLEX, 0.7, (0, 255, 0), 2)

        # Draw the extended line
        cv2.line(debug_img, right_top, right_bottom, (0, 255, 255), 2)

        print(f"[DEBUG] Right sideline corners: Top={right_top},
Bottom={right_bottom}")

    return corners

def find_corner_pair(sideline_segments, img_height, img_width, is_left=True):
    """
    Find the bottom corner and corresponding top corner for a sideline.

    Algorithm:
    1. Find the segment that contains the bottommost point
    2. Find all segments that are collinear with this segment
    3. Among collinear segments, find the one with the topmost point
    """
    if not sideline_segments:
        return None, None

    # Find the segment that contains the bottommost point
    bottom_point = None
    bottom_y = -1
    bottom_segment = None

    for seg in sideline_segments:
        x1, y1, x2, y2 = seg['line']

        # Check both endpoints
        for point in [(x1, y1), (x2, y2)]:
            if point[1] > bottom_y:
                bottom_y = point[1]
                bottom_point = point
                bottom_segment = seg

    if bottom_segment is None:
        return None, None

    # Find all segments that are collinear with the bottom segment
    collinear_segments = find_collinear_segments(bottom_segment,
sideline_segments)

    # Among collinear segments, find the topmost point
    top_point = find_topmost_point_from_collinear_segments(collinear_segments)

    return bottom_point, top_point

```

```

def find_collinear_segments(reference_segment, all_segments):
    """
    Find all segments that are collinear (on the same line) with the reference
    segment.
    """
    collinear_segments = [reference_segment]

    ref_x1, ref_y1, ref_x2, ref_y2 = reference_segment['line']
    ref_slope = reference_segment['slope']

    # Calculate reference line parameters
    # Line equation: ax + by + c = 0
    # For line through (x1,y1) and (x2,y2): (y2-y1)x - (x2-x1)y + (x2-x1)y1 -
    (y2-y1)x1 = 0
    a = ref_y2 - ref_y1
    b = ref_x1 - ref_x2
    c = (ref_x2 - ref_x1) * ref_y1 - (ref_y2 - ref_y1) * ref_x1

    # Normalize to avoid floating point issues
    norm = np.sqrt(a*a + b*b)
    if norm > 0:
        a, b, c = a/norm, b/norm, c/norm

    for seg in all_segments:
        if seg == reference_segment:
            continue

        x1, y1, x2, y2 = seg['line']

        # Check if both endpoints of this segment lie on the reference line
        # Distance from point (x,y) to line ax + by + c = 0 is |ax + by + c|
        dist1 = abs(a * x1 + b * y1 + c)

        dist2 = abs(a * x2 + b * y2 + c)

        # If both endpoints are close to the line (within tolerance), consider it
        # collinear
        tolerance = 10.0 # pixels
        if dist1 < tolerance and dist2 < tolerance:
            collinear_segments.append(seg)

    print(f"[DEBUG] Found {len(collinear_segments)} collinear segments")
    return collinear_segments

def find_topmost_point_from_collinear_segments(collinear_segments):
    """
    Find the topmost point among all collinear segments.
    """
    if not collinear_segments:
        return None

    topmost_point = None
    topmost_y = float('inf')

    for seg in collinear_segments:
        x1, y1, x2, y2 = seg['line']

        # Check both endpoints
        for point in [(x1, y1), (x2, y2)]:
            if point[1] < topmost_y: # Lower y value means higher on screen

```

```

        topmost_y = point[1]
        topmost_point = point

    return topmost_point

def extend_line_to_find_top_corner(segment, bottom_point, img_height, img_width):
    """
    This function is kept for compatibility but not used in the new approach.
    """
    x1, y1, x2, y2 = segment['line']

    # Simply return the topmost point of this specific segment
    if y1 < y2:
        return (x1, y1)
    else:
        return (x2, y2)

def get_ordered_corners(corners):
    """
    Return corners in a consistent order: [top-left, top-right, bottom-right,
    bottom-left]
    """

    if len(corners) != 4:
        return corners

    # Sort by y-coordinate to separate top and bottom
    corners = sorted(corners, key=lambda p: p[1])

    # Top two points
    top_points = corners[:2]
    bottom_points = corners[2:]

    # Sort by x-coordinate
    top_points = sorted(top_points, key=lambda p: p[0])
    bottom_points = sorted(bottom_points, key=lambda p: p[0])

    # Return in order: top-left, top-right, bottom-right, bottom-left
    return [top_points[0], top_points[1], bottom_points[1], bottom_points[0]]

# Example usage function
def process_frame(frame):
    """
    Process a single frame to detect tennis court corners.
    """
    corners = detect_court_corners_simple(frame, debug=True)

    if len(corners) == 4:
        ordered_corners = get_ordered_corners(corners)
        print(f"[RESULT] Court corners found:")
        print(f" Top-left: {ordered_corners[0]}")
        print(f" Top-right: {ordered_corners[1]}")
        print(f" Bottom-right: {ordered_corners[2]}")
        print(f" Bottom-left: {ordered_corners[3]}")
        return ordered_corners
    else:
        print(f"[RESULT] Could not find all 4 corners. Found {len(corners)}
corners.")
    return corners

```

```

# For testing with an image file
def test_with_image(image_path):
    """
    Test the corner detection with an image file.
    """
    frame = cv2.imread(image_path)
    if frame is None:
        print(f"Error: Could not load image from {image_path}")
        return

    corners = process_frame(frame)
    return corners

```

[FILE] create_tracking_boxes_video.py

Path: Detect_and_Track\create_tracking_boxes_video.py

```

from .deep_sort.tracker_implementation import trackingXl5
from .deep_sort.tracks_cleaning import clean_tracks
from .deep_sort.tracking_video import make_tracking_video
from .yoloV5.load_models import yoloV5l

def create_tracking_boxes_video(video_path , out_name):
    """
    this functions is to make video with objects tracked.

    Parameters
    -----
    video_path : string
        the path of the directory of the processed video.
    out_name : string
        the name to save the video with objects tracked with.
    """

    import cv2
    vid = cv2.VideoCapture(video_path)
    width = int(vid.get(cv2.CAP_PROP_FRAME_WIDTH))
    height = int(vid.get(cv2.CAP_PROP_FRAME_HEIGHT))
    vid.release()
    imgsz = min(width, height)
    modelv5l, ball_modelv5l = yoloV5l(imgsz=imgsz)
    iframes, itboxes, fps = trackingXl5(modelv5l, ball_modelv5l, video_path)
    make_tracking_video(iframes, itboxes, f'./Out/{out_name}_out_tracked.mp4',
    fps, draw=True)

```

[FILE] create_tracking_video_and_init_data.py

Path: Detect_and_Track\create_tracking_video_and_init_data.py


```

import pandas as pd
from .init_dataframe.create_df import creatInitDataFrame
from .get_tracks import get_video_tracks
from .deep_sort.tracker_implementation import trackingXl5
from .deep_sort.tracks_cleaning import clean_tracks
from .deep_sort.tracking_video import make_tracking_video
from .yoloV5.load_models import yoloV5l

def create_tracking_video_and_init_data(video_path, out_name, teams_colors,
ball_only=True):
    """
    Combined function to process video once and create both tracking video and
    init CSV.

    Parameters
    -----
    video_path : string
        the path of the directory of the processed video.
    out_name : string
        the name to save outputs with.
    teams_colors : list of strings
        list of the colors to classify the teams.
    ball_only : boolean
        whether or not to save only the frames with the ball detected in them.

    Returns
    -----
    init_df : pandas dataframe
        the initial unmapped dataframe.
    ziframes : list
        list of cleaned frames.
    zitboxes : list
        list of cleaned tracking boxes.
    """

    from Detect_and_Track.deep_sort.tracker_implementation import trackingXl5
    from Detect_and_Track.deep_sort.tracks_cleaning import clean_tracks
    from Detect_and_Track.deep_sort.tracking_video import make_tracking_video
    from Detect_and_Track.yoloV5.load_models import yoloV5l
    from Detect_and_Track.init_dataframe.create_df import creatInitDataFrame
    import os

    # Ensure output directory exists
    os.makedirs('./Out', exist_ok=True)

    import cv2
    vid = cv2.VideoCapture(video_path)
    width = int(vid.get(cv2.CAP_PROP_FRAME_WIDTH))
    height = int(vid.get(cv2.CAP_PROP_FRAME_HEIGHT))
    vid.release()
    imgsz = min(width, height)

    print("Loading YOLO models...")
    modelv5l, ball_modelv5l = yoloV5l(imgsz=imgsz)
    print("IMGSZ:", modelv5l.imgsz)

    print("Processing video with tracking and timing...")
    # This is where all the timing measurements happen
    iframes, itboxes, fps = trackingXl5(modelv5l, ball_modelv5l, video_path)

    print("Cleaning tracks...")

```

```

ziframes, zitboxes = clean_tracks(iframes, itboxes, ball_only)

print("Creating initial dataframe...")
# Create the init CSV
init_df = creatInitDataFrame(zitboxes, ziframes, teams_colors)
init_df.to_csv(f'./Out/{out_name}_init_df.csv', index=False)
print(f"Init dataframe saved to: ./Out/{out_name}_init_df.csv")

print("Creating tracking videos...")
# Create clean video (without bounding boxes)
make_tracking_video(ziframes, zitboxes, f'./Out/{out_name}_out.mp4', fps,
draw=False)
print(f"Clean video saved to: ./Out/{out_name}_out.mp4")

# Create tracking video (with bounding boxes)
make_tracking_video(ziframes, zitboxes, f'./Out/{out_name}_out_tracked.mp4',
fps, draw=True)
print(f"Tracking video saved to: ./Out/{out_name}_out_tracked.mp4")

print("All outputs created successfully!")
return init_df, ziframes, zitboxes

```

[FILE] demo_detection.py

Path: Detect_and_Track\demo_detection.py

```

# import cv2
import matplotlib.pyplot as plt
from .yoloV5.load_models import yoloV5l
from .yoloV5.detector import detectXl5

def detect_demo(path = "./Data/Capture.JPG"):
    '''
    this functions is to apply detection on an image.

    Parameters
    -----
    path : string
        the path of the directory of the processed image.
    '''

    model1 , ball_model = yoloV5l()
    img, res = detectXl5(model1, path, show=True)
    print(f'bounding boxes: {res}')

    ball_img, ball_res = detectXl5(ball_model, path, show=True)
    print(ball_res)

    #show one player
    player = res[5]
    print(f'player with index 5: {player}')
    # cv2.imshow(img[player[1] - 30:player[3] + 30, player[0] - 30 :player[2] +
30, :-1])
    plt.figure(figsize=(12, 8))
    plt.imshow(img[player[1] - 30:player[3] + 30, player[0] - 30 :player[2] + 30, :
])
    print("SADA BI TREBALO DA SE POKAZE")

```

```
plt.show()
return img, res
```

[FILE] demo_initial_df.py

Path: Detect_and_Track\demo_initial_df.py

```
import pandas as pd
import matplotlib.pyplot as plt
from .init_dataframe.create_df import creatInitDataFrame
from .yoloV5.load_models import yoloV5l
from .yoloV5.detector import detectXl5

def init_df_demo(path = "./Data/Capture.JPG", colors = []):
    """
    this functions is to create an initial unmapped dataframe demo on a single
    frame/ image,
    and plot the detected objects in the image.

    Parameters
    -----
    path : string
        the path of the directory of the processed image.
    """
    image_path = path
    modelV5l, _ = yoloV5l()
    img ,detections = detectXl5(modelV5l, image_path, False)
    init_df = creatInitDataFrame([detections], [img], frame_colors = colors)
    init_df = init_df.reset_index(drop=True)

    plt.figure(figsize=(12, 8))
    plt.imshow(img)
    for i in range(len(init_df)):
        plt.scatter(init_df.iloc[i][2], init_df.iloc[i][3], s= 20, c =
init_df.iloc[i][5])
        plt.annotate(init_df.iloc[i][1], (init_df.iloc[i][2] + 4, init_df.iloc[i][3]
+ 4), fontsize=15)
    plt.show()

    print(init_df)
```

[FILE] demo_tracking.py

Path: Detect_and_Track\demo_tracking.py

```
# import cv2
import matplotlib.pyplot as plt
from .get_tracks import get_video_tracks

def track_demo(video_path = "./Data/benz.mp4" , out_name = 'benz'):
```

```

'''
    this functions is to apply tracking on a video, and track certain player
    within 10 frames

    Parameters
    -----
    video_path : string
        the path of the directory of the processed video.
    out_name : string
        the name to save the video with objects tracked with. .
'''

iframes, itboxes = get_video_tracks(video_path, out_name = out_name)
plt.figure(figsize=(12, 8))
frame_num = 30
print(len(itboxes))
print(len(iframes))
print(iframes[frame_num].shape)
# cv2.imshow(iframes[frame_num][:, :, ::-1])
plt.imshow(iframes[frame_num])
plt.show()

t10 = itboxes[frame_num]
x = []
for t in t10:
    x.append([round(b) for b in t])

print(x)
xx = x[2]
# cv2.imshow(iframes[frame_num][xx[1]:xx[3],xx[0]:xx[2], ::-1])
plt.imshow(iframes[frame_num][xx[1]:xx[3],xx[0]:xx[2], :])
plt.show()

for i in range(frame_num, frame_num + 10):
    t10 = itboxes[i]
    x = []
    for t in t10:
        x.append([round(b) for b in t])

    xx = [xx for xx in x if xx[4] == 1][0]
    # cv2.imshow(iframes[i][xx[1]:xx[3],xx[0]:xx[2], ::-1])
    plt.imshow(iframes[i][xx[1]:xx[3],xx[0]:xx[2], :])
    plt.show()

```

[FILE] get_init_data.py

Path: Detect_and_Track\get_init_data.py

```

import pandas as pd
from .init_dataframe.create_df import creatInitDataFrame
from .get_tracks import get_video_tracks

def get_init_data(path, out_name, teams_colors, ball_only = True):
    '''
        this functions is to create initial unmapped dataframe
    '''

```

```

Parameters
-----
path : string
    the path of the directory of the processed video.
out_name : string
    the name to save the video with objects tracked with.
teams_colors : list of strings
    list of the colors to classify the teams.
ball_only : boolean
    whether or not to save only the frames with the ball detected in them.

Return
-----
init_df : pandas dataframe
    the initial unmapped dataframe.
'''

ziframes, zitboxes = get_video_tracks(video_path = path , out_name = out_name,
ball_only =
    ball_only)
init_df = creatInitDataFrame(zitboxes, ziframes, teams_colors)

#save the initial dataframe
init_df.to_csv(f'./Out/{out_name}_init_df.csv', index=False)

return init_df

```

[FILE] get_tracks.py

Path: Detect_and_Track\get_tracks.py

```

from .deep_sort.tracker_implementation import trackingXl5
from .deep_sort.tracks_cleaning import clean_tracks
from .deep_sort.tracking_video import make_tracking_video
from .yoloV5.load_models import yoloV5l

def get_video_tracks(video_path , out_name , ball_only = True):
    '''
    this functions is to implement tracking players and the ball in the video.

    Parameters
    -----
    video_path : string
        the path of the directory of the processed video.
    out_name : string
        the name to save the video with objects tracked with.
    ball_only : boolean
        whether or not to save only the frames with the ball detected in them.

    Return
    -----
    ziframes : list
        list of frames of the video with objects tracked.
    zitboxes :list
        list of every object tracked in every frame.
        object:[y1, x1, y2, x2, class of the object, id of the object]
    '''

```

```

    where y1, x1, y2, x2 are the coordinates of the box around the object
    ...

import cv2
vid = cv2.VideoCapture(video_path)
width = int(vid.get(cv2.CAP_PROP_FRAME_WIDTH))
height = int(vid.get(cv2.CAP_PROP_FRAME_HEIGHT))
vid.release()
imgsz = min(width, height)
modelv5l, ball_modelv5l = yoloV5l(imgsz=imgsz)
iframes, itboxes, fps = trackingXl5(modelv5l, ball_modelv5l, video_path)
ziframes, zitboxes = clean_tracks(iframes, itboxes, ball_only)
#make clean video
make_tracking_video(ziframes, zitboxes, f'./Out/{out_name}_out.mp4', fps,
draw=False)
return ziframes, zitboxes

```

[FILE] roi_main.py

Path: Detect_and_Track\roi_main.py

roi_main.py - Enhanced main functions for FAIR ROI tracking experiments

```

import os
import sys
import logging
import time
from pathlib import Path

def setup_logging(log_level=logging.INFO):
    """
    Setup logging configuration for ROI experiments.

    Parameters
    -----
    log_level : int
        Logging level (default: logging.INFO)

    Returns
    -----
    str
        Path to the log file
    """
    log_filename = f'./Out/roi_experiment_{int(time.time())}.log'

    # Create Out directory if it doesn't exist
    os.makedirs('./Out', exist_ok=True)

    logging.basicConfig(
        level=log_level,
        format='%(asctime)s - %(levelname)s - %(message)s',
        handlers=[
            logging.FileHandler(log_filename),
            logging.StreamHandler(sys.stdout)
        ]
    )

```

```

    )

    return log_filename

def validate_experiment_setup(video_path, teams_colors=None):
    """
    Validate that all required components are available for the experiment.

    Parameters
    -----
    video_path : str
        Path to video file
    teams_colors : list, optional
        Team colors list

    Returns
    -----
    tuple
        (is_valid: bool, issues: list)
    """

    issues = []

    # Check video file
    if not os.path.exists(video_path):
        issues.append(f"Video file not found: {video_path}")
    else:
        # Check if file is readable
        try:
            import cv2
            cap = cv2.VideoCapture(video_path)
            if not cap.isOpened():
                issues.append(f"Cannot open video file: {video_path}")
            else:
                frame_count = int(cap.get(cv2.CAP_PROP_FRAME_COUNT))
                if frame_count <= 0:
                    issues.append(f"Video appears to have no frames:
{video_path}")
            cap.release()
        except Exception as e:
            issues.append(f"Error checking video file: {str(e)}")

    # Check required directories and files
    required_dirs = ['./Detect_and_Track', './Out']
    for dir_path in required_dirs:
        if not os.path.exists(dir_path):
            issues.append(f"Required directory not found: {dir_path}")

    required_files = [
        './Detect_and_Track/model_data/mars-small128.pb',
        './Detect_and_Track/model_data/coco/coco.names'
    ]
    for file_path in required_files:
        if not os.path.exists(file_path):
            issues.append(f"Required model file not found: {file_path}")

    # Check team colors format
    if teams_colors is not None:
        if not isinstance(teams_colors, list) or len(teams_colors) != 6:
            issues.append("teams_colors must be a list of 6 color names")

```

```

# Try importing required modules
try:
    import torch
    import cv2
    import numpy as np

    import pandas as pd
except ImportError as e:
    issues.append(f"Required Python package not available: {str(e)}")

return len(issues) == 0, issues

def run_fair_roi_experiment(video_path, experiment_name, teams_colors=None,
                           target_player_id=1, ball_only=True,
save_roi_images=True,
                           roi_save_interval=15):
    """
    Main function to run FAIR ROI tracking experiment and comparison.
    Both approaches now track only ONE player for fair comparison.

    Parameters
    -----
    video_path : str
        Path to the video file to process
    experiment_name : str
        Name for this experiment (used in output files)
    teams_colors : list, optional
        List of team colors for classification. If None, uses default colors.
    target_player_id : int, optional
        ID of the player to track (default: 1)
    ball_only : bool, optional
        Whether to keep only frames with ball detected (default: True)
    save_roi_images : bool, optional
        Whether to save ROI images during tracking (default: True)
    roi_save_interval : int, optional
        Save ROI image every N frames (default: 15)

    Returns
    -----
    bool
        True if experiment completed successfully, False otherwise
    """

    # Setup logging
    log_file = setup_logging()
    logger = logging.getLogger(__name__)

    try:
        logger.info("="*60)
        logger.info("STARTING FAIR ROI TRACKING EXPERIMENT")
        logger.info("Both approaches track ONLY ONE player (no ball detection)")
        logger.info("="*60)
        logger.info(f"Video path: {video_path}")
        logger.info(f"Experiment name: {experiment_name}")
        logger.info(f"Target player ID: {target_player_id}")
        logger.info(f"Ball only mode: {ball_only}")

        logger.info(f"Save ROI images: {save_roi_images}")
        logger.info(f"ROI save interval: {roi_save_interval}")

```



```

# Validate setup
is_valid, issues = validate_experiment_setup(video_path, teams_colors)
if not is_valid:
    logger.error("Setup validation failed:")
    for issue in issues:
        logger.error(f" - {issue}")
    return False

# Set default team colors if not provided
if teams_colors is None:
    teams_colors = ['white', 'white', 'red', 'red', 'black', 'yellow']
logger.info("Using default team colors: ['white', 'white', 'red', 'red', 'black',
'yellow']")
else:
    logger.info(f"Using provided team colors: {teams_colors}")

# Import required modules
try:
    from Detect_and_Track.deep_sort.roi_tracking import
compare_tracking_approaches_fair
    logger.info("Successfully imported FAIR ROI tracking modules")
except ImportError as e:
    logger.error(f"Failed to import ROI tracking modules: {e}")
    return False

# Run the FAIR comparison experiment
logger.info("Starting FAIR tracking approaches comparison...")
start_time = time.time()

comparison_results = compare_tracking_approaches_fair(
    video_path=video_path,
    out_name=experiment_name,
    teams_colors=teams_colors,
    ball_only=ball_only,
    target_player_id=target_player_id,
    save_roi_images=save_roi_images,
    roi_save_interval=roi_save_interval
)

experiment_duration = time.time() - start_time

if comparison_results:
    logger.info("="*60)
    logger.info("FAIR EXPERIMENT COMPLETED SUCCESSFULLY")
    logger.info("="*60)
    logger.info(f"Total experiment duration: {experiment_duration:.2f}
seconds")

# Log key results

std_results = comparison_results['standard_approach_one_player']
roi_results = comparison_results['roi_approach_one_player']
improvements = comparison_results['improvements']

logger.info("SUMMARY OF FAIR RESULTS:")
logger.info(f"Standard approach (1 player) - Total time: {std_results[
'total_time']:.2f}s, "
f"Avg per frame: {std_results['avg_total_per_frame']:.6f}s")

logger.info(f"ROI approach (1 player) - Total time:
{roi_results['total_time']:.2f}s, "
f"Avg per frame: {roi_results['avg_total_per_frame']:.6f}s")

```

```

logger.info(f"Performance improvement: {improvements['total_time_improvement_pct']}
           :.2f}%")
logger.info(f"Overall speedup:
{improvements['overall_speedup']:.2f}x")
logger.info(f"ROI frames used: {roi_results['roi_frames_used']}/{roi_results[
'frames_processed']} "
f"({(roi_results['roi_frames_used']/roi_results['frames_processed'])
*100:.1f}%)")

# List generated files
logger.info("\nGENERATED FILES:")
output_files = [
    f"./Out/{experiment_name}_standard_one_player_init_df.csv",
    f"./Out/{experiment_name}_roi_one_player_init_df.csv",
    f"./Out/{experiment_name}_standard_one_player_out.mp4",
    f"./Out/{experiment_name}_standard_one_player_tracked.mp4",
    f"./Out/{experiment_name}_roi_one_player_out.mp4",
    f"./Out/{experiment_name}_roi_one_player_tracked.mp4",
    f"./Out/{experiment_name}_fair_tracking_comparison_report.txt"
]

for file_path in output_files:
    if os.path.exists(file_path):
        logger.info(f"[OK] {file_path}")
    else:
        logger.warning(f"[W] {file_path} (not found)")

# Check ROI images
if save_roi_images:
    roi_images_dir = f"./Out/{experiment_name}_roi_roi_images"
    if os.path.exists(roi_images_dir):
        roi_image_count = len([f for f in os.listdir(roi_images_dir) if
                                f.endswith('.jpg')])
        logger.info(f"[OK] ROI images: {roi_image_count} images in
{roi_images_dir}")
    else:
        logger.warning(f"[W] ROI images directory not found:
{roi_images_dir}")

    logger.info(f"\nExperiment log saved to: {log_file}")
    logger.info("="*60)
    logger.info("FAIR COMPARISON: Both approaches tracked ONLY ONE
player!")
    logger.info("Performance difference is due to ROI optimization, not workload

        difference.")
    logger.info("="*60)
    return True

else:
    logger.error("Comparison function returned None - experiment failed")
    return False

except Exception as e:
    logger.error(f"Experiment failed with error: {str(e)}")
    logger.exception("Full error traceback:")
    return False

def run_roi_only_experiment_fair(video_path, experiment_name, teams_colors=None,

```

```

        target_player_id=1, ball_only=True,
save_roi_images=True,
        roi_save_interval=15):
    """
    Run only ROI tracking for single player (without comparison to standard
    approach).
    Useful for quick testing of ROI parameters on single player tracking.

    Parameters
    -----
    video_path : str
        Path to the video file to process
    experiment_name : str
        Name for this experiment
    teams_colors : list, optional
        List of team colors for classification
    target_player_id : int, optional
        ID of the player to track with ROI (default: 1)
    ball_only : bool, optional
        Whether to keep only frames with ball detected (default: True)
    save_roi_images : bool, optional
        Whether to save ROI images during tracking (default: True)
    roi_save_interval : int, optional
        Save ROI image every N frames (default: 15)

    Returns
    -----
    tuple
        (success: bool, results: dict or None)
    """

    logger = logging.getLogger(__name__)

    try:
        logger.info("Running ROI-only experiment (single player, no ball)...")

        # Set default team colors if not provided

        if teams_colors is None:
            teams_colors = ['white', 'white', 'red', 'red', 'black', 'yellow']

        # Import required modules
        from Detect_and_Track.deep_sort.roi_tracking import
trackingXl5_ROI_single_player
        from Detect_and_Track.deep_sort.tracks_cleaning import clean_tracks
        from Detect_and_Track.deep_sort.tracking_video import make_tracking_video
        from Detect_and_Track.init_dataframe.create_df import creatInitDataFrame
        from Detect_and_Track.yoloV5.load_models import yoloV5l

        # Load models
        logger.info("Loading YOLO models...")
        modelv5l, ball_modelv5l = yoloV5l()

        # Run ROI tracking (single player)
        logger.info("Running ROI tracking (single player, no ball)...")
        start_time = time.time()
        iframes_roi, itboxes_roi, fps_roi = trackingXl5_ROI_single_player(
            modelv5l, ball_modelv5l, video_path, target_player_id,
            save_roi_images=save_roi_images, roi_save_interval=roi_save_interval,
            experiment_name=experiment_name
        )
        roi_total_time = time.time() - start_time

```

```

        # Clean and process results
        logger.info("Cleaning tracks and creating outputs...")
        ziframes_roi, zitboxes_roi = clean_tracks(iframes_roi, itboxes_roi,
ball_only)
        init_df_roi = creatInitDataFrame(zitboxes_roi, ziframes_roi, teams_colors)

        # Save outputs
        init_df_roi.to_csv(f'./Out/{experiment_name}_roi_only_single_player_init_df.csv',
            index=False)
        make_tracking_video(ziframes_roi, zitboxes_roi, f'./Out/{experiment_name}
            _roi_only_single_player_out.mp4', fps_roi, draw=False)
        make_tracking_video(ziframes_roi, zitboxes_roi, f'./Out/{experiment_name}
            _roi_only_single_player_tracked.mp4', fps_roi, draw=True)

        results = {
            'total_time': roi_total_time,
            'frames_processed': len(iframes_roi),
            'clean_frames': len(ziframes_roi),
            'fps': fps_roi,
            'avg_time_per_frame': roi_total_time / len(iframes_roi) if len(iframes_roi) > 0
else
            0,
            'target_player_id': target_player_id
        }

        logger.info(f"ROI-only experiment (single player) completed in
{roi_total_time:.2f}
seconds")
        logger.info(f"Processed {len(iframes_roi)} frames, {len(ziframes_roi)}
after cleaning")

        logger.info(f"Tracked player ID: {target_player_id}")

        return True, results

    except Exception as e:
        logger.error(f"ROI-only experiment failed: {str(e)}")
        return False, None

def run_standard_only_experiment(video_path, experiment_name, teams_colors=None,
                                target_player_id=1, ball_only=True):
    """
    Run only standard tracking for single player (for testing purposes).

    Parameters
    -----
    video_path : str
        Path to the video file to process
    experiment_name : str
        Name for this experiment
    teams_colors : list, optional
        List of team colors for classification
    target_player_id : int, optional
        ID of the player to track (default: 1)
    ball_only : bool, optional
        Whether to keep only frames with ball detected (default: True)

    Returns
    -----
    tuple

```

```

        (success: bool, results: dict or None)
    """

    logger = logging.getLogger(__name__)

    try:
        logger.info("Running standard-only experiment (single player, no
ball)...")

        # Set default team colors if not provided
        if teams_colors is None:
            teams_colors = ['white', 'white', 'red', 'red', 'black', 'yellow']

        # Import required modules
        from Detect_and_Track.deep_sort.standard_one_player import
standard_trackingXl5_one_player
        from Detect_and_Track.deep_sort.tracks_cleaning import clean_tracks
        from Detect_and_Track.deep_sort.tracking_video import make_tracking_video
        from Detect_and_Track.init_dataframe.create_df import creatInitDataFrame
        from Detect_and_Track.yoloV5.load_models import yoloV5l

        # Load models

        logger.info("Loading YOLO models...")
        modelv5l, ball_modelv5l = yoloV5l()

        # Run standard tracking (single player)
        logger.info("Running standard tracking (single player, no ball)...")
        start_time = time.time()
        iframes_std, itboxes_std, fps_std = standard_trackingXl5_one_player(
            modelv5l, ball_modelv5l, video_path, target_player_id,
            experiment_name=experiment_name
        )
        std_total_time = time.time() - start_time

        # Clean and process results
        logger.info("Cleaning tracks and creating outputs...")
        ziframes_std, zitboxes_std = clean_tracks(iframes_std, itboxes_std,
ball_only)
        init_df_std = creatInitDataFrame(zitboxes_std, ziframes_std, teams_colors)

        # Save outputs
        init_df_std.to_csv(f'./Out/{experiment_name}_standard_only_single_player_init_df.
csv',
            index=False)
        make_tracking_video(ziframes_std, zitboxes_std, f'./Out/{experiment_name}
_standard_only_single_player_out.mp4', fps_std, draw=False)
        make_tracking_video(ziframes_std, zitboxes_std, f'./Out/{experiment_name}
_standard_only_single_player_tracked.mp4', fps_std, draw=True)

        results = {
            'total_time': std_total_time,
            'frames_processed': len(iframes_std),
            'clean_frames': len(ziframes_std),
            'fps': fps_std,
            'avg_time_per_frame': std_total_time / len(iframes_std) if len(iframes_std) > 0
else
            0,
            'target_player_id': target_player_id
        }

        logger.info(f"Standard-only experiment (single player) completed in

```

```

{std_total_time:.2f}
    seconds")
    logger.info(f"Processed {len(iframes_std)} frames, {len(ziframes_std)}
after cleaning")
    logger.info(f"Tracked player ID: {target_player_id}")

    return True, results

except Exception as e:
    logger.error(f"Standard-only experiment failed: {str(e)}")
    return False, None

def quick_fair_roi_test(video_path, experiment_name="fair_test"):
    """
    Quick test function for immediate usage with fair comparison.

    Parameters
    -----
    video_path : str
        Path to video file
    experiment_name : str, optional
        Name for experiment outputs

    Returns
    -----
    bool
        Success status
    """

    print("Starting quick FAIR ROI test (both approaches track one player)...")

    # Validate setup
    is_valid, issues = validate_experiment_setup(video_path)
    if not is_valid:
        print("Setup validation failed:")
        for issue in issues:
            print(f" - {issue}")
        return False

    # Run fair experiment
    success = run_fair_roi_experiment(
        video_path=video_path,
        experiment_name=experiment_name,
        teams_colors=None, # Use defaults
        target_player_id=1,
        ball_only=True,
        save_roi_images=True,
        roi_save_interval=10
    )

    if success:
        print(f"Quick FAIR test completed successfully!")
        print(f"Check ./Out directory for results with prefix '{
experiment_name}'")
        print("Both approaches tracked only ONE player for fair comparison.")
    else:
        print("Quick fair test failed. Check logs for details.")

    return success

```

```

def run_parameter_sweep_fair(video_path, base_experiment_name,
target_player_ids=[1, 2, 3]):
    """
    Run FAIR ROI experiments with different target player IDs to find optimal
    parameters.
    Both approaches track only one player each.

    Parameters
    -----
    video_path : str
        Path to video file
    base_experiment_name : str
        Base name for experiments
    target_player_ids : list
        List of player IDs to test

    Returns
    -----
    dict
        Results for each parameter combination
    """

    logger = logging.getLogger(__name__)
    logger.info(f"Starting FAIR parameter sweep for player IDs:
{target_player_ids}")
    logger.info("Each test compares standard vs ROI for ONE player only")

    results = {}

    for player_id in target_player_ids:
        experiment_name = f"{base_experiment_name}_fair_player_{player_id}"
        logger.info(f"\nTesting FAIR comparison with target player ID:
{player_id}")

        success = run_fair_roi_experiment(
            video_path=video_path,
            experiment_name=experiment_name,
            target_player_id=player_id,
            ball_only=True,
            save_roi_images=False, # Save space during sweep
            roi_save_interval=30
        )

        results[player_id] = {
            'experiment_name': experiment_name,
            'success': success,
            'comparison_type': 'fair_one_player'
        }

        if success:
            logger.info(f"FAIR Player ID {player_id} test completed successfully")
        else:
            logger.warning(f"FAIR Player ID {player_id} test failed")

    logger.info("FAIR parameter sweep completed")
    return results

# Main execution functions for easy importing

```

```

def main_fair_roi_experiment(video_path, experiment_name="fair_roi_test",
target_player_id=1):
    """
    Main function for running a complete FAIR ROI experiment.
    Both approaches track only one player.
    """
    return run_fair_roi_experiment(
        video_path=video_path,
        experiment_name=experiment_name,
        target_player_id=target_player_id,
        ball_only=True,
        save_roi_images=True,
        roi_save_interval=15
    )

def main_roi_only_fair(video_path, experiment_name="roi_only_fair_test",
target_player_id=1):
    """
    Main function for running ROI-only tracking (single player).
    """
    setup_logging()
    success, results = run_roi_only_experiment_fair(
        video_path=video_path,
        experiment_name=experiment_name,
        target_player_id=target_player_id,
        ball_only=True,
        save_roi_images=True,
        roi_save_interval=15
    )
    return success, results

def main_standard_only_fair(video_path, experiment_name="standard_only_fair_test",
target_player_id=1):
    """
    Main function for running standard-only tracking (single player).
    """
    setup_logging()
    success, results = run_standard_only_experiment(
        video_path=video_path,
        experiment_name=experiment_name,
        target_player_id=target_player_id,
        ball_only=True
    )
    return success, results

```


[FOLDER] Detect_and_Track\deep_sort

Location: Detect_and_Track\deep_sort

[FILE] __init__.py

Path: Detect_and_Track\deep_sort__init__.py

File is empty or could not be read.

[FILE] detection.py

Path: Detect_and_Track\deep_sort\detection.py

```
import numpy as np

class Detection(object):
    """
    This class represents a bounding box detection in a single image.

    Parameters
    -----
    tlwh : array_like
        Bounding box in format `(x, y, w, h)`.
    confidence : float
        Detector confidence score.
    feature : array_like
        A feature vector that describes the object contained in this image.

    Attributes
    -----
    tlwh : ndarray
        Bounding box in format `(top left x, top left y, width, height)`.
    confidence : ndarray
        Detector confidence score.
    class_name : ndarray
        Detector class.
    feature : ndarray | NoneType
        A feature vector that describes the object contained in this image.

    """
    def __init__(self, tlwh, confidence, class_name, feature):
        self.tlwh = np.asarray(tlwh, dtype=float)
        self.confidence = float(confidence)
        self.class_name = class_name
        self.feature = np.asarray(feature, dtype=np.float32)

    def get_class(self):
```

```

        return self.class_name

    def to_tlbr(self):
        """Convert bounding box to format `(min x, min y, max x, max y)`, i.e.,
        `(top left, bottom right)`.
        """
        ret = self.tlwh.copy()
        ret[2:] += ret[:2]
        return ret

    def to_xyah(self):
        """Convert bounding box to format `(center x, center y, aspect ratio,
        height)`, where the aspect ratio is `width / height`.
        """

        ret = self.tlwh.copy()
        ret[:2] += ret[2:] / 2
        ret[2] /= ret[3]
        return ret

```

[FILE] generate_detections.py

Path: Detect_and_Track\deep_sort\generate_detections.py

```

import os
import errno
import argparse
import numpy as np
import cv2
import tensorflow.compat.v1 as tf

physical_devices = tf.config.experimental.list_physical_devices('GPU')
if len(physical_devices) > 0:
    tf.config.experimental.set_memory_growth(physical_devices[0], True)

def _run_in_batches(f, data_dict, out, batch_size):
    data_len = len(out)
    num_batches = int(data_len / batch_size)

    s, e = 0, 0
    for i in range(num_batches):
        s, e = i * batch_size, (i + 1) * batch_size
        batch_data_dict = {k: v[s:e] for k, v in data_dict.items()}
        out[s:e] = f(batch_data_dict)
    if e < len(out):
        batch_data_dict = {k: v[e:] for k, v in data_dict.items()}
        out[e:] = f(batch_data_dict)

def extract_image_patch(image, bbox, patch_shape):
    """Extract image patch from bounding box.

    Parameters
    -----
    image : ndarray
        The full image.
    bbox : array_like

```

The bounding box in format (x, y, width, height).
 patch_shape : Optional[array_like]
 This parameter can be used to enforce a desired patch shape (height, width). First, the `bbox` is adapted to the aspect ratio of the patch shape, then it is clipped at the image boundaries. If None, the shape is computed from :arg:`bbox`.

Returns

 ndarray | NoneType
 An image patch showing the :arg:`bbox`, optionally reshaped to :arg:`patch\_shape`.
 Returns None if the bounding box is empty or fully outside of the image boundaries.

```

"""
bbox = np.array(bbox)

if patch_shape is not None:
    # correct aspect ratio to patch shape
    target_aspect = float(patch_shape[1]) / patch_shape[0]
    new_width = target_aspect * bbox[3]
    bbox[0] -= (new_width - bbox[2]) / 2
    bbox[2] = new_width

# convert to top left, bottom right
bbox[2:] += bbox[:2]
bbox = bbox.astype(int)

# clip at image boundaries
bbox[:2] = np.maximum(0, bbox[:2])
bbox[2:] = np.minimum(np.asarray(image.shape[:2][::-1]) - 1, bbox[2:])
if np.any(bbox[:2] >= bbox[2:]):
    return None
sx, sy, ex, ey = bbox
image = image[sy:ey, sx:ex]
image = cv2.resize(image, tuple(patch_shape[::-1]))
return image

```

```
class ImageEncoder(object):
```

```

    def __init__(self, checkpoint_filename, input_name="images",
output_name="features"):
        self.session = tf.Session()
        with tf.gfile.GFile(checkpoint_filename, "rb") as file_handle:
            graph_def = tf.GraphDef()
            graph_def.ParseFromString(file_handle.read())
            tf.import_graph_def(graph_def)
        try:
            self.input_var = tf.get_default_graph().get_tensor_by_name(input_name)
            self.output_var = tf.get_default_graph().get_tensor_by_name(output_name)
        except KeyError:
            layers = [i.name for i in tf.get_default_graph().get_operations()]
            self.input_var = tf.get_default_graph().get_tensor_by_name(layers[0]+':'
0')
            self.output_var = tf.get_default_graph().get_tensor_by_name(layers[-1]
+':0')

        assert len(self.output_var.get_shape()) == 2
        assert len(self.input_var.get_shape()) == 4

```

```

        self.feature_dim = self.output_var.get_shape().as_list()[-1]
        self.image_shape = self.input_var.get_shape().as_list()[1:]

    def __call__(self, data_x, batch_size=32):
        out = np.zeros((len(data_x), self.feature_dim), np.float32)
        _run_in_batches(
            lambda x: self.session.run(self.output_var, feed_dict=x),
            {self.input_var: data_x}, out, batch_size)
        return out

def create_box_encoder(model_filename, input_name="images:0",
    output_name="features:0",
    batch_size=32):
    image_encoder = ImageEncoder(model_filename, input_name, output_name)
    image_shape = image_encoder.image_shape

    def encoder(image, boxes):
        image_patches = []
        for box in boxes:
            patch = extract_image_patch(image, box, image_shape[:2])
            if patch is None:
                print("WARNING: Failed to extract image patch: %s." % str(box))
                patch = np.random.uniform(0., 255., image_shape).astype(np.uint8)
            image_patches.append(patch)
        image_patches = np.asarray(image_patches)
        return image_encoder(image_patches, batch_size)

    return encoder

def generate_detections(encoder, mot_dir, output_dir, detection_dir=None):
    """Generate detections with features.

    Parameters
    -----
    encoder : Callable[image, ndarray] -> ndarray
        The encoder function takes as input a BGR color image and a matrix of
        bounding boxes in format `(x, y, w, h)` and returns a matrix of
        corresponding feature vectors.
    mot_dir : str
        Path to the MOTChallenge directory (can be either train or test).
    output_dir
        Path to the output directory. Will be created if it does not exist.
    detection_dir
        Path to custom detections. The directory structure should be the default
        MOTChallenge structure: `[sequence]/det/det.txt`. If None, uses the
        standard MOTChallenge detections.

    """
    if detection_dir is None:
        detection_dir = mot_dir
    try:
        os.makedirs(output_dir)
    except OSError as exception:
        if exception.errno == errno.EEXIST and os.path.isdir(output_dir):
            pass
        else:
            raise ValueError(
                "Failed to created output directory '%s'" % output_dir)

```

```

for sequence in os.listdir(mot_dir):
    print("Processing %s" % sequence)
    sequence_dir = os.path.join(mot_dir, sequence)

    image_dir = os.path.join(sequence_dir, "img1")
    image_filenames = {
        int(os.path.splitext(f)[0]): os.path.join(image_dir, f)
        for f in os.listdir(image_dir)}

    detection_file = os.path.join(
        detection_dir, sequence, "det/det.txt")
    detections_in = np.loadtxt(detection_file, delimiter=',')
    detections_out = []

    frame_indices = detections_in[:, 0].astype(np.int)
    min_frame_idx = frame_indices.astype(np.int).min()
    max_frame_idx = frame_indices.astype(np.int).max()
    for frame_idx in range(min_frame_idx, max_frame_idx + 1):
        print("Frame %05d/%05d" % (frame_idx, max_frame_idx))
        mask = frame_indices == frame_idx
        rows = detections_in[mask]

        if frame_idx not in image_filenames:
            print("WARNING could not find image for frame %d" % frame_idx)
            continue
        bgr_image = cv2.imread(
            image_filenames[frame_idx], cv2.IMREAD_COLOR)
        features = encoder(bgr_image, rows[:, 2:6].copy())
        detections_out += [np.r_[(row, feature)] for row, feature
            in zip(rows, features)]

    output_filename = os.path.join(output_dir, "%s.npy" % sequence)
    np.save(
        output_filename, np.asarray(detections_out), allow_pickle=False)

def parse_args():
    """Parse command line arguments.
    """
    parser = argparse.ArgumentParser(description="Re-ID feature extractor")
    parser.add_argument(
        "--model",
        default="resources/networks/mars-small128.pb",
        help="Path to freezed inference graph protobuf.")
    parser.add_argument(
        "--mot_dir", help="Path to MOTChallenge directory (train or test)",
        required=True)
    parser.add_argument(
        "--detection_dir", help="Path to custom detections. Defaults to "
        "standard MOT detections Directory structure should be the default "

        "MOTChallenge structure: [sequence]/det/det.txt", default=None)
    parser.add_argument(
        "--output_dir", help="Output directory. Will be created if it does not"
        " exist.", default="detections")
    return parser.parse_args()

def main():
    args = parse_args()
    encoder = create_box_encoder(args.model, batch_size=32)
    generate_detections(encoder, args.mot_dir, args.output_dir,

```

```
args.detection_dir)
```

```
if __name__ == "__main__":  
    main()
```

[FILE] iou_matching.py

Path: Detect_and_Track\deep_sort\iou_matching.py

```
from __future__ import absolute_import  
import numpy as np  
from . import linear_assignment  
  
def iou(bbox, candidates):  
    """Computer intersection over union.  
  
    Parameters  
    -----  
    bbox : ndarray  
        A bounding box in format `(top left x, top left y, width, height)`.  
    candidates : ndarray  
        A matrix of candidate bounding boxes (one per row) in the same format  
        as `bbox`.  
  
    Returns  
    -----  
    ndarray  
        The intersection over union in  $[0, 1]$  between the `bbox` and each  
        candidate. A higher score means a larger fraction of the `bbox` is  
        occluded by the candidate.  
  
    """  
    bbox_tl, bbox_br = bbox[:2], bbox[:2] + bbox[2:]  
    candidates_tl = candidates[:, :2]  
    candidates_br = candidates[:, :2] + candidates[:, 2:]  
  
    tl = np.c_[np.maximum(bbox_tl[0], candidates_tl[:, 0])[0], np.newaxis],  
              np.maximum(bbox_tl[1], candidates_tl[:, 1])[0], np.newaxis]  
    br = np.c_[np.minimum(bbox_br[0], candidates_br[:, 0])[0], np.newaxis],  
              np.minimum(bbox_br[1], candidates_br[:, 1])[0], np.newaxis]  
    wh = np.maximum(0., br - tl)  
  
    area_intersection = wh.prod(axis=1)  
    area_bbox = bbox[2:].prod()  
    area_candidates = candidates[:, 2:].prod(axis=1)  
    return area_intersection / (area_bbox + area_candidates - area_intersection)  
  
def iou_cost(tracks, detections, track_indices=None,  
             detection_indices=None):  
    """An intersection over union distance metric.  
  
    Parameters  
    -----  
    tracks : List[deep_sort.track.Track]
```

```

    A list of tracks.
detections : List[deep_sort.detection.Detection]
    A list of detections.

track_indices : Optional[List[int]]
    A list of indices to tracks that should be matched. Defaults to
    all `tracks`.
detection_indices : Optional[List[int]]
    A list of indices to detections that should be matched. Defaults
    to all `detections`.

Returns
-----
ndarray
    Returns a cost matrix of shape
    len(track_indices), len(detection_indices) where entry (i, j) is
    `1 - iou(tracks[track_indices[i]], detections[detection_indices[j]])`.

"""
if track_indices is None:
    track_indices = np.arange(len(tracks))
if detection_indices is None:
    detection_indices = np.arange(len(detections))

cost_matrix = np.zeros((len(track_indices), len(detection_indices)))
for row, track_idx in enumerate(track_indices):
    if tracks[track_idx].time_since_update > 1:
        cost_matrix[row, :] = linear_assignment.INFTY_COST
        continue

    bbox = tracks[track_idx].to_tlwh()
    candidates = np.asarray([detections[i].tlwh for i in detection_indices])
    cost_matrix[row, :] = 1. - iou(bbox, candidates)
return cost_matrix

```

[FILE] kalman_filter.py

Path: Detect_and_Track\deep_sort\kalman_filter.py

```

import numpy as np
import scipy.linalg

"""
Table for the 0.95 quantile of the chi-square distribution with N degrees of
freedom (contains values for N=1, ..., 9). Taken from MATLAB/Octave's chi2inv
function and used as Mahalanobis gating threshold.
"""
chi2inv95 = {
    1: 3.8415,
    2: 5.9915,
    3: 7.8147,
    4: 9.4877,
    5: 11.070,
    6: 12.592,
    7: 14.067,
    8: 15.507,

```

9: 16.919}

```
class KalmanFilter(object):
    """
    A simple Kalman filter for tracking bounding boxes in image space.

    The 8-dimensional state space

        x, y, a, h, vx, vy, va, vh

    contains the bounding box center position (x, y), aspect ratio a, height h,
    and their respective velocities.

    Object motion follows a constant velocity model. The bounding box location
    (x, y, a, h) is taken as direct observation of the state space (linear
    observation model).

    """
    def __init__(self):
        ndim, dt = 4, 1.

        # Create Kalman filter model matrices.
        self._motion_mat = np.eye(2 * ndim, 2 * ndim)
        for i in range(ndim):
            self._motion_mat[i, ndim + i] = dt
        self._update_mat = np.eye(ndim, 2 * ndim)

        # Motion and observation uncertainty are chosen relative to the current
        # state estimate. These weights control the amount of uncertainty in
        # the model. This is a bit hacky.

        self._std_weight_position = 1. / 20
        self._std_weight_velocity = 1. / 160

    def initiate(self, measurement):
        """Create track from unassociated measurement.

        Parameters
        -----
        measurement : ndarray
            Bounding box coordinates (x, y, a, h) with center position (x, y),
            aspect ratio a, and height h.

        Returns
        -----
        (ndarray, ndarray)
            Returns the mean vector (8 dimensional) and covariance matrix (8x8
            dimensional) of the new track. Unobserved velocities are initialized
            to 0 mean.

        """
        mean_pos = measurement
        mean_vel = np.zeros_like(mean_pos)
        mean = np.r_[mean_pos, mean_vel]

        std = [
            2 * self._std_weight_position * measurement[3],
            2 * self._std_weight_position * measurement[3],
            1e-2,
            2 * self._std_weight_position * measurement[3],
```



```

        10 * self._std_weight_velocity * measurement[3],
        10 * self._std_weight_velocity * measurement[3],
        1e-5,
        10 * self._std_weight_velocity * measurement[3]]
    covariance = np.diag(np.square(std))
    return mean, covariance

def predict(self, mean, covariance):
    """Run Kalman filter prediction step.

    Parameters
    -----
    mean : ndarray
        The 8 dimensional mean vector of the object state at the previous
        time step.
    covariance : ndarray
        The 8x8 dimensional covariance matrix of the object state at the
        previous time step.

    Returns
    -----
    (ndarray, ndarray)
        Returns the mean vector and covariance matrix of the predicted
        state. Unobserved velocities are initialized to 0 mean.

    """
    std_pos = [
        self._std_weight_position * mean[3],
        self._std_weight_position * mean[3],
        1e-2,
        self._std_weight_position * mean[3]]
    std_vel = [
        self._std_weight_velocity * mean[3],
        self._std_weight_velocity * mean[3],
        1e-5,
        self._std_weight_velocity * mean[3]]
    motion_cov = np.diag(np.square(np.r_[std_pos, std_vel]))

    mean = np.dot(self._motion_mat, mean)
    covariance = np.linalg.multi_dot((
        self._motion_mat, covariance, self._motion_mat.T)) + motion_cov

    return mean, covariance

def project(self, mean, covariance):
    """Project state distribution to measurement space.

    Parameters
    -----
    mean : ndarray
        The state's mean vector (8 dimensional array).
    covariance : ndarray
        The state's covariance matrix (8x8 dimensional).

    Returns
    -----
    (ndarray, ndarray)
        Returns the projected mean and covariance matrix of the given state
        estimate.

    """

```

```

std = [
    self._std_weight_position * mean[3],
    self._std_weight_position * mean[3],
    1e-1,
    self._std_weight_position * mean[3]]
innovation_cov = np.diag(np.square(std))

mean = np.dot(self._update_mat, mean)
covariance = np.linalg.multi_dot((
    self._update_mat, covariance, self._update_mat.T))

return mean, covariance + innovation_cov

def update(self, mean, covariance, measurement):
    """Run Kalman filter correction step.

    Parameters
    -----
    mean : ndarray
        The predicted state's mean vector (8 dimensional).
    covariance : ndarray
        The state's covariance matrix (8x8 dimensional).
    measurement : ndarray
        The 4 dimensional measurement vector (x, y, a, h), where (x, y)
        is the center position, a the aspect ratio, and h the height of the
        bounding box.

    Returns
    -----
    (ndarray, ndarray)
        Returns the measurement-corrected state distribution.

    """
    projected_mean, projected_cov = self.project(mean, covariance)

    chol_factor, lower = scipy.linalg.cho_factor(
        projected_cov, lower=True, check_finite=False)
    kalman_gain = scipy.linalg.cho_solve(
        (chol_factor, lower), np.dot(covariance, self._update_mat.T).T,
        check_finite=False).T
    innovation = measurement - projected_mean

    new_mean = mean + np.dot(innovation, kalman_gain.T)
    new_covariance = covariance - np.linalg.multi_dot((
        kalman_gain, projected_cov, kalman_gain.T))
    return new_mean, new_covariance

def gating_distance(self, mean, covariance, measurements,
                    only_position=False):
    """Compute gating distance between state distribution and measurements.

    A suitable distance threshold can be obtained from `chi2inv95`. If
    `only_position` is False, the chi-square distribution has 4 degrees of
    freedom, otherwise 2.

    Parameters
    -----
    mean : ndarray
        Mean vector over the state distribution (8 dimensional).
    covariance : ndarray
        Covariance of the state distribution (8x8 dimensional).

```

```

measurements : ndarray
    An Nx4 dimensional matrix of N measurements, each in
    format (x, y, a, h) where (x, y) is the bounding box center
    position, a the aspect ratio, and h the height.
only_position : Optional[bool]
    If True, distance computation is done with respect to the bounding
    box center position only.

Returns
-----
ndarray
    Returns an array of length N, where the i-th element contains the
    squared Mahalanobis distance between (mean, covariance) and
    `measurements[i]`.

"""
mean, covariance = self.project(mean, covariance)
if only_position:
    mean, covariance = mean[:2], covariance[:2, :2]
    measurements = measurements[:, :2]

cholesky_factor = np.linalg.cholesky(covariance)
d = measurements - mean
z = scipy.linalg.solve_triangular(
    cholesky_factor, d.T, lower=True, check_finite=False,
    overwrite_b=True)
squared_maha = np.sum(z * z, axis=0)
return squared_maha

```

[FILE] linear_assignment.py

Path: Detect_and_Track\deep_sort\linear_assignment.py

```

import numpy as np
from scipy.optimize import linear_sum_assignment
from . import kalman_filter

INFTY_COST = 1e+5

def min_cost_matching(
    distance_metric, max_distance, tracks, detections, track_indices=None,
    detection_indices=None):
    """Solve linear assignment problem.

    Parameters
    -----
    distance_metric : Callable[List[Track], List[Detection], List[int],
List[int]) -> ndarray
        The distance metric is given a list of tracks and detections as well as
        a list of N track indices and M detection indices. The metric should
        return the NxM dimensional cost matrix, where element (i, j) is the
        association cost between the i-th track in the given track indices and
        the j-th detection in the given detection_indices.
    max_distance : float
        Gating threshold. Associations with cost larger than this value are

```

```

        disregarded.
tracks : List[track.Track]
    A list of predicted tracks at the current time step.
detections : List[detection.Detection]
    A list of detections at the current time step.
track_indices : List[int]
    List of track indices that maps rows in `cost_matrix` to tracks in
    `tracks` (see description above).
detection_indices : List[int]
    List of detection indices that maps columns in `cost_matrix` to
    detections in `detections` (see description above).

Returns
-----
(List[(int, int)], List[int], List[int])
    Returns a tuple with the following three entries:
    * A list of matched track and detection indices.
    * A list of unmatched track indices.
    * A list of unmatched detection indices.

"""
if track_indices is None:
    track_indices = np.arange(len(tracks))
if detection_indices is None:
    detection_indices = np.arange(len(detections))

if len(detection_indices) == 0 or len(track_indices) == 0:

    return [], track_indices, detection_indices # Nothing to match.

cost_matrix = distance_metric(
    tracks, detections, track_indices, detection_indices)
cost_matrix[cost_matrix > max_distance] = max_distance + 1e-5
indices = linear_sum_assignment(cost_matrix)
indices = np.asarray(indices)
indices = np.transpose(indices)
matches, unmatched_tracks, unmatched_detections = [], [], []
for col, detection_idx in enumerate(detection_indices):
    if col not in indices[:, 1]:
        unmatched_detections.append(detection_idx)
for row, track_idx in enumerate(track_indices):
    if row not in indices[:, 0]:
        unmatched_tracks.append(track_idx)
for row, col in indices:
    track_idx = track_indices[row]
    detection_idx = detection_indices[col]
    if cost_matrix[row, col] > max_distance:
        unmatched_tracks.append(track_idx)
        unmatched_detections.append(detection_idx)
    else:
        matches.append((track_idx, detection_idx))
return matches, unmatched_tracks, unmatched_detections

def matching_cascade(
    distance_metric, max_distance, cascade_depth, tracks, detections,
    track_indices=None, detection_indices=None):
    """Run matching cascade.

Parameters
-----
distance_metric : Callable[List[Track], List[Detection], List[int],

```

```

List[int]) -> ndarray
    The distance metric is given a list of tracks and detections as well as
    a list of N track indices and M detection indices. The metric should
    return the NxM dimensional cost matrix, where element (i, j) is the
    association cost between the i-th track in the given track indices and
    the j-th detection in the given detection indices.
max_distance : float
    Gating threshold. Associations with cost larger than this value are
    disregarded.
cascade_depth: int
    The cascade depth, should be set to the maximum track age.
tracks : List[track.Track]
    A list of predicted tracks at the current time step.
detections : List[detection.Detection]
    A list of detections at the current time step.
track_indices : Optional[List[int]]
    List of track indices that maps rows in `cost_matrix` to tracks in
    `tracks` (see description above). Defaults to all tracks.
detection_indices : Optional[List[int]]
    List of detection indices that maps columns in `cost_matrix` to
    detections in `detections` (see description above). Defaults to all
    detections.

Returns
-----
(List[(int, int)], List[int], List[int])
    Returns a tuple with the following three entries:
    * A list of matched track and detection indices.
    * A list of unmatched track indices.
    * A list of unmatched detection indices.

"""
if track_indices is None:
    track_indices = list(range(len(tracks)))
if detection_indices is None:
    detection_indices = list(range(len(detections)))

unmatched_detections = detection_indices
matches = []
for level in range(cascade_depth):
    if len(unmatched_detections) == 0: # No detections left
        break

    track_indices_l = [
        k for k in track_indices
        if tracks[k].time_since_update == 1 + level
    ]
    if len(track_indices_l) == 0: # Nothing to match at this level
        continue

    matches_l, _, unmatched_detections = \
        min_cost_matching(
            distance_metric, max_distance, tracks, detections,
            track_indices_l, unmatched_detections)
    matches += matches_l
    unmatched_tracks = list(set(track_indices) - set(k for k, _ in matches))
    return matches, unmatched_tracks, unmatched_detections

def gate_cost_matrix(
    kf, cost_matrix, tracks, detections, track_indices, detection_indices,

```

```

        gated_cost=INFTY_COST, only_position=False):
    """Invalidate infeasible entries in cost matrix based on the state
    distributions obtained by Kalman filtering.

    Parameters
    -----

    kf : The Kalman filter.
    cost_matrix : ndarray
        The NxM dimensional cost matrix, where N is the number of track indices
        and M is the number of detection indices, such that entry (i, j) is the
        association cost between `tracks[track_indices[i]]` and
        `detections[detection_indices[j]]`.
    tracks : List[track.Track]
        A list of predicted tracks at the current time step.
    detections : List[detection.Detection]
        A list of detections at the current time step.
    track_indices : List[int]
        List of track indices that maps rows in `cost_matrix` to tracks in
        `tracks` (see description above).
    detection_indices : List[int]
        List of detection indices that maps columns in `cost_matrix` to
        detections in `detections` (see description above).
    gated_cost : Optional[float]
        Entries in the cost matrix corresponding to infeasible associations are
        set this value. Defaults to a very large value.
    only_position : Optional[bool]
        If True, only the x, y position of the state distribution is considered
        during gating. Defaults to False.

    Returns
    -----
    ndarray
        Returns the modified cost matrix.

    """
    gating_dim = 2 if only_position else 4
    gating_threshold = kalman_filter.chi2inv95[gating_dim]
    measurements = np.asarray(
        [detections[i].to_xyah() for i in detection_indices])
    for row, track_idx in enumerate(track_indices):
        track = tracks[track_idx]
        gating_distance = kf.gating_distance(
            track.mean, track.covariance, measurements, only_position)
        cost_matrix[row, gating_distance > gating_threshold] = gated_cost
    return cost_matrix

```

[FILE] nn_matching.py

Path: Detect_and_Track\deep_sort\nn_matching.py

```
import numpy as np
```

```
def _pdist(a, b):
    """Compute pair-wise squared distance between points in `a` and `b`.
```

```

Parameters
-----
a : array_like
    An NxM matrix of N samples of dimensionality M.
b : array_like
    An LxM matrix of L samples of dimensionality M.

Returns
-----
ndarray
    Returns a matrix of size len(a), len(b) such that element (i, j)
    contains the squared distance between `a[i]` and `b[j]`.

"""
a, b = np.asarray(a), np.asarray(b)
if len(a) == 0 or len(b) == 0:
    return np.zeros((len(a), len(b)))
a2, b2 = np.square(a).sum(axis=1), np.square(b).sum(axis=1)
r2 = -2. * np.dot(a, b.T) + a2[:, None] + b2[None, :]
r2 = np.clip(r2, 0., float(np.inf))
return r2

def _cosine_distance(a, b, data_is_normalized=False):
    """Compute pair-wise cosine distance between points in `a` and `b`.

Parameters
-----
a : array_like
    An NxM matrix of N samples of dimensionality M.
b : array_like
    An LxM matrix of L samples of dimensionality M.
data_is_normalized : Optional[bool]
    If True, assumes rows in a and b are unit length vectors.
    Otherwise, a and b are explicitly normalized to length 1.

Returns
-----
ndarray
    Returns a matrix of size len(a), len(b) such that element (i, j)
    contains the squared distance between `a[i]` and `b[j]`.

"""
if not data_is_normalized:
    a = np.asarray(a) / np.linalg.norm(a, axis=1, keepdims=True)
    b = np.asarray(b) / np.linalg.norm(b, axis=1, keepdims=True)
return 1. - np.dot(a, b.T)

def _nn_euclidean_distance(x, y):
    """ Helper function for nearest neighbor distance metric (Euclidean).

Parameters
-----
x : ndarray
    A matrix of N row-vectors (sample points).
y : ndarray
    A matrix of M row-vectors (query points).

Returns
-----

```

```

    ndarray
        A vector of length M that contains for each entry in `y` the
        smallest Euclidean distance to a sample in `x`.

    """
    distances = _pdist(x, y)
    return np.maximum(0.0, distances.min(axis=0))

def _nn_cosine_distance(x, y):
    """ Helper function for nearest neighbor distance metric (cosine).

    Parameters
    -----
    x : ndarray
        A matrix of N row-vectors (sample points).
    y : ndarray
        A matrix of M row-vectors (query points).

    Returns
    -----
    ndarray
        A vector of length M that contains for each entry in `y` the
        smallest cosine distance to a sample in `x`.

    """
    distances = _cosine_distance(x, y)
    return distances.min(axis=0)

class NearestNeighborDistanceMetric(object):
    """
    A nearest neighbor distance metric that, for each target, returns
    the closest distance to any sample that has been observed so far.

    Parameters
    -----
    metric : str
        Either "euclidean" or "cosine".
    matching_threshold: float
        The matching threshold. Samples with larger distance are considered an
        invalid match.
    budget : Optional[int]
        If not None, fix samples per class to at most this number. Removes
        the oldest samples when the budget is reached.

    Attributes
    -----
    samples : Dict[int -> List[ndarray]]
        A dictionary that maps from target identities to the list of samples
        that have been observed so far.

    """
    def __init__(self, metric, matching_threshold, budget=None):

        if metric == "euclidean":
            self._metric = _nn_euclidean_distance
        elif metric == "cosine":
            self._metric = _nn_cosine_distance

```



```

else:
    raise ValueError(
        "Invalid metric; must be either 'euclidean' or 'cosine'")
self.matching_threshold = matching_threshold
self.budget = budget
self.samples = {}

def partial_fit(self, features, targets, active_targets):
    """Update the distance metric with new data.

    Parameters
    -----
    features : ndarray
        An NxM matrix of N features of dimensionality M.
    targets : ndarray
        An integer array of associated target identities.
    active_targets : List[int]
        A list of targets that are currently present in the scene.

    """
    for feature, target in zip(features, targets):
        self.samples.setdefault(target, []).append(feature)

        if self.budget is not None:
            self.samples[target] = self.samples[target][-self.budget:]
    self.samples = {k: self.samples[k] for k in active_targets}

def distance(self, features, targets):
    """Compute distance between features and targets.

    Parameters
    -----
    features : ndarray
        An NxM matrix of N features of dimensionality M.
    targets : List[int]
        A list of targets to match the given `features` against.

    Returns
    -----
    ndarray
        Returns a cost matrix of shape len(targets), len(features), where
        element (i, j) contains the closest squared distance between
        `targets[i]` and `features[j]`.

    """
    cost_matrix = np.zeros((len(targets), len(features)))
    for i, target in enumerate(targets):
        cost_matrix[i, :] = self._metric(self.samples[target], features)
    return cost_matrix

```

[FILE] roi_tracking.py

Path: Detect_and_Track\deep_sort\roi_tracking.py

roi_tracking.py - Enhanced ROI tracking implementation with fair comparison

import os

```

import cv2
import numpy as np
import time
import csv
from .utils import read_class_names
from .nn_matching import NearestNeighborDistanceMetric
from .detection import Detection
from .tracker import Tracker
from .generate_detections import create_box_encoder

YOLO_COCO_CLASSES = "./Detect_and_Track/model_data/coco/coco.names"

def trackingXl5_ROI_single_player(Yolo_model, ball_model, video_path,
target_player_id=1,
                                save_roi_images=False, roi_save_interval=15,
                                experiment_name="roi_experiment"):
    """
    Enhanced ROI tracking that optimizes both detection and tracking steps.
    Tracks only one specific player using Kalman filter predictions with full ROI
    optimization.
    Ball detection and tracking are EXCLUDED for fair comparison with standard
    approach.

    Parameters
    -----
    Yolo_model : pytorch model
        pytorch YoloV5l model.
    ball_model : pytorch model
        pytorch YoloV5l model to detect the ball specifically (not used but kept
    for compatibility).
    video_path : string
        the path of the directory of the processed video.
    target_player_id : int
        ID of the player to track with ROI (default: 1)
    save_roi_images : bool
        Whether to save ROI images during tracking (default: False)
    roi_save_interval : int
        Save ROI image every N frames (default: 15)
    experiment_name : str
        Name for the experiment (used in ROI image saving)

    Return
    -----
    frames : list
        list of frames of the video with objects tracked.
    tboxes : list
        list of every object tracked in every frame.
    fps : int
        number of frames per second used in processing
    """

    # Definition of the parameters
    max_cosine_distance = 0.7
    nn_budget = None

    # Initialize deep sort object
    model_filename = './Detect_and_Track/model_data/mars-small128.pb'
    encoder = create_box_encoder(model_filename, batch_size=1)
    metric = NearestNeighborDistanceMetric("cosine", max_cosine_distance,
nn_budget)
    tracker = Tracker(metric)

```

```

if video_path:
    vid = cv2.VideoCapture(video_path)

    fps = int(vid.get(cv2.CAP_PROP_FPS))
    print(f'fps of the input video = {fps}')
    print('Please wait ... \n')
    NUM_CLASS = read_class_names(YOLO_COCO_CLASSES)
    key_list = list(NUM_CLASS.keys())
    val_list = list(NUM_CLASS.values())

    frames = []
    tboxes = []

    # ROI tracking variables
    target_track = None
    roi_active = False
    roi_frames_used = 0
    first_frame_processed = False
    # For adaptive ROI
    next_roi_coords = None # ROI to use for the next frame
    prev_center = None # For movement-based margin

    # Initialize timing variables
    frame_count = 0
    timing_data = []

    # Ensure output directory exists
    os.makedirs('./Out', exist_ok=True)

    # Create ROI images directory if needed
    if save_roi_images:
        roi_images_dir = f'./Out/{experiment_name}_roi_images'
        os.makedirs(roi_images_dir, exist_ok=True)

    # Create CSV filename
    csv_filename = f'./Out/roi_detailed_frame_timing_{int(time.time())}.csv'

    while True:
        # Start timing for this frame
        frame_start_time = time.time()

        _, frame = vid.read()
        try:
            original_frame = cv2.cvtColor(frame, cv2.COLOR_BGR2RGB)
            original_frame = cv2.resize(original_frame, (1280, 720))
            frames.append(original_frame)
        except:
            break

        frame_count += 1
        print(f"Frame {frame_count} - Processing started at:
{frame_start_time:.6f}")

        # Detection timing
        detection_start = time.time()

        # Use ROI from previous frame's prediction
        if roi_active and next_roi_coords is not None:
            x1, y1, x2, y2 = next_roi_coords
            roi_frame = original_frame[y1:y2, x1:x2]

```

```

        # Save ROI image if requested
        if save_roi_images and frame_count % roi_save_interval == 0:
            roi_image_path = f'{roi_images_dir}/frame_{frame_count:06d}_roi.jp
g'
            cv2.imwrite(roi_image_path, cv2.cvtColor(roi_frame,
cv2.COLOR_RGB2BGR))

        # Run detection on ROI
        results = Yolo_model(roi_frame)
        pred_bbox = results.xyxy[0].tolist()

        # Adjust coordinates back to full frame
        adjusted_bboxes = []
        for box in pred_bbox:
            adjusted_box = [
                box[0] + x1, # x1
                box[1] + y1, # y1
                box[2] + x1, # x2
                box[3] + y1, # y2
                box[4], # confidence
                box[5] # class
            ]
            adjusted_bboxes.append(adjusted_box)

        pred_bbox = adjusted_bboxes
        bboxes = [np.array(box) for box in pred_bbox]
        roi_frames_used += 1

        print(f"Frame {frame_count} - Using ROI: ({x1},{y1}) to ({x2},{y2})")
    else:
        # First frame or when target is lost - process full frame

        results = Yolo_model(original_frame)
        pred_bbox = results.xyxy[0].tolist()
        bboxes = [np.array(box) for box in pred_bbox]
        print(f"Frame {frame_count} - Processing full frame")

    # Extract bboxes to boxes (x, y, width, height), scores and names
    boxes, scores, names = [], [], []
    for bbox in bboxes:
        boxes.append([bbox[0].astype(int), bbox[1].astype(int),
                      bbox[2].astype(int)-bbox[0].astype(int),
                      bbox[3].astype(int)-bbox[1].astype(int)])
        scores.append(bbox[4])
        names.append(NUM_CLASS[int(bbox[5])])

    detection_end = time.time()

    # Tracking timing
    tracking_start = time.time()

    # ROI-optimized feature extraction
    if roi_active and next_roi_coords is not None and len(boxes) > 0:
        x1, y1, x2, y2 = next_roi_coords
        roi_frame_for_features = original_frame[y1:y2, x1:x2]

        # Adjust box coordinates relative to ROI for feature extraction
        roi_boxes = []
        valid_indices = []
        for i, box in enumerate(boxes):
            roi_x = max(0, box[0] - x1)

```

```

        roi_y = max(0, box[1] - y1)
        roi_w = min(x2 - x1 - roi_x, box[2])
        roi_h = min(y2 - y1 - roi_y, box[3])
        if roi_w > 10 and roi_h > 10:
            roi_boxes.append([roi_x, roi_y, roi_w, roi_h])
            valid_indices.append(i)
        else:
            roi_boxes.append([1, 1, 10, 10]) # Fallback for tiny boxes, ensures encoder
            # doesn't error
            valid_indices.append(i)
    try:
        if len(roi_boxes) > 0:
            features = np.array(encoder(roi_frame_for_features,
roi_boxes))
            print(f"Frame {frame_count} - ROI feature extraction:
{len(roi_boxes)} boxes")
        else:
            features = np.array([])
    except Exception as e:
        print(f"Frame {frame_count} - ROI feature extraction failed: {e}")
        features = np.array(encoder(original_frame, boxes))
        print(f"Frame {frame_count} - Fallback to full frame feature
extraction")
    else:
        features = np.array(encoder(original_frame, boxes))

    # Filter out ball detections (same as standard approach for fair
comparison)
    non_ball_detections_indices = [i for i, name in enumerate(names) if name !
= 'sports ball']
    filtered_boxes = [boxes[i] for i in non_ball_detections_indices]
    filtered_scores = [scores[i] for i in non_ball_detections_indices]
    filtered_names = [names[i] for i in non_ball_detections_indices]
    filtered_features = [features[i] for i in non_ball_detections_indices]

    # Create detections (excluding ball)
    detections = [Detection(bbox, score, class_name, feature)
for bbox, score, class_name, feature in zip(filtered_boxes, filtered_scores,
filtered_names, filtered_features)]

    # ROI-optimized tracking prediction and update
    tracker.predict()

    # Filter detections for ROI tracking optimization
    if roi_active and target_track is not None and next_roi_coords is not
None:
        relevant_detections = []
        x1, y1, x2, y2 = next_roi_coords
        for detection in detections:
            det_bbox = detection.to_tlbr()
            overlap_x1 = max(det_bbox[0], x1)
            overlap_y1 = max(det_bbox[1], y1)
            overlap_x2 = min(det_bbox[2], x2)
            overlap_y2 = min(det_bbox[3], y2)
            overlap_area = max(0, overlap_x2 - overlap_x1) * max(0,
overlap_y2 - overlap_y1)
            det_area = (det_bbox[2] - det_bbox[0]) * (det_bbox[3] -
det_bbox[1])
            # Only include detections with significant overlap (excluding
ball explicitly)
            if overlap_area > 0.2 * det_area:

```

```

        relevant_detections.append(detection)
        tracker.update(relevant_detections)
print(f"Frame {frame_count} - ROI tracking: {len(relevant_detections)}/{len(
        detections)} detections processed (excluding ball)")
    else:
        tracker.update(detections)

    # Obtain info from the tracks
    tracked_bboxes = []
    current_target_found = False

    for track in tracker.tracks:
        if not track.is_confirmed() or track.time_since_update > 5:
            continue

        bbox = track.to_tlbr()
        class_name = track.get_class()
        tracking_id = track.track_id
        index = key_list[val_list.index(class_name)]

        # Exclude ball (class index 32) from being added to tracked_bboxes
        if index == 32:
            continue

        # Check if this is our target player
        if tracking_id == target_player_id and not first_frame_processed:
            target_track = track
            roi_active = True
            first_frame_processed = True
            current_target_found = True
print(f"Frame {frame_count} - Target player {target_player_id} locked for ROI
        tracking")
            elif roi_active and track == target_track:
                current_target_found = True

        # Only track the target player (same as standard approach for fair
comparison)
        if target_track is not None and track == target_track:
            tracked_bboxes.append(bbox.tolist() + [tracking_id, index])

    # After processing tracks, set up ROI for next frame using Kalman
prediction
    if roi_active and target_track is not None:
        predicted_bbox = target_track.to_tlbr()
        obj_width = predicted_bbox[2] - predicted_bbox[0]
        obj_height = predicted_bbox[3] - predicted_bbox[1]
        obj_size = max(obj_width, obj_height)
        center_x = (predicted_bbox[0] + predicted_bbox[2]) / 2
        center_y = (predicted_bbox[1] + predicted_bbox[3]) / 2
        # Calculate movement (Euclidean distance) from previous center
        if prev_center is not None:
            movement = np.linalg.norm(np.array([center_x, center_y]) -
np.array(prev_center))
        else:
            movement = 0
        # Adaptive margins based on object size and movement
        min_margin = int(0.2 * obj_size + 0.5 * movement)
        max_margin = int(1.0 * obj_size + 1.0 * movement)
        # Use a margin between min and max (e.g., average)
        margin = int(0.5 * (min_margin + max_margin))
        x1 = max(0, int(predicted_bbox[0] - margin))

```

```

        y1 = max(0, int(predicted_bbox[1] - margin))
        x2 = min(1280, int(predicted_bbox[2] + margin))
        y2 = min(720, int(predicted_bbox[3] + margin))
        next_roi_coords = (x1, y1, x2, y2)
        prev_center = (center_x, center_y)
    else:
        next_roi_coords = None
        prev_center = None

    # If target is lost, disable ROI and fall back to full frame
    if roi_active and not current_target_found:

        print(f"Frame {frame_count} - Target player lost, switching back to
full frame")
        roi_active = False
        target_track = None
        next_roi_coords = None

    # Ball detection is EXCLUDED (same as standard approach for fair
comparison)
    # Removed the entire ball detection block here for fair comparison

    tboxes.append([[round(bb) for bb in tracked_bbox] for tracked_bbox in
tracked_bboxes])

    tracking_end = time.time()

    # End timing for this frame
    frame_end_time = time.time()
    total_processing_time = frame_end_time - frame_start_time
    detection_time = detection_end - detection_start
    tracking_time = tracking_end - tracking_start
    complete_detection_tracking_time = detection_time + tracking_time

    print(f"Frame {frame_count} - Completed at: {frame_end_time:.6f}")
    print(f"Frame {frame_count} - Total time: {total_processing_time:.6f}s "
        f"(Detection: {detection_time:.6f}s, Tracking:
{tracking_time:.6f}s, "
        f"Detection+Tracking: {complete_detection_tracking_time:.6f}s)")

    # Store timing data with ROI optimization indicator
    timing_data.append([frame_count, frame_start_time, frame_end_time,
        detection_time, tracking_time,
complete_detection_tracking_time,
        total_processing_time, roi_active])

    # Write timing data to CSV
    with open(csv_filename, 'w', newline='') as csvfile:
        writer = csv.writer(csvfile)
        writer.writerow(['Frame_Number', 'Start_Time (unix_timestamp)', 'End_Time
(unix_timestamp)',
            'Detection_Time (s)', 'Tracking_Time (s)', 'Complete_Detection_Tracking_Time
(s)',
            'Total_Processing_Time (s)', 'ROI_Active'])
        writer.writerows(timing_data)

    print(f"\nDetailed timing data saved to: {csv_filename}")
    print(f'Tracked {len(frames)} frames')
    print(f'ROI was used for {roi_frames_used} frames out of {len(frames)} total
frames')

    return frames, tboxes, fps

```

```

def compare_tracking_approaches_fair(video_path, out_name, teams_colors,
ball_only=False,
                                target_player_id=1, save_roi_images=True,
roi_save_interval=15):
    """
    FAIR comparison between standard tracking approach and ROI tracking approach.
    Both approaches now track only one player and exclude ball detection.

    Parameters
    -----
    video_path : str
        Path to the video file
    out_name : str
        Output name prefix for files
    teams_colors : list
        List of team colors for classification
    ball_only : bool
        Whether to keep only frames with ball detected (kept for compatibility)
    target_player_id : int
        ID of the player to track
    save_roi_images : bool
        Whether to save ROI images during tracking
    roi_save_interval : int
        Save ROI image every N frames

    Returns
    -----
    dict
        Comparison results with timing and performance metrics
    """

    from .standard_one_player import standard_trackingXl5_one_player
    from .tracks_cleaning import clean_tracks
    from .tracking_video import make_tracking_video
    from ..init_dataframe.create_df import creatInitDataFrame
    from ..yoloV5.load_models import yoloV5l

    print("="*60)
    print("FAIR TRACKING APPROACHES COMPARISON")
    print("Both approaches track only ONE player (no ball detection)")
    print("="*60)

    # Load models once
    print("Loading YOLO models...")
    modelv5l, ball_modelv5l = yoloV5l()

    # =====
    # Standard Approach (One Player)
    # =====
    print("\n" + "="*50)
    print("RUNNING STANDARD TRACKING APPROACH (ONE PLAYER)")
    print("="*50)

    std_start_time = time.time()

    # Run standard tracking (one player only)
    std_iframes, std_itboxes, std_fps = standard_trackingXl5_one_player(

```



```

        modelv5l, ball_modelv5l, video_path, target_player_id,
        experiment_name=f"{out_name}_standard"
    )

    std_total_time = time.time() - std_start_time

    # Clean and process standard results
    std_ziframes, std_zitboxes = clean_tracks(std_iframes, std_itboxes, ball_only)
    std_init_df = creatInitDataFrame(std_zitboxes, std_ziframes, teams_colors)

    # Save standard results
    std_init_df.to_csv(f'./Out/{out_name}_standard_one_player_init_df.csv',
index=False)
    make_tracking_video(std_ziframes, std_zitboxes,
f'./Out/{out_name}_standard_one_player_out.mp4',
        std_fps, draw=False)
    make_tracking_video(std_ziframes, std_zitboxes,
f'./Out/{out_name}_standard_one_player_tracked.m
        p4', std_fps, draw=True)

    print(f"Standard approach (one player) completed in {std_total_time:.2f}
seconds")
    print(f"Processed {len(std_iframes)} frames, {len(std_ziframes)} after
cleaning")

    # =====
    # ROI Approach (One Player)
    # =====
    print("\n" + "="*40)
    print("RUNNING ROI TRACKING APPROACH (ONE PLAYER)")
    print("="*40)

    roi_start_time = time.time()

    # Run ROI tracking (one player only)
    roi_iframes, roi_itboxes, roi_fps = trackingXl5_ROI_single_player(
        modelv5l, ball_modelv5l, video_path, target_player_id,
        save_roi_images=save_roi_images, roi_save_interval=roi_save_interval,
        experiment_name=f"{out_name}_roi"
    )

    roi_total_time = time.time() - roi_start_time

    # Clean and process ROI results
    roi_ziframes, roi_zitboxes = clean_tracks(roi_iframes, roi_itboxes, ball_only)
    roi_init_df = creatInitDataFrame(roi_zitboxes, roi_ziframes, teams_colors)

    # Save ROI results
    roi_init_df.to_csv(f'./Out/{out_name}_roi_one_player_init_df.csv',
index=False)
    make_tracking_video(roi_ziframes, roi_zitboxes,
f'./Out/{out_name}_roi_one_player_out.mp4',
        roi_fps, draw=False)
    make_tracking_video(roi_ziframes, roi_zitboxes,
f'./Out/{out_name}_roi_one_player_tracked.mp4',
        roi_fps, draw=True)

    print(f"ROI approach (one player) completed in {roi_total_time:.2f} seconds")

    print(f"Processed {len(roi_iframes)} frames, {len(roi_ziframes)} after
cleaning")

```

```

# =====
# FAIR Performance Analysis
# =====
print("\n" + "="*40)
print("FAIR PERFORMANCE COMPARISON ANALYSIS")
print("="*40)

# Calculate performance metrics
std_avg_per_frame = std_total_time / len(std_iframes) if len(std_iframes) > 0
else 0
roi_avg_per_frame = roi_total_time / len(roi_iframes) if len(roi_iframes) > 0
else 0

time_improvement = std_total_time - roi_total_time
time_improvement_pct = (time_improvement / std_total_time) * 100 if
std_total_time > 0 else 0
speedup_factor = std_total_time / roi_total_time if roi_total_time > 0 else 0

# Count ROI frames used (from CSV if available)
roi_frames_used = 0
try:
    csv_files = [f for f in os.listdir('./Out') if
f.startswith('roi_detailed_frame_timing')]
    if csv_files:
        latest_csv = max(csv_files, key=lambda x:
os.path.getctime(os.path.join('./Out', x)))
        import pandas as pd
        timing_df = pd.read_csv(f'./Out/{latest_csv}')
        roi_frames_used = timing_df['ROI_Active'].sum()
except:
    pass

# Create results dictionary
results = {
    'standard_approach_one_player': {
        'total_time': std_total_time,
        'frames_processed': len(std_iframes),
        'clean_frames': len(std_ziframes),
        'avg_total_per_frame': std_avg_per_frame,
        'fps': std_fps
    },
    'roi_approach_one_player': {
        'total_time': roi_total_time,
        'frames_processed': len(roi_iframes),
        'clean_frames': len(roi_ziframes),
        'avg_total_per_frame': roi_avg_per_frame,
        'fps': roi_fps,
        'roi_frames_used': roi_frames_used
    },
    'improvements': {
        'time_saved': time_improvement,
        'total_time_improvement_pct': time_improvement_pct,
        'overall_speedup': speedup_factor,

        'avg_per_frame_improvement': std_avg_per_frame - roi_avg_per_frame
    }
}

# Print summary
print(f"Standard Approach (One Player):")
print(f" - Total time: {std_total_time:.2f}s")
print(f" - Average per frame: {std_avg_per_frame:.6f}s")

```

```

print(f" - Frames processed: {len(std_iframes)}")
print(f" - Clean frames: {len(std_ziframes)}")

print(f"\nROI Approach (One Player):")
print(f" - Total time: {roi_total_time:.2f}s")
print(f" - Average per frame: {roi_avg_per_frame:.6f}s")
print(f" - Frames processed: {len(roi_iframes)}")
print(f" - Clean frames: {len(roi_ziframes)}")
print(f" - ROI frames used: {roi_frames_used}/{len(roi_iframes)}")

print(f"\nFair Performance Improvements:")
print(f" - Time saved: {time_improvement:.2f}s")
print(f" - Improvement: {time_improvement_pct:.2f}%")
print(f" - Overall speedup: {speedup_factor:.2f}x")

# Save detailed report
report_content = f"""
FAIR TRACKING APPROACHES COMPARISON REPORT
=====

Video: {video_path}
Target Player ID: {target_player_id}
Ball Only Mode: {ball_only}

COMPARISON SCOPE:
- Both approaches track ONLY ONE player (target_player_id={target_player_id})
- Ball detection and tracking EXCLUDED from both approaches
- Fair comparison focusing on detection and tracking optimization

STANDARD APPROACH RESULTS (ONE PLAYER):
-----
Total Processing Time: {std_total_time:.2f} seconds
Average Time per Frame: {std_avg_per_frame:.6f} seconds
Frames Processed: {len(std_iframes)}
Clean Frames: {len(std_ziframes)}
FPS: {std_fps}
Detection Method: Full frame processing (1280x720)
Feature Extraction: Full frame

ROI APPROACH RESULTS (ONE PLAYER):
-----
Total Processing Time: {roi_total_time:.2f} seconds

Average Time per Frame: {roi_avg_per_frame:.6f} seconds
Frames Processed: {len(roi_iframes)}
Clean Frames: {len(roi_ziframes)}
ROI Frames Used: {roi_frames_used}/{len(roi_iframes)} ({(roi_frames_used/len(
    roi_iframes))*100:.1f}%)
FPS: {roi_fps}
Detection Method: ROI-based processing (adaptive region size)
Feature Extraction: ROI-optimized

FAIR PERFORMANCE IMPROVEMENTS:
-----
Time Saved: {time_improvement:.2f} seconds
Performance Improvement: {time_improvement_pct:.2f}%
Overall Speedup: {speedup_factor:.2f}x
Per-frame Improvement: {std_avg_per_frame - roi_avg_per_frame:.6f} seconds

OPTIMIZATION ANALYSIS:
-----
The performance improvement is TRUE optimization comparison because:

```

1. Both approaches track the same number of objects (1 player)
2. Both approaches exclude ball detection
3. ROI approach reduces computational load through:
 - Smaller detection area ($\frac{\text{roi_frames_used}}{\text{len(roi_iframes)}}$ frames used ROI)
 - Optimized feature extraction on smaller regions
 - Targeted tracking updates

FILES GENERATED:

Standard Approach (One Player):

- {out_name}_standard_one_player_init_df.csv
- {out_name}_standard_one_player_out.mp4
- {out_name}_standard_one_player_tracked.mp4

ROI Approach (One Player):

- {out_name}_roi_one_player_init_df.csv
- {out_name}_roi_one_player_out.mp4
- {out_name}_roi_one_player_tracked.mp4

Analysis generated on: {time.strftime('%Y-%m-%d %H:%M:%S')}

"""

```
with open(f'./Out/{out_name}_fair_tracking_comparison_report.txt', 'w') as f:
    f.write(report_content)
```

```
print(f"\nFair comparison report saved to: .
/Out/{out_name}_fair_tracking_comparison_report.txt"
)
print("="*60)
print("This is now a FAIR comparison - both approaches track only one
player!")
print("="*60)
```

```
return results
```

[FILE] standard_one_player.py

Path: Detect_and_Track\deep_sort\standard_one_player.py

standard_one_player.py - Standard tracking implementation for single player

```
import os
import cv2
import numpy as np
import time
import csv
from .utils import read_class_names
from .nn_matching import NearestNeighborDistanceMetric
from .detection import Detection
from .tracker import Tracker
from .generate_detections import create_box_encoder

YOLO_COCO_CLASSES = "./Detect_and_Track/model_data/coco/coco.names"

def standard_trackingX15_one_player(Yolo_model, ball_model, video_path,
target_player_id=1,
```

```

        experiment_name="standard_one_player"):
    """
    Standard tracking approach that tracks only one specific player (no ball
    tracking).
    This is for fair comparison with ROI approach.
    Processes full frames but only tracks the target player.

    Parameters
    -----
    Yolo_model : pytorch model
        pytorch YoloV5l model.
    ball_model : pytorch model
        pytorch YoloV5l model to detect the ball specifically (not used but kept
    for compatibility).
    video_path : string
        the path of the directory of the processed video.
    target_player_id : int
        ID of the player to track (default: 1)
    experiment_name : str
        Name for the experiment (used in timing file naming)

    Return
    -----
    frames : list
        list of frames of the video with objects tracked.
    tboxes : list
        list of every object tracked in every frame.
    fps : int
        number of frames per second used in processing
    """

    # Definition of the parameters
    max_cosine_distance = 0.7
    nn_budget = None

    # Initialize deep sort object

    model_filename = './Detect_and_Track/model_data/mars-small128.pb'
    encoder = create_box_encoder(model_filename, batch_size=1)
    metric = NearestNeighborDistanceMetric("cosine", max_cosine_distance,
nn_budget)
    tracker = Tracker(metric)

    if video_path:
        vid = cv2.VideoCapture(video_path)

        fps = int(vid.get(cv2.CAP_PROP_FPS))
        print(f'fps of the input video = {fps}')
        print('Please wait ... \n')
        NUM_CLASS = read_class_names(YOLO_COCO_CLASSES)
        key_list = list(NUM_CLASS.keys())
        val_list = list(NUM_CLASS.values())

        frames = []
        tboxes = []

        # Standard tracking variables
        target_track = None
        first_frame_processed = False

        # Initialize timing variables
        frame_count = 0

```

```

timing_data = []

# Ensure output directory exists
os.makedirs('./Out', exist_ok=True)

# Create CSV filename
csv_filename = f'./Out/standard_one_player_detailed_frame_timing_{int(time.time())}.csv'

while True:
    # Start timing for this frame
    frame_start_time = time.time()

    _, frame = vid.read()
    try:
        original_frame = cv2.cvtColor(frame, cv2.COLOR_BGR2RGB)
        original_frame = cv2.resize(original_frame, (1280, 720))
        frames.append(original_frame)
    except:
        break

    frame_count += 1
    print(f"Frame {frame_count} - Processing started at: {frame_start_time:.6f}")

    # Detection timing - ALWAYS process full frame
    detection_start = time.time()

    results = Yolo_model(original_frame)
    pred_bbox = results.xyxy[0].tolist()
    bboxes = [np.array(box) for box in pred_bbox]

    # Extract bboxes to boxes (x, y, width, height), scores and names
    boxes, scores, names = [], [], []
    for bbox in bboxes:
        boxes.append([bbox[0].astype(int), bbox[1].astype(int),
                      bbox[2].astype(int)-bbox[0].astype(int),
                      bbox[3].astype(int)-bbox[1].astype(int)])
        scores.append(bbox[4])
        names.append(NUM_CLASS[int(bbox[5])])

    detection_end = time.time()

    # Tracking timing
    tracking_start = time.time()

    # ALWAYS use full frame for feature extraction (standard approach)
    features = np.array(encoder(original_frame, boxes))

    # Filter out ball detections (same as ROI approach for fair comparison)
    non_ball_detections_indices = [i for i, name in enumerate(names) if name != 'sports ball']
    filtered_boxes = [boxes[i] for i in non_ball_detections_indices]
    filtered_scores = [scores[i] for i in non_ball_detections_indices]
    filtered_names = [names[i] for i in non_ball_detections_indices]
    filtered_features = [features[i] for i in non_ball_detections_indices]

    # Create detections (excluding ball)
    detections = [Detection(bbox, score, class_name, feature)
                  for bbox, score, class_name, feature in zip(filtered_boxes, filtered_scores,
                                                              filtered_names, filtered_features)]

```

```

# Standard tracking prediction and update - process ALL detections
tracker.predict()
tracker.update(detections)

# Obtain info from the tracks
tracked_bboxes = []
current_target_found = False

for track in tracker.tracks:
    if not track.is_confirmed() or track.time_since_update > 5:
        continue

    bbox = track.to_tlbr()
    class_name = track.get_class()
    tracking_id = track.track_id
    index = key_list[val_list.index(class_name)]

    # Exclude ball (class index 32) from being added to tracked_bboxes
    if index == 32:
        continue

    # Check if this is our target player
    if tracking_id == target_player_id and not first_frame_processed:
        target_track = track
        first_frame_processed = True
        current_target_found = True
    print(f"Frame {frame_count} - Target player {target_player_id} locked for
standard
        tracking")
    elif target_track is not None and track == target_track:
        current_target_found = True

    # Only track the target player (same as ROI for fair comparison)
    if target_track is not None and track == target_track:
        tracked_bboxes.append(bbox.tolist() + [tracking_id, index])

    # If target is lost, reset target tracking
    if target_track is not None and not current_target_found:
        print(f"Frame {frame_count} - Target player lost in standard
tracking")
        target_track = None
        first_frame_processed = False

    # Ball detection is EXCLUDED (same as ROI approach)
    # Removed the entire ball detection block here for fair comparison

    tboxes.append([[round(bb) for bb in tracked_bbox] for tracked_bbox in
tracked_bboxes])

    tracking_end = time.time()

    # End timing for this frame
    frame_end_time = time.time()
    total_processing_time = frame_end_time - frame_start_time
    detection_time = detection_end - detection_start
    tracking_time = tracking_end - tracking_start
    complete_detection_tracking_time = detection_time + tracking_time

    print(f"Frame {frame_count} - Completed at: {frame_end_time:.6f}")
    print(f"Frame {frame_count} - Total time: {total_processing_time:.6f}s "
        f"(Detection: {detection_time:.6f}s, Tracking:

```

```

{tracking_time:.6f}s, "
    f"Detection+Tracking: {complete_detection_tracking_time:.6f}s)")

    # Store timing data
    timing_data.append([frame_count, frame_start_time, frame_end_time,
                        detection_time, tracking_time,
complete_detection_tracking_time,
                        total_processing_time])

    # Write timing data to CSV
    with open(csv_filename, 'w', newline='') as csvfile:

        writer = csv.writer(csvfile)
        writer.writerow(['Frame_Number', 'Start_Time (unix_timestamp)', 'End_Time
(unix_timestamp)',
'Detection_Time (s)', 'Tracking_Time (s)', 'Complete_Detection_Tracking_Time
(s)',
                        'Total_Processing_Time (s)'])
        writer.writerows(timing_data)

    print(f"\nDetailed timing data saved to: {csv_filename}")
    print(f'Tracked {len(frames)} frames')

    return frames, tboxes, fps

```

[FILE] track.py

Path: Detect_and_Track\deep_sort\track.py

```

class TrackState:
    """
    Enumeration type for the single target track state. Newly created tracks are
    classified as `tentative` until enough evidence has been collected. Then,
    the track state is changed to `confirmed`. Tracks that are no longer alive
    are classified as `deleted` to mark them for removal from the set of active
    tracks.

    """
    Tentative = 1
    Confirmed = 2
    Deleted = 3


class Track:
    """
    A single target track with state space `(x, y, a, h)` and associated
    velocities, where `(x, y)` is the center of the bounding box, `a` is the
    aspect ratio and `h` is the height.

    Parameters
    -----
    mean : ndarray
        Mean vector of the initial state distribution.
    covariance : ndarray
        Covariance matrix of the initial state distribution.
    track_id : int

```



```

    A unique track identifier.
n_init : int
    Number of consecutive detections before the track is confirmed. The
    track state is set to `Deleted` if a miss occurs within the first
    `n_init` frames.
max_age : int
    The maximum number of consecutive misses before the track state is
    set to `Deleted`.
feature : Optional[ndarray]
    Feature vector of the detection this track originates from. If not None,
    this feature is added to the `features` cache.

Attributes
-----
mean : ndarray
    Mean vector of the initial state distribution.
covariance : ndarray
    Covariance matrix of the initial state distribution.
track_id : int
    A unique track identifier.
hits : int
    Total number of measurement updates.

age : int
    Total number of frames since first occurrence.
time_since_update : int
    Total number of frames since last measurement update.
state : TrackState
    The current track state.
features : List[ndarray]
    A cache of features. On each measurement update, the associated feature
    vector is added to this list.

"""

def __init__(self, mean, covariance, track_id, n_init, max_age,
             feature=None, class_name=None):
    self.mean = mean
    self.covariance = covariance
    self.track_id = track_id
    self.hits = 1
    self.age = 1
    self.time_since_update = 0

    self.state = TrackState.Tentative
    self.features = []
    if feature is not None:
        self.features.append(feature)

    self._n_init = n_init
    self._max_age = max_age
    self.class_name = class_name

def to_tlwh(self):
    """Get current position in bounding box format `(top left x, top left y,
    width, height)`."""

    Returns
    -----
    ndarray
        The bounding box.

```

```

    """
    ret = self.mean[:4].copy()
    ret[2] *= ret[3]
    ret[:2] -= ret[2:] / 2
    return ret

def to_tlbr(self):
    """Get current position in bounding box format `(min x, miny, max x,
    max y)`.

    Returns
    -----
    ndarray
        The bounding box.

    """
    ret = self.to_tlwh()
    ret[2:] = ret[:2] + ret[2:]
    return ret

def get_class(self):
    return self.class_name

def predict(self, kf):
    """Propagate the state distribution to the current time step using a
    Kalman filter prediction step.

    Parameters
    -----
    kf : kalman_filter.KalmanFilter
        The Kalman filter.

    """
    self.mean, self.covariance = kf.predict(self.mean, self.covariance)
    self.age += 1
    self.time_since_update += 1

def update(self, kf, detection):
    """Perform Kalman filter measurement update step and update the feature
    cache.

    Parameters
    -----
    kf : kalman_filter.KalmanFilter
        The Kalman filter.
    detection : Detection
        The associated detection.

    """
    self.mean, self.covariance = kf.update(
        self.mean, self.covariance, detection.to_xyah())
    self.features.append(detection.feature)

    self.hits += 1
    self.time_since_update = 0
    if self.state == TrackState.Tentative and self.hits >= self._n_init:
        self.state = TrackState.Confirmed

def mark_missed(self):
    """Mark this track as missed (no association at the current time step).
    """

```

```

        if self.state == TrackState.Tentative:
            self.state = TrackState.Deleted
        elif self.time_since_update > self._max_age:
            self.state = TrackState.Deleted

    def is_tentative(self):
        """Returns True if this track is tentative (unconfirmed).
        """
        return self.state == TrackState.Tentative

    def is_confirmed(self):
        """Returns True if this track is confirmed."""
        return self.state == TrackState.Confirmed

    def is_deleted(self):
        """Returns True if this track is dead and should be deleted."""
        return self.state == TrackState.Deleted

```

[FILE] tracker.py

Path: Detect_and_Track\deep_sort\tracker.py

```

# vim: expandtab:ts=4:sw=4
from __future__ import absolute_import
import numpy as np
from . import kalman_filter
from . import linear_assignment
from . import iou_matching
from .track import Track

class Tracker:
    """
    This is the multi-target tracker.

    Parameters
    -----
    metric : nn_matching.NearestNeighborDistanceMetric
        A distance metric for measurement-to-track association.
    max_age : int
        Maximum number of missed misses before a track is deleted.
    n_init : int
        Number of consecutive detections before the track is confirmed. The
        track state is set to `Deleted` if a miss occurs within the first
        `n_init` frames.

    Attributes
    -----
    metric : nn_matching.NearestNeighborDistanceMetric
        The distance metric used for measurement to track association.
    max_age : int
        Maximum number of missed misses before a track is deleted.
    n_init : int
        Number of frames that a track remains in initialization phase.
    kf : kalman_filter.KalmanFilter
        A Kalman filter to filter target trajectories in image space.
    tracks : List[Track]

```

The list of active tracks at the current time step.

```
"""

def __init__(self, metric, max_iou_distance=0.7, max_age=30, n_init=3):
    self.metric = metric
    self.max_iou_distance = max_iou_distance
    self.max_age = max_age
    self.n_init = n_init

    self.kf = kalman_filter.KalmanFilter()
    self.tracks = []
    self._next_id = 1

def predict(self):
    """Propagate track state distributions one time step forward.

    This function should be called once every time step, before `update`.
    """
    for track in self.tracks:
        track.predict(self.kf)

def update(self, detections):
    """Perform measurement update and track management.

    Parameters
    -----
    detections : List[deep_sort.detection.Detection]
        A list of detections at the current time step.

    """
    # Run matching cascade.
    matches, unmatched_tracks, unmatched_detections = \
        self._match(detections)

    # Update track set.
    for track_idx, detection_idx in matches:
        self.tracks[track_idx].update(
            self.kf, detections[detection_idx])
    for track_idx in unmatched_tracks:
        self.tracks[track_idx].mark_missed()
    for detection_idx in unmatched_detections:
        self._initiate_track(detections[detection_idx])
    self.tracks = [t for t in self.tracks if not t.is_deleted()]

    # Update distance metric.
    active_targets = [t.track_id for t in self.tracks if t.is_confirmed()]
    features, targets = [], []
    for track in self.tracks:
        if not track.is_confirmed():
            continue
        features += track.features
        targets += [track.track_id for _ in track.features]
        track.features = []
    self.metric.partial_fit(
        np.asarray(features), np.asarray(targets), active_targets)

def _match(self, detections):

    def gated_metric(tracks, dets, track_indices, detection_indices):
        features = np.array([dets[i].feature for i in detection_indices])
```

```

        targets = np.array([tracks[i].track_id for i in track_indices])
        cost_matrix = self.metric.distance(features, targets)
        cost_matrix = linear_assignment.gate_cost_matrix(
            self.kf, cost_matrix, tracks, dets, track_indices,

            detection_indices)

    return cost_matrix

# Split track set into confirmed and unconfirmed tracks.
confirmed_tracks = [
    i for i, t in enumerate(self.tracks) if t.is_confirmed()]
unconfirmed_tracks = [
    i for i, t in enumerate(self.tracks) if not t.is_confirmed()]

# Associate confirmed tracks using appearance features.
matches_a, unmatched_tracks_a, unmatched_detections = \
    linear_assignment.matching_cascade(
        gated_metric, self.metric.matching_threshold, self.max_age,
        self.tracks, detections, confirmed_tracks)

# Associate remaining tracks together with unconfirmed tracks using IOU.
iou_track_candidates = unconfirmed_tracks + [
    k for k in unmatched_tracks_a if
        self.tracks[k].time_since_update == 1]
unmatched_tracks_a = [
    k for k in unmatched_tracks_a if
        self.tracks[k].time_since_update != 1]
matches_b, unmatched_tracks_b, unmatched_detections = \
    linear_assignment.min_cost_matching(
        iou_matching.iou_cost, self.max_iou_distance, self.tracks,
        detections, iou_track_candidates, unmatched_detections)

matches = matches_a + matches_b
unmatched_tracks = list(set(unmatched_tracks_a + unmatched_tracks_b))
return matches, unmatched_tracks, unmatched_detections

def _initiate_track(self, detection):
    mean, covariance = self.kf.initiate(detection.to_xyah())
    class_name = detection.get_class()
    self.tracks.append(Track(
        mean, covariance, self._next_id, self.n_init, self.max_age,
        detection.feature, class_name))
    self._next_id += 1

```

[FILE] tracker_implementation.py

Path: Detect_and_Track\deep_sort\tracker_implementation.py

```

import os
os.environ['CUDA_VISIBLE_DEVICES'] = '0'
import cv2
import numpy as np
import time
import csv
from .utils import read_class_names

```

```

from .nn_matching import NearestNeighborDistanceMetric
from .detection import Detection
from .tracker import Tracker
from .generate_detections import create_box_encoder
# import nn_matching
# import generate_detections as gdet

YOLO_COCO_CLASSES = "./Detect_and_Track/model_data/coco/coco.names"

def trackingXl5(Yolo_model, ball_model, video_path):
    """
    this functions is to track objects in a video by:
    1 - reading the video frames
    2 - detecting the objects in the frame
    3 - tracking objects frame by frame by the ID of each object.

    Parameters
    -----
    Yolo_model : pytorch model
        pytorch YoloV5l model.
    ball_model : pytorch model
        pytorch YoloV5l model to detect the ball specifically.
    video_path : string
        the path of the directory of the processed video.

    Return
    -----
    frames : list
        list of frames of the video with objects tracked.
    tboxes :list
        list of every object tracked in every frame.
        object:[y1, x1, y2, x2, class of the object, id of the object]
        where y1, x1, y2, x2 are the coordinates of the box around the object
    fps: int
        number of frames per second used in processing
    """

    # Definition of the parameters
    max_cosine_distance = 0.7
    nn_budget = None

    #initialize deep sort object

    model_filename = './Detect_and_Track/model_data/mars-small128.pb'
    encoder = create_box_encoder(model_filename, batch_size=1)
    metric = NearestNeighborDistanceMetric("cosine", max_cosine_distance,
nn_budget)
    tracker = Tracker(metric)

    if video_path:
        vid = cv2.VideoCapture(video_path) # detect on video

    fps = int(vid.get(cv2.CAP_PROP_FPS))
    print(f'fps of the input video = {fps}')
    print('Please wait ... \n')
    NUM_CLASS = read_class_names(YOLO_COCO_CLASSES)
    key_list = list(NUM_CLASS.keys())
    val_list = list(NUM_CLASS.values())

    frames = []
    tboxes = []

```

```

# Initialize timing variables
frame_count = 0
timing_data = []

# Ensure output directory exists
os.makedirs('./Out', exist_ok=True)

# Create CSV filename
csv_filename = f'./Out/detailed_frame_timing_{int(time.time())}.csv'

while True:
    # Start timing for this frame
    frame_start_time = time.time()

    _, frame = vid.read()
    try:
        original_frame = cv2.cvtColor(frame, cv2.COLOR_BGR2RGB)
        source_frame = original_frame
        original_frame = cv2.resize(original_frame, (1280,720))
        frames.append(original_frame)
    except:
        break

    frame_count += 1
    print(f"Frame {frame_count} - Processing started at:
{frame_start_time:.6f}")

    # Detection timing
    detection_start = time.time()
    # Determine the smaller dimension of the current frame
    frame_height, frame_width = source_frame.shape[:2]
    imgsz = min(frame_width, frame_height)
    # Pass imgsz/size to YOLO model if supported

    try:
        results = Yolo_model(original_frame, size=imgsz)
    except TypeError:
        # Fallback for models that do not accept 'size' argument
        for i in range(3):
            print("MODEL DO NOT ACCEPT THIS PARAMETER FOR SIZE\n")
            results = Yolo_model(original_frame)
        pred_bbox = results.xyxy[0].tolist()
        bboxes = [np.array(box) for box in pred_bbox]

    # extract bboxes to boxes (x, y, width, height), scores and names
    boxes, scores, names = [], [], []
    for bbox in bboxes:
        boxes.append([bbox[0].astype(int), bbox[1].astype(int), bbox[2].astype(
            int)-bbox[0].astype(int), bbox[3].astype(int)-bbox[1].astype(int)])

        scores.append(bbox[4])
        names.append(NUM_CLASS[int(bbox[5])])

    detection_end = time.time()

    # Tracking timing
    tracking_start = time.time()
    boxes = np.array(boxes)
    names = np.array(names)
    scores = np.array(scores)
    features = np.array(encoder(original_frame, boxes))
    detections = [Detection(bbox, score, class_name, feature) for bbox, score,

```

```

class_name,
    feature in zip(boxes, scores, names, features)]

    tracker.predict()
    tracker.update(detections)

    # Obtain info from the tracks
    tracked_bboxes = []
    for track in tracker.tracks:
        if not track.is_confirmed() or track.time_since_update > 5:
            continue

        bbox = track.to_tlbr() # Get the corrected/predicted bounding box
        class_name = track.get_class() #Get the class name of particular
object
        tracking_id = track.track_id # Get the ID for the particular track
        index = key_list[val_list.index(class_name)] # Get predicted object
index by object name
        # give id 0 to the ball
        if index == 32:
            tracked_bboxes.append(bbox.tolist() + [0, index]) # Structure data, that we could
                use it with our draw_bbox function
        else:
            tracked_bboxes.append(bbox.tolist() + [tracking_id, index]) # Structure data,
that
                we could use it with our draw_bbox function

    #detect the ball only
    if 32 not in [b[-1] for b in tracked_bboxes]:
        ball_results = ball_model(original_frame).xyxy[0].tolist()
        if len(ball_results) > 0:
            ball_pred_bbox = ball_results[0]
    ball = [round(ball_pred_bbox[0]), round(ball_pred_bbox[1]),
            round(ball_pred_bbox[2]), round(ball_pred_bbox[3]), 0, 32]
            tracked_bboxes.append(ball)

    tboxes.append([[round(bb) for bb in tracked_bbox] for tracked_bbox in
tracked_bboxes])

    tracking_end = time.time()

    # End timing for this frame
    frame_end_time = time.time()
    total_processing_time = frame_end_time - frame_start_time
    detection_time = detection_end - detection_start
    tracking_time = tracking_end - tracking_start
    complete_detection_tracking_time = detection_time + tracking_time

    print(f"Frame {frame_count} - Completed at: {frame_end_time:.6f}")
    print(f"Frame {frame_count} - Total time: {total_processing_time:.6f}s
(Detection:
{detection_time:.6f}s, Tracking: {tracking_time:.6f}s, Detection+Tracking:
{complete_detection_tracking_time:.6f}s)")

    # Store timing data
    timing_data.append([frame_count, frame_start_time, frame_end_time,
detection_time,
        tracking_time, complete_detection_tracking_time,
total_processing_time])

```



```

# Write timing data to CSV
with open(csv_filename, 'w', newline='') as csvfile:
    writer = csv.writer(csvfile)
writer.writerow(['Frame_Number', 'Start_Time (unix_timestamp)', 'End_Time
(unix_timestamp)', 'Detection_Time (s)', 'Tracking_Time (s)',
    'Complete_Detection_Tracking_Time (s)', 'Total_Processing_Time (s)'])
    writer.writerows(timing_data)

print(f"\nDetailed timing data saved to: {csv_filename}")
print(f'tracked {len(frames)} frames')

return frames, tboxes, fps

```

[FILE] tracking_video.py

Path: Detect_and_Track\deep_sort\tracking_video.py

```

import cv2
import time
from .utils import draw_bbox

YOLO_COCO_CLASSES = "./Detect_and_Track/model_data/coco/coco.names"

def make_tracking_video(frames, tboxes, output_path, fps, draw = True):
    """
    this functions is to create video with objects tracked, each object has its
    bounding box.

    Parameters
    -----
    frames : list
        list of frames to create the video of.
    tboxes : list
        list of coordinates of the bounding box of each object.
    output_path : string
        the path of the directory to save the output video.
    fps: int
        numder of frames per second of the output video.
    draw: boolen
        whether or not to draw the bounding boxes.
    """

    codec = cv2.VideoWriter_fourcc(*'XVID')
    out = cv2.VideoWriter(output_path, codec, fps, (1280,720)) # output_path must
    be .mp4
    times= []
    for frame, tracked_bboxes in zip(frames, tboxes):
        if draw:
            t1 = time.time()
            # draw detection on frame
            img = draw_bbox(frame, tracked_bboxes, CLASSES=YOLO_COCO_CLASSES,
tracking=True)
            t2 = time.time()
            times.append(t2-t1)

        times = times[-20:]

```

```

ms = max(sum(times)/len(times)*1000, 1e-6)
fps = 1000 / ms

img = cv2.putText(img, "Time: {:.1f} FPS".format(fps), (0, 30),
cv2.FONT_HERSHEY_COMPLEX_SMALL
, 1, (0, 0, 255), 2)

print("Time: {:.2f}ms, total FPS: {:.1f}".format(ms, fps))
out.write(img[:, :, ::-1])

else:
    out.write(frame[:, :, ::-1])

```

[FILE] tracks_cleaning.py

Path: Detect_and_Track\deep_sort\tracks_cleaning.py

```

def clean_tracks(tracking_frames, tracking_boxes, ball_only = True):
    """
    function to:
    -----
    1 - remove the zoomed in frames from the video
    2 - improve ball tracking by using a separate model
    3 - remove any object that is not completely visible in the frame

    Parameters
    -----
    tracking_frames : list
        list of frames of the input video with objects tracked.
    tracking_boxes : list
        list of every object tracked in every input frame.
        object:[y1, x1, y2, x2, class of the object, id of the object]
        where y1, x1, y2, x2 are the coordinates of the box around the object
    ball_only : boolean
        whether or not to save only the frames with the ball detected in them.

    Return
    -----
    new_tframes : list
        list of frames of the cleaned video with objects tracked.
    new_tboxes : list
        list of every object tracked in every cleaned frame.
        object:[y1, x1, y2, x2, class of the object, id of the object]
        where y1, x1, y2, x2 are the coordinates of the box around the object
    """

    print(f'length of the original frames = {len(tracking_frames)}')
    new_tframes = []
    new_tboxes = []

    out_boxes = 0
    for boxes, frame in zip(tracking_boxes, tracking_frames):
        zoom = False
        for box in boxes[:]:
            if (box[3] - box[1]) > 160:
                zoom = True
                break

```

```

if not (0 <= box[0] <= 1280) or not (0 <= box[2] < 1280) or not (0 <= box[1]
    <= 720) or not (0 <= box[3] < 720):
    boxes.remove(box)
    out_boxes += 1

if not zoom:
    new_tframes.append(frame)
    new_tboxes.append(boxes)

print(f'\n-----')

print(f'number of outliers removed = {out_boxes}')

print(f'length of zoomed out frames = {len(new_tframes)}')

#removing the frames that ball is not tracked in
if ball_only:
    bframes = []
    bboxes = []
    for frame, boxes in zip(new_tframes, new_tboxes):
        if 32 in [b[-1] for b in boxes]:
            bframes.append(frame)
            bboxes.append(boxes)

    print(f'length of frames only with ball tracked = {len(bframes)}')
    print(f'the final zoomed out cut = {round(len(bframes)/len(tracking_frames),
2)*100}% of the
        original')
    print(f'-----\n')
    return bframes, bboxes

else:
    print(f'length of the final zoomed out frames (with and without the ball) =
{len(new_tframes)}')
    print(f'the final zoomed out cut = {round(len(new_tframes)/len(tracking_frames),
2)*100} % of
        the original')

    return new_tframes, new_tboxes

```

[FILE] utils.py

Path: Detect_and_Track\deep_sort\utils.py

```

import cv2
import random
import colorsys
import numpy as np

YOLO_COCO_CLASSES = "./Detect_and_Track/model_data/coco/coco.names"

def read_class_names(class_file_name):
    # loads class name from a file
    names = {}
    with open(class_file_name, 'r') as data:
        for ID, name in enumerate(data):
            names[ID] = name.strip('\n')

```

```

return names

def draw_bbox(image, bboxes, CLASSES=YOLO_COCO_CLASSES, show_label=True,
show_confidence = True,
    Text_colors=(255,255,0), rectangle_colors='', tracking=False):
    NUM_CLASS = read_class_names(CLASSES)
    num_classes = len(NUM_CLASS)
    image_h, image_w, _ = image.shape
    hsv_tuples = [(1.0 * x / num_classes, 1., 1.) for x in range(num_classes)]
    #print("hsv_tuples", hsv_tuples)
    colors = list(map(lambda x: colorsys.hsv_to_rgb(*x), hsv_tuples))
    colors = list(map(lambda x: (int(x[0] * 255), int(x[1] * 255), int(x[2] *
255)), colors))

    random.seed(0)
    random.shuffle(colors)
    random.seed(None)

    for i, bbox in enumerate(bboxes):
        coor = np.array(bbox[:4], dtype=np.int32)
        score = bbox[4]
        class_ind = int(bbox[5])
        bbox_color = rectangle_colors if rectangle_colors != '' else
colors[class_ind]
        bbox_thick = int(0.6 * (image_h + image_w) / 1000)
        if bbox_thick < 1: bbox_thick = 1
        fontScale = 0.75 * bbox_thick
        (x1, y1), (x2, y2) = (coor[0], coor[1]), (coor[2], coor[3])

        # put object rectangle
        cv2.rectangle(image, (x1, y1), (x2, y2), bbox_color, bbox_thick*2)

        if show_label:
            # get text label
            score_str = " {:.2f}".format(score) if show_confidence else ""

            if tracking: score_str = " "+str(score)

            try:
                label = "{}".format(NUM_CLASS[class_ind]) + score_str
            except KeyError:
                print("You received KeyError, this might be that you are trying to use yolo
original weights")
                print("while using custom classes, if using custom model in configs.py set
YOLO_CUSTOM_WEIGHTS = True")

            # get text size
            (text_width, text_height), baseline = cv2.getTextSize(label,
cv2.FONT_HERSHEY_COMPLEX_SMALL,
                                                                    fontScale,
thickness=bbox_thick)
            # put filled text rectangle
            cv2.rectangle(image, (x1, y1), (x1 + text_width, y1 - text_height - baseline),
bbox_color, thickness=cv2.FILLED)

            # put text above rectangle
            cv2.putText(image, label, (x1, y1-4), cv2.FONT_HERSHEY_COMPLEX_SMALL,
fontScale, Text_colors, bbox_thick, lineType=cv2.LINE_AA)

    return image

```


[FOLDER] Detect_and_Track\init_dataframe

Location: Detect_and_Track\init_dataframe

[FILE] __init__.py

Path: Detect_and_Track\init_dataframe__init__.py

File is empty or could not be read.

[FILE] create_df.py

Path: Detect_and_Track\init_dataframe\create_df.py

```
import pandas as pd
from .teams_classification import classify_teams
from .players_positions import get_positions

def creatInitDataFrame(tracks, frames, frame_colors = None):
    """
    this functions is to create initial dataframe, with players coordinates
    relative to TV video,
    and classify them according to teams colors.

    Parameters
    -----
    tracks : dataframe
        list of every object tracked in every frame.
        object:[y1, x1, y2, x2, class of the object, id of the object]
        where y1, x1, y2, x2 are the coordinates of the box around the object
    frames : dataframe
        list of frames of the video with objects tracked.
    frame_colors : list of strings
        list of the colors to classify the teams in that order:
        [Defense team color, Defense goalkeeper color, Attack team color, Attack
        goalkeeper color,
         referee color, ball color]
        or None.

    Return
    -----
    df_points : dataframe
        the dataframe of the the tracked objects.
        consists of 7 columns:
        1 - frame: the index of the frame in the video
        2 - ID: the ID of the tracked object in the frame
        3 - X: the X coordinate of the lower center of the object
        4 - Y: the Y coordinate of the lower center of the object
        5 - Class: the type of the tracked object (ball or player)
        6 - Color: the color of the team of the tracked player
```

```

    7 - position: the position of the tracked object (Attack - Defense - ball -
Not a player)
'''

```

```

frame_list = []
X_list = []
Y_list = []
ID = []
Classes = []
colors = []
for frame_idx, frame_tracks in enumerate(tracks):
    for track in frame_tracks:
        x1 = track[0]
        x2 = track[2]
        y1 = track[1]
        y2 = track[3]

        frame_list.append(frame_idx + 1)
        X = (x1 + x2)/2
        X_list.append(round(X))
        Y_list.append(y2)
        ID.append(track[4])
        Classes.append(track[5])

        obj = frames[frame_idx][y1 : y2+1, x1 : x2+1, :]
        if obj.tolist() == []:
            print(f'frame_idx {frame_idx}')
            print(f'track {track}')
            obj = frames[frame_idx][y2:y1 , x2:x1, :]

        # frame_colors = [def, def goalkeeper, att, att goalkeeper, ref, ball]
        if track[5] == 32:
            colors.append(frame_colors[-1])
        else:
            color = classify_teams(obj, frame_colors[:-1])
            colors.append(color)

data = {'frame':frame_list,
        'ID':ID,
        'X':X_list,
        'Y':Y_list,
        'Class':Classes,
        'Color':colors}

df_points = pd.DataFrame(data)
df_points['Class'] = df_points['Class'].replace([0], 'Player')
df_points['Class'] = df_points['Class'].replace([32], 'Ball')
df_points.reset_index(drop=True, inplace=True)

df_points = get_positions(df_points, frame_colors)

return df_points

```

[FILE] players_positions.py

Path: Detect_and_Track\init_dataframe\players_positions.py

```

import pandas as pd

def get_positions(df, colors):
    '''
    this functions is to decide if the boject is in Attacking team or Defense team,
    or if it is a
        ball.

    Parameters
    -----
    df : dataframe
        the dataframe of the objects.
    colors : list of strings
        list of the colors to classify the teams.

    Return
    -----
    df : dataframe
        the dataframe of the objects and their positions.
    '''

    positions = []
    for i, c in enumerate(df['Color'].tolist()):
        if df['Class'][i] == 'Ball':
            positions.append('Ball')
            continue

        if df['Color'][i] == colors[0] or df['Color'][i] == colors[1]:
            positions.append('Defense')
        elif df['Color'][i] == colors[2] or df['Color'][i] == colors[3]:
            positions.append('Attack')
        else:
            positions.append('Not a player')

    df['Position'] = positions
    return df

```

[FILE] teams_classification.py

Path: Detect_and_Track\init_dataframe\teams_classification.py

```

import cv2
import numpy as np

def classify_teams(player, color):
    '''
    this functions is to classify players to teams based on thier colors.

    Parameters
    -----
    player : image of the player
        the dataframe of the objects.
    colors : list of strings
        list of the colors to choose the player color from.
        the colors available are : [white, black, red, blue, green, yellow, purple,
skyblue]

```



```

Return
-----
player choosen color: string
'''

player = cv2.cvtColor(player, cv2.COLOR_RGB2HSV)
masks = []
Hsv_boundaries = [
    ([0,0,185],[180,80,255], 'white'), #white
    ([0,0,0],[180,255,65], 'black'), #black
    ([0,120,20],[15,255,255], 'red'), #red
    ([163,100,20],[180,255,255], 'red'), #dark red/ pink
    ([90,100,20],[130,255,255], 'blue'), #blue
    ([50,100,20],[80,255,255], 'green'), #green
    ([17,150,20],[35,255,255], 'yellow'), #yellow
    ([131,60,20],[162,255,255], 'purple'), #purple
    ([80,36,20],[105,255,255], 'skyblue'), #skyblue
]

filtered_boundaries = list(filter(lambda boundary: boundary[2] in color,
Hsv_boundaries))
for boundary in filtered_boundaries:
    mask = cv2.inRange(player, np.array(boundary[0]) , np.array(boundary[1]))
    count = np.count_nonzero(cv2.bitwise_and(player, player, mask = mask))
    masks.append(count)

color_idx = masks.index(max(masks))

return filtered_boundaries[color_idx][2]

```

[FOLDER] Detect_and_Track\yoloV5

Location: Detect_and_Track\yoloV5

[FILE] __init__.py

Path: Detect_and_Track\yoloV5__init__.py

```
#
```

[FILE] detector.py

Path: Detect_and_Track\yoloV5\detector.py

```
import os
os.environ['CUDA_VISIBLE_DEVICES'] = '0'
# import cv2
import matplotlib.pyplot as plt
import numpy as np
from .utils import *

YOLO_COCO_CLASSES = "./Detect_and_Track/model_data/coco/coco.names"

def detectX15(model, image_path, show=True):
    """
    this functions is to detect objects in an image using Yolov5 models:

    Parameters
    -----
    model : pytorch model
            pytorch YoloV5l model.
    image_path : string
                the path of the directory of the processed image.
    show : boolen
            whether or not to draw the bounding boxes.

    Return
    -----
    image : image
            image with objects tracked.
    nbboxes : list
            list of every object tracked in the image.
            object:[y1, x1, y2, x2, class of the object]
            where y1, x1, y2, x2 are the coordinates of the box around the object
    """
    #image
    original_image = cv2.imread(image_path)
    original_image = cv2.resize(original_image, (1280, 720))
    original_image = cv2.cvtColor(original_image, cv2.COLOR_BGR2RGB)
```

```

# Inference
results = model(original_image)
pred_bbox = results.xyxy[0].tolist()
bboxes = [np.array(box) for box in pred_bbox]
print(f'image shape {original_image.shape[:-1]}')
print(f'number of objects detected = {len(bboxes)}')

if show:
    # Show the image
    image = draw_bbox(original_image, bboxes, CLASSES=YOLO_COCO_CLASSES,
                      rectangle_colors=(255,0,0))
    # cv2.imshow(image[:, :, :-1])
    plt.figure(figsize=(12, 8))
    plt.imshow(image)

    plt.show()
else:
    image = original_image

nbboxes = [[int(round(b)) if b != list(bbox)[4] else round(b, 2) for b in
list(bbox)] for bbox
            in bboxes]
return image, nbboxes

```

[FILE] load_models.py

Path: Detect_and_Track\yoloV5\load_models.py

```

import torch

def yoloV5l(imgsz=640):
    """
    this function loads YoloV5l pytorch models from torch hub with configurable
    input size

    Parameters
    -----
    imgsz : int or tuple, optional
        Input image size for YOLO model (default: 640)

    Return
    -----
    model1 : pytorch model
        pytorch YoloV5l model
    ball_model : pytorch model
        pytorch YoloV5l model to detect the ball specifically
    """

    # Load models without imgsz parameter
    model1 = torch.hub.load('ultralytics/yolov5', 'yolov5l')
    model1.classes = [0, 32]
    # Set image size after loading
    model1.imgsz = imgsz

    ball_model = torch.hub.load('ultralytics/yolov5', 'yolov5l')
    ball_model.classes = [32]
    ball_model.conf = 0.15

```

```

ball_model.max_det = 1
# Set image size after loading
ball_model.imgsz = imgsz

print('\n-----\n')

return model_i, ball_model

```

[FILE] utils.py

Path: Detect_and_Track\yoloV5\utils.py

```

import cv2
import random
import colorsys
import numpy as np

YOLO_COCO_CLASSES = "./model_data/coco/coco.names"

def read_class_names(class_file_name):
    # loads class name from a file
    names = {}
    with open(class_file_name, 'r') as data:
        for ID, name in enumerate(data):
            names[ID] = name.strip('\n')
    return names

def draw_bbox(image, bboxes, CLASSES=YOLO_COCO_CLASSES, show_label=True,
show_confidence = True,
Text_colors=(255,255,0), rectangle_colors='', tracking=False):
    NUM_CLASS = read_class_names(CLASSES)
    num_classes = len(NUM_CLASS)
    image_h, image_w, _ = image.shape
    hsv_tuples = [(1.0 * x / num_classes, 1., 1.) for x in range(num_classes)]
    #print("hsv_tuples", hsv_tuples)
    colors = list(map(lambda x: colorsys.hsv_to_rgb(*x), hsv_tuples))
    colors = list(map(lambda x: (int(x[0] * 255), int(x[1] * 255), int(x[2] *
255)), colors))

    random.seed(0)
    random.shuffle(colors)
    random.seed(None)

    for i, bbox in enumerate(bboxes):
        coor = np.array(bbox[:4], dtype=np.int32)
        score = bbox[4]
        class_ind = int(bbox[5])
        bbox_color = rectangle_colors if rectangle_colors != '' else
colors[class_ind]
        bbox_thick = int(0.6 * (image_h + image_w) / 1000)
        if bbox_thick < 1: bbox_thick = 1
        fontScale = 0.75 * bbox_thick
        (x1, y1), (x2, y2) = (coor[0], coor[1]), (coor[2], coor[3])

```

```

# put object rectangle
cv2.rectangle(image, (x1, y1), (x2, y2), bbox_color, bbox_thick*2)

if show_label:
    # get text label
    score_str = "{:.2f}".format(score) if show_confidence else ""

    if tracking: score_str = " "+str(score)

    try:
        label = "{}".format(NUM_CLASS[class_ind]) + score_str
    except KeyError:
        print("You received KeyError, this might be that you are trying to use yolo
              original weights")
        print("while using custom classes, if using custom model in configs.py set
              YOLO_CUSTOM_WEIGHTS = True")

    # get text size
    (text_width, text_height), baseline = cv2.getTextSize(label,
        cv2.FONT_HERSHEY_COMPLEX_SMALL,
                                                    fontScale,
thicknes=bbox_thick)
    # put filled text rectangle
    cv2.rectangle(image, (x1, y1), (x1 + text_width, y1 - text_height - baseline),
        bbox_color, thickness=cv2.FILLED)

    # put text above rectangle
    cv2.putText(image, label, (x1, y1-4), cv2.FONT_HERSHEY_COMPLEX_SMALL,
        fontScale, Text_colors, bbox_thick, lineType=cv2.LINE_AA)

return image

```

[FOLDER] Distance_measurement

Location: Distance_measurement

[FILE] __init__.py

Path: Distance_measurement__init__.py

```
"""
Distance Measurement Module for Tennis Court Tracking

This module provides comprehensive tennis court tracking capabilities including:
- Corner detection from video frames
- Coordinate mapping using homography
- Player distance and movement analysis
- Ball trajectory and distance calculation
- Dynamic homography management
"""

from .corner_detection import CornerDetector
from .coordinate_mapper import CoordinateMapper
from .player_distance_calculator import PlayerDistanceCalculator
from .ball_distance_calculator import BallDistanceCalculator
from .homography_manager import HomographyManager
from .tennis_court_tracker import (
    TennisCourtTracker,
    main_video_processing_pipeline,
    main_video_processing_pipeline_dynamic
)

__all__ = [
    'CornerDetector',
    'CoordinateMapper',
    'PlayerDistanceCalculator',
    'BallDistanceCalculator',
    'HomographyManager',
    'TennisCourtTracker',
    'main_video_processing_pipeline',
    'main_video_processing_pipeline_dynamic'
]

__version__ = "1.0.0"
```

[FILE] advanced_tennis_tracker.py

Path: Distance_measurement\advanced_tennis_tracker.py

```
"""
Advanced Tennis Tracker with Improved Distance Calculation and Outlier Handling
"""
```

```

import pandas as pd
import numpy as np
import cv2
from scipy import stats
from sklearn.cluster import DBSCAN
from sklearn.preprocessing import StandardScaler

class AdvancedTennisTracker:
    def __init__(self, court_width=10.97, court_length=23.77):
        self.court_width = court_width
        self.court_length = court_length

        # Define realistic boundaries with larger buffer for initial filtering
        self.extended_buffer = 10.0 # meters
        self.tight_buffer = 3.0 # meters for final filtering

        self.court_lines = {
            'baseline_near': 0,
            'service_line_near': 6.40,
            'net': court_length / 2,
            'service_line_far': court_length - 6.40,
            'baseline_far': court_length,
            'center_line': court_width / 2
        }

    def detect_and_fix_corner_issues(self, corners_data):
        """
        Detect and fix corner detection issues that lead to poor homography
        """
        print("\n=== CORNER ISSUE DETECTION ===")

        if len(corners_data) < 4:
            print("[ERROR] Not enough corners detected")
            return None

        # Analyze corner stability across frames
        corner_std = np.std(corners_data, axis=0)
        print(f"Corner stability (std dev): {corner_std}")

        # Check for suspicious perfect validation (0.000m errors)
        # This often indicates corner detection picked the same points repeatedly
        max_std = np.max(corner_std)
        if max_std < 1.0: # Very low variation
            print("[WARNING] Corners show very low variation - may be detection artifacts")
            print("Consider manual corner specification or different detection parameters")

            # Use median corners instead of mean for robustness
            median_corners = np.median(corners_data, axis=0)
            print(f"Using median corners: {median_corners}")

            return median_corners

    def robust_outlier_detection(self, df):
        """
        Advanced outlier detection using multiple methods
        """
        print("\n=== ADVANCED OUTLIER DETECTION ===")

```

```

# Method 1: Statistical outliers (Z-score)
z_scores_x = np.abs(stats.zscore(df['X_court']))
z_scores_y = np.abs(stats.zscore(df['Y_court']))
statistical_outliers = (z_scores_x > 3) | (z_scores_y > 3)

# Method 2: IQR-based outliers
Q1_x, Q3_x = df['X_court'].quantile([0.25, 0.75])
Q1_y, Q3_y = df['Y_court'].quantile([0.25, 0.75])
IQR_x, IQR_y = Q3_x - Q1_x, Q3_y - Q1_y

iqr_outliers = (
    (df['X_court'] < Q1_x - 1.5 * IQR_x) |
    (df['X_court'] > Q3_x + 1.5 * IQR_x) |
    (df['Y_court'] < Q1_y - 1.5 * IQR_y) |
    (df['Y_court'] > Q3_y + 1.5 * IQR_y)
)

# Method 3: Court boundary outliers
court_outliers = (
    (df['X_court'] < -self.extended_buffer) |
    (df['X_court'] > self.court_width + self.extended_buffer) |
    (df['Y_court'] < -self.extended_buffer) |
    (df['Y_court'] > self.court_length + self.extended_buffer)
)

# Method 4: DBSCAN clustering to find spatial outliers
coordinates = df[['X_court', 'Y_court']].values
scaler = StandardScaler()
coordinates_scaled = scaler.fit_transform(coordinates)

clustering = DBSCAN(eps=0.5, min_samples=3).fit(coordinates_scaled)
cluster_outliers = clustering.labels_ == -1

# Combine methods
combined_outliers = statistical_outliers | iqr_outliers | court_outliers
| cluster_outliers

print(f"Statistical outliers: {statistical_outliers.sum()}
({statistical_outliers.sum()/len(
    df)*100:.1f}%)")

print(f"IQR outliers: {iqr_outliers.sum()}
({iqr_outliers.sum()/len(df)*100:.1f}%)")
print(f"Court boundary outliers: {court_outliers.sum()}
({court_outliers.sum()/len(
    df)*100:.1f}%)")
print(f"Clustering outliers: {cluster_outliers.sum()}
({cluster_outliers.sum()/len(
    df)*100:.1f}%)")
print(f"Combined outliers: {combined_outliers.sum()}
({combined_outliers.sum()/len(
    df)*100:.1f}%)")

return combined_outliers

def interpolate_missing_positions(self, df, id_col='ID'):
    """
    Interpolate missing positions for continuous tracking
    """
    print("\n=== POSITION INTERPOLATION ===")

    df_interpolated = df.copy()

```



```

for obj_id in df['ID'].unique():
    obj_data = df[df[id_col] == obj_id].sort_values('frame')

    if len(obj_data) < 2:
        continue

    # Create frame range for this object
    frame_min, frame_max = obj_data['frame'].min(),
obj_data['frame'].max()
    all_frames = pd.DataFrame({'frame': range(frame_min, frame_max + 1)})

    # Merge with object data
    obj_complete = pd.merge(all_frames, obj_data, on='frame', how='left')

    # Interpolate missing positions
    obj_complete['X_court_interp'] =
obj_complete['X_court'].interpolate(method='linear')
    obj_complete['Y_court_interp'] =
obj_complete['Y_court'].interpolate(method='linear')

    # Fill other columns
    obj_complete['ID'] = obj_complete['ID'].fillna(obj_id)
    obj_complete['Class'] = obj_complete['Class'].fillna(method='ffill').fillna(
        method='bfill')

    # Only keep interpolated frames that were missing
    missing_frames = obj_complete[obj_complete['X_court'].isna() &
        obj_complete['X_court_interp'].notna()]

    if len(missing_frames) > 0:
        print(f"Object {obj_id}: Interpolated {len(missing_frames)}
missing positions")

        # Add interpolated data back to main dataframe
        for _, row in missing_frames.iterrows():
            new_row = {

                'frame': row['frame'],
                'ID': obj_id,
                'X_court': row['X_court_interp'],
                'Y_court': row['Y_court_interp'],
                'Class': row['Class'],
                'interpolated': True
            }
        df_interpolated = pd.concat([df_interpolated, pd.DataFrame([new_row])],
            ignore_index=True)

    return df_interpolated.sort_values(['frame', 'ID'])

def calculate_corrected_distances(self, df, fps=25.0, id_col='ID',
class_col='Class'):
    """
    Calculate distances with outlier filtering and interpolation
    """
    print("\n=== CORRECTED DISTANCE CALCULATION ===")

    # Step 1: Remove outliers
    outliers = self.robust_outlier_detection(df)
    df_clean = df[~outliers].copy()

    print(f"Removed {outliers.sum()} outliers, {len(df_clean)} points

```

```

remaining")

    # Step 2: Interpolate missing positions
    df_interpolated = self.interpolate_missing_positions(df_clean, id_col)

    # Step 3: Calculate distances for each object
    results = []

    for obj_id in df_interpolated[id_col].unique():
        obj_data = df_interpolated[df_interpolated[id_col] ==
obj_id].sort_values('frame')

        if len(obj_data) < 2:
            continue

        # Calculate frame-to-frame distances
        positions = obj_data[['X_court', 'Y_court']].values
        distances = np.sqrt(np.sum(np.diff(positions, axis=0)**2, axis=1))

        # Calculate time intervals (handle variable frame rates)
        frame_intervals = np.diff(obj_data['frame'].values) / fps

        # Calculate speeds (with outlier filtering)
        speeds = distances / frame_intervals # m/s
        speeds_kmh = speeds * 3.6 # km/h

        # Filter unrealistic speeds (> 50 km/h for players, > 200 km/h for
ball)
        obj_class = obj_data[class_col].iloc[0] if class_col in
obj_data.columns else 'Unknown'
        max_speed = 200 if obj_class == 'Ball' else 50

        realistic_speeds = speeds_kmh[speeds_kmh <= max_speed]
        realistic_distances = distances[speeds_kmh <= max_speed]

        # Calculate statistics
        total_distance = realistic_distances.sum()
        avg_speed = realistic_speeds.mean() if len(realistic_speeds) > 0 else
0
        max_speed_actual = realistic_speeds.max() if len(realistic_speeds) >
0 else 0

        # Calculate time span
        time_span = (obj_data['frame'].max() - obj_data['frame'].min()) / fps

        # Calculate court coverage (area of bounding box)
        x_range = obj_data['X_court'].max() - obj_data['X_court'].min()
        y_range = obj_data['Y_court'].max() - obj_data['Y_court'].min()
        court_coverage = x_range * y_range

        results.append({
            'ID': obj_id,
            'Class': obj_class,
            'TotalDistance': total_distance,
            'AverageSpeed': avg_speed,
            'MaxSpeed': max_speed_actual,
            'TimeSpan': time_span,
            'CourtCoverage': court_coverage,
            'DataPoints': len(obj_data),
            'InterpolatedPoints': obj_data.get('interpolated', False).sum(),
            'OutlierRate': outliers[df[id_col] == obj_id].sum() / len(df[df[id_col] ==

```

```

obj_id])
        * 100
    })

    return pd.DataFrame(results), df_clean, df_interpolated

def validate_and_suggest_improvements(self, results_df, original_df):
    """
    Validate results and suggest improvements
    """
    print("\n=== VALIDATION AND SUGGESTIONS ===")

    issues = []
    suggestions = []

    # Check for unrealistic distances
    player_results = results_df[results_df['Class'] == 'Player']

    if len(player_results) > 0:
        avg_distance = player_results['TotalDistance'].mean()
        max_distance = player_results['TotalDistance'].max()

        print(f"Player distance statistics:")

        print(f" Average total distance: {avg_distance:.2f}m")
        print(f" Maximum total distance: {max_distance:.2f}m")
        print(f" Average time span: {player_results['TimeSpan'].mean():.2f}s")

        # Check if distances are too low (common issue)
        if avg_distance < 5.0:
            issues.append("Player distances appear too low")
            suggestions.append("1. Check homography computation - corners may be incorrectly
            detected")
            suggestions.append("2. Verify frame rate setting (currently
            assuming 25 fps)")
            suggestions.append("3. Consider manual corner specification")

        # Check outlier rates
        high_outlier_objects = results_df[results_df['OutlierRate'] > 30]
        if len(high_outlier_objects) > 0:
            issues.append(f"{len(high_outlier_objects)} objects have >30%
            outlier rate")
            suggestions.append("4. Improve corner detection stability")
            suggestions.append("5. Use dynamic homography updates")

        # Check coordinate ranges
        x_range = original_df['X_court'].max() - original_df['X_court'].min()
        y_range = original_df['Y_court'].max() - original_df['Y_court'].min()

        if x_range > 30 or y_range > 40:
            issues.append("Coordinate ranges are too large")
            suggestions.append("6. Recalibrate court corner detection")
            suggestions.append("7. Check for camera movement or distortion")

    if issues:
        print("\n[WARNING] Issues detected:")
        for issue in issues:
            print(f" - {issue}")
        print("\n[SUGGESTIONS] Improvement suggestions:")
        for suggestion in suggestions:
            print(f" {suggestion}")
    else:

```

```

        print("\n[SUCCESS] Results look reasonable!")

    return issues, suggestions

def manual_corner_correction(self, image_path=None):
    """
    Provide manual corner specification when automatic detection fails
    """
    print("\n=== MANUAL CORNER CORRECTION ===")
    print("Based on your court image, try these manually specified corners:")
    print("(Update these coordinates based on your specific court image)")

    # These are example coordinates - you should update based on your court
image    manual_corners = [

        (136.9, 615.0), # Bottom-left baseline
        (1223.5, 621.0), # Bottom-right baseline
        (890.6, 107.1), # Top-right baseline
        (375.4, 227.4) # Top-left baseline
    ]

    print("Suggested manual corners:")
    labels = ["Bottom-Left", "Bottom-Right", "Top-Right", "Top-Left"]
    for corner, label in zip(manual_corners, labels):
        print(f" {label}: {corner}")

    return manual_corners

def recompute_homography_with_manual_corners(self, manual_corners):
    """
    Recompute homography using manually specified corners
    """
    print("\n=== RECOMPUTING HOMOGRAPHY ===")

    # Real court corners (standard)
    real_corners = np.array([
        [0, 0], # Bottom-left
        [self.court_width, 0], # Bottom-right
        [self.court_width, self.court_length], # Top-right
        [0, self.court_length] # Top-left
    ], dtype=np.float32)

    pixel_corners = np.array(manual_corners, dtype=np.float32)

    # Compute homography
    H, status = cv2.findHomography(pixel_corners, real_corners, cv2.RANSAC,
5.0)

    if H is None:
        print("[ERROR] Failed to compute homography with manual corners")
        return None

    # Validate
    transformed = cv2.perspectiveTransform(pixel_corners.reshape(-1, 1, 2),
H).reshape(-1, 2)
    errors = np.linalg.norm(transformed - real_corners, axis=1)

    print("Manual corner validation:")
    for i, (label, error) in enumerate(zip(["BL", "BR", "TR", "TL"], errors)):
        print(f" {label}: {error:.3f}m error")

```

```

        max_error = np.max(errors)
        if max_error < 0.5:
            print(f"[SUCCESS] Manual homography looks good (max error:
{max_error:.3f}m)")
        else:
            print(f"[WARNING] Manual homography has high error (max error:
{max_error:.3f}m)")

    return H

def process_with_advanced_tracking(data_csv_path, fps=25.0,
use_manual_corners=False):
    """
    Process tennis tracking data with advanced outlier handling and distance
    correction
    """
    print("=== ADVANCED TENNIS TRACKING PROCESSING ===")

    # Load data
    df = pd.read_csv(data_csv_path)
    print(f"Loaded {len(df)} tracking points")

    # Initialize advanced tracker
    tracker = AdvancedTennisTracker()

    # Process with advanced methods
    results_df, df_clean, df_interpolated =
tracker.calculate_corrected_distances(df, fps=fps)

    # Validate and get suggestions
    issues, suggestions = tracker.validate_and_suggest_improvements(results_df,
df)

    # If manual corner correction is needed
    if use_manual_corners or len(issues) > 0:
        print("\n" + "="*60)
        print("MANUAL CORNER CORRECTION RECOMMENDED")
        print("="*60)

        manual_corners = tracker.manual_corner_correction()

        # You can uncomment and modify this section to apply manual correction:
        # H_manual = tracker.recompute_homography_with_manual_corners(manual_corne
rs)

        # if H_manual is not None:
        # # Reapply mapping with manual homography
        # # ... (implement remapping logic)
        # pass

    # Save results
    output_path = data_csv_path.replace('.csv', '_advanced_corrected.csv')
    df_interpolated.to_csv(output_path, index=False)

    results_path = data_csv_path.replace('.csv', '_advanced_results.csv')
    results_df.to_csv(results_path, index=False)

    print(f"\n[SAVE] Advanced results saved to: {output_path}")
    print(f"[SAVE] Distance analysis saved to: {results_path}")

    return results_df, df_clean, df_interpolated, issues, suggestions

```

```

# Usage example
def fix_distance_calculation(csv_path):
    """
    Main function to fix distance calculation issues
    """
    # Process with advanced tracking
    results, clean_df, interpolated_df, issues, suggestions =
process_with_advanced_tracking(
    csv_path,
    fps=25.0, # Adjust based on your video frame rate
    use_manual_corners=True # Set to True if automatic corner detection is
poor
)

    print("\n=== FINAL RESULTS ===")
    print("Player movement analysis:")
    player_results = results[results['Class'] == 'Player']

    for _, player in player_results.iterrows():
        if player['TotalDistance'] > 0.1: # Only show players with significant
movement
            print(f"\nPlayer {player['ID']}:")
            print(f" Total distance: {player['TotalDistance']:.2f}m")
            print(f" Average speed: {player['AverageSpeed']:.1f} km/h")
            print(f" Max speed: {player['MaxSpeed']:.1f} km/h")
            print(f" Time span: {player['TimeSpan']:.1f}s")
            print(f" Outlier rate: {player['OutlierRate']:.1f}%")

    return results, clean_df, interpolated_df

```

[FILE] ball_distance_calculator.py

Path: Distance_measurement\ball_distance_calculator.py

```

"""
Ball Distance Calculator Module for Tennis Court Tracking
"""
import numpy as np
import pandas as pd

class BallDistanceCalculator:
    def __init__(self):
        # Tennis court dimensions in meters (doubles)
        self.court_width = 10.97
        self.court_length = 23.77

    def calculate_ball_distance_improved(self, df, x_col='X_court',
y_col='Y_court',
class_col='Class', frame_col='frame',
fps=30.0):
    """
    Improved ball distance calculation with temporal awareness and trajectory
estimation

    Args:

```

```

df: DataFrame with tracking data
x_col, y_col: Column names for court coordinates
class_col: Column name for object classification
frame_col: Column name for frame numbers
fps: Video frame rate (frames per second)

Returns:
dict: Contains total distance, average speed, max speed, and
trajectory info
"""
ball_data = df[df[class_col] == 'Ball'].copy()

if ball_data.empty or len(ball_data) < 2:
    return {
        'total_distance': 0.0,
        'average_speed': 0.0,
        'max_speed': 0.0,
        'trajectory_segments': 0,
        'detection_gaps': 0,
        'valid_points': 0
    }

# Sort by frame number
ball_data = ball_data.sort_values(frame_col).reset_index(drop=True)

# Calculate time intervals between detections
frame_diffs = np.diff(ball_data[frame_col].values)
time_intervals = frame_diffs / fps # Convert to seconds

# Get coordinate differences
coords = ball_data[[x_col, y_col]].values
coord_diffs = np.diff(coords, axis=0)

frame_distances = np.linalg.norm(coord_diffs, axis=1)

# Calculate instantaneous speeds (m/s)
speeds = np.divide(frame_distances, time_intervals,
                    out=np.zeros_like(frame_distances),
                    where=time_intervals != 0)

# Filter out unrealistic speeds
max_realistic_speed = 70.0 # m/s
valid_speed_mask = (speeds > 0) & (speeds <= max_realistic_speed)

# Detect gaps in detection (where frame difference > 1)
detection_gaps = np.sum(frame_diffs > 1)

# Handle gaps by estimating trajectory
total_distance = 0.0
trajectory_segments = 0

for i in range(len(frame_distances)):
    if valid_speed_mask[i]:
        if frame_diffs[i] == 1:
            # Consecutive frames - use direct distance
            total_distance += frame_distances[i]
        else:
            # Gap in detection - estimate trajectory
            gap_frames = frame_diffs[i]
            gap_time = time_intervals[i]

            # Simple linear interpolation for gaps

```

```

        estimated_distance = frame_distances[i]
        total_distance += estimated_distance

print(f"INFO: Gap detected {gap_frames} frames, estimated distance
      {estimated_distance:.2f}m")

    trajectory_segments += 1

    # Calculate statistics
    valid_speeds = speeds[valid_speed_mask]
    average_speed = np.mean(valid_speeds) if len(valid_speeds) > 0 else 0.0
    max_speed = np.max(valid_speeds) if len(valid_speeds) > 0 else 0.0

    # Convert speeds to km/h for reporting
    average_speed_kmh = average_speed * 3.6
    max_speed_kmh = max_speed * 3.6

    print(f"BALL ANALYSIS: Total distance {total_distance:.2f}m")
    print(f"BALL ANALYSIS: Average speed {average_speed_kmh:.1f} km/h")
    print(f"BALL ANALYSIS: Max speed {max_speed_kmh:.1f} km/h")
    print(f"BALL ANALYSIS: Detection gaps {detection_gaps}")

    print(f"BALL ANALYSIS: Valid trajectory segments {trajectory_segments}")

    return {
        'total_distance': total_distance,
        'average_speed': average_speed,
        'max_speed': max_speed,
        'average_speed_kmh': average_speed_kmh,
        'max_speed_kmh': max_speed_kmh,
        'trajectory_segments': trajectory_segments,
        'detection_gaps': detection_gaps,
        'valid_points': len(ball_data),
        'filtered_unrealistic': np.sum(~valid_speed_mask)
    }

def calculate_ball_distance_with_trajectory_estimation(self, df, x_col='X_court',
    y_col='Y_court',
    class_col='Class', frame_col='frame',
                                                    fps=30.0):
    """
    Advanced ball distance calculation with parabolic trajectory estimation

    Args:
        df: DataFrame with tracking data
        x_col, y_col: Column names for court coordinates
        class_col: Column name for object classification
        frame_col: Column name for frame numbers
        fps: Video frame rate (frames per second)

    Returns:
        dict: Contains total distance and trajectory estimation info
    """
    ball_data = df[df[class_col] == 'Ball'].copy()

    if ball_data.empty or len(ball_data) < 3:
    return self.calculate_ball_distance_improved(df, x_col, y_col, class_col,
frame_col,
        fps)

    ball_data = ball_data.sort_values(frame_col).reset_index(drop=True)

```



```

total_distance = 0.0
coords = ball_data[[x_col, y_col]].values
frames = ball_data[frame_col].values

i = 0
while i < len(coords) - 1:
    current_frame = frames[i]
    next_frame = frames[i + 1]

    if next_frame - current_frame == 1:
        # Consecutive frames - direct distance

        distance = np.linalg.norm(coords[i + 1] - coords[i])
        total_distance += distance
        i += 1
    else:
        # Gap in detection - try to estimate trajectory
        gap_size = next_frame - current_frame

        if gap_size <= 5 and i + 1 < len(coords): # Only estimate for
small gaps
            # Use parabolic trajectory estimation
            p1 = coords[i]
            p2 = coords[i + 1]

            # Estimate trajectory length (accounting for parabolic path)
            straight_distance = np.linalg.norm(p2 - p1)

            # Approximate parabolic path as 1.1-1.3 times straight
distance
            trajectory_factor = 1.2 # Conservative estimate
            estimated_distance = straight_distance * trajectory_factor

            total_distance += estimated_distance

        print(f"TRAJECTORY: Gap of {gap_size} frames, estimated distance
            {estimated_distance:.2f}m")
        else:
            # Large gap - use direct distance
            distance = np.linalg.norm(coords[i + 1] - coords[i])
            total_distance += distance

            print(f"LARGE GAP: {gap_size} frames, using direct distance
{distance:.2f}m")

            i += 1

    return {
        'total_distance': total_distance,
        'method': 'trajectory_estimation',
        'gaps_estimated': True
    }

def validate_ball_measurements(self, ball_results,
rally_duration_seconds=None):
    """
    Validate ball measurements against realistic tennis expectations

    Args:
        ball_results: Dictionary with ball movement statistics
        rally_duration_seconds: Duration of the rally in seconds (optional)

```

```

Returns:
    The original ball_results dictionary
    """
print("\n=== BALL MEASUREMENT VALIDATION ===")

total_distance = ball_results['total_distance']
avg_speed = ball_results.get('average_speed_kmh', 0)
max_speed = ball_results.get('max_speed_kmh', 0)

# Tennis ball physics validation
print(f"Total ball distance: {total_distance:.2f}m")

if rally_duration_seconds:
    rally_average_speed = (total_distance / rally_duration_seconds) * 3.6
    print(f"Rally average speed: {rally_average_speed:.1f} km/h")

    # Realistic ranges for tennis
    if 10 <= rally_average_speed <= 150:
        print("[OK] Rally average speed is realistic")
    else:
        print("[W] Rally average speed seems unrealistic")

if avg_speed > 0:
    print(f"Ball average speed: {avg_speed:.1f} km/h")
    if 20 <= avg_speed <= 200:
        print("[OK] Average speed is realistic")
    else:
        print("[W] Average speed seems unrealistic")

if max_speed > 0:
    print(f"Ball max speed: {max_speed:.1f} km/h")
    if 50 <= max_speed <= 250:
        print("[OK] Max speed is realistic")
    else:
        print("[W] Max speed seems unrealistic")

# Distance validation
if total_distance > 0:
    if 50 <= total_distance <= 2000: # Typical rally distances
        print("[OK] Total distance is realistic")
    else:
        print("[W] Total distance seems unrealistic")

return ball_results

def analyze_ball_trajectory(self, df, x_col='X_court', y_col='Y_court',
                           class_col='Class', frame_col='frame'):
    """
    Analyze ball trajectory patterns

    Args:
        df: DataFrame with tracking data
        x_col, y_col: Column names for court coordinates
        class_col: Column name for object classification

        frame_col: Column name for frame numbers

    Returns:
        Dictionary with trajectory analysis
        """
    ball_data = df[df[class_col] == 'Ball'].copy()

```

```

if ball_data.empty or len(ball_data) < 3:
    return {}

ball_data = ball_data.sort_values(frame_col).reset_index(drop=True)
coords = ball_data[[x_col, y_col]].values

# Calculate velocity vectors
velocities = np.diff(coords, axis=0)
speeds = np.linalg.norm(velocities, axis=1)

# Calculate acceleration vectors
accelerations = np.diff(velocities, axis=0)
acceleration_magnitudes = np.linalg.norm(accelerations, axis=1)

# Analyze direction changes
velocity_angles = np.arctan2(velocities[:, 1], velocities[:, 0])
angle_changes = np.diff(velocity_angles)

# Identify bounces (sudden direction changes)
angle_change_threshold = np.pi / 4 # 45 degrees
potential_bounces = np.where(np.abs(angle_changes) >
angle_change_threshold)[0]

# Calculate trajectory statistics
analysis = {
    'total_points': len(coords),
    'average_speed': float(np.mean(speeds)),
    'max_speed': float(np.max(speeds)),
    'speed_variability': float(np.std(speeds)),
    'average_acceleration': float(np.mean(acceleration_magnitudes)),
    'max_acceleration': float(np.max(acceleration_magnitudes)),
    'potential_bounces': len(potential_bounces),
    'trajectory_smoothness': float(1.0 / (1.0 + np.std(angle_changes))),
    'court_crossings': self._count_court_crossings(coords)
}

return analysis

def _count_court_crossings(self, coords):
    """
    Count how many times the ball crosses the court

    Args:
        coords: Array of ball coordinates

    Returns:
        Number of court crossings
    """
    if len(coords) < 2:
        return 0

    # Check crossings of the net (middle of the court)
    net_y = self.court_length / 2
    crossings = 0

    for i in range(len(coords) - 1):
        y1, y2 = coords[i][1], coords[i + 1][1]

        # Check if ball crossed the net line
        if (y1 < net_y < y2) or (y1 > net_y > y2):

```

```

        crossings += 1

    return crossings

```

[FILE] coordinate_mapper.py

Path: Distance_measurement\coordinate_mapper.py

```

"""
Coordinate Mapping Module for Tennis Court Tracking
"""

import cv2
import numpy as np
import pandas as pd

class CoordinateMapper:
    def __init__(self):
        # Tennis court dimensions in meters (doubles)
        self.court_width = 10.97
        self.court_length = 23.77
        self.real_corners = [
            (0, 0),
            (self.court_width, 0),
            (self.court_width, self.court_length),
            (0, self.court_length)
        ]

    def get_ordered_corners(self, corners):
        """Order corners in a consistent manner"""
        if len(corners) != 4:
            return corners
        corners = sorted(corners, key=lambda p: p[1])
        top_points = sorted(corners[:2], key=lambda p: p[0])
        bottom_points = sorted(corners[2:], key=lambda p: p[0])
        return [top_points[0], top_points[1], bottom_points[1], bottom_points[0]]

    def compute_homography(self, pixel_corners):
        """
        Compute homography matrix to map pixel coordinates to court coordinates

        Args:
            pixel_corners: List of 4 corner points in pixel coordinates

        Returns:
            Homography matrix H
        """
        if len(pixel_corners) != 4:
            raise ValueError("Exactly 4 corner points required")

        ordered_corners = self.get_ordered_corners(pixel_corners)
        pixel_pts = np.array(ordered_corners, dtype=np.float32)
        real_pts = np.array(self.real_corners, dtype=np.float32)

        H, status = cv2.findHomography(pixel_pts, real_pts, cv2.RANSAC, 5.0)

        if H is None:

```

```

        raise ValueError("Could not compute homography matrix")

    return H

def validate_homography(self, H, pixel_corners):
    """
    Validate the computed homography matrix

    Args:
        H: Homography matrix
        pixel_corners: Original pixel corners used to compute H

    Returns:
        bool: True if homography is valid
    """
    ordered_corners = self.get_ordered_corners(pixel_corners)
    pixel_pts = np.array(ordered_corners, dtype=np.float32)

    # Transform pixel corners to court coordinates
    pixel_pts_h = np.concatenate([pixel_pts, np.ones((4, 1))], axis=1)
    mapped_pts = (H @ pixel_pts_h.T).T
    mapped_pts = mapped_pts / mapped_pts[:, 2:3]

    # Compare with expected real corners
    real_pts = np.array(self.real_corners, dtype=np.float32)
    errors = np.linalg.norm(mapped_pts[:, :2] - real_pts, axis=1)

    print("Homography validation:")
    corner_names = ["Top-Left", "Top-Right", "Bottom-Right", "Bottom-Left"]
    for i, (name, pixel, real, mapped, error) in enumerate(zip(corner_names,
ordered_corners,
        self.real_corners, mapped_pts[:, :2], errors)):
    print(f"{name}: Pixel {pixel} -> Expected {real} -> Mapped {mapped} (Error:
        {error:.3f}m)")

    return np.all(errors < 1.0)

def map_pixels_to_court(self, df, H, x_col='X', y_col='Y'):
    """
    Map pixel coordinates to court coordinates using homography

    Args:
        df: DataFrame with pixel coordinates
        H: Homography matrix
        x_col: Column name for X pixel coordinates
        y_col: Column name for Y pixel coordinates

    Returns:
        DataFrame with added court coordinates
    """
    points = df[[x_col, y_col]].values.astype(np.float32)

    # Add homogeneous coordinate
    points_h = np.concatenate([points, np.ones((points.shape[0], 1),
dtype=np.float32)], axis=1)

    # Apply homography transformation
    mapped = (H @ points_h.T).T
    mapped = mapped / mapped[:, 2:3] # Normalize by homogeneous coordinate

```

```

# Create new DataFrame with court coordinates
df_mapped = df.copy()
df_mapped['X_court'] = mapped[:, 0]
df_mapped['Y_court'] = mapped[:, 1]

return df_mapped

def map_single_point(self, point, H):
    """
    Map a single pixel point to court coordinates

    Args:
        point: Tuple (x, y) in pixel coordinates
        H: Homography matrix

    Returns:
        Tuple (x_court, y_court) in court coordinates
    """
    point_h = np.array([point[0], point[1], 1.0], dtype=np.float32)
    mapped = H @ point_h
    mapped = mapped / mapped[2] # Normalize

    return (mapped[0], mapped[1])

def map_court_to_pixels(self, df, H, x_col='X_court', y_col='Y_court'):
    """
    Map court coordinates back to pixel coordinates (inverse transformation)

    Args:
        df: DataFrame with court coordinates
        H: Homography matrix
        x_col: Column name for X court coordinates
        y_col: Column name for Y court coordinates

    Returns:
        DataFrame with added pixel coordinates
    """
    # Compute inverse homography
    H_inv = np.linalg.inv(H)

    points = df[[x_col, y_col]].values.astype(np.float32)

    # Add homogeneous coordinate

    points_h = np.concatenate([points, np.ones((points.shape[0], 1),
dtype=np.float32)], axis=1)

    # Apply inverse homography transformation
    mapped = (H_inv @ points_h.T).T
    mapped = mapped / mapped[:, 2:3] # Normalize by homogeneous coordinate

    # Create new DataFrame with pixel coordinates
    df_mapped = df.copy()
    df_mapped['X_pixel'] = mapped[:, 0]
    df_mapped['Y_pixel'] = mapped[:, 1]

    return df_mapped

def debug_coordinates(self, df, x_col='X_court', y_col='Y_court',
sample_size=10):
    """
    Debug coordinate mapping by checking ranges and displaying samples

```

```

    Args:
        df: DataFrame with court coordinates
        x_col: Column name for X court coordinates
        y_col: Column name for Y court coordinates
        sample_size: Number of sample coordinates to display
    """
    print("Tennis Court Coordinate Statistics:")
    print(f"X_court range: {df[x_col].min():.3f} to {df[x_col].max():.3f}
meters")
    print(f"Y_court range: {df[y_col].min():.3f} to {df[y_col].max():.3f}
meters")
    print(f"Expected court dimensions: {self.court_width}m x
{self.court_length}m")
    print(f"\nSample of court coordinates:")
    print(df[['frame', x_col, y_col, 'Class']].head(sample_size))

    # Check for points outside court boundaries
    x_outside = df[(df[x_col] < -2) | (df[x_col] > self.court_width + 2)]
    y_outside = df[(df[y_col] < -2) | (df[y_col] > self.court_length + 2)]

    if not x_outside.empty:
        print(f"\nWARNING: {len(x_outside)} points outside court X
boundaries")
    if not y_outside.empty:
        print(f"WARNING: {len(y_outside)} points outside court Y boundaries")

    # Check for unrealistic coordinate ranges
    if df[x_col].max() - df[x_col].min() > 50:
        print("WARNING: X coordinate range seems too large")
    if df[y_col].max() - df[y_col].min() > 50:
        print("WARNING: Y coordinate range seems too large")

```

[FILE] corner_detection.py

Path: Distance_measurement\corner_detection.py

```

"""
Corner Detection Module for Tennis Court Tracking
"""
import cv2
import numpy as np
import urllib.request
import os
from sklearn.cluster import DBSCAN

class CornerDetector:
    def __init__(self):
        # Tennis court dimensions in meters (doubles)
        self.court_width = 10.97
        self.court_length = 23.77
        self.real_corners = [
            (0, 0),
            (self.court_width, 0),
            (self.court_width, self.court_length),
            (0, self.court_length)

```

```

]

def download_video(self, url, output_path="downloaded_video.mp4"):
    """Download video from URL if it's a web URL"""
    try:
        if url.startswith(('http://', 'https://')):
            print(f"[DEBUG] Downloading video from: {url}")
            urllib.request.urlretrieve(url, output_path)
            print(f"[DEBUG] Video downloaded to: {output_path}")
            return output_path
        else:
            # Local file path
            if os.path.exists(url):
                return url
            else:
                raise FileNotFoundError(f"Local video file not found: {url}")
    except Exception as e:
        print(f"[ERROR] Failed to download video: {e}")
        raise

def detect_corners_from_video(self, video_path, sample_interval=1.0,
max_frames=30,
    debug=False):
    """
    Detect court corners from multiple frames in a video

    Args:
        video_path: Path to video file or URL
        sample_interval: Interval in seconds between frame samples
        max_frames: Maximum number of frames to process
        debug: Whether to show debug information

    Returns:
        tuple: (average_corners, all_frame_corners, frame_info,
representative_frame)
    """
    # Download video if it's a URL
    if video_path.startswith(('http://', 'https://')):
        video_path = self.download_video(video_path)

    cap = cv2.VideoCapture(video_path)
    if not cap.isOpened():
        raise ValueError(f"Could not open video: {video_path}")

    # Get video properties
    fps = cap.get(cv2.CAP_PROP_FPS)
    total_frames = int(cap.get(cv2.CAP_PROP_FRAME_COUNT))
    duration = total_frames / fps

    print(f"[DEBUG] Video info: {total_frames} frames, {fps:.2f} FPS,
{duration:.2f}s duration")

    # Calculate frame sampling
    frame_interval = int(fps * sample_interval)
    frames_to_process = min(max_frames, int(duration / sample_interval))

    print(f"[DEBUG] Will process {frames_to_process} frames, sampling every
{frame_interval}
        frames ({sample_interval}s)")

    all_frame_corners = []

```



```

frame_info = []
processed_count = 0
representative_frame = None

for i in range(frames_to_process):
    frame_number = int(i * frame_interval) # Ensure it's a Python int
    timestamp = frame_number / fps

    # Seek to the specific frame
    cap.set(cv2.CAP_PROP_POS_FRAMES, frame_number)
    ret, frame = cap.read()

    if not ret:
        print(f"[WARNING] Could not read frame {frame_number}")
        continue

    print(f"[DEBUG] Processing frame {frame_number} at {timestamp:.2f}s")

    # Detect corners in this frame
    corners = self.detect_court_corners(frame, debug=False)

    frame_data = {
        'frame_number': frame_number,

        'timestamp': timestamp,
        'corners_found': len(corners),
        'corners': corners
    }

    if len(corners) == 4:
        all_frame_corners.append(corners)
        frame_data['valid'] = True
        processed_count += 1
        print(f"[DEBUG] Frame {frame_number}: Found 4 corners [OK]")

        # Select representative frame (middle of the sequence with good
corners)
        if processed_count == max(1, len(all_frame_corners) // 2):
            representative_frame = frame.copy()
            print(f"[DEBUG] Selected frame {frame_number} as
representative frame")

        else:
            frame_data['valid'] = False
            print(f"[DEBUG] Frame {frame_number}: Found {len(corners)}
corners [FAILED]")

    frame_info.append(frame_data)

    # Save debug image for first few frames if debug is enabled
    if debug and i < 5:
        debug_frame = frame.copy()
        if corners:
            self.draw_corners_on_frame(debug_frame, corners)
            output_path = f'debug_frame_{frame_number}.jpg'
            cv2.imwrite(output_path, debug_frame)
            print(f"[DEBUG] Saved debug frame to: {output_path}")

cap.release()

print(f"[DEBUG] Successfully processed
{processed_count}/{frames_to_process} frames")

```

```

        if processed_count == 0:
            print("[ERROR] No valid corners found in any frame")
            return None, all_frame_corners, frame_info, None

        # Calculate average corners
        average_corners = self._calculate_average_corners(all_frame_corners)

        # If no representative frame was selected, use the first valid frame
        if representative_frame is None and all_frame_corners:
            print("[DEBUG] No representative frame selected, using first valid
frame")
            # Get the first valid frame
            cap = cv2.VideoCapture(video_path)
            for info in frame_info:
                if info['valid']:
                    cap.set(cv2.CAP_PROP_POS_FRAMES, info['frame_number'])

                    ret, representative_frame = cap.read()
                    if ret:
                        break
            cap.release()

        # Clean up downloaded video if it was a URL
        if video_path.startswith('downloaded_video'):
            try:
                os.remove(video_path)
                print(f"[DEBUG] Cleaned up downloaded video: {video_path}")
            except:
                pass

        return average_corners, all_frame_corners, frame_info,
representative_frame

def detect_court_corners(self, frame, debug=False):
    """Original corner detection method for single frame"""
    if debug:
        print("[DEBUG] Converting to grayscale...")
        gray = cv2.cvtColor(frame, cv2.COLOR_BGR2GRAY)
        blurred = cv2.GaussianBlur(gray, (5, 5), 0)
        edges = cv2.Canny(blurred, 50, 150, apertureSize=3)
    lines = cv2.HoughLinesP(edges, 1, np.pi / 180, threshold=80, minLineLength=100,
maxLineGap=50)

    h, w = frame.shape[:2]
    if lines is None:
        return []

    sideline_segments = self._find_sideline_segments(lines, w, h)
    if len(sideline_segments) < 2:
        return []

    left_sideline, right_sideline = self._group_sideline_segments(sideline_segments,
w,
        frame.copy())
    if not left_sideline or not right_sideline:
        return []

    corners = self._find_corners_by_line_tracing(left_sideline, right_sideline,
frame.copy(),
        h, w)

```

```

        return corners

def get_ordered_corners(self, corners):
    """Order corners in a consistent manner"""
    if len(corners) != 4:
        return corners
    corners = sorted(corners, key=lambda p: p[1])
    top_points = sorted(corners[:2], key=lambda p: p[0])
    bottom_points = sorted(corners[2:], key=lambda p: p[0])

    return [top_points[0], top_points[1], bottom_points[1], bottom_points[0]]

def visualize_average_corners(self, frame, average_corners, output_path=
    "average_corners_visualization.jpg", show_image=True):
    """
    Visualize average corners on a representative frame
    """
    if frame is None:
        print("[ERROR] No frame provided for visualization")
        return

    if not average_corners or len(average_corners) != 4:
        print(f"[ERROR] Invalid average corners: expected 4, got {len(average_corners) if
            average_corners else 0}")
        return

    # Create a copy of the frame for visualization
    vis_frame = frame.copy()

    # Draw the average corners
    self.draw_corners_on_frame(vis_frame, average_corners)

    # Add additional information
    cv2.putText(vis_frame, 'AVERAGE CORNERS FROM MULTIPLE FRAMES', (10, 110),
        cv2.FONT_HERSHEY_SIMPLEX, 0.7, (0, 255, 255), 2)

    # Add corner coordinates as text
    ordered_corners = self.get_ordered_corners(average_corners)
    corner_labels = ['TL', 'TR', 'BR', 'BL']
    for i, (corner, label) in enumerate(zip(ordered_corners, corner_labels)):
        coord_text = f'{label}: ({corner[0]:.1f}, {corner[1]:.1f})'
        cv2.putText(vis_frame, coord_text, (10, 150 + i * 25),
            cv2.FONT_HERSHEY_SIMPLEX, 0.5, (255, 255, 255), 1)

    # Save the visualization
    try:
        cv2.imwrite(output_path, vis_frame)
        print(f"[DEBUG] Average corners visualization saved to:
{output_path}")
    except Exception as e:
        print(f"[ERROR] Failed to save visualization: {e}")

    # Display the image if requested
    if show_image:
        try:
            cv2.imshow('Average Tennis Court Corners', vis_frame)
            print("[DEBUG] Press any key to close the image window...")
            cv2.waitKey(0)
            cv2.destroyAllWindows()
        except Exception as e:
            print(f"[WARNING] Could not display image: {e}")

```

```

        print("Image saved to file instead.")

    def draw_corners_on_frame(self, frame, corners):
        """Draw detected corners on the frame with labels"""
        if len(corners) != 4:
            # If we don't have exactly 4 corners, just draw them as red circles
            for i, corner in enumerate(corners):
                x, y = int(corner[0]), int(corner[1])
                cv2.circle(frame, (x, y), 8, (0, 0, 255), -1) # Red filled circle
                cv2.putText(frame, f'C{i+1}', (x + 10, y - 10),
                            cv2.FONT_HERSHEY_SIMPLEX, 0.7, (0, 0, 255), 2)
        else:
            # Order corners and draw them with proper labels
            ordered_corners = self.get_ordered_corners(corners)
            corner_labels = ['TL', 'TR', 'BR', 'BL'] # Top-Left, Top-Right, Bottom-Right,
            Bottom-Left
            colors = [(255, 0, 0), (0, 255, 0), (0, 0, 255), (255, 255, 0)] # Blue, Green,
            Red,
            Cyan

            for i, (corner, label, color) in enumerate(zip(ordered_corners,
            corner_labels, colors)):
                x, y = int(corner[0]), int(corner[1])
                cv2.circle(frame, (x, y), 8, color, -1) # Filled circle
                cv2.circle(frame, (x, y), 12, color, 2) # Outer circle
                cv2.putText(frame, label, (x + 15, y - 10),
                            cv2.FONT_HERSHEY_SIMPLEX, 0.8, color, 2)

            # Draw lines connecting the corners to show the court outline
            for i in range(4):
                pt1 = (int(ordered_corners[i][0]), int(ordered_corners[i][1]))
                pt2 = (int(ordered_corners[(i+1)%4][0]),
            int(ordered_corners[(i+1)%4][1]))
                cv2.line(frame, pt1, pt2, (255, 255, 255), 2) # White lines

            # Add title
            cv2.putText(frame, 'Tennis Court Corners Detection', (10, 30),
                cv2.FONT_HERSHEY_SIMPLEX, 1, (255, 255, 255), 2)

            # Add corner count info
            cv2.putText(frame, f'Corners found: {len(corners)}/4', (10, 70),
                cv2.FONT_HERSHEY_SIMPLEX, 0.7, (255, 255, 255), 2)

    def _calculate_average_corners(self, all_frame_corners):
        """Calculate average corner positions from multiple frames"""
        if not all_frame_corners:
            return []

        print(f"[DEBUG] Calculating average corners from {len(all_frame_corners)}
        valid frames")

        # Group corners by their likely position (using simple clustering)
        corner_groups = [[] for _ in range(4)]

        for frame_corners in all_frame_corners:
            # Order corners consistently for this frame
            ordered_corners = self.get_ordered_corners(frame_corners)

            # Add to respective groups
            for i, corner in enumerate(ordered_corners):
                corner_groups[i].append(corner)

```

```

# Calculate average for each corner group
average_corners = []
corner_names = ["Top-Left", "Top-Right", "Bottom-Right", "Bottom-Left"]

for i, (group, name) in enumerate(zip(corner_groups, corner_names)):
    if group:
        # Convert to regular Python floats to avoid numpy compatibility
        x_coords = [float(corner[0]) for corner in group]
        y_coords = [float(corner[1]) for corner in group]

        # Use numpy for statistics instead of statistics module
        avg_x = np.mean(x_coords)
        avg_y = np.mean(y_coords)
        std_x = np.std(x_coords) if len(x_coords) > 1 else 0
        std_y = np.std(y_coords) if len(y_coords) > 1 else 0

        average_corners.append((avg_x, avg_y))
        print(f"[DEBUG] {name}: ({avg_x:.1f}, {avg_y:.1f}) +/-
({std_x:.1f}, {std_y:.1f})")

    return average_corners

def _detect_significant_corner_change(self, current_corners,
previous_corners, threshold=15.0):
    """
    Detect if corner positions have changed significantly
    """
    if not current_corners or not previous_corners or len(current_corners) != 4 or
        len(previous_corners) != 4:
        return True

    current_ordered = self.get_ordered_corners(current_corners)
    previous_ordered = self.get_ordered_corners(previous_corners)

    for curr, prev in zip(current_ordered, previous_ordered):
        distance = np.linalg.norm(np.array(curr) - np.array(prev))
        if distance > threshold:
            return True

    return False

def _find_sideline_segments(self, lines, img_width, img_height):
    """Find line segments that could be sidelines"""
    sideline_segments = []

    for line in lines:
        x1, y1, x2, y2 = line[0]
        dx = x2 - x1
        dy = y2 - y1
        length = np.sqrt(dx**2 + dy**2)
        if abs(dx) < 1e-6:
            angle = 90.0
        else:
            angle = abs(np.arctan(dy / dx) * 180 / np.pi)
        if abs(dx) < 1e-6:
            slope = float('inf')
        else:
            slope = dy / dx
        min_length = max(80, min(img_width, img_height) * 0.15)
        if length > min_length and 30 < angle < 85:

```

```

        center_x = (x1 + x2) / 2
        center_y = (y1 + y2) / 2
        sideline_segments.append({
            'line': (x1, y1, x2, y2),
            'length': length,
            'angle': angle,
            'slope': slope,
            'center_x': center_x,
            'center_y': center_y,
            'top_y': min(y1, y2),
            'bottom_y': max(y1, y2),
            'top_point': (x1, y1) if y1 < y2 else (x2, y2),
            'bottom_point': (x1, y1) if y1 > y2 else (x2, y2)
        })
    return sideline_segments

def _group_sideline_segments(self, segments, img_width, debug_img):
    """Group sideline segments into left and right sidelines"""
    if len(segments) < 2:
        return [], []
    x_coords = np.array([seg['center_x'] for seg in segments])
    clustering = DBSCAN(eps=img_width * 0.1, min_samples=1).fit(x_coords)
    labels = clustering.labels_
    clusters = {}
    for i, label in enumerate(labels):
        if label not in clusters:
            clusters[label] = []
        clusters[label].append(segments[i])
    cluster_info = []
    for label, cluster_segments in clusters.items():
        if len(cluster_segments) >= 1:
            avg_x = np.mean([seg['center_x'] for seg in cluster_segments])
            total_length = sum([seg['length'] for seg in cluster_segments])
            cluster_info.append((label, avg_x, total_length,
cluster_segments))
    cluster_info.sort(key=lambda x: x[2], reverse=True)

    if len(cluster_info) < 2:
        return [], []
    cluster1 = cluster_info[0]
    cluster2 = cluster_info[1]
    if cluster1[1] < cluster2[1]:
        left_sideline = cluster1[3]
        right_sideline = cluster2[3]
    else:
        left_sideline = cluster2[3]
        right_sideline = cluster1[3]
    return left_sideline, right_sideline

def _find_corners_by_line_tracing(self, left_sideline, right_sideline,
debug_img, img_height,
img_width):
    """Find corners by tracing sideline segments"""
    corners = []
    if left_sideline:
        left_bottom, left_top = self._find_corner_pair(left_sideline, img_height,
img_width,
is_left=True)
        if left_bottom and left_top:
            corners.extend([left_top, left_bottom])
    if right_sideline:
        right_bottom, right_top = self._find_corner_pair(right_sideline, img_height,

```

```

img_width,
        is_left=False)
    if right_bottom and right_top:
        corners.extend([right_top, right_bottom])
    return corners

def _find_corner_pair(self, sideline_segments, img_height, img_width,
is_left=True):
    """Find top and bottom corner pair for a sideline"""
    if not sideline_segments:
        return None, None
    bottom_point = None
    bottom_y = -1
    bottom_segment = None
    for seg in sideline_segments:
        x1, y1, x2, y2 = seg['line']
        for point in [(x1, y1), (x2, y2)]:
            if point[1] > bottom_y:
                bottom_y = point[1]
                bottom_point = point
                bottom_segment = seg
    if bottom_segment is None:
        return None, None
    collinear_segments = self._find_collinear_segments(bottom_segment,
sideline_segments)
    top_point = self._find_topmost_point_from_collinear_segments(collinear_seg
ments)
    return bottom_point, top_point

def _find_collinear_segments(self, reference_segment, all_segments):
    """Find segments that are collinear with the reference segment"""

    collinear_segments = [reference_segment]
    ref_x1, ref_y1, ref_x2, ref_y2 = reference_segment['line']
    a = ref_y2 - ref_y1
    b = ref_x1 - ref_x2
    c = (ref_x2 - ref_x1) * ref_y1 - (ref_y2 - ref_y1) * ref_x1
    norm = np.sqrt(a*a + b*b)
    if norm > 0:
        a, b, c = a/norm, b/norm, c/norm
    for seg in all_segments:
        if seg == reference_segment:
            continue
        x1, y1, x2, y2 = seg['line']
        dist1 = abs(a * x1 + b * y1 + c)
        dist2 = abs(a * x2 + b * y2 + c)
        tolerance = 10.0
        if dist1 < tolerance and dist2 < tolerance:
            collinear_segments.append(seg)
    return collinear_segments

def _find_topmost_point_from_collinear_segments(self, collinear_segments):
    """Find the topmost point from collinear segments"""
    if not collinear_segments:
        return None
    topmost_point = None
    topmost_y = float('inf')
    for seg in collinear_segments:
        x1, y1, x2, y2 = seg['line']
        for point in [(x1, y1), (x2, y2)]:
            if point[1] < topmost_y:
                topmost_y = point[1]

```

```

        topmost_point = point
    return topmost_point

```

[FILE] court_corners_average.py

Path: Distance_measurement\court_corners_average.py

File is empty or could not be read.

[FILE] distance_calculation.py

Path: Distance_measurement\distance_calculation.py

File is empty or could not be read.

[FILE] enhanced_tennis_court_tracker.py

Path: Distance_measurement\enhanced_tennis_court_tracker.py

```

"""
Enhanced Tennis Court Tracker with Improved Coordinate Mapping - FIXED VERSION
Copy this entire code to replace your enhanced_tennis_court_tracker.py
"""

import pandas as pd
import os
import json
import numpy as np

# You'll need to make sure these imports match your actual file structure
try:
    from .corner_detection import CornerDetector
    from .improved_coordinate_mapper import ImprovedCoordinateMapper
    from .player_distance_calculator import PlayerDistanceCalculator
    from .ball_distance_calculator import BallDistanceCalculator
    from .homography_manager import HomographyManager
except ImportError:
    # If relative imports fail, try absolute imports
    from corner_detection import CornerDetector
    from improved_coordinate_mapper import ImprovedCoordinateMapper
    from player_distance_calculator import PlayerDistanceCalculator
    from ball_distance_calculator import BallDistanceCalculator
    from homography_manager import HomographyManager

class EnhancedTennisCourtTracker:
    def __init__(self):
        # Initialize all modules with improved coordinate mapper

```



```

        self.corner_detector = CornerDetector()
        self.coordinate_mapper = ImprovedCoordinateMapper()
        self.player_calculator = PlayerDistanceCalculator()
        self.ball_calculator = BallDistanceCalculator()

    # Corner Detection Methods
    def detect_corners_from_video(self, video_path, sample_interval=1.0,
max_frames=30,
        debug=False):
        return self.corner_detector.detect_corners_from_video(video_path,
sample_interval,
            max_frames, debug)

    def detect_court_corners(self, frame, debug=False):
        return self.corner_detector.detect_court_corners(frame, debug)

    def visualize_average_corners(self, frame, average_corners, output_path=
        "average_corners_visualization.jpg", show_image=True):
        return self.corner_detector.visualize_average_corners(frame, average_corners,
output_path,
            show_image)

    # Enhanced Coordinate Mapping Methods
    def compute_homography(self, pixel_corners):
        try:
            H, ordered_corners = self.coordinate_mapper.compute_homography(pixel_c
orners)
            return H, ordered_corners
        except Exception as e:
            print(f"Enhanced homography computation failed: {e}")
            # Fallback to basic computation if needed
            try:
                H = self.coordinate_mapper.compute_homography_basic(pixel_corners)
                return H, None
            except:
                # If that also fails, use the original method
                return self.coordinate_mapper.compute_homography(pixel_corners)

    def validate_homography(self, H, pixel_corners):
        return self.coordinate_mapper.validate_homography(H, pixel_corners)

    def map_pixels_to_court(self, df, H, x_col='X', y_col='Y'):
        # Use the improved mapping method
        df_mapped = self.coordinate_mapper.map_pixels_to_court(df, H, x_col,
y_col)

        # Apply court constraints and add semantic information
        df_constrained = self.coordinate_mapper.apply_court_constraints(df_mapped)
        df_with_zones = self.coordinate_mapper.add_court_zones(df_constrained)
        df_final = self.coordinate_mapper.get_distance_to_lines(df_with_zones)

        return df_final

    def debug_coordinates(self, df, x_col='X_court', y_col='Y_court',
sample_size=10):
        return self.coordinate_mapper.debug_coordinates(df, sample_size)

    def diagnose_mapping_quality(self, df, pixel_corners):
        print("\n=== MAPPING QUALITY DIAGNOSIS ===")

        # Check coordinate ranges

```

```

x_coords = df['X_court'].dropna()
y_coords = df['Y_court'].dropna()

issues = []

# Range checks
if x_coords.max() - x_coords.min() > 50:
    issues.append("X coordinate range too large - possible homography
error")
if y_coords.max() - y_coords.min() > 50:
    issues.append("Y coordinate range too large - possible homography
error")

# Boundary checks
court_width = self.coordinate_mapper.court_width
court_length = self.coordinate_mapper.court_length

x_outliers = ((x_coords < -5) | (x_coords > court_width + 5)).sum()
y_outliers = ((y_coords < -5) | (y_coords > court_length + 5)).sum()

outlier_percentage = (x_outliers + y_outliers) / (len(x_coords) +
len(y_coords)) * 100

if outlier_percentage > 20:
    issues.append(f"High outlier percentage ({outlier_percentage:.1f}%) - check
corner
detection")

if issues:
    print("[WARNING] Issues detected:")
    for issue in issues:
        print(f" - {issue}")

    print("\n[SUGGESTION] Suggestions:")
    print(" 1. Verify corner detection accuracy")
    print(" 2. Check corner ordering (should be: BL, BR, TR, TL)")
    print(" 3. Consider using frame-by-frame corner detection")
    print(" 4. Apply coordinate constraints to filter outliers")
else:
    print("[SUCCESS] Mapping quality looks good!")

return issues

# Player Distance Methods
def calculate_player_distances_improved(self, df, id_col='ID', x_col='X_court',
y_col='Y_court',
class_col='Class', frame_col='frame',
fps=30.0):
    return self.player_calculator.calculate_player_distances_improved(df, id_col,
x_col, y_col,
class_col, frame_col, fps)

def analyze_player_movement_patterns(self, df, id_col='ID', x_col='X_court',
y_col='Y_court',
class_col='Class', frame_col='frame'):
    return self.player_calculator.analyze_player_movement_patterns(df, id_col,
x_col, y_col,
class_col, frame_col)

def validate_player_measurements(self, player_results,
rally_duration_seconds=None):

```

```

return self.player_calculator.validate_player_measurements(player_results,
    rally_duration_seconds)

    # Ball Distance Methods
    def calculate_ball_distance_improved(self, df, x_col='X_court',
y_col='Y_court',
                                class_col='Class', frame_col='frame',
fps=30.0):
    return self.ball_calculator.calculate_ball_distance_improved(df, x_col, y_col,
class_col,
        frame_col, fps)

def calculate_ball_distance_with_trajectory_estimation(self, df, x_col='X_court',
    y_col='Y_court',
class_col='Class', frame_col='frame',
                                fps=30.0):
return self.ball_calculator.calculate_ball_distance_with_trajectory_estimation(df,
x_col,
    y_col, class_col, frame_col, fps)

    def validate_ball_measurements(self, ball_results,
rally_duration_seconds=None):
    return self.ball_calculator.validate_ball_measurements(ball_results,
rally_duration_seconds)

    def analyze_ball_trajectory(self, df, x_col='X_court', y_col='Y_court',
                                class_col='Class', frame_col='frame'):
    return self.ball_calculator.analyze_ball_trajectory(df, x_col, y_col,
class_col, frame_col)

def enhanced_video_processing_pipeline(video_path, data_csv_path,
sample_interval=1.0,
    max_frames=30, use_corner_diagnostics=True):
    """
    Enhanced processing pipeline with improved coordinate mapping and diagnostics
    """
    tracker = EnhancedTennisCourtTracker()

    print("=== ENHANCED TENNIS COURT PROCESSING PIPELINE ===")
    print("=== STEP 1: Detecting Court Corners from Video ===")

    try:
        average_corners, all_frame_corners, frame_info, representative_frame =
            tracker.detect_corners_from_video(
                video_path, sample_interval=sample_interval, max_frames=max_frames,
debug=True
            )

        if average_corners is None:
            print("[ERROR] Error: Could not detect corners from video")
            return None, None, None, None, None, None

        print(f"[SUCCESS] Successfully calculated average corners from
{len(all_frame_corners)}
        frames")

        # Print corner analysis
        valid_frames = sum(1 for info in frame_info if info['valid'])
        print(f"[INFO] Frame processing: {valid_frames}/{len(frame_info)} frames
had valid corners")

```

```

# Visualize corners
if representative_frame is not None:
    print("=== STEP 1.5: Visualizing Average Corners ===")
    csv_dir = os.path.dirname(data_csv_path)
    csv_filename = os.path.basename(data_csv_path)
    image_filename = csv_filename.replace('.csv', '_enhanced_corners.jpg')
    image_output_path = os.path.join(csv_dir, image_filename)

    tracker.visualize_average_corners(
        representative_frame,
        average_corners,
        output_path=image_output_path,
        show_image=False
    )

    print(f"[INFO] Corner visualization saved to: {image_output_path}")

except Exception as e:
    print(f"[ERROR] Error processing video: {e}")
    return None, None, None, None, None, None

print("=== STEP 2: Computing Enhanced Homography ===")
try:
    # Use enhanced homography computation
    result = tracker.compute_homography(average_corners)
    if isinstance(result, tuple):
        H, ordered_corners = result
        print("[SUCCESS] Enhanced homography computed with corner ordering")
    else:
        H = result
        ordered_corners = None
        print("[SUCCESS] Basic homography computed")

    # Validate homography
    is_valid = tracker.validate_homography(H, average_corners)
    if not is_valid:
        print("[WARNING] WARNING: Homography validation failed - results may
be unrealistic")
    else:
        print("[SUCCESS] Homography validation passed")

except Exception as e:
    print(f"[ERROR] Error computing homography: {e}")
    return None, None, None, None, None, None

print("=== STEP 3: Loading and Processing Tracking Data ===")
df = pd.read_csv(data_csv_path)
print(f"[INFO] Loaded {len(df)} tracking points")

# Apply enhanced mapping
df_mapped = tracker.map_pixels_to_court(df, H)
print("[SUCCESS] Applied enhanced coordinate mapping with court constraints")

# Debug and diagnose
if use_corner_diagnostics:
    print("=== STEP 3.5: Diagnostic Analysis ===")
    tracker.debug_coordinates(df_mapped)
    mapping_issues = tracker.diagnose_mapping_quality(df_mapped,
average_corners)
else:
    mapping_issues = []

```

```

print("=== STEP 4: Calculating Enhanced Distance Metrics ===")

# Use appropriate coordinate columns based on what's available
x_col = 'X_court_clamped' if 'X_court_clamped' in df_mapped.columns else '
X_court'
y_col = 'Y_court_clamped' if 'Y_court_clamped' in df_mapped.columns else '
Y_court'

player_distances = tracker.calculate_player_distances_improved(
    df_mapped, x_col=x_col, y_col=y_col
)
print("[INFO] Player distances calculated:")
print(player_distances)

ball_results = tracker.calculate_ball_distance_improved(
    df_mapped, x_col=x_col, y_col=y_col, fps=30.0
)
print("[INFO] Ball trajectory analyzed:")
tracker.validate_ball_measurements(ball_results)

print("=== STEP 5: Saving Enhanced Results ===")

# Save main results
output_path = data_csv_path.replace('.csv', '_enhanced_court_coords.csv')
df_mapped.to_csv(output_path, index=False, encoding='utf-8')
print(f"[SAVE] Enhanced results saved to: {output_path}")

# Save diagnostics if enabled
if use_corner_diagnostics:
    diagnostics = {
        'mapping_issues': mapping_issues,
        'coordinate_ranges': {
            'x_min': float(df_mapped['X_court'].min()),
            'x_max': float(df_mapped['X_court'].max()),
            'y_min': float(df_mapped['Y_court'].min()),
            'y_max': float(df_mapped['Y_court'].max()),
        },
        'outlier_count': int(df_mapped['is_outlier'].sum()) if 'is_outlier' in
df_mapped.columns else 0,
        'total_points': len(df_mapped),
        'homography_valid': is_valid
    }

    diagnostics_path = data_csv_path.replace('.csv', '
_mapping_diagnostics.json')
    with open(diagnostics_path, 'w') as f:
        json.dump(diagnostics, f, indent=2)
    print(f"[SAVE] Diagnostics saved to: {diagnostics_path}")
else:
    diagnostics = None

# Save additional analysis files
corner_analysis_path = data_csv_path.replace('.csv', '_corner_analysis.csv')
corner_df = pd.DataFrame(frame_info)
corner_df.to_csv(corner_analysis_path, index=False, encoding='utf-8')

player_analysis_path = data_csv_path.replace('.csv', '_player_analysis.csv')
player_distances.to_csv(player_analysis_path, index=False, encoding='utf-8')

```

```

ball_analysis_path = data_csv_path.replace('.csv', '_ball_analysis.csv')
ball_df = pd.DataFrame([ball_results])
ball_df.to_csv(ball_analysis_path, index=False, encoding='utf-8')

print("[SUCCESS] All analysis files saved")
print("\n[COMPLETE] Enhanced Processing Complete!")
print(f" [INFO] Main results: {output_path}")
print(f" [INFO] Use columns: {x_col}, {y_col}")
print(f" [INFO] Court zones available: {'court_zone' in df_mapped.columns}")
print(f" [INFO] Outliers detected: {'is_outlier' in df_mapped.columns}")

return df_mapped, player_distances, ball_results, average_corners,
frame_info, diagnostics

def process_tennis_video(video_path, data_csv_path, method='enhanced', **kwargs):
    """
    Convenience function to process tennis video with the best available method
    """
    if method == 'dynamic':
        # For now, just use the enhanced method since dynamic has more complexity
        print("[INFO] Dynamic method not fully implemented, using enhanced
method")
        return enhanced_video_processing_pipeline(video_path, data_csv_path,
**kwargs)
    else:
        return enhanced_video_processing_pipeline(video_path, data_csv_path,
**kwargs)

```

[FILE] homography_manager.py

Path: Distance_measurement\homography_manager.py

```

"""
Homography Manager Module for Dynamic Tennis Court Tracking
"""

class HomographyManager:
    def __init__(self, tennis_tracker):
        """
        Initialize HomographyManager

        Args:
            tennis_tracker: Instance of TennisCourtTracker
        """
        self.tracker = tennis_tracker
        self.current_homography = None
        self.current_corners = None
        self.homography_history = []
        self.corner_change_threshold = 15.0

    def update_homography_if_needed(self, frame_corners, frame_number):
        """
        Update homography if corners have changed significantly

```

```

    Args:
        frame_corners: List of 4 corner points for current frame
        frame_number: Current frame number

    Returns:
        Current homography matrix (updated or existing)
    """
    if not frame_corners or len(frame_corners) != 4:
        return self.current_homography

    # Check if this is first frame or significant change
    if (self.current_corners is None or
self._detect_significant_corner_change(frame_corners, self.current_corners,
self.corner_change_threshold)):

        try:
            new_homography = self.tracker.compute_homography(frame_corners)
            is_valid = self.tracker.validate_homography(new_homography,
frame_corners)

            if is_valid:
                self.current_homography = new_homography
                self.current_corners = frame_corners
                self.homography_history.append({
                    'frame': frame_number,
                    'homography': new_homography,
                    'corners': frame_corners.copy()
                })

                print(f"[DEBUG] Updated homography at frame {frame_number}")
                return new_homography
            else:
                print(f"[WARNING] Invalid homography at frame {frame_number},
keeping previous")

        except Exception as e:
            print(f"[ERROR] Failed to compute homography at frame
{frame_number}: {e}")

            return self.current_homography

    def _detect_significant_corner_change(self, current_corners,
previous_corners, threshold=15.0):
    """
    Detect if corner positions have changed significantly

    Args:
        current_corners: Current frame corners
        previous_corners: Previous frame corners
        threshold: Pixel distance threshold for significant change

    Returns:
        bool: True if significant change detected
    """
    if not current_corners or not previous_corners or len(current_corners) != 4 or
len(previous_corners) != 4:
        return True

    current_ordered = self.tracker.get_ordered_corners(current_corners)
    previous_ordered = self.tracker.get_ordered_corners(previous_corners)

    import numpy as np

```

```

        for curr, prev in zip(current_ordered, previous_ordered):
            distance = np.linalg.norm(np.array(curr) - np.array(prev))
            if distance > threshold:
                return True

        return False

def get_homography_stats(self):
    """
    Get statistics about homography updates

    Returns:
        Dictionary with homography statistics
    """
    return {
        'total_updates': len(self.homography_history),
        'current_corners': self.current_corners,
        'threshold': self.corner_change_threshold,
        'update_frames': [h['frame'] for h in self.homography_history]
    }

def set_corner_change_threshold(self, threshold):
    """
    Set the threshold for detecting significant corner changes

    Args:
        threshold: New threshold value in pixels
    """
    self.corner_change_threshold = threshold
    print(f"[DEBUG] Corner change threshold set to {threshold} pixels")

```

[FILE] improved_coordinate_mapper.py

Path: Distance_measurement\improved_coordinate_mapper.py

```

"""
Improved Coordinate Mapping Module for Tennis Court Tracking
"""
import cv2
import numpy as np
import pandas as pd

class ImprovedCoordinateMapper:
    def __init__(self):
        # Tennis court dimensions in meters (doubles court)
        self.court_width = 10.97 # Side to side
        self.court_length = 23.77 # Baseline to baseline

        # Define court coordinate system with origin at bottom-left baseline
        # This matches tennis conventions where baselines are at y=0 and y=23.77
        self.real_corners = np.array([
            [0, 0], # Bottom-left (BL)
            [self.court_width, 0], # Bottom-right (BR)
            [self.court_width, self.court_length], # Top-right (TR)
            [0, self.court_length] # Top-left (TL)

```



```

], dtype=np.float32)

# Define key court lines for validation
self.court_lines = {
    'baseline_near': 0,
    'service_line_near': 6.40,
    'net': self.court_length / 2, # 11.885m
    'service_line_far': self.court_length - 6.40,
    'baseline_far': self.court_length,
    'singles_left': 1.37,
    'singles_right': self.court_width - 1.37,
    'center_line': self.court_width / 2
}

def order_corners_by_position(self, corners):
    """
    Order corners based on their position in the image
    Expected order: [Bottom-Left, Bottom-Right, Top-Right, Top-Left]
    """
    if len(corners) != 4:
        raise ValueError("Exactly 4 corners required")

    corners = np.array(corners)

    # Sort by y-coordinate (bottom to top in image coordinates)
    sorted_by_y = corners[np.argsort(corners[:, 1])]

    # Bottom two points (smaller y values in image)
    bottom_points = sorted_by_y[:2]

    bottom_left = bottom_points[np.argmin(bottom_points[:, 0])]
    bottom_right = bottom_points[np.argmax(bottom_points[:, 0])]

    # Top two points (larger y values in image)
    top_points = sorted_by_y[2:]
    top_left = top_points[np.argmin(top_points[:, 0])]
    top_right = top_points[np.argmax(top_points[:, 0])]

    return np.array([bottom_left, bottom_right, top_right, top_left],
dtype=np.float32)

def compute_homography(self, pixel_corners):
    """
    Compute homography matrix with improved corner ordering

    Args:
        pixel_corners: List of 4 corner points in pixel coordinates

    Returns:
        Homography matrix H
    """
    if len(pixel_corners) != 4:
        raise ValueError("Exactly 4 corner points required")

    # Order corners consistently
    ordered_pixel_corners = self.order_corners_by_position(pixel_corners)

    # Compute homography
    H, status = cv2.findHomography(
        ordered_pixel_corners,
        self.real_corners,
        cv2.RANSAC,

```

```

        5.0
    )

    if H is None:
        raise ValueError("Could not compute homography matrix")

    return H, ordered_pixel_corners

def validate_homography(self, H, pixel_corners):
    """
    Validate homography and provide detailed feedback
    """
    ordered_corners = self.order_corners_by_position(pixel_corners)

    # Transform pixel corners to court coordinates
    transformed = cv2.perspectiveTransform(
        ordered_corners.reshape(-1, 1, 2), H
    ).reshape(-1, 2)

    # Calculate errors
    errors = np.linalg.norm(transformed - self.real_corners, axis=1)

    print("=== Homography Validation ===")
    corner_names = ["Bottom-Left", "Bottom-Right", "Top-Right", "Top-Left"]

    for i, (name, pixel, expected, actual, error) in enumerate(
        zip(corner_names, ordered_corners, self.real_corners, transformed,
errors)
    ):
        print(f"{name}: Pixel {pixel} -> Expected {expected} -> Actual {actual} (Error:
            {error:.3f}m)")

    max_error = np.max(errors)
    avg_error = np.mean(errors)

    print(f"\nError Statistics:")
    print(f"Max error: {max_error:.3f}m")
    print(f"Average error: {avg_error:.3f}m")

    is_valid = max_error < 1.0 # Allow up to 1m error
    print(f"Validation: {'PASSED' if is_valid else 'FAILED'}")

    return is_valid

def map_pixels_to_court(self, df, H, x_col='X', y_col='Y'):
    """
    Map pixel coordinates to court coordinates
    """
    points = df[[x_col, y_col]].values.astype(np.float32)

    # Transform points using OpenCV (more robust)
    transformed = cv2.perspectiveTransform(
        points.reshape(-1, 1, 2), H
    ).reshape(-1, 2)

    # Create result DataFrame
    result_df = df.copy()
    result_df['X_court'] = transformed[:, 0]
    result_df['Y_court'] = transformed[:, 1]

    return result_df

```

```

def apply_court_constraints(self, df, buffer=2.0):
    """
    Apply realistic constraints to court coordinates

    Args:
        df: DataFrame with X_court and Y_court columns
        buffer: Buffer zone around court in meters

    Returns:
        DataFrame with constrained coordinates and outlier flags
    """
    result_df = df.copy()

    # Define valid ranges with buffer
    x_min, x_max = -buffer, self.court_width + buffer
    y_min, y_max = -buffer, self.court_length + buffer

    # Flag outliers
    result_df['is_outlier'] = (
        (result_df['X_court'] < x_min) |
        (result_df['X_court'] > x_max) |
        (result_df['Y_court'] < y_min) |
        (result_df['Y_court'] > y_max)
    )

    # Option 1: Clamp coordinates to valid range
    result_df['X_court_clamped'] = np.clip(
        result_df['X_court'], x_min, x_max
    )
    result_df['Y_court_clamped'] = np.clip(
        result_df['Y_court'], y_min, y_max
    )

    return result_df

def add_court_zones(self, df):
    """
    Add semantic court zone information
    """
    result_df = df.copy()

    # Determine court zones
    conditions = [
        (result_df['Y_court'] <= self.court_lines['service_line_near']),
        (result_df['Y_court'] <= self.court_lines['net']),
        (result_df['Y_court'] <= self.court_lines['service_line_far']),
        (result_df['Y_court'] <= self.court_lines['baseline_far'])
    ]

    choices = ['Near Service Box', 'Near Court', 'Far Court', 'Far Service
Box']

    result_df['court_zone'] = np.select(conditions, choices, default='Out of
Bounds')

    # Add side information
    result_df['court_side'] = np.where(
        result_df['X_court'] < self.court_lines['center_line'],
        'Left', 'Right'
    )

```

```

        return result_df

    def get_distance_to_lines(self, df):
        """
        Calculate distances to key court lines
        """
        result_df = df.copy()

        # Distance to baselines
        result_df['dist_to_near_baseline'] = np.abs(result_df['Y_court'] -
            self.court_lines['baseline_near'])
        result_df['dist_to_far_baseline'] = np.abs(result_df['Y_court'] -
            self.court_lines['baseline_far'])

        # Distance to net
        result_df['dist_to_net'] = np.abs(result_df['Y_court'] -
self.court_lines['net'])

        # Distance to sidelines
        result_df['dist_to_left_sideline'] = np.abs(result_df['X_court'] - 0)
        result_df['dist_to_right_sideline'] = np.abs(result_df['X_court'] -
self.court_width)

        # Distance to center line
        result_df['dist_to_center'] = np.abs(result_df['X_court'] -
self.court_lines['center_line'])

        return result_df

    def debug_coordinates(self, df, sample_size=10):
        """
        Enhanced debugging with court-specific analysis
        """
        print("=== Tennis Court Coordinate Analysis ===")
        print(f"Court dimensions: {self.court_width}m × {self.court_length}m")
        print(f"Total data points: {len(df)}")

        # Basic statistics
        print(f"\nCoordinate Ranges:")
        print(f"X_court: {df['X_court'].min():.2f} to {df['X_court'].max():.2f}m")
        print(f"Y_court: {df['Y_court'].min():.2f} to {df['Y_court'].max():.2f}m")

        # Check for outliers
        x_outliers = df[(df['X_court'] < -2) | (df['X_court'] > self.court_width
+ 2)]
        y_outliers = df[(df['Y_court'] < -2) | (df['Y_court'] > self.court_length
+ 2)]

        print(f"\nOutlier Analysis:")
        print(f"X outliers: {len(x_outliers)}
({len(x_outliers)/len(df)*100:.1f}%)")
        print(f"Y outliers: {len(y_outliers)}
({len(y_outliers)/len(df)*100:.1f}%)")

        # Sample coordinates by class
        if 'Class' in df.columns:

            print(f"\nSample coordinates by class:")
            for class_name in df['Class'].unique():
                class_data = df[df['Class'] == class_name]
                print(f"\n{class_name} ({len(class_data)} points):")

```

```

        sample = class_data[['frame', 'X_court', 'Y_court']].head(5)
        print(sample.to_string(index=False))

        # Check coordinate distribution
        print(f"\nCoordinate Distribution:")
        print(f"Points in near court (Y < {self.court_lines['net']:.1f}):
{len(df[df['Y_court']
        < self.court_lines['net']])}")
        print(f"Points in far court (Y > {self.court_lines['net']:.1f}):
{len(df[df['Y_court']
        > self.court_lines['net']])}")
        print(f"Points on left side (X < {self.court_lines['center_line']:.1f}):
        {len(df[df['X_court'] < self.court_lines['center_line']])}")
        print(f"Points on right side (X > {self.court_lines['center_line']:.1f}):
        {len(df[df['X_court'] > self.court_lines['center_line']])}")

    def create_court_visualization_data(self, df):
        """
        Create data for court visualization
        """
        # Court boundaries
        court_bounds = {
            'outer_bounds': [
                [0, 0], [self.court_width, 0],
                [self.court_width, self.court_length], [0, self.court_length],
                [0, 0]
            ],
            'service_boxes': [
                # Near service boxes
                [[0, 0], [self.court_width/2, 0], [self.court_width/2, self.court_lines[
                    'service_line_near']], [0,
self.court_lines['service_line_near']], [0, 0]],
                [[self.court_width/2, 0], [self.court_width, 0], [self.court_width,
self.court_lines['service_line_near']], [self.court_width/2,
self.court_lines['service_line_near']], [self.court_width/2,
0]],
                # Far service boxes
                [[0, self.court_lines['service_line_far']], [self.court_width/2,
self.court_lines['service_line_far']], [self.court_width/2, self.court_length],
                [0, self.court_length], [0,
self.court_lines['service_line_far']],
                [[self.court_width/2, self.court_lines['service_line_far']], [self.court_width,
self.court_lines['service_line_far']], [self.court_width, self.court_length],
                [self.court_width/2, self.court_length], [self.court_width/2,
self.court_lines['service_line_far']]
            ],
            'net_line': [[0, self.court_lines['net']], [self.court_width,
self.court_lines['net']]],
            'center_line': [[self.court_width/2, 0], [self.court_width/2,
self.court_length]]
        }

        return court_bounds

# Example usage function
def process_tennis_data(csv_file, pixel_corners):
    """
    Complete pipeline for processing tennis tracking data

    Args:
        csv_file: Path to CSV file with tracking data

```

```

        pixel_corners: List of 4 corner points in pixel coordinates
                        Order: [Bottom-Left, Bottom-Right, Top-Right, Top-Left]

Returns:
    Processed DataFrame with court coordinates
    """
    # Initialize mapper
    mapper = ImprovedCoordinateMapper()

    # Load data
    df = pd.read_csv(csv_file)

    # Compute homography
    H, ordered_corners = mapper.compute_homography(pixel_corners)

    # Validate homography
    if not mapper.validate_homography(H, pixel_corners):
        print("WARNING: Homography validation failed!")

    # Map coordinates
    df_mapped = mapper.map_pixels_to_court(df, H)

    # Apply constraints and add analysis
    df_constrained = mapper.apply_court_constraints(df_mapped)
    df_with_zones = mapper.add_court_zones(df_constrained)
    df_final = mapper.get_distance_to_lines(df_with_zones)

    # Debug analysis
    mapper.debug_coordinates(df_final)

    return df_final, H, mapper

```

[FILE] intelligent_court_mapper.py

Path: Distance_measurement\intelligent_court_mapper.py

```

"""
Intelligent Court Coordinate Mapper
This class remaps all coordinates (including negative and out-of-bounds) to
realistic court
positions
"""
import numpy as np
import pandas as pd
from scipy import interpolate
from scipy.spatial.distance import cdist
from sklearn.preprocessing import MinMaxScaler
from sklearn.cluster import KMeans
import cv2

class IntelligentCourtMapper:
    def __init__(self, court_width=10.97, court_length=23.77):
        self.court_width = court_width
        self.court_length = court_length

        # Define court zones for intelligent mapping

```

```

self.court_zones = {
    'baseline_near': (0, 6.40),
    'service_area_near': (6.40, 11.885),
    'service_area_far': (11.885, 17.37),
    'baseline_far': (17.37, 23.77)
}

# Expected coordinate distribution based on tennis gameplay
self.expected_distribution = {
    'baseline_zones': 0.4, # 40% of play near baselines
    'service_zones': 0.3, # 30% in service areas
    'net_zone': 0.2, # 20% near net
    'out_of_bounds': 0.1 # 10% slightly out of bounds
}

def analyze_coordinate_distribution(self, df):
    """
    Analyze the current coordinate distribution to understand mapping issues
    """
    print("=== COORDINATE DISTRIBUTION ANALYSIS ===")

    x_coords = df['X_court'].values
    y_coords = df['Y_court'].values

    # Basic statistics
    x_stats = {
        'min': np.min(x_coords),
        'max': np.max(x_coords),
        'mean': np.mean(x_coords),
        'std': np.std(x_coords),

        'range': np.max(x_coords) - np.min(x_coords)
    }

    y_stats = {
        'min': np.min(y_coords),
        'max': np.max(y_coords),
        'mean': np.mean(y_coords),
        'std': np.std(y_coords),
        'range': np.max(y_coords) - np.min(y_coords)
    }

    print(f"X coordinates: min={x_stats['min']:.2f},
max={x_stats['max']:.2f}, "
        f"mean={x_stats['mean']:.2f}, std={x_stats['std']:.2f}")
    print(f"Y coordinates: min={y_stats['min']:.2f},
max={y_stats['max']:.2f}, "
        f"mean={y_stats['mean']:.2f}, std={y_stats['std']:.2f}")

    # Identify coordinate clusters for intelligent remapping
    valid_mask = (
        (x_coords > -100) & (x_coords < 100) & # Remove extreme outliers
        (y_coords > -100) & (y_coords < 100)
    )

    if valid_mask.sum() > 10:
        coords = np.column_stack([x_coords[valid_mask], y_coords[valid_mask]])

        # Use clustering to identify main court area
        n_clusters = min(5, len(coords) // 20) # Adaptive number of clusters
        if n_clusters >= 2:
            kmeans = KMeans(n_clusters=n_clusters, random_state=42, n_init=10)

```

```

        clusters = kmeans.fit_predict(coords)

        print(f"\nIdentified {n_clusters} coordinate clusters:")
        for i in range(n_clusters):
            cluster_coords = coords[clusters == i]
            cluster_center = np.mean(cluster_coords, axis=0)
            cluster_size = len(cluster_coords)
        print(f" Cluster {i}: center=({cluster_center[0]:.2f},
            {cluster_center[1]:.2f}), "
            f"size={cluster_size} points")

    return x_stats, y_stats

def create_intelligent_mapping_function(self, df):
    """
    Create an intelligent mapping function that transforms coordinates to realistic
    court
        positions
    """
    print("\n=== CREATING INTELLIGENT MAPPING FUNCTION ===")

    x_coords = df['X_court'].values
    y_coords = df['Y_court'].values

    # Method 1: Linear rescaling based on data distribution
    def linear_rescale_mapping(x, y):
        # Find the main data cluster (remove extreme outliers)
        x_clean = x[(x > np.percentile(x, 5)) & (x < np.percentile(x, 95))]
        y_clean = y[(y > np.percentile(y, 5)) & (y < np.percentile(y, 95))]

        # Scale to court dimensions with some buffer
        x_scaler = MinMaxScaler(feature_range=(-2, self.court_width + 2))
        y_scaler = MinMaxScaler(feature_range=(-3, self.court_length + 3))

        x_scaled = x_scaler.fit_transform(x_clean.reshape(-1, 1)).flatten()
        y_scaled = y_scaler.fit_transform(y_clean.reshape(-1, 1)).flatten()

        return x_scaled, y_scaled

    # Method 2: Percentile-based mapping for robust scaling
    def percentile_mapping(x, y):
        # Use percentiles to handle outliers
        x_p5, x_p95 = np.percentile(x, [5, 95])
        y_p5, y_p95 = np.percentile(y, [5, 95])

        # Map percentile range to court dimensions
        x_mapped = np.interp(x, [x_p5, x_p95], [0, self.court_width])
        y_mapped = np.interp(y, [y_p5, y_p95], [0, self.court_length])

        return x_mapped, y_mapped

    # Method 3: Distribution-aware mapping
    def distribution_aware_mapping(x, y):
        # Analyze data distribution and map to expected tennis distribution

        # Find main playing area (central 80% of data)
        x_sorted = np.sort(x)
        y_sorted = np.sort(y)

        x_10, x_90 = np.percentile(x_sorted, [10, 90])
        y_10, y_90 = np.percentile(y_sorted, [10, 90])

```



```

# Map core playing area to court with realistic margins
x_core_mapped = np.interp(x, [x_10, x_90], [1, self.court_width - 1])
y_core_mapped = np.interp(y, [y_10, y_90], [2, self.court_length - 2])

# Handle outliers by extending mapping
x_mapped = np.where(
    x < x_10,
    np.interp(x, [np.min(x), x_10], [-3, 1]),
    np.where(
        x > x_90,
        np.interp(x, [x_90, np.max(x)], [self.court_width - 1,
self.court_width + 3]),
        x_core_mapped
    )
)

y_mapped = np.where(
    y < y_10,
    np.interp(y, [np.min(y), y_10], [-4, 2]),
    np.where(
        y > y_90,
        np.interp(y, [y_90, np.max(y)], [self.court_length - 2,
self.court_length + 4]),
        y_core_mapped
    )
)

return x_mapped, y_mapped

# Method 4: Physics-aware mapping (considers tennis movement patterns)
def physics_aware_mapping(x, y):
    # Group points by object/player for trajectory-aware mapping
    if 'ID' in df.columns:
        x_mapped = np.zeros_like(x)
        y_mapped = np.zeros_like(y)

        for obj_id in df['ID'].unique():
            obj_mask = df['ID'] == obj_id
            obj_x = x[obj_mask]
            obj_y = y[obj_mask]

            if len(obj_x) > 1:
                # For each object, maintain relative movement patterns
                # but scale to court dimensions

                # Calculate movement vectors
                dx = np.diff(obj_x)
                dy = np.diff(obj_y)

                # Scale movements to realistic court speeds
                # Typical tennis movement: 0.1-2.0 meters per frame
                movement_magnitudes = np.sqrt(dx**2 + dy**2)

                if len(movement_magnitudes) > 0:
                    # Scale extreme movements
                    reasonable_movement =
np.percentile(movement_magnitudes, 90)
                    if reasonable_movement > 5: # If movements are too
large
                        scale_factor = 2.0 / reasonable_movement

```

```

        dx *= scale_factor
        dy *= scale_factor

        # Reconstruct positions starting from court center

        start_x = self.court_width / 2
        start_y = self.court_length / 2

        obj_x_new = np.cumsum(np.concatenate([[start_x], dx]))
        obj_y_new = np.cumsum(np.concatenate([[start_y], dy]))

        x_mapped[obj_mask] = obj_x_new
        y_mapped[obj_mask] = obj_y_new
    else:
        # Single point - place in court center
        x_mapped[obj_mask] = self.court_width / 2
        y_mapped[obj_mask] = self.court_length / 2

    return x_mapped, y_mapped
else:
    # Fallback to distribution-aware mapping
    return distribution_aware_mapping(x, y)

# Test all methods and choose the best one
methods = {
    'linear_rescale': linear_rescale_mapping,
    'percentile': percentile_mapping,
    'distribution_aware': distribution_aware_mapping,
    'physics_aware': physics_aware_mapping
}

best_method = None
best_score = float('inf')

print("Testing mapping methods:")

for method_name, method_func in methods.items():
    try:
        x_test, y_test = method_func(x_coords, y_coords)

        # Score based on how well it fits court dimensions
        x_range = np.max(x_test) - np.min(x_test)
        y_range = np.max(y_test) - np.min(y_test)

        # Penalties for being too far from expected court dimensions
        x_penalty = abs(x_range - self.court_width) / self.court_width
        y_penalty = abs(y_range - self.court_length) / self.court_length

        # Penalty for too many extreme outliers
        x_outliers = np.sum((x_test < -5) | (x_test > self.court_width +
5))
        y_outliers = np.sum((y_test < -5) | (y_test > self.court_length +
5))
        outlier_penalty = (x_outliers + y_outliers) / len(x_test)

        total_score = x_penalty + y_penalty + outlier_penalty

        print(f" {method_name}: score={total_score:.3f}, "
              f"x_range={x_range:.2f}, y_range={y_range:.2f}, "
              f"outliers={outlier_penalty:.3f}")

```

```

        if total_score < best_score:
            best_score = total_score
            best_method = method_func
            best_method_name = method_name

    except Exception as e:
        print(f" {method_name}: failed ({e})")

    if best_method:
        print(f"\nSelected method: {best_method_name} (score:
{best_score:.3f})")

    return best_method, best_method_name

def apply_intelligent_mapping(self, df):
    """
    Apply intelligent coordinate mapping to the dataframe
    """
    print("\n=== APPLYING INTELLIGENT MAPPING ===")

    # Analyze current distribution
    x_stats, y_stats = self.analyze_coordinate_distribution(df)

    # Create mapping function
    mapping_func, method_name = self.create_intelligent_mapping_function(df)

    if mapping_func is None:
        print("[ERROR] Could not create mapping function")
        return df

    # Apply mapping
    x_mapped, y_mapped = mapping_func(df['X_court'].values,
df['Y_court'].values)

    # Create result dataframe
    df_mapped = df.copy()
    df_mapped['X_court_intelligent'] = x_mapped
    df_mapped['Y_court_intelligent'] = y_mapped

    # Add quality metrics
    df_mapped['in_court_bounds'] = (
        (x_mapped >= -3) & (x_mapped <= self.court_width + 3) &
        (y_mapped >= -4) & (y_mapped <= self.court_length + 4)
    )

    df_mapped['in_strict_bounds'] = (
        (x_mapped >= 0) & (x_mapped <= self.court_width) &
        (y_mapped >= 0) & (y_mapped <= self.court_length)
    )

    # Apply final smoothing and constraints
    x_final, y_final = self.apply_final_constraints(x_mapped, y_mapped,
df_mapped)
    df_mapped['X_court_final'] = x_final
    df_mapped['Y_court_final'] = y_final

    # Add court zones
    df_mapped = self.add_court_zone_information(df_mapped)

    # Print results
    print(f"\nMapping results using {method_name}:")

```

```

print(f"Original X range: {x_stats['min']:.2f} to {x_stats['max']:.2f}")
print(f"Mapped X range: {np.min(x_final):.2f} to {np.max(x_final):.2f}")
print(f"Original Y range: {y_stats['min']:.2f} to {y_stats['max']:.2f}")
print(f"Mapped Y range: {np.min(y_final):.2f} to {np.max(y_final):.2f}")

in_bounds = df_mapped['in_court_bounds'].sum()
strictly_in_bounds = df_mapped['in_strict_bounds'].sum()
print(f"Points in court bounds: {in_bounds}/{len(df_mapped)} ({in_bounds/len(df_mapped)*100:.1f}%)")
print(f"Points strictly in court: {strictly_in_bounds}/{len(df_mapped)} ({strictly_in_bounds/len(df_mapped)*100:.1f}%)")

return df_mapped

def apply_final_constraints(self, x_mapped, y_mapped, df_mapped):
    """
    Apply final smoothing and physical constraints
    """
    x_final = x_mapped.copy()
    y_final = y_mapped.copy()

    # Smooth trajectories for each object
    if 'ID' in df_mapped.columns:
        for obj_id in df_mapped['ID'].unique():
            obj_mask = df_mapped['ID'] == obj_id
            obj_indices = np.where(obj_mask)[0]

            if len(obj_indices) > 3: # Need at least 4 points for smoothing
                obj_x = x_final[obj_mask]
                obj_y = y_final[obj_mask]
                obj_frames = df_mapped.loc[obj_mask, 'frame'].values

                # Sort by frame
                sort_idx = np.argsort(obj_frames)
                obj_x_sorted = obj_x[sort_idx]
                obj_y_sorted = obj_y[sort_idx]
                obj_frames_sorted = obj_frames[sort_idx]

                # Apply light smoothing (reduce jitter while preserving
                movement)
                try:
                    # Use interpolation to smooth the trajectory
                    f_x = interpolate.UnivariateSpline(obj_frames_sorted,
obj_x_sorted, s=0.5)
                    f_y = interpolate.UnivariateSpline(obj_frames_sorted,
obj_y_sorted, s=0.5)

                    obj_x_smooth = f_x(obj_frames_sorted)
                    obj_y_smooth = f_y(obj_frames_sorted)

                    # Update final coordinates
                    x_final[obj_indices[sort_idx]] = obj_x_smooth
                    y_final[obj_indices[sort_idx]] = obj_y_smooth

                except Exception:
                    # If smoothing fails, use simple moving average
                    window = min(3, len(obj_x_sorted) // 2)
                    if window >= 1:
obj_x_smooth = np.convolve(obj_x_sorted, np.ones(window)/window,
mode='same')
obj_y_smooth = np.convolve(obj_y_sorted, np.ones(window)/window,

```

```

        mode='same')

        x_final[obj_indices[sort_idx]] = obj_x_smooth
        y_final[obj_indices[sort_idx]] = obj_y_smooth

    # Apply soft constraints (allow some out-of-bounds but penalize extreme
values)
    buffer_soft = 4.0
    x_final = np.clip(x_final, -buffer_soft, self.court_width + buffer_soft)
    y_final = np.clip(y_final, -buffer_soft, self.court_length + buffer_soft)

    return x_final, y_final

def add_court_zone_information(self, df_mapped):
    """
    Add semantic court zone information
    """
    df_result = df_mapped.copy()

    # Determine court zones based on Y coordinate
    y_coords = df_result['Y_court_final']

    conditions = [
        y_coords < 6.40, # Near baseline
        (y_coords >= 6.40) & (y_coords < 11.885), # Near service area
        (y_coords >= 11.885) & (y_coords < 17.37), # Far service area
        (y_coords >= 17.37) & (y_coords <= 23.77), # Far baseline
    ]

    choices = ['Near Baseline', 'Near Service', 'Far Service', 'Far Baseline']

    df_result['court_zone'] = np.select(conditions, choices, default='Out of
Bounds')

    # Add side information
    x_coords = df_result['X_court_final']
    df_result['court_side'] = np.where(x_coords < self.court_width/2, 'Left', '
Right')

    return df_result

def calculate_realistic_distances(self, df_mapped, fps=25.0):
    """
    Calculate distances using the intelligently mapped coordinates
    """
    print("\n=== CALCULATING REALISTIC DISTANCES ===")

    results = []

    for obj_id in df_mapped['ID'].unique():
        obj_data = df_mapped[df_mapped['ID'] == obj_id].sort_values('frame')

        if len(obj_data) < 2:
            continue

        # Use final mapped coordinates
        positions = obj_data[['X_court_final', 'Y_court_final']].values

        # Calculate frame-to-frame distances
        distances = np.sqrt(np.sum(np.diff(positions, axis=0)**2, axis=1))

```

```

# Calculate time intervals
frame_intervals = np.diff(obj_data['frame'].values) / fps

# Calculate speeds
speeds_ms = distances / frame_intervals
speeds_kmh = speeds_ms * 3.6

# Apply realistic speed limits
obj_class = obj_data['Class'].iloc[0] if 'Class' in obj_data.columns
else 'Unknown'
if obj_class == 'Ball':
    max_speed = 200 # km/h
    typical_speed = 80
else:
    max_speed = 45 # km/h for players
    typical_speed = 15

# Filter unrealistic speeds but be more lenient
realistic_mask = speeds_kmh <= max_speed
realistic_distances = distances[realistic_mask]
realistic_speeds = speeds_kmh[realistic_mask]

# Calculate statistics

total_distance = realistic_distances.sum()
avg_speed = realistic_speeds.mean() if len(realistic_speeds) > 0 else
0
max_speed_actual = realistic_speeds.max() if len(realistic_speeds) >
0 else 0

# Time span and coverage
time_span = (obj_data['frame'].max() - obj_data['frame'].min()) / fps

x_coverage = obj_data['X_court_final'].max() -
obj_data['X_court_final'].min()
y_coverage = obj_data['Y_court_final'].max() -
obj_data['Y_court_final'].min()
court_coverage = x_coverage * y_coverage

# Quality metrics
in_bounds_ratio = obj_data['in_court_bounds'].mean()

results.append({
    'ID': obj_id,
    'Class': obj_class,
    'TotalDistance': total_distance,
    'AverageSpeed': avg_speed,
    'MaxSpeed': max_speed_actual,
    'TimeSpan': time_span,
    'CourtCoverage': court_coverage,
    'InBoundsRatio': in_bounds_ratio,
    'DataPoints': len(obj_data),
    'ValidMovements': len(realistic_distances)
})

return pd.DataFrame(results)

def intelligent_court_mapping_pipeline(csv_path, fps=25.0):
    """
    Complete pipeline for intelligent court coordinate mapping
    """

```

```

print("=== INTELLIGENT COURT MAPPING PIPELINE ===")

# Load data
df = pd.read_csv(csv_path)
print(f"Loaded {len(df)} tracking points")

# Initialize intelligent mapper
mapper = IntelligentCourtMapper()

# Apply intelligent mapping
df_mapped = mapper.apply_intelligent_mapping(df)

# Calculate realistic distances
distance_results = mapper.calculate_realistic_distances(df_mapped, fps)

# Display results

print("\n=== INTELLIGENT MAPPING RESULTS ===")
active_objects = distance_results[distance_results['TotalDistance'] > 0.5]

if len(active_objects) > 0:
    print("Active objects with significant movement:")
    for _, obj in active_objects.iterrows():
        print(f"\n{obj['Class']} {obj['ID']}:")
        print(f" Total distance: {obj['TotalDistance']:.2f}m")
        print(f" Average speed: {obj['AverageSpeed']:.1f} km/h")
        print(f" Max speed: {obj['MaxSpeed']:.1f} km/h")
        print(f" Time span: {obj['TimeSpan']:.1f}s")
        print(f" Court coverage: {obj['CourtCoverage']:.2f}m²")
        print(f" In bounds: {obj['InBoundsRatio']*100:.1f}%")

# Save results
output_path = csv_path.replace('.csv', '_intelligent_mapped.csv')
df_mapped.to_csv(output_path, index=False)

results_path = csv_path.replace('.csv', '_intelligent_distances.csv')
distance_results.to_csv(results_path, index=False)

print(f"\n[SAVE] Mapped data saved to: {output_path}")
print(f"[SAVE] Distance results saved to: {results_path}")

return df_mapped, distance_results

# Usage example
if __name__ == "__main__":
    csv_path = "./Out/match_point_init_df_with_video_court_coords.csv"
    df_mapped, results = intelligent_court_mapping_pipeline(csv_path, fps=25.12)

```

[FILE] main_court_tracker.py

Path: Distance_measurement\main_court_tracker.py

```

"""
Main Tennis Court Tracker Module - Refactored
"""
import pandas as pd

```

```

import os
import json
from .corner_detection import CornerDetector
from .coordinate_mapper import CoordinateMapper
from .player_distance_calculator import PlayerDistanceCalculator
from .ball_distance_calculator import BallDistanceCalculator
from .homography_manager import HomographyManager

class TennisCourtTracker:
    def __init__(self):
        # Initialize all modules
        self.corner_detector = CornerDetector()
        self.coordinate_mapper = CoordinateMapper()
        self.player_calculator = PlayerDistanceCalculator()
        self.ball_calculator = BallDistanceCalculator()

        # Corner Detection Methods (delegated to CornerDetector)
    def detect_corners_from_video(self, video_path, sample_interval=1.0,
max_frames=30,
        debug=False):
        """Detect court corners from multiple frames in a video"""
        return self.corner_detector.detect_corners_from_video(video_path,
sample_interval,
            max_frames, debug)

    def detect_court_corners(self, frame, debug=False):
        """Detect court corners from a single frame"""
        return self.corner_detector.detect_court_corners(frame, debug)

    def visualize_average_corners(self, frame, average_corners, output_path=
        "average_corners_visualization.jpg", show_image=True):
        """Visualize average corners on a representative frame"""
        return self.corner_detector.visualize_average_corners(frame, average_corners,
output_path,
            show_image)

    def draw_corners_on_frame(self, frame, corners):
        """Draw detected corners on frame"""
        return self.corner_detector.draw_corners_on_frame(frame, corners)

    def get_ordered_corners(self, corners):
        """Order corners in a consistent manner"""
        return self.corner_detector.get_ordered_corners(corners)

        # Coordinate Mapping Methods (delegated to CoordinateMapper)
    def compute_homography(self, pixel_corners):
        """Compute homography matrix to map pixel coordinates to court
coordinates"""
        return self.coordinate_mapper.compute_homography(pixel_corners)

    def validate_homography(self, H, pixel_corners):
        """Validate the computed homography matrix"""
        return self.coordinate_mapper.validate_homography(H, pixel_corners)

    def map_pixels_to_court(self, df, H, x_col='X', y_col='Y'):
        """Map pixel coordinates to court coordinates using homography"""
        return self.coordinate_mapper.map_pixels_to_court(df, H, x_col, y_col)

    def debug_coordinates(self, df, x_col='X_court', y_col='Y_court',
sample_size=10):

```



```

        """Debug coordinate mapping"""
        return self.coordinate_mapper.debug_coordinates(df, x_col, y_col,
sample_size)

    # Player Distance Methods (delegated to PlayerDistanceCalculator)
    def calculate_player_distances_improved(self, df, id_col='ID', x_col='X_court',
y_col='Y_court',
class_col='Class', frame_col='frame',
fps=30.0):
        """Calculate improved player distances with temporal awareness"""
        return self.player_calculator.calculate_player_distances_improved(df, id_col,
x_col, y_col,
class_col, frame_col, fps)

    def analyze_player_movement_patterns(self, df, id_col='ID', x_col='X_court',
y_col='Y_court',
class_col='Class', frame_col='frame'):
        """Analyze detailed movement patterns for each player"""
        return self.player_calculator.analyze_player_movement_patterns(df, id_col,
x_col, y_col,
class_col, frame_col)

    def validate_player_measurements(self, player_results,
rally_duration_seconds=None):
        """Validate player measurements against realistic tennis movement
patterns"""
        return self.player_calculator.validate_player_measurements(player_results,
rally_duration_seconds)

    # Ball Distance Methods (delegated to BallDistanceCalculator)
    def calculate_ball_distance_improved(self, df, x_col='X_court',
y_col='Y_court',
class_col='Class', frame_col='frame',
fps=30.0):
        """Calculate improved ball distance with temporal awareness"""
        return self.ball_calculator.calculate_ball_distance_improved(df, x_col, y_col,
class_col,
frame_col, fps)

    def calculate_ball_distance_with_trajectory_estimation(self, df, x_col='X_court',
y_col='Y_court',
class_col='Class', frame_col='frame',
fps=30.0):
        """Calculate ball distance with parabolic trajectory estimation"""
        return self.ball_calculator.calculate_ball_distance_with_trajectory_estimation(df,
x_col,
y_col, class_col, frame_col, fps)

    def validate_ball_measurements(self, ball_results,
rally_duration_seconds=None):
        """Validate ball measurements against realistic tennis expectations"""
        return self.ball_calculator.validate_ball_measurements(ball_results,
rally_duration_seconds)

    def analyze_ball_trajectory(self, df, x_col='X_court', y_col='Y_court',
class_col='Class', frame_col='frame'):
        """Analyze ball trajectory patterns"""
        return self.ball_calculator.analyze_ball_trajectory(df, x_col, y_col,
class_col, frame_col)

    # Legacy method for backward compatibility

```

```

def _draw_corners_on_frame(self, frame, corners):
    """Legacy method - redirects to draw_corners_on_frame"""
    return self.draw_corners_on_frame(frame, corners)

def main_video_processing_pipeline(video_path, data_csv_path,
sample_interval=1.0, max_frames=30):
    """
    Main processing pipeline for video-based corner detection

    Args:
        video_path: Path to video file or URL
        data_csv_path: Path to CSV file with tracking data
        sample_interval: Interval in seconds between frame samples
        max_frames: Maximum number of frames to process

    Returns:
        Tuple of (df_mapped, player_distances, ball_results, average_corners,
frame_info)
    """
    tracker = TennisCourtTracker()

    print("=== STEP 1: Detecting Court Corners from Video ===")
    try:
        average_corners, all_frame_corners, frame_info, representative_frame =
            tracker.detect_corners_from_video(
                video_path, sample_interval=sample_interval, max_frames=max_frames,
debug=True
            )

        if average_corners is None:
            print("Error: Could not detect corners from video")
            return None, None, None, None, None

        print(f"Successfully calculated average corners from
{len(all_frame_corners)} frames")

        # Print summary statistics
        valid_frames = sum(1 for info in frame_info if info['valid'])
        print(f"Frame processing summary: {valid_frames}/{len(frame_info)} frames had
valid
            corners")

        # Optional: Visualize the average corners on the representative frame
        if representative_frame is not None:
            print("=== STEP 1.5: Visualizing Average Corners ===")
            # Extract directory and filename from data_csv_path to match the Out
folder structure
            csv_dir = os.path.dirname(data_csv_path)

            csv_filename = os.path.basename(data_csv_path)
            image_filename = csv_filename.replace('.csv', '_average_corners.jpg')
            image_output_path = os.path.join(csv_dir, image_filename)

            tracker.visualize_average_corners(
                representative_frame,
                average_corners,
                output_path=image_output_path,
                show_image=False
            )

    except Exception as e:

```

```

        print(f"Error processing video: {e}")
        return None, None, None, None, None

print("=== STEP 2: Computing Homography with Average Corners ===")
try:
    H = tracker.compute_homography(average_corners)
    print("Homography matrix computed successfully using average corners")
    is_valid = tracker.validate_homography(H, average_corners)
    if not is_valid:
        print("WARNING: Homography validation failed")
except Exception as e:
    print(f"Error computing homography: {e}")
    return None, None, None, None, None

print("=== STEP 3: Loading and Processing Tracking Data ===")
df = pd.read_csv(data_csv_path)
print(f"Loaded {len(df)} tracking points")
df_mapped = tracker.map_pixels_to_court(df, H)
tracker.debug_coordinates(df_mapped)

print("=== STEP 4: Calculating Distances ===")
player_distances = tracker.calculate_player_distances_improved(df_mapped)
print("Player distances:")
print(player_distances)

ball_results = tracker.calculate_ball_distance_improved(df_mapped, fps=30.0)
tracker.validate_ball_measurements(ball_results)

# Save results
output_path = data_csv_path.replace('.csv', '_with_video_court_coords.csv')
df_mapped.to_csv(output_path, index=False, encoding='utf-8')
print(f"\nResults saved to: {output_path}")

# Save corner analysis results
corner_analysis_path = data_csv_path.replace('.csv', '_corner_analysis.csv')
corner_df = pd.DataFrame(frame_info)
corner_df.to_csv(corner_analysis_path, index=False, encoding='utf-8')
print(f"Corner analysis saved to: {corner_analysis_path}")

# Save player movement analysis
player_analysis_path = data_csv_path.replace('.csv', '_player_movement_analysis.csv')
player_distances.to_csv(player_analysis_path, index=False, encoding='utf-8')
print(f"Player movement analysis saved to: {player_analysis_path}")

# Save ball movement analysis
ball_analysis_path = data_csv_path.replace('.csv', '_ball_movement_analysis.csv')
ball_df = pd.DataFrame([ball_results]) # Convert dict to DataFrame
ball_df.to_csv(ball_analysis_path, index=False, encoding='utf-8')
print(f"Ball movement analysis saved to: {ball_analysis_path}")

# Save detailed movement patterns (optional)
movement_patterns = tracker.analyze_player_movement_patterns(df_mapped)
if movement_patterns:
    patterns_path = data_csv_path.replace('.csv', '_movement_patterns.json')
    with open(patterns_path, 'w') as f:
        json.dump(movement_patterns, f, indent=2)
    print(f"Movement patterns saved to: {patterns_path}")

return df_mapped, player_distances, ball_results, average_corners, frame_info

```

```

def main_video_processing_pipeline_dynamic(video_path, data_csv_path,
sample_interval=1.0,
max_frames=30):
    """
    Main processing pipeline with dynamic homography updates

    Args:
        video_path: Path to video file or URL
        data_csv_path: Path to CSV file with tracking data
        sample_interval: Interval in seconds between frame samples
        max_frames: Maximum number of frames to process

    Returns:
        Tuple of (df_mapped, player_distances, ball_results, average_corners,
frame_info)
    """
    tracker = TennisCourtTracker()

    print("=== STEP 1: Processing Video with Dynamic Homography ===")
    try:
        # Get frame analysis (keep existing corner detection)
        average_corners, all_frame_corners, frame_info, representative_frame =
            tracker.detect_corners_from_video(
                video_path, sample_interval=sample_interval, max_frames=max_frames,
debug=True
            )

        if average_corners is None:
            print("Error: Could not detect corners from video")
            return None, None, None, None, None

    except Exception as e:
        print(f"Error processing video: {e}")
        return None, None, None, None, None

    print("=== STEP 2: Setting up Dynamic Homography Processing ===")
    homography_manager = HomographyManager(tracker)

    # Initialize with first valid frame
    initial_homography = None
    for info in frame_info:
        if info['valid']:
            initial_homography = homography_manager.update_homography_if_needed(
                info['corners'], info['frame_number']
            )
            break

    if initial_homography is None:
        print("Error: Could not initialize homography")
        return None, None, None, None, None

    print("=== STEP 3: Loading and Processing Tracking Data with Dynamic
Homography ===")
    df = pd.read_csv(data_csv_path)
    print(f"Loaded {len(df)} tracking points")

    # Process data with dynamic homography updates
    df_mapped = df.copy()
    df_mapped['X_court'] = 0.0 # Initialize court coordinates

```

```

df_mapped['Y_court'] = 0.0
current_homography = initial_homography

# Group by frame and apply appropriate homography
unique_frames = sorted(df['frame'].unique())
homography_updates = 0

for frame_num in unique_frames:
    frame_data = df[df['frame'] == frame_num]

    # Check if we have corner data for this frame
    frame_corners = None
    for info in frame_info:
        if info['frame_number'] == frame_num and info['valid']:
            frame_corners = info['corners']
            break

    # Update homography if needed
    if frame_corners is not None:
        updated_homography = homography_manager.update_homography_if_needed(
            frame_corners, frame_num
        )

        if updated_homography is not current_homography:
            current_homography = updated_homography
            homography_updates += 1

    # Apply current homography to this frame's data
    if current_homography is not None:
        frame_mapped = tracker.map_pixels_to_court(frame_data,
current_homography)
        df_mapped.loc[df['frame'] == frame_num, ['X_court', 'Y_court']] =
            frame_mapped[['X_court', 'Y_court']]

    print(f"[DEBUG] Applied {homography_updates} homography updates across
{len(unique_frames)}
frames")
    tracker.debug_coordinates(df_mapped)

    print("=== STEP 4: Calculating Distances with Dynamic Mapping ===")
    player_distances = tracker.calculate_player_distances_improved(df_mapped)
    print("Player distances:")
    print(player_distances)

    ball_results = tracker.calculate_ball_distance_improved(df_mapped, fps=30.0)
    tracker.validate_ball_measurements(ball_results)

    # Save results
    output_path = data_csv_path.replace('.csv', '_with_dynamic_court_coords.csv')
    df_mapped.to_csv(output_path, index=False, encoding='utf-8')
    print(f"\nResults saved to: {output_path}")

    # Save corner analysis results
    corner_analysis_path = data_csv_path.replace('.csv', '_corner_analysis.csv')
    corner_df = pd.DataFrame(frame_info)
    corner_df.to_csv(corner_analysis_path, index=False, encoding='utf-8')
    print(f"Corner analysis saved to: {corner_analysis_path}")

    # Save player movement analysis
    player_analysis_path = data_csv_path.replace('.csv', '
_player_movement_analysis.csv')
    player_distances.to_csv(player_analysis_path, index=False, encoding='utf-8')

```

```

print(f"Player movement analysis saved to: {player_analysis_path}")

# Save ball movement analysis
ball_analysis_path = data_csv_path.replace('.csv', '
_ball_movement_analysis.csv')
ball_df = pd.DataFrame([ball_results]) # Convert dict to DataFrame
ball_df.to_csv(ball_analysis_path, index=False, encoding='utf-8')
print(f"Ball movement analysis saved to: {ball_analysis_path}")

# Save detailed movement patterns (optional)
movement_patterns = tracker.analyze_player_movement_patterns(df_mapped)
if movement_patterns:
    patterns_path = data_csv_path.replace('.csv', '_movement_patterns.json')
    with open(patterns_path, 'w') as f:
        json.dump(movement_patterns, f, indent=2)

    print(f"Movement patterns saved to: {patterns_path}")

# Save homography statistics
homography_stats = homography_manager.get_homography_stats()
homography_stats_path = data_csv_path.replace('.csv', '
_homography_stats.json')
with open(homography_stats_path, 'w') as f:
    # Convert numpy arrays and types to JSON serializable format
    stats_serializable = {}
    for key, value in homography_stats.items():
        if key == 'current_corners' and value is not None:
            # Convert corners to regular Python lists/floats
            stats_serializable[key] = [[float(corner[0]), float(corner[1])]
for corner in value]
        elif key == 'update_frames':
            # Convert numpy integers to regular Python integers
            stats_serializable[key] = [int(frame) for frame in value]
        elif key == 'total_updates':
            # Convert numpy integer to regular Python integer
            stats_serializable[key] = int(value)
        elif key == 'threshold':
            # Convert to regular Python float
            stats_serializable[key] = float(value)
        else:
            stats_serializable[key] = value
    json.dump(stats_serializable, f, indent=2)
print(f"Homography statistics saved to: {homography_stats_path}")

return df_mapped, player_distances, ball_results, average_corners, frame_info

```

[FILE] pixel_to_court_mapping.py

Path: Distance_measurement\pixel_to_court_mapping.py

File is empty or could not be read.

[FILE] player_distance_calculator.py

Path: Distance_measurement\player_distance_calculator.py

```
"""
Player Distance Calculator Module for Tennis Court Tracking
"""
import numpy as np
import pandas as pd
from collections import Counter

class PlayerDistanceCalculator:
    def __init__(self):
        # Tennis court dimensions in meters (doubles)
        self.court_width = 10.97
        self.court_length = 23.77

    def calculate_player_distances_improved(self, df, id_col='ID', x_col='X_court',
                                           y_col='Y_court',
                                           class_col='Class', frame_col='frame',
                                           fps=30.0):
        """
        Improved player distance calculation with temporal awareness and movement
        analysis

        Args:
            df: DataFrame with tracking data
            id_col: Column name for player ID
            x_col, y_col: Column names for court coordinates
            class_col: Column name for object classification
            frame_col: Column name for frame numbers
            fps: Video frame rate (frames per second)

        Returns:
            DataFrame with comprehensive player movement statistics
        """
        players = df[df[class_col] == 'Player'].copy()

        if players.empty:
            return pd.DataFrame(columns=[
                'ID', 'TotalDistance', 'AverageSpeed', 'MaxSpeed', 'FrameCount',
                'ValidMovements', 'FilteredMovements', 'DetectionGaps', '
MovementSegments',
                'CourtCoverage', 'TimeActive'
            ])

        results = []

        for pid, group in players.groupby(id_col):
            # Sort by frame number
            group = group.sort_values(frame_col).reset_index(drop=True)
            coords = group[[x_col, y_col]].values
            frames = group[frame_col].values

            if len(coords) < 2:

                results.append({
                    'ID': pid,
```

```

        'TotalDistance': 0.0,
        'AverageSpeed': 0.0,
        'MaxSpeed': 0.0,
        'FrameCount': len(coords),
        'ValidMovements': 0,
        'FilteredMovements': 0,
        'DetectionGaps': 0,
        'MovementSegments': 0,
        'CourtCoverage': 0.0,
        'TimeActive': 0.0
    })
    continue

# Calculate frame differences and time intervals
frame_diffs = np.diff(frames)
time_intervals = frame_diffs / fps

# Calculate coordinate differences and distances
coord_diffs = np.diff(coords, axis=0)
frame_distances = np.linalg.norm(coord_diffs, axis=1)

# Calculate instantaneous speeds (m/s)
speeds = np.divide(frame_distances, time_intervals,
                    out=np.zeros_like(frame_distances),
                    where=time_intervals != 0)

# Filter unrealistic movements
max_realistic_speed = 12.0 # m/s (top tennis players can reach ~11
m/s)
min_movement_threshold = 0.05 # m (ignore tiny movements due to
detection noise)

# Create validity mask
valid_mask = (
    (speeds > 0) &
    (speeds <= max_realistic_speed) &
    (frame_distances >= min_movement_threshold)
)

# Handle detection gaps
detection_gaps = np.sum(frame_diffs > 1)

# Calculate movement segments (continuous tracking periods)
movement_segments = self._calculate_movement_segments(frames)

# Calculate total distance with gap handling
total_distance = 0.0
valid_movements = 0

for i in range(len(frame_distances)):
    if valid_mask[i]:
        if frame_diffs[i] == 1:
            # Consecutive frames - use direct distance
            total_distance += frame_distances[i]
            valid_movements += 1
        elif frame_diffs[i] <= 3: # Small gap - estimate movement
            # For small gaps, use linear interpolation
            estimated_distance = frame_distances[i] * (frame_diffs[i]
/ frame_diffs[i])
            total_distance += estimated_distance
            valid_movements += 1

```



```

        else:
            # Large gap - player likely stationary or off-court
            # Don't count this movement
            pass

    # Calculate speeds for valid movements only
    valid_speeds = speeds[valid_mask]
    average_speed = np.mean(valid_speeds) if len(valid_speeds) > 0 else
0.0
    max_speed = np.max(valid_speeds) if len(valid_speeds) > 0 else 0.0

    # Calculate court coverage (area covered by player)
    court_coverage = self._calculate_court_coverage(coords)

    # Calculate active time (time with valid detections)
    time_active = (frames[-1] - frames[0]) / fps if len(frames) > 1 else
0.0

    # Count filtered movements
    filtered_movements = np.sum(~valid_mask)

    results.append({
        'ID': pid,
        'TotalDistance': total_distance,
        'AverageSpeed': average_speed,
        'MaxSpeed': max_speed,
        'AverageSpeedKmh': average_speed * 3.6,
        'MaxSpeedKmh': max_speed * 3.6,
        'FrameCount': len(coords),
        'ValidMovements': valid_movements,
        'FilteredMovements': filtered_movements,
        'DetectionGaps': detection_gaps,
        'MovementSegments': movement_segments,
        'CourtCoverage': court_coverage,
        'TimeActive': time_active,
'DistancePerMinute': (total_distance / time_active * 60) if time_active > 0 else
0.0
    })

    result_df = pd.DataFrame(results)

    # Print analysis

    print("\n=== PLAYER MOVEMENT ANALYSIS ===")
    for _, row in result_df.iterrows():
        print(f"\nPlayer {row['ID']}:")
        print(f" Total distance: {row['TotalDistance']:.1f}m")
        print(f" Average speed: {row['AverageSpeedKmh']:.1f} km/h")
        print(f" Max speed: {row['MaxSpeedKmh']:.1f} km/h")
        print(f" Court coverage: {row['CourtCoverage']:.1f}m^2")
        print(f" Active time: {row['TimeActive']:.1f}s")
        print(f" Distance per minute: {row['DistancePerMinute']:.1f}m/min")
        print(f" Detection gaps: {row['DetectionGaps']}")
        print(f" Movement segments: {row['MovementSegments']}")

    return result_df

    def analyze_player_movement_patterns(self, df, id_col='ID', x_col='X_court',
y_col='Y_court',
                                     class_col='Class', frame_col='frame'):
        """
        Analyze detailed movement patterns for each player

```

Args:

df: DataFrame with tracking data
id_col: Column name for player ID
x_col, y_col: Column names for court coordinates
class_col: Column name for object classification
frame_col: Column name for frame numbers

Returns:

Dictionary with movement pattern analysis for each player

"""

```
players = df[df[class_col] == 'Player'].copy()
```

```
if players.empty:
    return {}
```

```
analysis = {}
```

```
for pid, group in players.groupby(id_col):
    group = group.sort_values(frame_col)
    coords = group[[x_col, y_col]].values
```

```
    if len(coords) < 10: # Need sufficient data points
        continue
```

```
    # Calculate movement vectors
    movements = np.diff(coords, axis=0)
    movement_magnitudes = np.linalg.norm(movements, axis=1)
```

```
    # Analyze movement directions
    movement_angles = np.arctan2(movements[:, 1], movements[:, 0])
```

```
    # Calculate acceleration (change in speed)
    speed_changes = np.diff(movement_magnitudes)
```

```
    # Identify different movement types
    stationary_threshold = 0.1 # m
    walking_threshold = 1.0 # m/frame
    running_threshold = 3.0 # m/frame
```

```
    stationary_frames = np.sum(movement_magnitudes < stationary_threshold)
    walking_frames = np.sum((movement_magnitudes >= stationary_threshold)
```

&

```
        (movement_magnitudes < walking_threshold))
    running_frames = np.sum(movement_magnitudes >= walking_threshold)
```

```
    analysis[pid] = {
        'total_frames': int(len(coords)),
        'stationary_frames': int(stationary_frames),
        'walking_frames': int(walking_frames),
        'running_frames': int(running_frames),
        'dominant_direction': self._get_dominant_direction(movement_angles)
```

,

```
        'movement_variability': float(np.std(movement_magnitudes)),
        'avg_acceleration': float(np.mean(np.abs(speed_changes))),
        'position_heat_map': self._create_position_heatmap(coords)
    }
```

```
    return analysis
```

```
def validate_player_measurements(self, player_results,
```

```

rally_duration_seconds=None):
    """
    Validate player measurements against realistic tennis movement patterns

    Args:
        player_results: DataFrame with player movement statistics
        rally_duration_seconds: Duration of the rally in seconds (optional)
    """
    print("\n=== PLAYER MEASUREMENT VALIDATION ===")

    for _, player in player_results.iterrows():
        pid = player['ID']
        total_distance = player['TotalDistance']
        avg_speed = player['AverageSpeedKmh']
        max_speed = player['MaxSpeedKmh']

        print(f"\nPlayer {pid}:")
        print(f" Total distance: {total_distance:.1f}m")

        # Validate against realistic tennis movement
        if rally_duration_seconds and rally_duration_seconds > 0:
            distance_per_second = total_distance / rally_duration_seconds
            print(f" Distance per second: {distance_per_second:.1f}m/s")

            if 0.5 <= distance_per_second <= 8.0:
                print(" [OK] Distance per second is realistic")
            else:
                print(" [W] Distance per second seems unrealistic")

        # Speed validation
        if 0 <= avg_speed <= 25: # km/h
            print(f" [OK] Average speed ({avg_speed:.1f} km/h) is realistic")
        else:
            print(f" [W] Average speed ({avg_speed:.1f} km/h) seems
unrealistic")

        if 0 <= max_speed <= 45: # km/h
            print(f" [OK] Max speed ({max_speed:.1f} km/h) is realistic")
        else:
            print(f" [W] Max speed ({max_speed:.1f} km/h) seems unrealistic")

        # Court coverage validation
        court_coverage = player['CourtCoverage']
        total_court_area = self.court_width * self.court_length # ~260 m²
        coverage_percentage = (court_coverage / total_court_area) * 100

        print(f" Court coverage: {court_coverage:.1f}m² ({coverage_percentage:.1f}% of
        court)")

        if 1 <= coverage_percentage <= 50:
            print(" [OK] Court coverage is realistic")
        else:
            print(" ■ Court coverage seems unrealistic")

    def _calculate_movement_segments(self, frames):
        """
        Calculate the number of continuous movement segments

        Args:
            frames: Array of frame numbers

```

```

Returns:
    Number of movement segments
"""
if len(frames) <= 1:
    return 0

segments = 1
for i in range(1, len(frames)):
    if frames[i] - frames[i-1] > 1: # Gap detected
        segments += 1

return segments

def _calculate_court_coverage(self, coords):
    """
    Calculate the area covered by player movement (simplified as convex hull
area)

    Args:
        coords: Array of coordinate positions

    Returns:
        Area covered in square meters
    """
    if len(coords) < 3:
        return 0.0

    try:
        from scipy.spatial import ConvexHull
        hull = ConvexHull(coords)
        return hull.volume # In 2D, volume is actually area
    except:
        # Fallback: use bounding box area
        x_range = np.max(coords[:, 0]) - np.min(coords[:, 0])
        y_range = np.max(coords[:, 1]) - np.min(coords[:, 1])
        return x_range * y_range

def _get_dominant_direction(self, angles):
    """
    Determine the dominant movement direction

    Args:
        angles: Array of movement angles in radians

    Returns:
        String representing the dominant direction
    """
    # Convert to degrees and categorize
    angles_deg = np.degrees(angles)

    # Categorize into 8 directions
    directions = []
    for angle in angles_deg:
        if -22.5 <= angle < 22.5:
            directions.append('E')
        elif 22.5 <= angle < 67.5:
            directions.append('NE')
        elif 67.5 <= angle < 112.5:
            directions.append('N')
        elif 112.5 <= angle < 157.5:
            directions.append('NW')

```

```

        elif 157.5 <= angle <= 180 or -180 <= angle < -157.5:
            directions.append('W')
        elif -157.5 <= angle < -112.5:
            directions.append('SW')

        elif -112.5 <= angle < -67.5:
            directions.append('S')
        elif -67.5 <= angle < -22.5:
            directions.append('SE')

    # Find most common direction
    if directions:
        return Counter(directions).most_common(1)[0][0]
    return 'Unknown'

def _create_position_heatmap(self, coords):
    """
    Create a simplified position heatmap (JSON-safe)

    Args:
        coords: Array of coordinate positions

    Returns:
        Dictionary with heatmap information
    """
    # Divide court into grid and count positions
    x_bins = np.linspace(0, self.court_width, 11) # 10 divisions
    y_bins = np.linspace(0, self.court_length, 24) # 23 divisions

    heatmap, _, _ = np.histogram2d(coords[:, 0], coords[:, 1], bins=[x_bins,
y_bins])

    # Find the most occupied cell
    max_cell = np.unravel_index(np.argmax(heatmap), heatmap.shape)
    max_cell = tuple(int(i) for i in max_cell) # Convert to native ints

    position_spread = [float(x) for x in np.std(coords, axis=0)]
    center_of_activity = [float(x) for x in np.mean(coords, axis=0)]

    return {
        'most_occupied_region': max_cell,
        'position_spread': position_spread,
        'center_of_activity': center_of_activity
    }

```

[FILE] tennis_court_diagnostics.py

Path: Distance_measurement\tennis_court_diagnostics.py

```

"""
Tennis Court Mapping Diagnostics and Correction Tool
"""
import cv2
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from matplotlib.patches import Rectangle

```

```

from matplotlib.collections import LineCollection

class TennisCourtDiagnostics:
    def __init__(self):
        self.court_width = 10.97 # meters
        self.court_length = 23.77 # meters

        # Define standard court coordinate system
        # Origin at bottom-left baseline, Y increases toward far baseline
        self.standard_corners = np.array([
            [0, 0], # Bottom-left baseline
            [self.court_width, 0], # Bottom-right baseline
            [self.court_width, self.court_length], # Top-right baseline
            [0, self.court_length] # Top-left baseline
        ], dtype=np.float32)

        # Key court lines for validation
        self.court_lines = {
            'baseline_near': 0,
            'service_line_near': 6.40,
            'net': 11.885, # Court length / 2
            'service_line_far': 17.37, # 23.77 - 6.40
            'baseline_far': 23.77,
            'center_line': 5.485, # Court width / 2
            'singles_sideline_left': 1.37,
            'singles_sideline_right': 9.60, # 10.97 - 1.37
            'doubles_sideline_left': 0,
            'doubles_sideline_right': 10.97
        }

    def analyze_corner_configuration(self, pixel_corners):
        """
        Analyze and suggest corrections for corner configuration
        """
        print("=== Corner Configuration Analysis ===")

        if len(pixel_corners) != 4:
            print(f"■ Expected 4 corners, got {len(pixel_corners)}")
            return False

        corners = np.array(pixel_corners)

        print(f"Input corners: {corners}")

        # Check if corners form a reasonable quadrilateral
        centroid = np.mean(corners, axis=0)
        print(f"Centroid: {centroid}")

        # Calculate angles and distances
        for i, corner in enumerate(corners):
            next_corner = corners[(i + 1) % 4]
            distance = np.linalg.norm(next_corner - corner)
            print(f"Side {i}-{(i+1)%4}: distance = {distance:.1f} pixels")

        # Suggest proper ordering based on image coordinates
        # In image coordinates: (0,0) is top-left, Y increases downward
        ordered_corners = self.order_corners_for_tennis_court(corners)
        print(f"\nSuggested corner order:")
        labels = ["Bottom-Left", "Bottom-Right", "Top-Right", "Top-Left"]
        for i, (corner, label) in enumerate(zip(ordered_corners, labels)):
            print(f"{label}: {corner}")

```

```

        return ordered_corners

def order_corners_for_tennis_court(self, corners):
    """
    Order corners specifically for tennis court mapping
    """
    corners = np.array(corners)

    # Sort by y-coordinate (top to bottom in image coordinates)
    y_sorted = corners[np.argsort(corners[:, 1])]

    # Top two corners (smaller y values - closer to net in tennis view)
    top_two = y_sorted[:2]
    top_left = top_two[np.argmin(top_two[:, 0])] # Leftmost of top two
    top_right = top_two[np.argmax(top_two[:, 0])] # Rightmost of top two

    # Bottom two corners (larger y values - baseline in tennis view)
    bottom_two = y_sorted[2:]
    bottom_left = bottom_two[np.argmin(bottom_two[:, 0])] # Leftmost of
bottom two
    bottom_right = bottom_two[np.argmax(bottom_two[:, 0])] # Rightmost of
bottom two

    # Return in order: BL, BR, TR, TL (matches standard_corners)
    return np.array([bottom_left, bottom_right, top_right, top_left],
dtype=np.float32)

def compute_robust_homography(self, pixel_corners):
    """
    Compute homography with multiple validation steps
    """
    print("\n=== Homography Computation ===")

    # Order corners properly
    ordered_corners = self.order_corners_for_tennis_court(pixel_corners)
    print(f"Ordered pixel corners: {ordered_corners}")
    print(f"Target court corners: {self.standard_corners}")

    # Compute homography
    H, mask = cv2.findHomography(
        ordered_corners,
        self.standard_corners,
        cv2.RANSAC,
        5.0
    )

    if H is None:
        print("■ Failed to compute homography")
        return None, None

    print("■ Homography computed successfully")

    # Validate by transforming corners back
    transformed_corners = cv2.perspectiveTransform(
        ordered_corners.reshape(-1, 1, 2), H
    ).reshape(-1, 2)

    print("\nValidation - Corner transformation:")
    labels = ["Bottom-Left", "Bottom-Right", "Top-Right", "Top-Left"]
    max_error = 0

```

```

for i, (label, expected, actual) in enumerate(zip(labels, self.standard_corners,
transformed_corners)):
    error = np.linalg.norm(actual - expected)
    max_error = max(max_error, error)
    print(f"{label}: Expected {expected} → Got {actual} (Error:
{error:.3f}m)")

    if max_error > 1.0:
        print(f"■■ High transformation error: {max_error:.3f}m")
    else:
        print(f"■ Transformation error acceptable: {max_error:.3f}m")

    return H, ordered_corners

def diagnose_coordinate_issues(self, df):
    """
    Diagnose issues with current coordinate mapping
    """
    print("\n=== Coordinate Mapping Diagnosis ===")

    x_coords = df['X_court'].dropna()
    y_coords = df['Y_court'].dropna()

    print(f"Current coordinate ranges:")
    print(f"X: {x_coords.min():.3f} to {x_coords.max():.3f} (range: {x_coords.max() -
x_coords.min():.3f})")
    print(f"Y: {y_coords.min():.3f} to {y_coords.max():.3f} (range: {y_coords.max() -
y_coords.min():.3f})")

    # Expected ranges
    print(f"\nExpected coordinate ranges:")
    print(f"X: 0 to {self.court_width} (range: {self.court_width})")
    print(f"Y: 0 to {self.court_length} (range: {self.court_length})")

    # Identify issues
    issues = []

    if x_coords.min() < -5:
        issues.append(f"Large negative X values (min: {x_coords.min():.3f})")
    if y_coords.min() < -5:
        issues.append(f"Large negative Y values (min: {y_coords.min():.3f})")
    if x_coords.max() > self.court_width + 5:
        issues.append(f"X values too large (max: {x_coords.max():.3f})")
    if y_coords.max() > self.court_length + 5:
        issues.append(f"Y values too large (max: {y_coords.max():.3f})")
    if (x_coords.max() - x_coords.min()) > 50:
        issues.append(f"X range too large ({x_coords.max() -
x_coords.min():.3f})")
    if (y_coords.max() - y_coords.min()) > 50:
        issues.append(f"Y range too large ({y_coords.max() -
y_coords.min():.3f})")

    if issues:
        print("\n■ Issues detected:")
        for issue in issues:
            print(f" - {issue}")
    else:
        print("\n■ Coordinates look reasonable")

    return issues

```



```

def correct_mapping(self, df, new_homography):
    """
    Apply corrected homography mapping
    """
    print("\n=== Applying Corrected Mapping ===")

    # Extract pixel coordinates
    pixel_coords = df[['X', 'Y']].values.astype(np.float32)

    # Apply homography transformation
    court_coords = cv2.perspectiveTransform(
        pixel_coords.reshape(-1, 1, 2), new_homography
    ).reshape(-1, 2)

    # Create corrected dataframe
    df_corrected = df.copy()
    df_corrected['X_court_corrected'] = court_coords[:, 0]
    df_corrected['Y_court_corrected'] = court_coords[:, 1]

    # Add outlier detection
    x_outliers = (court_coords[:, 0] < -2) | (court_coords[:, 0] >
self.court_width + 2)
    y_outliers = (court_coords[:, 1] < -2) | (court_coords[:, 1] >
self.court_length + 2)
    df_corrected['is_outlier'] = x_outliers | y_outliers

    # Statistics
    x_corr = df_corrected['X_court_corrected']
    y_corr = df_corrected['Y_court_corrected']

    print(f"Corrected coordinate ranges:")
    print(f"X: {x_corr.min():.3f} to {x_corr.max():.3f}")
    print(f"Y: {y_corr.min():.3f} to {y_corr.max():.3f}")

    outlier_count = df_corrected['is_outlier'].sum()
    print(f"Outliers detected: {outlier_count}
({outlier_count/len(df_corrected)*100:.1f}%)")

    return df_corrected

def visualize_court_mapping(self, df, title="Tennis Court Mapping"):
    """
    Create visualization of court mapping
    """
    fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(16, 8))

    # Original coordinates
    ax1.scatter(df['X_court'], df['Y_court'], c='red', alpha=0.6, s=20)
    ax1.set_title('Original Mapping')
    ax1.set_xlabel('X (meters)')
    ax1.set_ylabel('Y (meters)')
    ax1.grid(True, alpha=0.3)
    ax1.set_aspect('equal')

    # Corrected coordinates (if available)
    if 'X_court_corrected' in df.columns:
        # Plot court boundaries
        court_rect = Rectangle((0, 0), self.court_width, self.court_length,
                               linewidth=2, edgecolor='black',
facecolor='lightgreen', alpha=0.3)

```

```

ax2.add_patch(court_rect)

# Plot court lines
# Net line
ax2.plot([0, self.court_width], [self.court_lines['net'],
self.court_lines['net']],
        'k-', linewidth=2, label='Net')

# Service lines

ax2.plot([0, self.court_width],
        [self.court_lines['service_line_near'],
self.court_lines['service_line_near']],
        'b--', linewidth=1, label='Service Lines')
ax2.plot([0, self.court_width],
        [self.court_lines['service_line_far'],
self.court_lines['service_line_far']],
        'b--', linewidth=1)

# Center line
ax2.plot([self.court_lines['center_line'],
self.court_lines['center_line']],
        [0, self.court_length], 'g--', linewidth=1, label='Center
Line')

# Singles sidelines
ax2.plot([self.court_lines['singles_sideline_left'], self.court_lines[
'singles_sideline_left']],
        [0, self.court_length], 'r:', linewidth=1, label='Singles
Lines')
ax2.plot([self.court_lines['singles_sideline_right'], self.court_lines[
'singles_sideline_right']],
        [0, self.court_length], 'r:', linewidth=1)

# Plot points
non_outliers = df[~df['is_outlier']] if 'is_outlier' in df.columns
else df
outliers = df[df['is_outlier']] if 'is_outlier' in df.columns else
pd.DataFrame()

if len(non_outliers) > 0:
    ax2.scatter(non_outliers['X_court_corrected'],
non_outliers['Y_court_corrected'],
                c='blue', alpha=0.6, s=20, label='Valid Points')

if len(outliers) > 0:
    ax2.scatter(outliers['X_court_corrected'],
outliers['Y_court_corrected'],
                c='red', alpha=0.8, s=30, marker='x', label='Outliers')

ax2.set_title('Corrected Mapping')
ax2.set_xlabel('X (meters)')
ax2.set_ylabel('Y (meters)')
ax2.legend()
ax2.grid(True, alpha=0.3)
ax2.set_aspect('equal')
ax2.set_xlim(-2, self.court_width + 2)
ax2.set_ylim(-2, self.court_length + 2)

plt.tight_layout()
plt.show()

```

```

def generate_correction_report(self, df_original, df_corrected):
    """
    Generate a comprehensive correction report
    """
    print("\n" + "="*60)
    print("TENNIS COURT MAPPING CORRECTION REPORT")
    print("="*60)

    # Before/After comparison
    print("\n1. COORDINATE RANGE COMPARISON")
    print("-" * 40)

    orig_x = df_original['X_court']
    orig_y = df_original['Y_court']
    corr_x = df_corrected['X_court_corrected']
    corr_y = df_corrected['Y_court_corrected']

    print(f"{'Metric':<20} {'Original':<15} {'Corrected':<15}"
          {'Expected':<15}")
    print("-" * 65)
    print(f"{'X Range':<20} {orig_x.max()-orig_x.min():<15.2f}"
          {corr_x.max()-corr_x.min():<15.2f} {self.court_width:<15.2f}")
    print(f"{'Y Range':<20} {orig_y.max()-orig_y.min():<15.2f}"
          {corr_y.max()-corr_y.min():<15.2f} {self.court_length:<15.2f}")
    print(f"{'X Min':<20} {orig_x.min():<15.2f} {corr_x.min():<15.2f}"
          {'~0':<15}")
    print(f"{'X Max':<20} {orig_x.max():<15.2f} {corr_x.max():<15.2f}"
          {self.court_width:<15.2f}")
    print(f"{'Y Min':<20} {orig_y.min():<15.2f} {corr_y.min():<15.2f}"
          {'~0':<15}")
    print(f"{'Y Max':<20} {orig_y.max():<15.2f} {corr_y.max():<15.2f}"
          {self.court_length:<15.2f}")

    # Outlier analysis
    print("\n2. OUTLIER ANALYSIS")
    print("-" * 40)

    if 'is_outlier' in df_corrected.columns:
        outlier_count = df_corrected['is_outlier'].sum()
        total_count = len(df_corrected)
        outlier_percentage = (outlier_count / total_count) * 100

        print(f"Total points: {total_count}")
        print(f"Outliers: {outlier_count} ({outlier_percentage:.1f}%)")
        print(f"Valid points: {total_count - outlier_count} ({100 -"
              outlier_percentage:.1f}%)")

    # Class-wise analysis
    print("\n3. CLASS-WISE ANALYSIS")
    print("-" * 40)

    if 'Class' in df_corrected.columns:
        for class_name in df_corrected['Class'].unique():
            class_data = df_corrected[df_corrected['Class'] == class_name]
            class_outliers = class_data['is_outlier'].sum() if 'is_outlier' in
            class_data.columns else 0
            class_valid = len(class_data) - class_outliers

            print(f"\n{class_name}:")
            print(f" Total points: {len(class_data)}")
            print(f" Valid points: {class_valid}")

```

```

        print(f" Outliers: {class_outliers}")
print(f" X range: {class_data['X_court_corrected'].min():.2f} to
      {class_data['X_court_corrected'].max():.2f}")
print(f" Y range: {class_data['Y_court_corrected'].min():.2f} to
      {class_data['Y_court_corrected'].max():.2f}")

# Recommendations
print("\n4. RECOMMENDATIONS")
print("-" * 40)

recommendations = []

if outlier_count > total_count * 0.1:
    recommendations.append("High outlier percentage - consider reviewing
corner detection")

    if corr_x.min() < -1 or corr_x.max() > self.court_width + 1:
recommendations.append("X coordinates still outside expected range - check corner
ordering")

    if corr_y.min() < -1 or corr_y.max() > self.court_length + 1:
recommendations.append("Y coordinates still outside expected range - verify court
orientation")

    if abs((corr_x.max() - corr_x.min()) - self.court_width) > 2:
recommendations.append("X range doesn't match court width - homography may be
distorted")

    if abs((corr_y.max() - corr_y.min()) - self.court_length) > 2:
recommendations.append("Y range doesn't match court length - homography may be
distorted")

    if recommendations:
        for i, rec in enumerate(recommendations, 1):
            print(f"{i}. {rec}")
    else:
        print("■ Mapping appears to be working correctly!")

    return df_corrected

def fix_tennis_mapping(csv_file, pixel_corners):
    """
    Complete pipeline to diagnose and fix tennis court mapping issues

    Args:
        csv_file: Path to CSV file with tracking data
        pixel_corners: List of 4 corner points from your image
                      Format: [(x1,y1), (x2,y2), (x3,y3), (x4,y4)]

    Returns:
        Corrected DataFrame with proper court coordinates
    """
    print("■ TENNIS COURT MAPPING CORRECTION PIPELINE")
    print("="*50)

    # Initialize diagnostics
    diagnostics = TennisCourtDiagnostics()

    # Load data
    df = pd.read_csv(csv_file)

```

```

print(f"Loaded {len(df)} tracking points")

# Step 1: Analyze corner configuration
ordered_corners = diagnostics.analyze_corner_configuration(pixel_corners)
if ordered_corners is None:
    print("■ Failed to analyze corners")
    return None

# Step 2: Diagnose current mapping issues
issues = diagnostics.diagnose_coordinate_issues(df)

# Step 3: Compute corrected homography
H_corrected, final_corners = diagnostics.compute_robust_homography(pixel_corners)
rs)
if H_corrected is None:
    print("■ Failed to compute corrected homography")
    return None

# Step 4: Apply corrected mapping
df_corrected = diagnostics.correct_mapping(df, H_corrected)

# Step 5: Generate report
df_final = diagnostics.generate_correction_report(df, df_corrected)

# Step 6: Visualize results (optional - requires matplotlib)
# diagnostics.visualize_court_mapping(df_final, "Corrected Tennis Court Mapping")

print("\n■ Mapping correction completed!")
print("Use the 'X_court_corrected' and 'Y_court_corrected' columns for analysis")

return df_final, H_corrected, final_corners

# Example usage based on your image
def example_usage():
    """
    Example of how to use the correction pipeline with your data
    """
    # Your corner coordinates from the image (you'll need to provide these)
    # Based on your image, these should be the pixel coordinates of the court corners

    # Order them as: Bottom-Left, Bottom-Right, Top-Right, Top-Left
    pixel_corners = [
        (436.9, 615.0), # Bottom-Left (BL)
        (1223.5, 621.0), # Bottom-Right (BR)
        (890.5, 107.1), # Top-Right (TR)
        (375.4, 227.4) # Top-Left (TL)
    ]

    # Run the correction pipeline
    df_corrected, homography, corners = fix_tennis_mapping(
        'match_point_init_df_with_video_court_coords.csv',
        pixel_corners
    )

    return df_corrected, homography, corners

```

[FILE] tennis_court_tracker.py

Path: Distance_measurement\tennis_court_tracker.py

```
import cv2
import numpy as np
import pandas as pd
from sklearn.cluster import DBSCAN
import urllib.request
import os
from collections import defaultdict
from .homography_manager import HomographyManager

class TennisCourtTracker:
    def __init__(self):
        # Tennis court dimensions in meters (doubles)
        self.court_width = 10.97
        self.court_length = 23.77
        self.real_corners = [
            (0, 0),
            (self.court_width, 0),
            (self.court_width, self.court_length),
            (0, self.court_length)
        ]

    def download_video(self, url, output_path="downloaded_video.mp4"):
        """Download video from URL if it's a web URL"""
        try:
            if url.startswith(('http://', 'https://')):
                print(f"[DEBUG] Downloading video from: {url}")
                urllib.request.urlretrieve(url, output_path)
                print(f"[DEBUG] Video downloaded to: {output_path}")
                return output_path
            else:
                # Local file path
                if os.path.exists(url):
                    return url
                else:
                    raise FileNotFoundError(f"Local video file not found: {url}")
        except Exception as e:
            print(f"[ERROR] Failed to download video: {e}")
            raise

    def detect_corners_from_video(self, video_path, sample_interval=1.0,
max_frames=30,
debug=False):
        """
        Detect court corners from multiple frames in a video

        Args:
            video_path: Path to video file or URL
            sample_interval: Interval in seconds between frame samples
            max_frames: Maximum number of frames to process
            debug: Whether to show debug information
        """
```

```

Returns:
    tuple: (average_corners, all_frame_corners, frame_info,
representative_frame)
"""
# Download video if it's a URL
if video_path.startswith(('http://', 'https://')):
    video_path = self.download_video(video_path)

cap = cv2.VideoCapture(video_path)
if not cap.isOpened():
    raise ValueError(f"Could not open video: {video_path}")

# Get video properties
fps = cap.get(cv2.CAP_PROP_FPS)
total_frames = int(cap.get(cv2.CAP_PROP_FRAME_COUNT))
duration = total_frames / fps

print(f"[DEBUG] Video info: {total_frames} frames, {fps:.2f} FPS,
{duration:.2f}s duration")

# Calculate frame sampling
frame_interval = int(fps * sample_interval)
frames_to_process = min(max_frames, int(duration / sample_interval))

print(f"[DEBUG] Will process {frames_to_process} frames, sampling every
{frame_interval}
frames ({sample_interval}s)")

all_frame_corners = []
frame_info = []
processed_count = 0
representative_frame = None
best_frame_score = 0

for i in range(frames_to_process):
    frame_number = int(i * frame_interval) # Ensure it's a Python int
    timestamp = frame_number / fps

    # Seek to the specific frame
    cap.set(cv2.CAP_PROP_POS_FRAMES, frame_number)
    ret, frame = cap.read()

    if not ret:
        print(f"[WARNING] Could not read frame {frame_number}")
        continue

    print(f"[DEBUG] Processing frame {frame_number} at {timestamp:.2f}s")

    # Detect corners in this frame
    corners = self.detect_court_corners(frame, debug=False)

    frame_data = {
        'frame_number': frame_number,

        'timestamp': timestamp,
        'corners_found': len(corners),
        'corners': corners
    }

    if len(corners) == 4:
        all_frame_corners.append(corners)
        frame_data['valid'] = True

```

```

        processed_count += 1
        print(f"[DEBUG] Frame {frame_number}: Found 4 corners [OK]")

        # Select representative frame (middle of the sequence with good
corners)
        if processed_count == max(1, len(all_frame_corners) // 2):
            representative_frame = frame.copy()
            print(f"[DEBUG] Selected frame {frame_number} as
representative frame")

        else:
            frame_data['valid'] = False
            print(f"[DEBUG] Frame {frame_number}: Found {len(corners)}
corners [FAILED]")

        frame_info.append(frame_data)

        # Save debug image for first few frames if debug is enabled
        if debug and i < 5:
            debug_frame = frame.copy()
            if corners:
                self._draw_corners_on_frame(debug_frame, corners)
                output_path = f'debug_frame_{frame_number}.jpg'
                cv2.imwrite(output_path, debug_frame)
                print(f"[DEBUG] Saved debug frame to: {output_path}")

        cap.release()

        print(f"[DEBUG] Successfully processed
{processed_count}/{frames_to_process} frames")

        if processed_count == 0:
            print("[ERROR] No valid corners found in any frame")
            return None, all_frame_corners, frame_info, None

        # Calculate average corners
        average_corners = self._calculate_average_corners(all_frame_corners)

        # If no representative frame was selected, use the first valid frame
        if representative_frame is None and all_frame_corners:
            print("[DEBUG] No representative frame selected, using first valid
frame")

            # Get the first valid frame
            cap = cv2.VideoCapture(video_path)
            for info in frame_info:
                if info['valid']:
                    cap.set(cv2.CAP_PROP_POS_FRAMES, info['frame_number'])

                    ret, representative_frame = cap.read()
                    if ret:
                        break
            cap.release()

        # Clean up downloaded video if it was a URL
        if video_path.startswith('downloaded_video'):
            try:
                os.remove(video_path)
                print(f"[DEBUG] Cleaned up downloaded video: {video_path}")
            except:
                pass

        return average_corners, all_frame_corners, frame_info,

```


representative_frame

```
def _calculate_average_corners(self, all_frame_corners):
    """Calculate average corner positions from multiple frames"""
    if not all_frame_corners:
        return []

    print(f"[DEBUG] Calculating average corners from {len(all_frame_corners)}
valid frames")

    # Group corners by their likely position (using simple clustering)
    corner_groups = [[] for _ in range(4)]

    for frame_corners in all_frame_corners:
        # Order corners consistently for this frame
        ordered_corners = self.get_ordered_corners(frame_corners)

        # Add to respective groups
        for i, corner in enumerate(ordered_corners):
            corner_groups[i].append(corner)

    # Calculate average for each corner group
    average_corners = []
    corner_names = ["Top-Left", "Top-Right", "Bottom-Right", "Bottom-Left"]

    for i, (group, name) in enumerate(zip(corner_groups, corner_names)):
        if group:
            # Convert to regular Python floats to avoid numpy compatibility
            issues

            x_coords = [float(corner[0]) for corner in group]
            y_coords = [float(corner[1]) for corner in group]

            # Use numpy for statistics instead of statistics module
            avg_x = np.mean(x_coords)
            avg_y = np.mean(y_coords)
            std_x = np.std(x_coords) if len(x_coords) > 1 else 0
            std_y = np.std(y_coords) if len(y_coords) > 1 else 0

            average_corners.append((avg_x, avg_y))

            print(f"[DEBUG] {name}: ({avg_x:.1f}, {avg_y:.1f}) +/-
({std_x:.1f}, {std_y:.1f})")

    return average_corners

def visualize_average_corners(self, frame, average_corners, output_path=
"average_corners_visualization.jpg", show_image=True):
    """
    Visualize average corners on a representative frame

    Args:
        frame: The frame to draw on
        average_corners: List of average corner coordinates
        output_path: Path to save the visualization
        show_image: Whether to display the image using cv2.imshow
    """
    if frame is None:
        print("[ERROR] No frame provided for visualization")
        return

    if not average_corners or len(average_corners) != 4:
        print(f"[ERROR] Invalid average corners: expected 4, got {len(average_corners)} if
```

```

        average_corners else 0}"))
    return

# Create a copy of the frame for visualization
vis_frame = frame.copy()

# Draw the average corners
self._draw_corners_on_frame(vis_frame, average_corners)

# Add additional information
cv2.putText(vis_frame, 'AVERAGE CORNERS FROM MULTIPLE FRAMES', (10, 110),
            cv2.FONT_HERSHEY_SIMPLEX, 0.7, (0, 255, 255), 2)

# Add corner coordinates as text
ordered_corners = self.get_ordered_corners(average_corners)
corner_labels = ['TL', 'TR', 'BR', 'BL']
for i, (corner, label) in enumerate(zip(ordered_corners, corner_labels)):
    coord_text = f'{label}: ({corner[0]:.1f}, {corner[1]:.1f})'
    cv2.putText(vis_frame, coord_text, (10, 150 + i * 25),
                cv2.FONT_HERSHEY_SIMPLEX, 0.5, (255, 255, 255), 1)

# Save the visualization
try:
    cv2.imwrite(output_path, vis_frame)
    print(f"[DEBUG] Average corners visualization saved to:
{output_path}")
except Exception as e:
    print(f"[ERROR] Failed to save visualization: {e}")

# Display the image if requested

if show_image:
    try:
        cv2.imshow('Average Tennis Court Corners', vis_frame)
        print("[DEBUG] Press any key to close the image window...")
        cv2.waitKey(0)
        cv2.destroyAllWindows()
    except Exception as e:
        print(f"[WARNING] Could not display image: {e}")
        print("Image saved to file instead.")

def detect_court_corners(self, frame, debug=False):
    """Original corner detection method for single frame"""
    if debug:
        print("[DEBUG] Converting to grayscale...")
    gray = cv2.cvtColor(frame, cv2.COLOR_BGR2GRAY)
    blurred = cv2.GaussianBlur(gray, (5, 5), 0)
    edges = cv2.Canny(blurred, 50, 150, apertureSize=3)
    lines = cv2.HoughLinesP(edges, 1, np.pi / 180, threshold=80, minLineLength=100,
                             maxLineGap=50)

    h, w = frame.shape[:2]
    if lines is None:
        return []

    sideline_segments = self._find_sideline_segments(lines, w, h)
    if len(sideline_segments) < 2:
        return []

    left_sideline, right_sideline = self._group_sideline_segments(sideline_segments,
w,
                             frame.copy())

```

```

        if not left_sideline or not right_sideline:
            return []

    corners = self._find_corners_by_line_tracing(left_sideline, right_sideline,
        frame.copy(),
        h, w)

    return corners

    def _detect_significant_corner_change(self, current_corners,
previous_corners, threshold=15.0):
    """
    Detect if corner positions have changed significantly

    Args:
        current_corners: Current frame corners
        previous_corners: Previous frame corners
        threshold: Pixel distance threshold for significant change

    Returns:
        bool: True if significant change detected
    """

    if not current_corners or not previous_corners or len(current_corners) != 4 or
        len(previous_corners) != 4:
        return True

    current_ordered = self.get_ordered_corners(current_corners)
    previous_ordered = self.get_ordered_corners(previous_corners)

    for curr, prev in zip(current_ordered, previous_ordered):
        distance = np.linalg.norm(np.array(curr) - np.array(prev))
        if distance > threshold:
            return True

    return False

    def _draw_corners_on_frame(self, frame, corners):
    """Draw detected corners on the frame with labels"""
    if len(corners) != 4:
        # If we don't have exactly 4 corners, just draw them as red circles
        for i, corner in enumerate(corners):
            x, y = int(corner[0]), int(corner[1])
            cv2.circle(frame, (x, y), 8, (0, 0, 255), -1) # Red filled circle
            cv2.putText(frame, f'C{i+1}', (x + 10, y - 10),
                cv2.FONT_HERSHEY_SIMPLEX, 0.7, (0, 0, 255), 2)
    else:
        # Order corners and draw them with proper labels
        ordered_corners = self.get_ordered_corners(corners)
        corner_labels = ['TL', 'TR', 'BR', 'BL'] # Top-Left, Top-Right, Bottom-Right,
            Bottom-Left
        colors = [(255, 0, 0), (0, 255, 0), (0, 0, 255), (255, 255, 0)] # Blue, Green,
            Red,
            Cyan

        for i, (corner, label, color) in enumerate(zip(ordered_corners,
corner_labels, colors)):
            x, y = int(corner[0]), int(corner[1])
            cv2.circle(frame, (x, y), 8, color, -1) # Filled circle
            cv2.circle(frame, (x, y), 12, color, 2) # Outer circle
            cv2.putText(frame, label, (x + 15, y - 10),
                cv2.FONT_HERSHEY_SIMPLEX, 0.8, color, 2)

```

```

        # Draw lines connecting the corners to show the court outline
        for i in range(4):
            pt1 = (int(ordered_corners[i][0]), int(ordered_corners[i][1]))
            pt2 = (int(ordered_corners[(i+1)%4][0]),
int(ordered_corners[(i+1)%4][1]))
            cv2.line(frame, pt1, pt2, (255, 255, 255), 2) # White lines

    # Add title
    cv2.putText(frame, 'Tennis Court Corners Detection', (10, 30),
                cv2.FONT_HERSHEY_SIMPLEX, 1, (255, 255, 255), 2)

    # Add corner count info
    cv2.putText(frame, f'Corners found: {len(corners)}/4', (10, 70),

                cv2.FONT_HERSHEY_SIMPLEX, 0.7, (255, 255, 255), 2)

def _find_sideline_segments(self, lines, img_width, img_height):
    sideline_segments = []
    for line in lines:
        x1, y1, x2, y2 = line[0]
        dx = x2 - x1
        dy = y2 - y1
        length = np.sqrt(dx**2 + dy**2)
        if abs(dx) < 1e-6:
            angle = 90.0
        else:
            angle = abs(np.arctan(dy / dx) * 180 / np.pi)
        if abs(dx) < 1e-6:
            slope = float('inf')
        else:
            slope = dy / dx
        min_length = max(80, min(img_width, img_height) * 0.15)
        if length > min_length and 30 < angle < 85:
            center_x = (x1 + x2) / 2
            center_y = (y1 + y2) / 2
            sideline_segments.append({
                'line': (x1, y1, x2, y2),
                'length': length,
                'angle': angle,
                'slope': slope,
                'center_x': center_x,
                'center_y': center_y,
                'top_y': min(y1, y2),
                'bottom_y': max(y1, y2),
                'top_point': (x1, y1) if y1 < y2 else (x2, y2),
                'bottom_point': (x1, y1) if y1 > y2 else (x2, y2)
            })
    return sideline_segments

def _group_sideline_segments(self, segments, img_width, debug_img):
    if len(segments) < 2:
        return [], []
    x_coords = np.array([[seg['center_x']] for seg in segments])
    clustering = DBSCAN(eps=img_width * 0.1, min_samples=1).fit(x_coords)
    labels = clustering.labels_
    clusters = {}
    for i, label in enumerate(labels):
        if label not in clusters:
            clusters[label] = []
        clusters[label].append(segments[i])
    cluster_info = []

```

```

        for label, cluster_segments in clusters.items():
            if len(cluster_segments) >= 1:
                avg_x = np.mean([seg['center_x'] for seg in cluster_segments])

                total_length = sum([seg['length'] for seg in cluster_segments])
                cluster_info.append((label, avg_x, total_length,
cluster_segments))
            cluster_info.sort(key=lambda x: x[2], reverse=True)
            if len(cluster_info) < 2:
                return [], []
            cluster1 = cluster_info[0]
            cluster2 = cluster_info[1]
            if cluster1[1] < cluster2[1]:
                left_sideline = cluster1[3]
                right_sideline = cluster2[3]
            else:
                left_sideline = cluster2[3]
                right_sideline = cluster1[3]
            return left_sideline, right_sideline

def _find_corners_by_line_tracing(self, left_sideline, right_sideline,
debug_img, img_height,
img_width):
    corners = []
    if left_sideline:
        left_bottom, left_top = self._find_corner_pair(left_sideline, img_height,
img_width,
is_left=True)
        if left_bottom and left_top:
            corners.extend([left_top, left_bottom])
    if right_sideline:
        right_bottom, right_top = self._find_corner_pair(right_sideline, img_height,
img_width,
is_left=False)
        if right_bottom and right_top:
            corners.extend([right_top, right_bottom])
    return corners

def _find_corner_pair(self, sideline_segments, img_height, img_width,
is_left=True):
    if not sideline_segments:
        return None, None
    bottom_point = None
    bottom_y = -1
    bottom_segment = None
    for seg in sideline_segments:
        x1, y1, x2, y2 = seg['line']
        for point in [(x1, y1), (x2, y2)]:
            if point[1] > bottom_y:
                bottom_y = point[1]
                bottom_point = point
                bottom_segment = seg
    if bottom_segment is None:
        return None, None
    collinear_segments = self._find_collinear_segments(bottom_segment,
sideline_segments)
    top_point = self._find_topmost_point_from_collinear_segments(collinear_seg
ments)
    return bottom_point, top_point

def _find_collinear_segments(self, reference_segment, all_segments):

```

```

collinear_segments = [reference_segment]
ref_x1, ref_y1, ref_x2, ref_y2 = reference_segment['line']
a = ref_y2 - ref_y1
b = ref_x1 - ref_x2
c = (ref_x2 - ref_x1) * ref_y1 - (ref_y2 - ref_y1) * ref_x1
norm = np.sqrt(a*a + b*b)
if norm > 0:
    a, b, c = a/norm, b/norm, c/norm
for seg in all_segments:
    if seg == reference_segment:
        continue
    x1, y1, x2, y2 = seg['line']
    dist1 = abs(a * x1 + b * y1 + c)
    dist2 = abs(a * x2 + b * y2 + c)
    tolerance = 10.0
    if dist1 < tolerance and dist2 < tolerance:
        collinear_segments.append(seg)
return collinear_segments

def _find_topmost_point_from_collinear_segments(self, collinear_segments):
    if not collinear_segments:
        return None
    topmost_point = None
    topmost_y = float('inf')
    for seg in collinear_segments:
        x1, y1, x2, y2 = seg['line']
        for point in [(x1, y1), (x2, y2)]:
            if point[1] < topmost_y:
                topmost_y = point[1]
                topmost_point = point
    return topmost_point

def get_ordered_corners(self, corners):
    if len(corners) != 4:
        return corners
    corners = sorted(corners, key=lambda p: p[1])
    top_points = sorted(corners[:2], key=lambda p: p[0])
    bottom_points = sorted(corners[2:], key=lambda p: p[0])
    return [top_points[0], top_points[1], bottom_points[1], bottom_points[0]]

def compute_homography(self, pixel_corners):
    if len(pixel_corners) != 4:
        raise ValueError("Exactly 4 corner points required")
    ordered_corners = self.get_ordered_corners(pixel_corners)
    pixel_pts = np.array(ordered_corners, dtype=np.float32)
    real_pts = np.array(self.real_corners, dtype=np.float32)
    H, status = cv2.findHomography(pixel_pts, real_pts, cv2.RANSAC, 5.0)
    if H is None:
        raise ValueError("Could not compute homography matrix")
    return H

def validate_homography(self, H, pixel_corners):
    ordered_corners = self.get_ordered_corners(pixel_corners)
    pixel_pts = np.array(ordered_corners, dtype=np.float32)
    pixel_pts_h = np.concatenate([pixel_pts, np.ones((4, 1))], axis=1)
    mapped_pts = (H @ pixel_pts_h.T).T
    mapped_pts = mapped_pts / mapped_pts[:, 2:3]
    real_pts = np.array(self.real_corners, dtype=np.float32)
    errors = np.linalg.norm(mapped_pts[:, :2] - real_pts, axis=1)
    print("Homography validation:")
    corner_names = ["Top-Left", "Top-Right", "Bottom-Right", "Bottom-Left"]

```

```

for i, (name, pixel, real, mapped, error) in enumerate(zip(corner_names,
ordered_corners,
    self.real_corners, mapped_pts[:, :2], errors)):
    print(f"{name}: Pixel {pixel} -> Expected {real} -> Mapped {mapped} (Error:
        {error:.3f}m)")
    return np.all(errors < 1.0)

def map_pixels_to_court(self, df, H, x_col='X', y_col='Y'):
    points = df[[x_col, y_col]].values.astype(np.float32)
    points_h = np.concatenate([points, np.ones((points.shape[0], 1),
dtype=np.float32)], axis=1)
    mapped = (H @ points_h.T).T
    mapped = mapped / mapped[:, 2:3]
    df_mapped = df.copy()
    df_mapped['X_court'] = mapped[:, 0]
    df_mapped['Y_court'] = mapped[:, 1]
    return df_mapped
...

def calculate_player_distances(self, df, id_col='ID', x_col='X_court',
y_col='Y_court',
    class_col='Class', frame_col='frame'):
    players = df[df[class_col] == 'Player'].copy()
    if players.empty:
        return pd.DataFrame(columns=['ID', 'TotalDistance', 'FrameCount', '
FilteredFrames'])
    distances = []
    for pid, group in players.groupby(id_col):
        group = group.sort_values(frame_col)
        coords = group[[x_col, y_col]].values
        if len(coords) < 2:
            distances.append({'ID': pid, 'TotalDistance': 0.0, 'FrameCount': len(coords),
                'FilteredFrames': 0})
            continue
        diffs = np.diff(coords, axis=0)
        frame_distances = np.linalg.norm(diffs, axis=1)
        max_speed_per_frame = 0.5
        valid_mask = frame_distances <= max_speed_per_frame
        valid_distances = frame_distances[valid_mask]
        total_distance = np.sum(valid_distances)
        filtered_count = np.sum(~valid_mask)
        distances.append({
            'ID': pid,
            'TotalDistance': total_distance,

            'FrameCount': len(coords),
            'FilteredFrames': filtered_count
        })
    return pd.DataFrame(distances)
...

def calculate_player_distances_improved(self, df, id_col='ID', x_col='X_court',
y_col='Y_court',
    class_col='Class', frame_col='frame',
fps=30.0):
    """
    Improved player distance calculation with temporal awareness and movement
analysis

    Args:
    df: DataFrame with tracking data
    id_col: Column name for player ID
    x_col, y_col: Column names for court coordinates

```

```

class_col: Column name for object classification
frame_col: Column name for frame numbers
fps: Video frame rate (frames per second)

Returns:
DataFrame with comprehensive player movement statistics
"""
players = df[df[class_col] == 'Player'].copy()

if players.empty:
    return pd.DataFrame(columns=[
        'ID', 'TotalDistance', 'AverageSpeed', 'MaxSpeed', 'FrameCount',
        'ValidMovements', 'FilteredMovements', 'DetectionGaps', '
MovementSegments',
        'CourtCoverage', 'TimeActive'
    ])

results = []

for pid, group in players.groupby(id_col):
    # Sort by frame number
    group = group.sort_values(frame_col).reset_index(drop=True)
    coords = group[[x_col, y_col]].values
    frames = group[frame_col].values

    if len(coords) < 2:
        results.append({
            'ID': pid,
            'TotalDistance': 0.0,
            'AverageSpeed': 0.0,
            'MaxSpeed': 0.0,
            'FrameCount': len(coords),
            'ValidMovements': 0,
            'FilteredMovements': 0,
            'DetectionGaps': 0,

            'MovementSegments': 0,
            'CourtCoverage': 0.0,
            'TimeActive': 0.0
        })
        continue

    # Calculate frame differences and time intervals
    frame_diffs = np.diff(frames)
    time_intervals = frame_diffs / fps

    # Calculate coordinate differences and distances
    coord_diffs = np.diff(coords, axis=0)
    frame_distances = np.linalg.norm(coord_diffs, axis=1)

    # Calculate instantaneous speeds (m/s)
    speeds = np.divide(frame_distances, time_intervals,
                       out=np.zeros_like(frame_distances),
                       where=time_intervals != 0)

    # Filter unrealistic movements
    max_realistic_speed = 12.0 # m/s (top tennis players can reach ~11
m/s)
    min_movement_threshold = 0.05 # m (ignore tiny movements due to
detection noise)

    # Create validity mask

```



```

valid_mask = (
    (speeds > 0) &
    (speeds <= max_realistic_speed) &
    (frame_distances >= min_movement_threshold)
)

# Handle detection gaps
detection_gaps = np.sum(frame_diffs > 1)

# Calculate movement segments (continuous tracking periods)
movement_segments = self._calculate_movement_segments(frames)

# Calculate total distance with gap handling
total_distance = 0.0
valid_movements = 0

for i in range(len(frame_distances)):
    if valid_mask[i]:
        if frame_diffs[i] == 1:
            # Consecutive frames - use direct distance
            total_distance += frame_distances[i]
            valid_movements += 1
        elif frame_diffs[i] <= 3: # Small gap - estimate movement
            # For small gaps, use linear interpolation
            estimated_distance = frame_distances[i] * (frame_diffs[i]
/ frame_diffs[i])
            total_distance += estimated_distance

            valid_movements += 1
        else:
            # Large gap - player likely stationary or off-court
            # Don't count this movement
            pass

# Calculate speeds for valid movements only
valid_speeds = speeds[valid_mask]
average_speed = np.mean(valid_speeds) if len(valid_speeds) > 0 else
0.0

max_speed = np.max(valid_speeds) if len(valid_speeds) > 0 else 0.0

# Calculate court coverage (area covered by player)
court_coverage = self._calculate_court_coverage(coords)

# Calculate active time (time with valid detections)
time_active = (frames[-1] - frames[0]) / fps if len(frames) > 1 else
0.0

# Count filtered movements
filtered_movements = np.sum(~valid_mask)

results.append({
    'ID': pid,
    'TotalDistance': total_distance,
    'AverageSpeed': average_speed,
    'MaxSpeed': max_speed,
    'AverageSpeedKmh': average_speed * 3.6,
    'MaxSpeedKmh': max_speed * 3.6,
    'FrameCount': len(coords),
    'ValidMovements': valid_movements,
    'FilteredMovements': filtered_movements,
    'DetectionGaps': detection_gaps,
    'MovementSegments': movement_segments,

```

```

        'CourtCoverage': court_coverage,
        'TimeActive': time_active,
'DistancePerMinute': (total_distance / time_active * 60) if time_active > 0 else
        0.0
    })

    result_df = pd.DataFrame(results)

    # Print analysis
    print("\n=== PLAYER MOVEMENT ANALYSIS ===")
    for _, row in result_df.iterrows():
        print(f"\nPlayer {row['ID']}:")
        print(f" Total distance: {row['TotalDistance']:.1f}m")
        print(f" Average speed: {row['AverageSpeedKmh']:.1f} km/h")
        print(f" Max speed: {row['MaxSpeedKmh']:.1f} km/h")
        print(f" Court coverage: {row['CourtCoverage']:.1f}m^2")
        print(f" Active time: {row['TimeActive']:.1f}s")
        print(f" Distance per minute: {row['DistancePerMinute']:.1f}m/min")

        print(f" Detection gaps: {row['DetectionGaps']}")
        print(f" Movement segments: {row['MovementSegments']}")

    return result_df

def _calculate_movement_segments(self, frames):
    """
    Calculate the number of continuous movement segments
    """
    if len(frames) <= 1:
        return 0

    segments = 1
    for i in range(1, len(frames)):
        if frames[i] - frames[i-1] > 1: # Gap detected
            segments += 1

    return segments

def _calculate_court_coverage(self, coords):
    """
    Calculate the area covered by player movement (simplified as convex hull
area)
    """
    if len(coords) < 3:
        return 0.0

    try:
        from scipy.spatial import ConvexHull
        hull = ConvexHull(coords)
        return hull.volume # In 2D, volume is actually area
    except:
        # Fallback: use bounding box area
        x_range = np.max(coords[:, 0]) - np.min(coords[:, 0])
        y_range = np.max(coords[:, 1]) - np.min(coords[:, 1])
        return x_range * y_range

    def validate_player_measurements(self, player_results,
rally_duration_seconds=None):
    """
    Validate player measurements against realistic tennis movement patterns
    """
    print("\n=== PLAYER MEASUREMENT VALIDATION ===")

```

```

for _, player in player_results.iterrows():
    pid = player['ID']
    total_distance = player['TotalDistance']
    avg_speed = player['AverageSpeedKmh']
    max_speed = player['MaxSpeedKmh']

    print(f"\nPlayer {pid}:")
    print(f" Total distance: {total_distance:.1f}m")

    # Validate against realistic tennis movement
    if rally_duration_seconds and rally_duration_seconds > 0:
        distance_per_second = total_distance / rally_duration_seconds
        print(f" Distance per second: {distance_per_second:.1f}m/s")

        if 0.5 <= distance_per_second <= 8.0:
            print(" [OK] Distance per second is realistic")
        else:
            print(" [W] Distance per second seems unrealistic")

    # Speed validation
    if 0 <= avg_speed <= 25: # km/h
        print(f" [OK] Average speed ({avg_speed:.1f} km/h) is realistic")
    else:
        print(f" [W] Average speed ({avg_speed:.1f} km/h) seems
unrealistic")

    if 0 <= max_speed <= 45: # km/h
        print(f" [OK] Max speed ({max_speed:.1f} km/h) is realistic")
    else:
        print(f" [W] Max speed ({max_speed:.1f} km/h) seems unrealistic")

    # Court coverage validation
    court_coverage = player['CourtCoverage']
    total_court_area = self.court_width * self.court_length # ~260 m²
    coverage_percentage = (court_coverage / total_court_area) * 100

    print(f" Court coverage: {court_coverage:.1f}m² ({coverage_percentage:.1f}% of
        court)")

    if 1 <= coverage_percentage <= 50:
        print(" [OK] Court coverage is realistic")
    else:
        print(" ■ Court coverage seems unrealistic")

    def analyze_player_movement_patterns(self, df, id_col='ID', x_col='X_court',
y_col='Y_court',
                                     class_col='Class', frame_col='frame'):
        """
        Analyze detailed movement patterns for each player
        """
        players = df[df[class_col] == 'Player'].copy()

        if players.empty:
            return {}

        analysis = {}

        for pid, group in players.groupby(id_col):
            group = group.sort_values(frame_col)
            coords = group[[x_col, y_col]].values

```

```

        if len(coords) < 10: # Need sufficient data points
            continue

        # Calculate movement vectors
        movements = np.diff(coords, axis=0)
        movement_magnitudes = np.linalg.norm(movements, axis=1)

        # Analyze movement directions
        movement_angles = np.arctan2(movements[:, 1], movements[:, 0])

        # Calculate acceleration (change in speed)
        speed_changes = np.diff(movement_magnitudes)

        # Identify different movement types
        stationary_threshold = 0.1 # m
        walking_threshold = 1.0 # m/frame
        running_threshold = 3.0 # m/frame

        stationary_frames = np.sum(movement_magnitudes < stationary_threshold)
        walking_frames = np.sum((movement_magnitudes >= stationary_threshold)
                                &
                                (movement_magnitudes < walking_threshold))
        running_frames = np.sum(movement_magnitudes >= walking_threshold)

        analysis[pid] = {
            'total_frames': int(len(coords)),
            'stationary_frames': int(stationary_frames),
            'walking_frames': int(walking_frames),
            'running_frames': int(running_frames),
            'dominant_direction': self._get_dominant_direction(movement_angles)
        },

        'movement_variability': float(np.std(movement_magnitudes)),
        'avg_acceleration': float(np.mean(np.abs(speed_changes))),
        'position_heat_map': self._create_position_heatmap(coords)
    }

    return analysis

def _get_dominant_direction(self, angles):
    """
    Determine the dominant movement direction
    """
    # Convert to degrees and categorize
    angles_deg = np.degrees(angles)

    # Categorize into 8 directions
    directions = []
    for angle in angles_deg:
        if -22.5 <= angle < 22.5:
            directions.append('E')
        elif 22.5 <= angle < 67.5:
            directions.append('NE')
        elif 67.5 <= angle < 112.5:
            directions.append('N')
        elif 112.5 <= angle < 157.5:
            directions.append('NW')
        elif 157.5 <= angle <= 180 or -180 <= angle < -157.5:
            directions.append('W')
        elif -157.5 <= angle < -112.5:
            directions.append('SW')

```

```

        elif -112.5 <= angle < -67.5:
            directions.append('S')
        elif -67.5 <= angle < -22.5:
            directions.append('SE')

    # Find most common direction
    if directions:
        from collections import Counter
        return Counter(directions).most_common(1)[0][0]
    return 'Unknown'

def _create_position_heatmap(self, coords):
    """
    Create a simplified position heatmap (JSON-safe)
    """
    # Divide court into grid and count positions
    x_bins = np.linspace(0, self.court_width, 11) # 10 divisions
    y_bins = np.linspace(0, self.court_length, 24) # 23 divisions

    heatmap, _, _ = np.histogram2d(coords[:, 0], coords[:, 1], bins=[x_bins,
y_bins])

    # Find the most occupied cell
    max_cell = np.unravel_index(np.argmax(heatmap), heatmap.shape)
    max_cell = tuple(int(i) for i in max_cell) # Convert to native ints

    position_spread = [float(x) for x in np.std(coords, axis=0)]
    center_of_activity = [float(x) for x in np.mean(coords, axis=0)]

    return {
        'most_occupied_region': max_cell,
        'position_spread': position_spread,
        'center_of_activity': center_of_activity
    }

def calculate_ball_distance_improved(self, df, x_col='X_court', y_col='Y_court',
    class_col='Class', frame_col='frame', fps=30.0):
    """
    Improved ball distance calculation with temporal awareness and trajectory
    estimation

    Args:
        df: DataFrame with tracking data

        x_col, y_col: Column names for court coordinates
        class_col: Column name for object classification
        frame_col: Column name for frame numbers
        fps: Video frame rate (frames per second)

    Returns:
        dict: Contains total distance, average speed, max speed, and
trajectory info
    """
    ball_data = df[df[class_col] == 'Ball'].copy()

    if ball_data.empty or len(ball_data) < 2:
        return {
            'total_distance': 0.0,
            'average_speed': 0.0,
            'max_speed': 0.0,
            'trajectory_segments': 0,
            'detection_gaps': 0,

```

```

        'valid_points': 0
    }

# Sort by frame number
ball_data = ball_data.sort_values(frame_col).reset_index(drop=True)

# Calculate time intervals between detections
frame_diffs = np.diff(ball_data[frame_col].values)
time_intervals = frame_diffs / fps # Convert to seconds

# Get coordinate differences
coords = ball_data[[x_col, y_col]].values
coord_diffs = np.diff(coords, axis=0)
frame_distances = np.linalg.norm(coord_diffs, axis=1)

# Calculate instantaneous speeds (m/s)
speeds = np.divide(frame_distances, time_intervals,
                   out=np.zeros_like(frame_distances),
                   where=time_intervals != 0)

# Filter out unrealistic speeds
max_realistic_speed = 70.0 # m/s
valid_speed_mask = (speeds > 0) & (speeds <= max_realistic_speed)

# Detect gaps in detection (where frame difference > 1)
detection_gaps = np.sum(frame_diffs > 1)

# Handle gaps by estimating trajectory
total_distance = 0.0
trajectory_segments = 0

for i in range(len(frame_distances)):
    if valid_speed_mask[i]:

        if frame_diffs[i] == 1:
            # Consecutive frames - use direct distance
            total_distance += frame_distances[i]
        else:
            # Gap in detection - estimate trajectory
            gap_frames = frame_diffs[i]
            gap_time = time_intervals[i]

            # Simple linear interpolation for gaps
            estimated_distance = frame_distances[i]
            total_distance += estimated_distance

print(f"INFO Gap detected {gap_frames} frames, estimated distance
      {estimated_distance:.2f}m")

    trajectory_segments += 1

# Calculate statistics
valid_speeds = speeds[valid_speed_mask]
average_speed = np.mean(valid_speeds) if len(valid_speeds) > 0 else 0.0
max_speed = np.max(valid_speeds) if len(valid_speeds) > 0 else 0.0

# Convert speeds to km/h for reporting
average_speed_kmh = average_speed * 3.6
max_speed_kmh = max_speed * 3.6

print(f"BALL ANALYSIS Total distance {total_distance:.2f}m")
print(f"BALL ANALYSIS Average speed {average_speed_kmh:.1f} km/h")

```

```

print(f"BALL ANALYSIS Max speed {max_speed_kmh:.1f} km/h")
print(f"BALL ANALYSIS Detection gaps {detection_gaps}")
print(f"BALL ANALYSIS Valid trajectory segments {trajectory_segments}")

return {
    'total_distance': total_distance,
    'average_speed': average_speed,
    'max_speed': max_speed,
    'average_speed_kmh': average_speed_kmh,
    'max_speed_kmh': max_speed_kmh,
    'trajectory_segments': trajectory_segments,
    'detection_gaps': detection_gaps,
    'valid_points': len(ball_data),
    'filtered_unrealistic': np.sum(~valid_speed_mask)
}

def calculate_ball_distance_with_trajectory_estimation(self, df, x_col='X_court',
    y_col='Y_court',
    class_col='Class', frame_col='frame',
    fps=30.0):
    """
    Advanced ball distance calculation with parabolic trajectory estimation
    """
    ball_data = df[df[class_col] == 'Ball'].copy()

    if ball_data.empty or len(ball_data) < 3:
        return self.calculate_ball_distance_improved(df, x_col, y_col, class_col,
            frame_col,
            fps)

    ball_data = ball_data.sort_values(frame_col).reset_index(drop=True)

    total_distance = 0.0
    coords = ball_data[[x_col, y_col]].values
    frames = ball_data[frame_col].values

    i = 0
    while i < len(coords) - 1:
        current_frame = frames[i]
        next_frame = frames[i + 1]

        if next_frame - current_frame == 1:
            # Consecutive frames - direct distance
            distance = np.linalg.norm(coords[i + 1] - coords[i])
            total_distance += distance
            i += 1
        else:
            # Gap in detection - try to estimate trajectory
            gap_size = next_frame - current_frame

            if gap_size <= 5 and i + 1 < len(coords): # Only estimate for
small gaps
                # Use parabolic trajectory estimation
                p1 = coords[i]
                p2 = coords[i + 1]

                # Estimate trajectory length (accounting for parabolic path)
                straight_distance = np.linalg.norm(p2 - p1)

                # Approximate parabolic path as 1.1-1.3 times straight
distance

```

```

        trajectory_factor = 1.2 # Conservative estimate
        estimated_distance = straight_distance * trajectory_factor

        total_distance += estimated_distance

    print(f"TRAJECTORY Gap of {gap_size} frames, estimated distance
          {estimated_distance:.2f}m")
    else:
        # Large gap - use direct distance
        distance = np.linalg.norm(coords[i + 1] - coords[i])
        total_distance += distance

        print(f"LARGE GAP {gap_size} frames, using direct distance
              {distance:.2f}m")

        i += 1

    return {
        'total_distance': total_distance,
        'method': 'trajectory_estimation',
        'gaps_estimated': True
    }

def validate_ball_measurements(self, ball_results,
    rally_duration_seconds=None):
    """
    Validate ball measurements against realistic tennis expectations
    """
    print("\n=== BALL MEASUREMENT VALIDATION ===")

    total_distance = ball_results['total_distance']
    avg_speed = ball_results.get('average_speed_kmh', 0)
    max_speed = ball_results.get('max_speed_kmh', 0)

    # Tennis ball physics validation
    print(f"Total ball distance {total_distance:.2f}m")

    if rally_duration_seconds:
        rally_average_speed = (total_distance / rally_duration_seconds) * 3.6
        print(f"Rally average speed {rally_average_speed:.1f} km/h")

        # Realistic ranges for tennis
        if 10 <= rally_average_speed <= 150:
            print("Rally average speed is realistic")
        else:
            print("Rally average speed seems unrealistic")

    if avg_speed > 0:
        print(f"Ball average speed {avg_speed:.1f} km/h")
        if 20 <= avg_speed <= 200:
            print("Average speed is realistic")
        else:
            print("Average speed seems unrealistic")

    if max_speed > 0:
        print(f"Ball max speed {max_speed:.1f} km/h")
        if 50 <= max_speed <= 250:
            print("Max speed is realistic")
        else:
            print("Max speed seems unrealistic")

    # Distance validation

```



```

        if total_distance > 0:
            if 50 <= total_distance <= 2000: # Typical rally distances
                print("Total distance is realistic")
            else:

                print("Total distance seems unrealistic")

        return ball_results

    def debug_coordinates(self, df, x_col='X_court', y_col='Y_court',
sample_size=10):
        print("Tennis Court Coordinate Statistics:")
        print(f"X_court range: {df[x_col].min():.3f} to {df[x_col].max():.3f}
meters")
        print(f"Y_court range: {df[y_col].min():.3f} to {df[y_col].max():.3f}
meters")
        print(f"Expected court dimensions: {self.court_width}m x
{self.court_length}m")
        print(f"\nSample of court coordinates:")
        print(df[['frame', x_col, y_col, 'Class']].head(sample_size))
        x_outside = df[(df[x_col] < -2) | (df[x_col] > self.court_width + 2)]
        y_outside = df[(df[y_col] < -2) | (df[y_col] > self.court_length + 2)]
        if not x_outside.empty:
            print(f"\nWARNING: {len(x_outside)} points outside court X
boundaries")
        if not y_outside.empty:
            print(f"WARNING: {len(y_outside)} points outside court Y boundaries")
        if df[x_col].max() - df[x_col].min() > 50:
            print("WARNING: X coordinate range seems too large")
        if df[y_col].max() - df[y_col].min() > 50:
            print("WARNING: Y coordinate range seems too large")

def main_video_processing_pipeline(video_path, data_csv_path,
sample_interval=1.0, max_frames=30):
    """
    Main processing pipeline for video-based corner detection

    Args:
        video_path: Path to video file or URL
        data_csv_path: Path to CSV file with tracking data
        sample_interval: Interval in seconds between frame samples
        max_frames: Maximum number of frames to process
    """
    tracker = TennisCourtTracker()

    print("=== STEP 1: Detecting Court Corners from Video ===")
    try:
        # Fixed: Now unpacking 4 values instead of 3
        average_corners, all_frame_corners, frame_info, representative_frame =
            tracker.detect_corners_from_video(
                video_path, sample_interval=sample_interval, max_frames=max_frames,
debug=True
            )

        if average_corners is None:
            print("Error: Could not detect corners from video")
            return

        print(f"Successfully calculated average corners from
{len(all_frame_corners)} frames")

```

```

        # Print summary statistics
        valid_frames = sum(1 for info in frame_info if info['valid'])
        print(f"Frame processing summary: {valid_frames}/{len(frame_info)} frames had
valid
        corners")

        # Optional: Visualize the average corners on the representative frame
        if representative_frame is not None:
            print("=== STEP 1.5: Visualizing Average Corners ===")
            # Extract directory and filename from data_csv_path to match the Out
            folder structure
            import os
            csv_dir = os.path.dirname(data_csv_path) # Gets './Out'
            csv_filename = os.path.basename(data_csv_path) # Gets '
nadal_verdasco_init_df.csv'
            image_filename = csv_filename.replace('.csv', '_average_corners.jpg')
            image_output_path = os.path.join(csv_dir, image_filename)

            tracker.visualize_average_corners(
                representative_frame,
                average_corners,
                output_path=image_output_path,
                show_image=False # Set to True if you want to display the image
            )

        except Exception as e:
            print(f"Error processing video: {e}")
            return

        print("=== STEP 2: Computing Homography with Average Corners ===")
        try:
            H = tracker.compute_homography(average_corners)
            print("Homography matrix computed successfully using average corners")
            is_valid = tracker.validate_homography(H, average_corners)
            if not is_valid:
                print("WARNING: Homography validation failed")
        except Exception as e:
            print(f"Error computing homography: {e}")
            return

        print("=== STEP 3: Loading and Processing Tracking Data ===")
        df = pd.read_csv(data_csv_path)
        print(f"Loaded {len(df)} tracking points")
        df_mapped = tracker.map_pixels_to_court(df, H)
        tracker.debug_coordinates(df_mapped)

        print("=== STEP 4: Calculating Distances ===")
        player_distances = tracker.calculate_player_distances_improved(df_mapped)
        print("Player distances:")
        print(player_distances)

        ball_results = tracker.calculate_ball_distance_improved(df_mapped, fps=30.0)
        tracker.validate_ball_measurements(ball_results)

        #print(f"\nBall total distance: {ball_distance:.2f} meters")

        # Save results
        output_path = data_csv_path.replace('.csv', '_with_video_court_coords.csv')
        df_mapped.to_csv(output_path, index=False, encoding='utf-8')
        print(f"\nResults saved to: {output_path}")

```

```

# Save corner analysis results
corner_analysis_path = data_csv_path.replace('.csv', '_corner_analysis.csv')
corner_df = pd.DataFrame(frame_info)
corner_df.to_csv(corner_analysis_path, index=False, encoding='utf-8')
print(f"Corner analysis saved to: {corner_analysis_path}")

return df_mapped, player_distances, ball_results, average_corners, frame_info

def main_video_processing_pipeline_dynamic(video_path, data_csv_path,
sample_interval=1.0,
max_frames=30):
    """
    Main processing pipeline with dynamic homography updates
    """
    tracker = TennisCourtTracker()

    print("=== STEP 1: Processing Video with Dynamic Homography ===")
    try:
        # Get frame analysis (keep existing corner detection)
        average_corners, all_frame_corners, frame_info, representative_frame =
            tracker.detect_corners_from_video(
                video_path, sample_interval=sample_interval, max_frames=max_frames,
debug=True
            )

        if average_corners is None:
            print("Error: Could not detect corners from video")
            return

    except Exception as e:
        print(f"Error processing video: {e}")
        return

    print("=== STEP 2: Setting up Dynamic Homography Processing ===")
    homography_manager = HomographyManager(tracker)

    # Initialize with first valid frame
    for info in frame_info:
        if info['valid']:
            initial_homography = homography_manager.update_homography_if_needed(
                info['corners'], info['frame_number']
            )
            break

    print("=== STEP 3: Loading and Processing Tracking Data with Dynamic
Homography ===")

    df = pd.read_csv(data_csv_path)
    print(f"Loaded {len(df)} tracking points")

    # Process data with dynamic homography updates
    df_mapped = df.copy()
    current_homography = initial_homography

    # Group by frame and apply appropriate homography
    for frame_num in sorted(df['frame'].unique()):
        frame_data = df[df['frame'] == frame_num]

        # Check if we have corner data for this frame
        frame_corners = None
        for info in frame_info:

```

```

        if info['frame_number'] == frame_num and info['valid']:
            frame_corners = info['corners']
            break

    # Update homography if needed
    if frame_corners is not None:
        current_homography = homography_manager.update_homography_if_needed(
            frame_corners, frame_num
        )

    # Apply current homography to this frame's data
    if current_homography is not None:
        frame_mapped = tracker.map_pixels_to_court(frame_data,
current_homography)
    df_mapped.loc[df['frame'] == frame_num, ['X_court', 'Y_court']] =
        frame_mapped[['X_court', 'Y_court']]

    tracker.debug_coordinates(df_mapped)

    print("=== STEP 4: Calculating Distances ===")
    player_distances = tracker.calculate_player_distances_improved(df_mapped)
    print("Player distances:")
    print(player_distances)

    ball_results = tracker.calculate_ball_distance_improved(df_mapped, fps=30.0)
    tracker.validate_ball_measurements(ball_results)

    # Save results (existing saves)
    output_path = data_csv_path.replace('.csv', '_with_video_court_coords.csv')
    df_mapped.to_csv(output_path, index=False, encoding='utf-8')
    print(f"\nResults saved to: {output_path}")

    # Save corner analysis results (existing)
    corner_analysis_path = data_csv_path.replace('.csv', '_corner_analysis.csv')
    corner_df = pd.DataFrame(frame_info)
    corner_df.to_csv(corner_analysis_path, index=False, encoding='utf-8')
    print(f"Corner analysis saved to: {corner_analysis_path}")

    # NEW: Save player movement analysis
    player_analysis_path = data_csv_path.replace('.csv', '
_player_movement_analysis.csv')
    player_distances.to_csv(player_analysis_path, index=False, encoding='utf-8')
    print(f"Player movement analysis saved to: {player_analysis_path}")

    # NEW: Save ball movement analysis
    ball_analysis_path = data_csv_path.replace('.csv', '
_ball_movement_analysis.csv')
    ball_df = pd.DataFrame([ball_results]) # Convert dict to DataFrame
    ball_df.to_csv(ball_analysis_path, index=False, encoding='utf-8')
    print(f"Ball movement analysis saved to: {ball_analysis_path}")

    # NEW: Save detailed movement patterns (optional)
    movement_patterns = tracker.analyze_player_movement_patterns(df_mapped)
    if movement_patterns:
        patterns_path = data_csv_path.replace('.csv', '_movement_patterns.json')
        import json
        with open(patterns_path, 'w') as f:
            json.dump(movement_patterns, f, indent=2)
        print(f"Movement patterns saved to: {patterns_path}")

    return df_mapped, player_distances, ball_results, average_corners, frame_info

```

[FILE] tracker_custom_frame.py

Path: Distance_measurement\tracker_custom_frame.py

File is empty or could not be read.

[FOLDER] Root Directory

[FILE] generate_code_pdf.py

Path: generate_code_pdf.py

```
#!/usr/bin/env python3
"""
Script to generate a PDF document containing all Python code files
organized by folders with proper formatting and table of contents.
"""

import os
import glob
from pathlib import Path
from datetime import datetime
from reportlab.lib.pagesizes import A4
from reportlab.lib.styles import getSampleStyleSheet, ParagraphStyle
from reportlab.lib.units import inch
from reportlab.platypus import SimpleDocTemplate, Paragraph, Spacer, PageBreak,
Table, TableStyle
from reportlab.lib import colors
from reportlab.lib.enums import TA_LEFT, TA_CENTER
from reportlab.pdfgen import canvas
from reportlab.platypus.tableofcontents import TableOfContents
import sys

class NumberedCanvas(canvas.Canvas):
    def __init__(self, *args, **kwargs):
        canvas.Canvas.__init__(self, *args, **kwargs)
        self._saved_page_states = []

    def showPage(self):
        self._saved_page_states.append(dict(self.__dict__))
        self._startPage()

    def save(self):
        num_pages = len(self._saved_page_states)
        for (page_num, page_state) in enumerate(self._saved_page_states):
            self.__dict__.update(page_state)
            self.draw_page_number(page_num + 1, num_pages)
            canvas.Canvas.showPage(self)
            canvas.Canvas.save(self)

    def draw_page_number(self, self, page_num, total_pages):
        self.setFont("Helvetica", 9)
        self.drawRightString(A4[0] - 0.75*inch, 0.75*inch,
                             f"Page {page_num} of {total_pages}")

def collect_python_files():
    """Collect all Python files organized by directory."""
    base_path = Path(".")
    files_by_folder = {}

    # Get all Python files
    python_files = list(base_path.glob("**/*.py"))
```

```

for file_path in python_files:
    # Get relative path from base
    rel_path = file_path.relative_to(base_path)
    folder = str(rel_path.parent)

    if folder == ".":
        folder = "Root Directory"

    if folder not in files_by_folder:
        files_by_folder[folder] = []

    files_by_folder[folder].append(file_path)

return files_by_folder

def read_file_content(file_path):
    """Read file content with proper encoding handling."""
    try:
        with open(file_path, 'r', encoding='utf-8') as f:
            return f.read()
    except UnicodeDecodeError:
        try:
            with open(file_path, 'r', encoding='latin-1') as f:
                return f.read()
        except Exception as e:
            return f"Error reading file: {str(e)}"
    except Exception as e:
        return f"Error reading file: {str(e)}"

def create_pdf_document():
    """Create the PDF document with all code files."""

    # Collect files
    files_by_folder = collect_python_files()

    # Setup PDF
    output_filename = "Tennis_Court_Tracking_Code_Documentation.pdf"
    doc = SimpleDocTemplate(output_filename, pagesize=A4,
                           rightMargin=0.75*inch, leftMargin=0.75*inch,
                           topMargin=1*inch, bottomMargin=1*inch)

    # Setup styles
    styles = getSampleStyleSheet()

    # Custom styles
    title_style = ParagraphStyle(
        'CustomTitle',
        parent=styles['Title'],
        fontSize=24,
        spaceAfter=30,

        alignment=TA_CENTER,
        textColor=colors.darkblue
    )

    folder_style = ParagraphStyle(
        'FolderTitle',
        parent=styles['Heading1'],
        fontSize=18,
        spaceAfter=20,

```

```

        spaceBefore=30,
        textColor=colors.darkred,
        keepWithNext=1
    )

    file_style = ParagraphStyle(
        'FileTitle',
        parent=styles['Heading2'],
        fontSize=14,
        spaceAfter=12,
        spaceBefore=20,
        textColor=colors.darkgreen,
        keepWithNext=1
    )

    code_style = ParagraphStyle(
        'CodeStyle',
        parent=styles['Code'],
        fontSize=9,
        spaceAfter=12,
        fontName='Courier-Bold',
        leftIndent=15,
        rightIndent=15,
        backgroundColor=colors.lightgrey,
        leading=11,
        wordWrap='CJK'
    )

    # Build content
    story = []

    # Title page
    story.append(Paragraph("Tennis Court Tracking System", title_style))
    story.append(Spacer(1, 12))
    story.append(Paragraph("Complete Code Documentation", styles['Heading2']))
    story.append(Spacer(1, 12))
    story.append(Paragraph(f"Generated on: {datetime.now().strftime('%B %d, %Y at %H:%M')}",
        styles['Normal']))
    story.append(Spacer(1, 24))

    # Project overview

    overview_text = """
    This document contains all Python code files from the Tennis Court Tracking
    project.
    The project implements computer vision techniques for tennis court detection,
    player tracking,
    and distance measurement using YOLO object detection and DeepSORT tracking
    algorithms.

    The code is organized into the following main components:
    • Detection and Tracking modules for player and ball tracking
    • Distance Measurement modules for court mapping and coordinate
    transformation
    • Supporting utilities and helper functions
    """
    story.append(Paragraph("Project Overview", styles['Heading2']))
    story.append(Paragraph(overview_text, styles['Normal']))
    story.append(PageBreak())

    # Table of Contents

```



```

story.append(Paragraph("Table of Contents", styles['Heading1']))
story.append(Spacer(1, 12))

toc_data = [["Folder/File", "Page"]]
current_page = 3 # Starting after title and TOC

# Calculate approximate pages for TOC
for folder, files in sorted(files_by_folder.items()):
    toc_data.append([f"[FOLDER] {folder}", str(current_page)])
    current_page += 1

    for file_path in sorted(files):
        filename = file_path.name
        content = read_file_content(file_path)
        # Rough estimate: 50 lines per page
        estimated_pages = max(1, len(content.split('\n'))) // 50
        toc_data.append([f" {filename}", str(current_page)])
        current_page += estimated_pages

toc_table = Table(toc_data, colwidths=[4*inch, 1*inch])
toc_table.setStyle(TableStyle([
    ('BACKGROUND', (0, 0), (-1, 0), colors.grey),
    ('TEXTCOLOR', (0, 0), (-1, 0), colors.whitesmoke),
    ('ALIGN', (0, 0), (-1, -1), 'LEFT'),
    ('FONTNAME', (0, 0), (-1, 0), 'Helvetica-Bold'),
    ('FONTSIZE', (0, 0), (-1, 0), 12),
    ('BOTTOMPADDING', (0, 0), (-1, 0), 12),
    ('BACKGROUND', (0, 1), (-1, -1), colors.beige),
    ('GRID', (0, 0), (-1, -1), 1, colors.black)
]))

story.append(toc_table)
story.append(PageBreak())

# Add code content

for folder, files in sorted(files_by_folder.items()):
    # Folder header
    story.append(Paragraph(f"[FOLDER] {folder}", folder_style))
    story.append(Spacer(1, 12))

    if folder != "Root Directory":
        story.append(Paragraph(f"Location: {folder}", styles['Italic']))
        story.append(Spacer(1, 12))

    # Files in folder
    for file_path in sorted(files):
        filename = file_path.name
        story.append(Paragraph(f"[FILE] {filename}", file_style))
        story.append(Spacer(1, 6))

        # File path
        story.append(Paragraph(f"Path: {file_path}", styles['Italic']))
        story.append(Spacer(1, 6))

        # File content
        content = read_file_content(file_path)

        if content:
            # Process content to preserve indentation
            lines = content.split('\n')
            processed_lines = []

```

```

        for line in lines:
            # Convert tabs to 4 spaces for consistent indentation
            line = line.expandtabs(4)
            # Escape special characters for reportlab
            line = line.replace('&', '&').replace('<',
                '<').replace('>', '>')

            # Handle very long lines by breaking them intelligently
            max_line_length = 100
            if len(line) > max_line_length:
                # Try to break at logical points (spaces, commas,
operators)
                indent = len(line) - len(line.lstrip())
                while len(line) > max_line_length:
                    break_point = max_line_length
                    # Look for good break points
                    for char in [' ', ',', '(', ')', '[', ']', '{', '}', '
=, '+, '-']:
                        pos = line.rfind(char, 0, max_line_length)
                        if pos > max_line_length * 0.7: # Don't break too
early
                            break_point = pos + 1
                            break

                    processed_lines.append(line[:break_point].rstrip())
                    line = ' ' * (indent + 4) +
line[break_point:].lstrip()

                # Preserve leading spaces by converting them to non-breaking
spaces
                leading_spaces = len(line) - len(line.lstrip())
                if leading_spaces > 0:
                    line = '&nbsp;' * leading_spaces + line.lstrip()
                    processed_lines.append(line)

            # Split content into chunks to avoid reportlab issues with very
long paragraphs
            chunk_size = 50 # lines per chunk (reduced for better formatting)

            for i in range(0, len(processed_lines), chunk_size):
                chunk_lines = processed_lines[i:i+chunk_size]
                # Join lines with <br/> tags to preserve line breaks
                chunk = '<br/>'.join(chunk_lines)
                story.append(Paragraph(f"<font name='Courier' size='9'>{
                    chunk}</font>", code_style))
            else:
                story.append(Paragraph("File is empty or could not be read.",
styles['Italic']))

            story.append(Spacer(1, 20))

            story.append(PageBreak())

        # Build PDF
        print(f"Generating PDF: {output_filename}")
        doc.build(story, canvasmaker=NumberedCanvas)
        print(f"PDF generated successfully: {output_filename}")

        return output_filename

```

```

if __name__ == "__main__":
    try:
        # Check if reportlab is available
        import reportlab
        output_file = create_pdf_document()
        print(f"\nSUCCESS! PDF created: {output_file}")
        print(f"Total Python files processed:
{len(list(Path('.').glob('**/*.py')))}")

    except ImportError:
        print("ERROR: reportlab library is required to generate PDF.")
        print("Install it with: pip install reportlab")
        sys.exit(1)
    except Exception as e:
        print(f"ERROR generating PDF: {str(e)}")
        sys.exit(1)

```

[FILE] try_optimization.py

Path: try_optimization.py

```

from Time_Optimization.resolution_optimizer import ResolutionPerformanceOptimizer

# Initialize optimizer
optimizer = ResolutionPerformanceOptimizer(target_fps=30.0,
min_detection_threshold=2)

# Run interactive optimization - this will show you all available resolutions
# and let you choose which ones to test
results = optimizer.optimize_resolution_interactive(
    video_path="./test_videos/melbourne2.mp4",
    out_name="IMGSZ_test",
    teams_colors=['white', 'white', 'blue', 'blue', 'black', 'yellow'],
    ball_only=True,
    step_factor=0.8, # 20% reduction each step
    min_width=320 # Minimum width to generate
)

# Generate reports and plots
if results and 'error' not in results:
    print("\n" + "="*60)
    print("GENERATING REPORTS AND VISUALIZATIONS")
    print("="*60)

    # Generate comprehensive report
    optimizer.generate_performance_report(save_path="./Out/interactive_optimizatio
n_report.txt")

    # Create performance plots
    optimizer.plot_performance_analysis(save_path="./Out/interactive_performance_p
lots.png")

    # Create timing analysis plots
    optimizer.plot_comprehensive_timing_analysis(save_path="./Out/interactive_timi
ng_analysis.png")

    # Print final recommendation

```

```

    if results.get('recommended_resolution'):
        rec = results['recommended_resolution']
        print(f"\n FINAL RECOMMENDATION: {rec['resolution_str']}")
        print(f" Expected FPS: {rec.get('avg_fps', 0):.2f}")
        print(f" Real-time capability: {rec.get('real_time_percentage', 0):.1f}%")
        print(f" Average detections per frame:
{rec.get('avg_detections_per_frame', 0):.1f}")
        print(f" Ball detection rate: {rec.get('ball_detection_rate', 0):.1f}%")

    print(f"\nTesting completed! Check the ./Out/ folder for detailed reports and
plots.")
else:
    print("Optimization was cancelled or encountered an error.")

```

[FILE] try_roi.py

Path: try_roi.py

```

from Detect_and_Track.roi_main import *

video_path="./test_videos/melbourne2.mp4"
experiment_name="tennis_test"

success = run_fair_roi_experiment(
    video_path="./test_videos/melbourne2.mp4",
    experiment_name="tennis_test",
    teams_colors=['white', 'white', 'red', 'red', 'black', 'yellow'],
    target_player_id=2,
    ball_only=False,
    save_roi_images=True, # Enable ROI image saving
    roi_save_interval=15 # Save every 30 frames
)

'''
success1 = run_roi_only_experiment_fair(video_path, experiment_name,
teams_colors=None,
                                target_player_id=1, ball_only=False,
save_roi_images=True,
                                roi_save_interval=15)
'''
from Detect_and_Track.roi_main import quick_fair_roi_test

# Fer pore■enje
success = quick_fair_roi_test("./test_videos/melbourne2.mp4", "melbourne_fair")
'''

```

[FILE] try_tracking.py

Path: try_tracking.py

```

#from Distance_measurement.tennis_court_tracker import *
from Distance_measurement.main_court_tracker import *
from Distance_measurement.corner_detection import *
# --- PARAMETERS ---
video_path = "./test_videos/melbourne2.mp4"
out_name = "melbourne_kratak_poen"
init_df_path = f"./Out/{out_name}_init_df.csv"

# --- 1. Get initial data (creates a dataframe, usually saved as CSV) ---
# (Uncomment if you want to regenerate the initial dataframe)
from Detect_and_Track.get_init_data import get_init_data
from Detect_and_Track.get_tracks import get_video_tracks
from Detect_and_Track.create_tracking_boxes_video import
create_tracking_boxes_video
from Detect_and_Track.create_tracking_video_and_init_data import
create_tracking_video_and_init_data
teams_colors = ["red", "blue"]
ball_only = False
#get_init_data(video_path, out_name, teams_colors, ball_only)

#get_video_tracks(video_path, out_name)
#create_tracking_boxes_video(video_path, out_name)

# Process video once to get both tracking video and init CSV
init_df, ziframes, zitboxes = create_tracking_video_and_init_data(
    video_path=video_path,
    out_name=out_name,
    teams_colors=teams_colors,
    ball_only=ball_only
)

'''
# Standard pipeline (uses average corners)
results = main_video_processing_pipeline(
    video_path=video_path,
    data_csv_path=init_df_path
)
'''

'''
# Dynamic pipeline (updates homography per frame)
results = main_video_processing_pipeline_dynamic(
    video_path=video_path,
    data_csv_path=init_df_path
)
'''

'''
frame_path="./test_videos/frame_for_detection4.png"

frame = cv2.imread(frame_path)

cd=CornerDetector()
corners = cd.detect_court_corners(frame)
cd.draw_corners_on_frame(frame, corners)
cv2.imwrite("./Out/TEST4.png", frame)
'''

```


[FOLDER] Time_Optimization

Location: Time_Optimization

[FILE] __init__.py

Path: Time_Optimization__init__.py

File is empty or could not be read.

[FILE] resolution_optimizer.py

Path: Time_Optimization\resolution_optimizer.py

```
import os
import cv2
import time
import pandas as pd
import numpy as np
from typing import List, Tuple, Dict, Optional
import matplotlib.pyplot as plt
import glob

class ResolutionPerformanceOptimizer:
    """
    Module for testing tennis court tracking at multiple resolutions to find
    optimal
    performance vs accuracy balance for real-time processing.
    """

    def __init__(self, target_fps: float = 30.0, min_detection_threshold: int =
2):
        """
        Initialize the optimizer.

        Parameters
        -----
        target_fps : float
            Target FPS for real-time processing (default: 30.0)
        min_detection_threshold : int
            Minimum number of detections per frame to consider valid (default: 2)
        """
        self.target_fps = target_fps
        self.min_detection_threshold = min_detection_threshold
        self.results = []

        def generate_resolution_list(self, original_width: int, original_height: int,
step_factor: float = 0.8, min_width: int = 320) ->
            List[Tuple[int, int]]:
            """
```

down. Generate list of resolutions to test, starting from original and scaling

```
Parameters
-----
original_width : int
    Original video width
original_height : int
    Original video height
step_factor : float
    Factor to scale down resolution each step (default: 0.8 = 20%
reduction)
min_width : int
    Minimum width to test (default: 320)

Returns
-----
List[Tuple[int, int]]

    List of (width, height) tuples to test
"""
resolutions = []
current_width = original_width
current_height = original_height

while current_width >= min_width:
    # Ensure even dimensions for video encoding compatibility
    width = int(current_width // 2) * 2
    height = int(current_height // 2) * 2
    resolutions.append((width, height))

    current_width *= step_factor
    current_height *= step_factor

return resolutions

def interactive_resolution_selection(self, video_path: str, step_factor:
float = 0.8,
                                min_width: int = 320) -> List[Tuple[int,
int]]:
    """
    Interactive resolution selection allowing user to choose which
    resolutions to test.

    Parameters
    -----
    video_path : str
        Path to input video
    step_factor : float
        Resolution reduction factor between options
    min_width : int
        Minimum width to generate

    Returns
    -----
    List[Tuple[int, int]]
        List of selected resolutions to test
    """
    # Get original video dimensions
    cap = cv2.VideoCapture(video_path)
    original_width = int(cap.get(cv2.CAP_PROP_FRAME_WIDTH))
    original_height = int(cap.get(cv2.CAP_PROP_FRAME_HEIGHT))
```



```

cap.release()

print(f"\nOriginal video resolution: {original_width}x{original_height}")

# Generate all possible resolutions
all_resolutions = self.generate_resolution_list(
    original_width, original_height, step_factor, min_width
)

print(f"\nAvailable resolutions to test:")

print("=" * 50)
for i, (width, height) in enumerate(all_resolutions, 1):
    # Calculate percentage of original
    percentage = (width * height) / (original_width * original_height) *
100
    print(f"{i:2d}. {width:4d}x{height:4d} ({percentage:5.1f}% of
original)")

    print("\nResolution Selection Options:")
    print("1. Enter specific numbers (e.g., '1 3 5' to test resolutions 1, 3,
and 5)")
    print("2. Enter 'all' to test all resolutions")
    print("3. Enter 'range X-Y' to test resolutions from X to Y (e.g., 'range
1-5')")
    print("4. Enter 'top N' to test the N highest resolutions (e.g., 'top
3')")
    print("5. Enter 'bottom N' to test the N lowest resolutions (e.g., '
bottom 3')")
    print("6. Enter 'every N' to test every Nth resolution (e.g., 'every 2')")

    while True:
        try:
            user_input = input("\nEnter your selection: ").strip().lower()

            if user_input == 'all':
                selected_resolutions = all_resolutions
                print(f"Selected all {len(selected_resolutions)} resolutions
for testing.")
                break

            elif user_input.startswith('range '):
                # Parse range input (e.g., "range 1-5")
                range_part = user_input.replace('range ', '')
                if '-' in range_part:
                    start_str, end_str = range_part.split('-')
                    start_idx = int(start_str) - 1 # Convert to 0-based
indexing
                    end_idx = int(end_str) # End is exclusive in slicing

                    if 0 <= start_idx < len(all_resolutions) and 0 < end_idx <=
len(all_resolutions):
                        selected_resolutions =
all_resolutions[start_idx:end_idx]
                        print(f"Selected resolutions {start_str} to {end_str}
({len(selected_resolutions)} resolutions).")
                        break
                    else:
                        print(f"Invalid range. Please use numbers between 1 and
{len(all_resolutions)}.")
                else:
                    print("Invalid range format. Use 'range X-Y' (e.g., '

```

```

range 1-5').")

    elif user_input.startswith('top '):
        # Take top N resolutions (highest resolution)
        n = int(user_input.replace('top ', ''))
        if 1 <= n <= len(all_resolutions):
            selected_resolutions = all_resolutions[:n]
            print(f"Selected top {n} highest resolutions.")
            break

        else:
            print(f"Invalid number. Please use a number between 1 and
                    {len(all_resolutions)}.")

    elif user_input.startswith('bottom '):
        # Take bottom N resolutions (lowest resolution)
        n = int(user_input.replace('bottom ', ''))
        if 1 <= n <= len(all_resolutions):
            selected_resolutions = all_resolutions[-n:]
            print(f"Selected bottom {n} lowest resolutions.")
            break

        else:
            print(f"Invalid number. Please use a number between 1 and
                    {len(all_resolutions)}.")

    elif user_input.startswith('every '):
        # Take every Nth resolution
        n = int(user_input.replace('every ', ''))
        if n >= 1:
            selected_resolutions = all_resolutions[::n]
            print(f"Selected every {n}{'st' if n == 1 else 'nd' if n == 2 else 'rd' if
n == 3 else 'th'} resolution ({len(selected_resolutions)}
            resolutions).")
            break

        else:
            print("Invalid number. Please use a number >= 1.")

    else:
        # Parse individual numbers (e.g., "1 3 5")
        indices = [int(x.strip()) for x in user_input.split()]

        # Validate indices
        invalid_indices = [i for i in indices if i < 1 or i >
                            len(all_resolutions)]
        if invalid_indices:
            print(f"Invalid indices: {invalid_indices}. Please use numbers between 1
                    and {len(all_resolutions)}.")
            continue

        # Convert to 0-based indexing and get resolutions
        selected_indices = [i - 1 for i in indices]
        selected_resolutions = [all_resolutions[i] for i in
selected_indices]

        print(f"Selected {len(selected_resolutions)} resolutions:
{indices}")

        break

except ValueError:
    print("Invalid input. Please enter numbers, 'all', or use the
specified formats.")
except Exception as e:

```

```

        print(f"Error parsing input: {e}")

    # Display selected resolutions
    print(f"\nSelected resolutions for testing:")
    print("-" * 40)
    for i, (width, height) in enumerate(selected_resolutions, 1):
        percentage = (width * height) / (original_width * original_height) *
100
        print(f"{i:2d}. {width:4d}x{height:4d} ({percentage:5.1f}% of
original)")

    # Confirm selection
    while True:
        confirm = input(f"\nProceed with testing these {len(selected_resolutions)}
resolutions?
        (y/n): ").strip().lower()
        if confirm in ['y', 'yes']:
            break
        elif confirm in ['n', 'no']:
            print("Selection cancelled. Please run again to make a new
selection.")
            return []
        else:
            print("Please enter 'y' or 'n'.")

    return selected_resolutions

def create_resized_video(self, input_video_path: str, output_video_path: str,
                        target_resolution: Tuple[int, int]) -> str:
    """
    Create a resized version of the input video.

    Parameters
    -----
    input_video_path : str
        Path to original video
    output_video_path : str
        Path for resized video output
    target_resolution : Tuple[int, int]
        Target (width, height) for resizing

    Returns
    -----
    str
        Path to created resized video
    """
    cap = cv2.VideoCapture(input_video_path)

    # Get original video properties
    fps = int(cap.get(cv2.CAP_PROP_FPS))
    frame_count = int(cap.get(cv2.CAP_PROP_FRAME_COUNT))

    # Create video writer for resized video
    fourcc = cv2.VideoWriter_fourcc(*'mp4v')
    out = cv2.VideoWriter(output_video_path, fourcc, fps, target_resolution)

    print(f"Creating resized video: {target_resolution[0]}x{target_resolution[
1]}")
    print(f"Processing {frame_count} frames...")

```

```

frame_idx = 0
while True:
    ret, frame = cap.read()
    if not ret:
        break

    # Resize frame
    resized_frame = cv2.resize(frame, target_resolution)
    out.write(resized_frame)

    frame_idx += 1
    if frame_idx % 100 == 0:
        print(f"Processed {frame_idx}/{frame_count} frames")

cap.release()
out.release()

print(f"Resized video saved: {output_video_path}")
return output_video_path

def analyze_timing_csv(self, csv_path: str) -> Dict:
    """
    Analyze the timing CSV file to extract performance metrics.

    Parameters
    -----
    csv_path : str
        Path to the detailed_frame_timing CSV file

    Returns
    -----
    Dict
        Dictionary containing performance metrics
    """
    if not os.path.exists(csv_path):
        print(f"Warning: Timing CSV not found: {csv_path}")
        return {
            'avg_total_time': float('inf'),
            'avg_detection_time': float('inf'),
            'avg_tracking_time': float('inf'),
            'avg_fps': 0.0,
            'frame_count': 0,
            'real_time_percentage': 0.0
        }

    try:
        df = pd.read_csv(csv_path)

        # Calculate metrics
        avg_total_time = df['Total_Processing_Time (s)'].mean()
        avg_detection_time = df['Detection_Time (s)'].mean()
        avg_tracking_time = df['Tracking_Time (s)'].mean()
        frame_count = len(df)

        # Calculate average FPS (1 / avg_total_time)
        avg_fps = 1.0 / avg_total_time if avg_total_time > 0 else 0.0

        # Calculate what percentage of real-time this achieves
        real_time_percentage = (avg_fps / self.target_fps) * 100 if self.target_fps > 0
    else
        0.0

```

```

        return {
            'avg_total_time': avg_total_time,
            'avg_detection_time': avg_detection_time,
            'avg_tracking_time': avg_tracking_time,
            'avg_fps': avg_fps,
            'frame_count': frame_count,
            'real_time_percentage': real_time_percentage,
            'csv_path': csv_path
        }

    except Exception as e:
        print(f"Error analyzing timing CSV {csv_path}: {e}")
        return {
            'avg_total_time': float('inf'),
            'avg_detection_time': float('inf'),
            'avg_tracking_time': float('inf'),
            'avg_fps': 0.0,
            'frame_count': 0,
            'real_time_percentage': 0.0
        }

def count_detections_in_results(self, ziframes: List, zitboxes: List) -> Dict:
    """
    Analyze detection quality from the tracking results.

    Parameters
    -----
    ziframes : List
        List of processed frames
    zitboxes : List
        List of tracking boxes per frame

    Returns
    -----
    Dict
        Dictionary containing detection quality metrics
    """
    if not zitboxes:
        return {
            'avg_detections_per_frame': 0.0,
            'avg_player_detections': 0.0,
            'avg_ball_detections': 0.0,
            'frames_with_ball': 0,
            'ball_detection_rate': 0.0,
            'total_frames': len(ziframes)
        }

    total_detections = []
    player_detections = []
    ball_detections = []
    frames_with_ball = 0

    for frame_boxes in zitboxes:
        frame_total = len(frame_boxes)
        frame_players = len([box for box in frame_boxes if box[-1] != 32]) #
        Not ball
        frame_balls = len([box for box in frame_boxes if box[-1] == 32]) #
        Ball

        total_detections.append(frame_total)

```

```

        player_detections.append(frame_players)
        ball_detections.append(frame_balls)

        if frame_balls > 0:
            frames_with_ball += 1

    return {
        'avg_detections_per_frame': np.mean(total_detections) if
total_detections else 0.0,
        'avg_player_detections': np.mean(player_detections) if
player_detections else 0.0,
        'avg_ball_detections': np.mean(ball_detections) if ball_detections
else 0.0,
        'frames_with_ball': frames_with_ball,
        'ball_detection_rate': (frames_with_ball / len(zitboxes)) * 100 if
zitboxes else 0.0,
        'total_frames': len(ziframes)
    }

def run_resolution_test(self, video_path: str, resolution: Tuple[int, int],
                        out_name: str, teams_colors: List[str],
                        ball_only: bool = True) -> Dict:
    """
    Run tracking test at a specific resolution.

    Parameters
    -----
    video_path : str
        Path to video file
    resolution : Tuple[int, int]
        Target resolution (width, height)

    out_name : str
        Output name prefix
    teams_colors : List[str]
        Team colors for classification
    ball_only : bool
        Whether to keep only frames with ball

    Returns
    -----
    Dict
        Combined results from timing and detection analysis
    """
    # Import the tracking function from the correct location
    import sys
    import os

    # Add parent directory to path to access Detect_and_Track folder
    current_dir = os.path.dirname(os.path.abspath(__file__))
    parent_dir = os.path.dirname(current_dir)
    if parent_dir not in sys.path:
        sys.path.insert(0, parent_dir)

    try:
        from Detect_and_Track.create_tracking_video_and_init_data import
            create_tracking_video_and_init_data
    except ImportError as e:
        print(f"Import error: {e}")
        print(f"Current working directory: {os.getcwd()}")
        print(f"Python path: {sys.path}")
        print(f"Looking for: Detect_and_Track/create_tracking_video_and_init_d

```

```

ata.py")
        raise

    print(f"\n{'='*60}")
    print(f"Testing resolution: {resolution[0]}x{resolution[1]}")
    print(f"{'='*60}")

    # Create resized video if needed
    original_cap = cv2.VideoCapture(video_path)
    original_width = int(original_cap.get(cv2.CAP_PROP_FRAME_WIDTH))
    original_height = int(original_cap.get(cv2.CAP_PROP_FRAME_HEIGHT))
    original_cap.release()

    test_video_path = video_path
    if (original_width, original_height) != resolution:
        # Create resized video
        resized_video_name = f"temp_resized_{resolution[0]}x{resolution[1]}_{int(time.time())}.mp4"
        resized_video_path = f"./Out/{resized_video_name}"
        test_video_path = self.create_resized_video(video_path,
        resized_video_path, resolution)

    try:
        # Run the tracking
        start_time = time.time()
        init_df, ziframes, zitboxes = create_tracking_video_and_init_data(
            test_video_path, f"{out_name}_{resolution[0]}x{resolution[1]}",
            teams_colors, ball_only
        )
        end_time = time.time()

        # Find the timing CSV file
        timing_csv_pattern = f"./Out/detailed_frame_timing*.csv"
        timing_files = glob.glob(timing_csv_pattern)
        latest_timing_file = max(timing_files, key=os.path.getctime) if
timing_files else None

        # Analyze timing performance
        timing_metrics = self.analyze_timing_csv(latest_timing_file) if
latest_timing_file else
        {}

        # Analyze detection quality
        detection_metrics = self.count_detections_in_results(ziframes,
zitboxes)

        # Calculate overall processing time
        total_processing_time = end_time - start_time

        # Combine results
        result = {
            'resolution': resolution,
            'resolution_str': f"{resolution[0]}x{resolution[1]}",
            'total_processing_time': total_processing_time,
            'video_path': test_video_path,
            **timing_metrics,
            **detection_metrics
        }

        # Clean up temporary resized video

```

```

        if test_video_path != video_path and os.path.exists(test_video_path):
            os.remove(test_video_path)
            print(f"Cleaned up temporary video: {test_video_path}")

        return result

    except Exception as e:
        print(f"Error testing resolution {resolution}: {e}")
        # Clean up on error
        if test_video_path != video_path and os.path.exists(test_video_path):
            os.remove(test_video_path)
        return {
            'resolution': resolution,
            'resolution_str': f"{resolution[0]}x{resolution[1]}",
            'error': str(e),

            'avg_fps': 0.0,
            'real_time_percentage': 0.0
        }

    def optimize_resolution_interactive(self, video_path: str, out_name: str,
teams_colors:
        List[str],
                                ball_only: bool = True, step_factor: float
= 0.8,
                                min_width: int = 320) -> Dict:
        """
        Run optimization with interactive resolution selection.

        Parameters
        -----
        video_path : str
            Path to input video
        out_name : str
            Output name prefix
        teams_colors : List[str]
            Team colors for classification
        ball_only : bool
            Whether to keep only frames with ball
        step_factor : float
            Resolution reduction factor between options
        min_width : int
            Minimum width to generate

        Returns
        -----
        Dict
            Optimization results with recommendations
        """
        print("INTERACTIVE RESOLUTION OPTIMIZATION")
        print("=" * 50)

        # Interactive resolution selection
        selected_resolutions = self.interactive_resolution_selection(
            video_path, step_factor, min_width
        )

        if not selected_resolutions:
            return {"error": "No resolutions selected"}

        print(f"\nStarting optimization with {len(selected_resolutions)} selected
        resolutions...")

```



```

# Test each selected resolution
results = []
for i, resolution in enumerate(selected_resolutions, 1):
    print(f"\nProgress: {i}/{len(selected_resolutions)}")
    result = self.run_resolution_test(
        video_path, resolution, out_name, teams_colors, ball_only

    )
    results.append(result)

# Print summary for this resolution
if 'error' not in result:
    print(f"\nResults for {result['resolution_str']}:")
    print(f" Average FPS: {result.get('avg_fps', 0):.2f}")
    print(f" Real-time %: {result.get('real_time_percentage',
0):.1f}%")
    print(f" Avg detections/frame:
{result.get('avg_detections_per_frame', 0):.1f}")
    print(f" Ball detection rate: {result.get('ball_detection_rate',
0):.1f}%")
    else:
        print(f"\nError for {result['resolution_str']}:
{result['error']}")

self.results = results

# Analyze results and make recommendations
return self.analyze_optimization_results()

# [Include all other methods from the original class -
analyze_optimization_results,
# generate_performance_report, plot_performance_analysis, etc.]
# ... (keeping the rest of the methods unchanged for brevity)

def extract_detailed_timing_statistics(self, csv_path: str) -> Dict:
    """
    Extract detailed timing statistics from CSV file including median and
    standard deviation.

    Parameters
    -----
    csv_path : str
        Path to the timing CSV file

    Returns
    -----
    Dict
        Detailed timing statistics
    """
    if not os.path.exists(csv_path):
        return {
            'total_time_stats': {'mean': 0, 'median': 0, 'std': 0},
            'detection_time_stats': {'mean': 0, 'median': 0, 'std': 0},
            'tracking_time_stats': {'mean': 0, 'median': 0, 'std': 0},
            'frame_count': 0,
            'total_processing_time': 0,
            'total_detection_time': 0,
            'total_tracking_time': 0
        }
    try:

```

```

df = pd.read_csv(csv_path)

# Per-frame statistics
total_times = df['Total_Processing_Time (s)'].values
detection_times = df['Detection_Time (s)'].values
tracking_times = df['Tracking_Time (s)'].values

# Calculate statistics
total_time_stats = {
    'mean': np.mean(total_times),
    'median': np.median(total_times),
    'std': np.std(total_times)
}

detection_time_stats = {
    'mean': np.mean(detection_times),
    'median': np.median(detection_times),
    'std': np.std(detection_times)
}

tracking_time_stats = {
    'mean': np.mean(tracking_times),
    'median': np.median(tracking_times),
    'std': np.std(tracking_times)
}

# Total times
frame_count = len(df)
total_processing_time = np.sum(total_times)
total_detection_time = np.sum(detection_times)
total_tracking_time = np.sum(tracking_times)

return {
    'total_time_stats': total_time_stats,
    'detection_time_stats': detection_time_stats,
    'tracking_time_stats': tracking_time_stats,
    'frame_count': frame_count,
    'total_processing_time': total_processing_time,
    'total_detection_time': total_detection_time,
    'total_tracking_time': total_tracking_time
}

except Exception as e:
    print(f"Error extracting detailed statistics from {csv_path}: {e}")
    return {
        'total_time_stats': {'mean': 0, 'median': 0, 'std': 0},
        'detection_time_stats': {'mean': 0, 'median': 0, 'std': 0},
        'tracking_time_stats': {'mean': 0, 'median': 0, 'std': 0},
        'frame_count': 0,
        'total_processing_time': 0,
        'total_detection_time': 0,
        'total_tracking_time': 0
    }

}

def analyze_optimization_results(self) -> Dict:
    """
    Analyze the optimization results and provide recommendations.

    Returns
    -----

```

```

Dict
    Analysis results with recommendations
    """
    if not self.results:
        return {"error": "No results to analyze"}

    # Filter out error results
    valid_results = [r for r in self.results if 'error' not in r and
r.get('avg_fps', 0) > 0]

    if not valid_results:
        return {"error": "No valid results found"}

    # Find best options based on different criteria
    best_fps = max(valid_results, key=lambda x: x.get('avg_fps', 0))
    best_real_time = max(valid_results, key=lambda x:
x.get('real_time_percentage', 0))
    best_detections = max(valid_results, key=lambda x:
x.get('avg_detections_per_frame', 0))

    # Find resolutions that achieve real-time or close to it
    real_time_candidates = [
        r for r in valid_results
        if r.get('real_time_percentage', 0) >= 80 # Within 80% of target FPS
    ]

    # Find balanced option (good FPS with reasonable detection quality)
    balanced_candidates = [
        r for r in valid_results
        if (r.get('real_time_percentage', 0) >= 60 and
            r.get('avg_detections_per_frame', 0) >=
self.min_detection_threshold)
    ]

    recommended = None
    if real_time_candidates:
        # Choose the highest quality among real-time candidates
        recommended = max(real_time_candidates, key=lambda x:
x.get('avg_detections_per_frame',
0))
    elif balanced_candidates:
        # Choose the fastest among balanced candidates
        recommended = max(balanced_candidates, key=lambda x: x.get('avg_fps',
0))
    else:
        # Fall back to best FPS
        recommended = best_fps

    return {
        'total_resolutions_tested': len(self.results),
        'valid_results_count': len(valid_results),
        'target_fps': self.target_fps,

        'best_fps': best_fps,
        'best_real_time_percentage': best_real_time,
        'best_detection_quality': best_detections,

        'real_time_candidates': real_time_candidates,
        'balanced_candidates': balanced_candidates,

        'recommended_resolution': recommended,

```

```

        'all_results': valid_results
    }
def extract_comprehensive_timing_data(self) -> Dict:
    """
    Extract comprehensive timing data from all tested resolutions.

    Returns
    -----
    Dict
        Comprehensive timing analysis
    """
    timing_analysis = {
        'resolutions': [],
        'resolution_strings': [],
        'avg_time_per_frame': [],
        'total_processing_time': [],
        'avg_detection_time': [],
        'avg_tracking_time': [],
        'total_detection_time': [],
        'total_tracking_time': [],
        'frame_counts': [],
        'estimated_fps': []
    }

    for result in self.results:
        if 'error' in result:
            continue

        resolution_str = result.get('resolution_str', 'Unknown')
        resolution = result.get('resolution', (0, 0))

        # Extract timing data
        avg_total_time = result.get('avg_total_time', 0)
        avg_detection_time = result.get('avg_detection_time', 0)
        avg_tracking_time = result.get('avg_tracking_time', 0)
        frame_count = result.get('frame_count', 0)

        # Calculate totals
        total_detection = avg_detection_time * frame_count if frame_count > 0
    else 0
        total_tracking = avg_tracking_time * frame_count if frame_count > 0
    else 0
        total_processing = avg_total_time * frame_count if frame_count > 0
    else 0

        # Calculate FPS
        fps = 1.0 / avg_total_time if avg_total_time > 0 else 0

        timing_analysis['resolutions'].append(resolution)
        timing_analysis['resolution_strings'].append(resolution_str)
        timing_analysis['avg_time_per_frame'].append(avg_total_time)
        timing_analysis['total_processing_time'].append(total_processing)
        timing_analysis['avg_detection_time'].append(avg_detection_time)
        timing_analysis['avg_tracking_time'].append(avg_tracking_time)
        timing_analysis['total_detection_time'].append(total_detection)
        timing_analysis['total_tracking_time'].append(total_tracking)
        timing_analysis['frame_counts'].append(frame_count)
        timing_analysis['estimated_fps'].append(fps)

    return timing_analysis

```

```

def analyze_optimization_results(self) -> Dict:
    """
    Analyze the optimization results and provide recommendations.

    Returns
    -----
    Dict
        Analysis results with recommendations
    """
    if not self.results:
        return {"error": "No results to analyze"}

    # Filter out error results
    valid_results = [r for r in self.results if 'error' not in r and
r.get('avg_fps', 0) > 0]

    if not valid_results:
        return {"error": "No valid results found"}

    # Find best options based on different criteria
    best_fps = max(valid_results, key=lambda x: x.get('avg_fps', 0))
    best_real_time = max(valid_results, key=lambda x:
x.get('real_time_percentage', 0))
    best_detections = max(valid_results, key=lambda x:
x.get('avg_detections_per_frame', 0))

    # Find resolutions that achieve real-time or close to it
    real_time_candidates = [
        r for r in valid_results
        if r.get('real_time_percentage', 0) >= 80 # Within 80% of target FPS
    ]

    # Find balanced option (good FPS with reasonable detection quality)
    balanced_candidates = [
        r for r in valid_results
        if (r.get('real_time_percentage', 0) >= 60 and
            r.get('avg_detections_per_frame', 0) >=
self.min_detection_threshold)
    ]

    recommended = None
    if real_time_candidates:
        # Choose the highest quality among real-time candidates
        recommended = max(real_time_candidates, key=lambda x:
x.get('avg_detections_per_frame',
0))
    elif balanced_candidates:
        # Choose the fastest among balanced candidates
        recommended = max(balanced_candidates, key=lambda x: x.get('avg_fps',
0))
    else:
        # Fall back to best FPS
        recommended = best_fps

    return {
        'total_resolutions_tested': len(self.results),
        'valid_results_count': len(valid_results),
        'target_fps': self.target_fps,

        'best_fps': best_fps,
        'best_real_time_percentage': best_real_time,

```

```

        'best_detection_quality': best_detections,

        'real_time_candidates': real_time_candidates,
        'balanced_candidates': balanced_candidates,

        'recommended_resolution': recommended,
        'all_results': valid_results
    }

def generate_performance_report(self, save_path: Optional[str] = None) -> str:
    """
    Generate a detailed performance report.

    Parameters
    -----
    save_path : Optional[str]
        Path to save the report (if None, prints to console)

    Returns
    -----
    str
        The generated report
    """
    analysis = self.analyze_optimization_results()

    if 'error' in analysis:
        return f"Error generating report: {analysis['error']}"

    report = []
    report.append("="*80)
    report.append("TENNIS COURT TRACKING - RESOLUTION OPTIMIZATION REPORT")
    report.append("="*80)
    report.append(f"Target FPS for real-time processing: {self.target_fps}")
    report.append(f"Total resolutions tested: {analysis['total_resolutions_tested']}")
    report.append(f"Valid results: {analysis['valid_results_count']}")
    report.append("")

    # Recommended resolution
    if analysis['recommended_resolution']:
        rec = analysis['recommended_resolution']
        report.append("RECOMMENDED RESOLUTION:")
        report.append("-" * 30)
        report.append(f"Resolution: {rec['resolution_str']}")
        report.append(f"Average FPS: {rec.get('avg_fps', 0):.2f}")
        report.append(f"Real-time percentage: {rec.get('real_time_percentage', 0):.1f}%")
        report.append(f"Average detections per frame: {rec.get('avg_detections_per_frame', 0):.1f}")
        report.append(f"Ball detection rate: {rec.get('ball_detection_rate', 0):.1f}%")
        report.append("")

    # Real-time candidates
    if analysis['real_time_candidates']:
        report.append("REAL-TIME CAPABLE RESOLUTIONS:")
        report.append("-" * 40)
        for candidate in analysis['real_time_candidates']:
            report.append(f"{candidate['resolution_str']} - {candidate.get('avg_fps', 0):.2f}")

```

```

        FPS")
        report.append("")

# Detailed per-resolution analysis
report.append("DETAILED RESOLUTION ANALYSIS:")
report.append("="*80)
report.append("")

for i, result in enumerate(self.results):
    if 'error' in result:
        report.append(f"RESOLUTION {i+1}: {result['resolution_str']} -
ERROR")
        report.append("-" * 60)
        report.append(f"Error: {result['error']}")
        report.append("")
        continue

# Get detailed timing statistics
csv_path = result.get('csv_path')

detailed_stats = self.extract_detailed_timing_statistics(csv_path) if
csv_path else None

resolution_str = result.get('resolution_str', 'Unknown')
report.append(f"RESOLUTION {i+1}: {resolution_str}")
report.append("-" * 60)

# Basic performance metrics
fps = result.get('avg_fps', 0)
real_time_pct = result.get('real_time_percentage', 0)
detections = result.get('avg_detections_per_frame', 0)
ball_rate = result.get('ball_detection_rate', 0)

report.append(f"Performance Summary:")
report.append(f" - Average FPS: {fps:.2f}")
report.append(f" - Real-time capability: {real_time_pct:.1f}%")
report.append(f" - Average detections per frame: {detections:.1f}")
report.append(f" - Ball detection rate: {ball_rate:.1f}%")
report.append("")

# Detailed timing analysis
if detailed_stats:
    frame_count = detailed_stats['frame_count']

    report.append(f"Frame Count: {frame_count}")
    report.append("")

    # Per-frame timing statistics
    report.append("PER-FRAME TIMING STATISTICS:")
    report.append(" Total Processing Time per Frame:")
report.append(f" - Mean: {detailed_stats['total_time_stats']['mean']:.4f}
seconds")
report.append(f" - Median: {detailed_stats['total_time_stats']['median']:.4f}
seconds")
report.append(f" - Std Dev: {detailed_stats['total_time_stats']['std']:.4f}
seconds")
    report.append("")

    report.append(" Detection Time per Frame:")
report.append(f" - Mean: {detailed_stats['detection_time_stats']['
'mean']:.4f} seconds")
report.append(f" - Median: {detailed_stats['detection_time_stats']['

```

```

        'median']:.4f} seconds")
report.append(f" - Std Dev: {detailed_stats['detection_time_stats']['std']:.4f}
seconds")
report.append("")

report.append(" Tracking Time per Frame:")
report.append(f" - Mean: {detailed_stats['tracking_time_stats']['mean']:.4f}
seconds")
report.append(f" - Median: {detailed_stats['tracking_time_stats']["

        'median']:.4f} seconds")
report.append(f" - Std Dev: {detailed_stats['tracking_time_stats']['std']:.4f}
seconds")
report.append("")

# Total timing for all frames
report.append("TOTAL PROCESSING TIME FOR ALL FRAMES:")
report.append(f" - Total Processing Time: {detailed_stats[
'total_processing_time']:.2f} seconds")
report.append(f" - Total Detection Time: {detailed_stats[
'total_detection_time']:.2f} seconds")
report.append(f" - Total Tracking Time: {detailed_stats[
'total_tracking_time']:.2f} seconds")
report.append("")

# Percentage breakdown
if detailed_stats['total_processing_time'] > 0:
det_pct = (detailed_stats['total_detection_time'] /
detailed_stats['total_processing_time']) * 100
track_pct = (detailed_stats['total_tracking_time'] /
detailed_stats['total_processing_time']) * 100
other_pct = 100 - det_pct - track_pct

report.append("TIME DISTRIBUTION:")
report.append(f" - Detection: {det_pct:.1f}%")
report.append(f" - Tracking: {track_pct:.1f}%")
report.append(f" - Other: {other_pct:.1f}%")
report.append("")
else:
# Fallback to basic timing info
avg_total_time = result.get('avg_total_time', 0)
avg_detection_time = result.get('avg_detection_time', 0)
avg_tracking_time = result.get('avg_tracking_time', 0)
frame_count = result.get('frame_count', 0)

report.append(f"Frame Count: {frame_count}")
report.append("")
report.append("BASIC TIMING INFORMATION:")
report.append(f" - Average total time per frame:
{avg_total_time:.4f} seconds")
report.append(f" - Average detection time per frame: {avg_detection_time:.4f}
seconds")
report.append(f" - Average tracking time per frame: {avg_tracking_time:.4f}
seconds")

if frame_count > 0:
total_proc = avg_total_time * frame_count
total_det = avg_detection_time * frame_count
total_track = avg_tracking_time * frame_count

report.append("")

```



```

seconds")
        report.append(f" - Total processing time: {total_proc:.2f}
seconds")
        report.append(f" - Total detection time: {total_det:.2f}
seconds")
        report.append(f" - Total tracking time: {total_track:.2f}
seconds")
        report.append("")

        report.append("="*60)
        report.append("")

        # Add comprehensive timing analysis
        timing_data = self.extract_comprehensive_timing_data()
        if timing_data['resolutions']:
            report.append("COMPREHENSIVE TIMING ANALYSIS:")
            report.append("=" * 50)

            # Summary statistics
            total_resolutions = len(timing_data['resolutions'])
            avg_fps_all = np.mean(timing_data['estimated_fps']) if
timing_data['estimated_fps']
            else 0
            fastest_idx = np.argmax(timing_data['estimated_fps']) if
timing_data['estimated_fps']
            else 0
            slowest_idx = np.argmin(timing_data['estimated_fps']) if
timing_data['estimated_fps']
            else 0

            report.append(f"Total resolutions tested: {total_resolutions}")
            report.append(f"Average FPS across all resolutions:
{avg_fps_all:.2f}")
            report.append(f"Fastest resolution: {timing_data['resolution_strings'][fastest_id
x]}
({timing_data['estimated_fps'][fastest_idx]:.2f} FPS)")
            report.append(f"Slowest resolution: {timing_data['resolution_strings'][slowest_id
x]}
({timing_data['estimated_fps'][slowest_idx]:.2f} FPS)")
            report.append("")

            # Detailed timing table
            report.append("DETAILED TIMING BREAKDOWN:")
            report.append("-" * 80)
            report.append(f"{'Resolution':<12} {'Frames':<8} {'Avg/Frame(s)':<12}
{'Total(s)':<10} {'Det/Frame(s)':<12} {'Track/Frame(s)':<13}
{'FPS':<8}")
            report.append("-" * 80)

            for i in range(len(timing_data['resolution_strings'])):
                res_str = timing_data['resolution_strings'][i]
                frames = timing_data['frame_counts'][i]
                avg_frame = timing_data['avg_time_per_frame'][i]
                total_time = timing_data['total_processing_time'][i]
                avg_det = timing_data['avg_detection_time'][i]
                avg_track = timing_data['avg_tracking_time'][i]
                fps = timing_data['estimated_fps'][i]

            report.append(f"{'res_str':<12} {'frames':<8} {'avg_frame':<12.4f}
{'total_time':<10.2f} {'avg_det':<12.4f} {'avg_track':<13.4f}
{'fps':<8.2f}")

```

```

report.append("")

# Total time breakdown
report.append("TOTAL TIME BREAKDOWN BY COMPONENT:")
report.append("-" * 50)
report.append(f"{'Resolution':<12} {'Total Detect(s)':<15} {'Total Track(s)':<14} {'Total Process(s)':<15}")
report.append("-" * 50)

for i in range(len(timing_data['resolution_strings'])):
    res_str = timing_data['resolution_strings'][i]
    total_det = timing_data['total_detection_time'][i]
    total_track = timing_data['total_tracking_time'][i]
    total_proc = timing_data['total_processing_time'][i]

report.append(f"{'res_str':<12} {'total_det':<15.2f} {'total_track':<14.2f} {'total_proc':<15.2f}")

report.append("")

# Performance insights
report.append("PERFORMANCE INSIGHTS:")
report.append("-" * 30)

if len(timing_data['estimated_fps']) >= 2:
    fps_improvement = timing_data['estimated_fps'][-1] /
    timing_data['estimated_fps'][
        0] if timing_data['estimated_fps'][0] > 0 else 0
    report.append(f"FPS improvement (lowest vs highest res):
{fps_improvement:.2f}x")

# Calculate detection vs tracking time ratios
avg_det_ratio = np.mean([timing_data['avg_detection_time'][i] /
    timing_data['avg_time_per_frame'][i]
    for i in
range(len(timing_data['avg_time_per_frame']))
    if timing_data['avg_time_per_frame'][i] >
0]) * 100
avg_track_ratio = np.mean([timing_data['avg_tracking_time'][i] /
    timing_data['avg_time_per_frame'][i]
    for i in
range(len(timing_data['avg_time_per_frame']))
    if timing_data['avg_time_per_frame'][i]
> 0]) * 100

report.append(f"Average time spent on detection:
{avg_det_ratio:.1f}%")
report.append(f"Average time spent on tracking:
{avg_track_ratio:.1f}%")

report.append("")

report_text = "\n".join(report)

if save_path:
    with open(save_path, 'w') as f:
        f.write(report_text)
    print(f"Report saved to: {save_path}")
else:
    print(report_text)

```

```

        return report_text

def plot_performance_analysis(self, save_path: Optional[str] = None):
    """
    Create visualization plots of the performance analysis.

    Parameters
    -----
    save_path : Optional[str]
        Path to save the plot (if None, displays interactively)
    """
    analysis = self.analyze_optimization_results()

    if 'error' in analysis or not analysis['all_results']:
        print("No valid results to plot")
        return

    results = analysis['all_results']

    # Extract data for plotting
    resolutions = [r['resolution_str'] for r in results]
    fps_values = [r.get('avg_fps', 0) for r in results]
    detection_values = [r.get('avg_detections_per_frame', 0) for r in results]
    ball_detection_rates = [r.get('ball_detection_rate', 0) for r in results]

    # Create subplots
    fig, ((ax1, ax2), (ax3, ax4)) = plt.subplots(2, 2, figsize=(15, 10))
    fig.suptitle('Tennis Court Tracking - Resolution Performance Analysis',
    fontsize=16)

    # Plot 1: FPS vs Resolution
    ax1.bar(resolutions, fps_values, color='skyblue', alpha=0.7)
    ax1.axhline(y=self.target_fps, color='red', linestyle='--', label=f'Target FPS
    ({self.target_fps})')
    ax1.set_title('Average FPS by Resolution')
    ax1.set_ylabel('FPS')
    ax1.set_xlabel('Resolution')
    ax1.tick_params(axis='x', rotation=45)
    ax1.legend()
    ax1.grid(True, alpha=0.3)

    # Plot 2: Detection Quality vs Resolution
    ax2.bar(resolutions, detection_values, color='lightgreen', alpha=0.7)
    ax2.axhline(y=self.min_detection_threshold, color='orange',
    linestyle='--',
    label=f'Min Threshold ({self.min_detection_threshold})')
    ax2.set_title('Average Detections per Frame')

    ax2.set_ylabel('Detections per Frame')
    ax2.set_xlabel('Resolution')
    ax2.tick_params(axis='x', rotation=45)
    ax2.legend()
    ax2.grid(True, alpha=0.3)

    # Plot 3: Ball Detection Rate
    ax3.bar(resolutions, ball_detection_rates, color='coral', alpha=0.7)
    ax3.set_title('Ball Detection Rate (%)')
    ax3.set_ylabel('Ball Detection Rate (%)')
    ax3.set_xlabel('Resolution')
    ax3.tick_params(axis='x', rotation=45)
    ax3.grid(True, alpha=0.3)

```

```

# Plot 4: FPS vs Detection Quality (scatter)
ax4.scatter(detection_values, fps_values, s=100, alpha=0.7, c='purple')
for i, res in enumerate(resolutions):
    ax4.annotate(res, (detection_values[i], fps_values[i]),
                  xytext=(5, 5), textcoords='offset points', fontsize=8)
ax4.axhline(y=self.target_fps, color='red', linestyle='--', alpha=0.7)
ax4.axvline(x=self.min_detection_threshold, color='orange',
linestyle='--', alpha=0.7)
ax4.set_title('FPS vs Detection Quality')
ax4.set_xlabel('Average Detections per Frame')
ax4.set_ylabel('FPS')
ax4.grid(True, alpha=0.3)

# Highlight recommended resolution if available
if analysis['recommended_resolution']:
    rec = analysis['recommended_resolution']
    rec_res = rec['resolution_str']
    if rec_res in resolutions:
        idx = resolutions.index(rec_res)
        # Highlight in all plots
        ax1.bar(rec_res, fps_values[idx], color='gold', alpha=0.9,
label='Recommended')
        ax2.bar(rec_res, detection_values[idx], color='gold', alpha=0.9)
        ax3.bar(rec_res, ball_detection_rates[idx], color='gold',
alpha=0.9)
        ax1.legend()

plt.tight_layout()

if save_path:
    plt.savefig(save_path, dpi=300, bbox_inches='tight')
    print(f"Performance plots saved to: {save_path}")
else:
    plt.show()

def plot_comprehensive_timing_analysis(self, save_path: Optional[str] = None):
    """
    Create comprehensive timing analysis plots.

    Parameters
    -----
    save_path : Optional[str]
        Path to save the plot (if None, displays interactively)
    """
    timing_data = self.extract_comprehensive_timing_data()

    if not timing_data['resolutions']:
        print("No timing data to plot")
        return

    # Create comprehensive timing plots
    fig, ((ax1, ax2), (ax3, ax4), (ax5, ax6)) = plt.subplots(3, 2,
figsize=(16, 18))
    fig.suptitle('Comprehensive Timing Analysis - Tennis Court Tracking',
fontsize=16,
fontweight='bold')

    resolutions = timing_data['resolution_strings']

    # Plot 1: Average Time per Frame
    ax1.bar(resolutions, timing_data['avg_time_per_frame'],
color='lightblue', alpha=0.7)

```

```

ax1.set_title('Average Processing Time per Frame')
ax1.set_ylabel('Time (seconds)')
ax1.set_xlabel('Resolution')
ax1.tick_params(axis='x', rotation=45)
ax1.grid(True, alpha=0.3)

# Add value labels on bars
for i, v in enumerate(timing_data['avg_time_per_frame']):
    ax1.text(i, v + max(timing_data['avg_time_per_frame']) * 0.01,
f'{v:.3f}s',
            ha='center', va='bottom', fontsize=8)

# Plot 2: FPS Comparison
ax2.bar(resolutions, timing_data['estimated_fps'], color='lightgreen',
alpha=0.7)
ax2.axhline(y=self.target_fps, color='red', linestyle='--', label=f'Target FPS
({self.target_fps})')
ax2.set_title('Estimated FPS by Resolution')
ax2.set_ylabel('FPS')
ax2.set_xlabel('Resolution')
ax2.tick_params(axis='x', rotation=45)
ax2.legend()
ax2.grid(True, alpha=0.3)

# Add value labels
for i, v in enumerate(timing_data['estimated_fps']):
    ax2.text(i, v + max(timing_data['estimated_fps']) * 0.01, f'{v:.1f}',
            ha='center', va='bottom', fontsize=8)

# Plot 3: Detection vs Tracking Time (Stacked Bar)
ax3.bar(resolutions, timing_data['avg_detection_time'], label='Detection
Time',
        color='royalblue', alpha=0.7)

ax3.bar(resolutions, timing_data['avg_tracking_time'],
        bottom=timing_data['avg_detection_time'], label='Tracking Time',
        color='orange', alpha=0.7)
ax3.set_title('Detection vs Tracking Time per Frame')
ax3.set_ylabel('Time (seconds)')
ax3.set_xlabel('Resolution')
ax3.tick_params(axis='x', rotation=45)
ax3.legend()
ax3.grid(True, alpha=0.3)

# Plot 4: Total Processing Time
ax4.bar(resolutions, timing_data['total_processing_time'], color='gold',
alpha=0.7)
ax4.set_title('Total Processing Time for All Frames')
ax4.set_ylabel('Total Time (seconds)')
ax4.set_xlabel('Resolution')
ax4.tick_params(axis='x', rotation=45)
ax4.grid(True, alpha=0.3)

# Add value labels
for i, v in enumerate(timing_data['total_processing_time']):
    ax4.text(i, v + max(timing_data['total_processing_time']) * 0.01,
f'{v:.1f}s',
            ha='center', va='bottom', fontsize=8)

# Plot 5: Component Time Distribution (Pie charts for highest and lowest
resolution)
if len(timing_data['resolutions']) >= 2:

```

```

        # Highest resolution (first)
        det_time_high = timing_data['total_detection_time'][0]
        track_time_high = timing_data['total_tracking_time'][0]
        other_time_high = timing_data['total_processing_time'][0] - det_time_high -
            track_time_high

        sizes_high = [det_time_high, track_time_high, max(0, other_time_high)]
        labels = ['Detection', 'Tracking', 'Other']
        colors = ['royalblue', 'orange', 'lightgray']

        # Remove zero or negative values
        sizes_high = [max(0, s) for s in sizes_high]
        valid_data_high = [(s, l, c) for s, l, c in zip(sizes_high, labels,
colors) if s > 0]
        if valid_data_high:
            sizes_high, labels_high, colors_high = zip(*valid_data_high)
        ax5.pie(sizes_high, labels=labels_high, colors=colors_high, autopct='%1.1f%%',
            startangle=90)
        ax5.set_title(f'Time Distribution\n{resolutions[0]} (Highest Res)')

        # Lowest resolution (last)
        det_time_low = timing_data['total_detection_time'][-1]
        track_time_low = timing_data['total_tracking_time'][-1]
        other_time_low = timing_data['total_processing_time'][-1] - det_time_low -
            track_time_low

        sizes_low = [det_time_low, track_time_low, max(0, other_time_low)]
        sizes_low = [max(0, s) for s in sizes_low]
        valid_data_low = [(s, l, c) for s, l, c in zip(sizes_low, labels,
colors) if s > 0]
        if valid_data_low:
            sizes_low, labels_low, colors_low = zip(*valid_data_low)
        ax6.pie(sizes_low, labels=labels_low, colors=colors_low, autopct='%1.1f%%',
            startangle=90)
        ax6.set_title(f'Time Distribution\n{resolutions[-1]} (Lowest Res)')
    else:
        ax5.text(0.5, 0.5, 'Insufficient data', ha='center', va='center',
            transform=ax5.transAxes)
        ax6.text(0.5, 0.5, 'Insufficient data', ha='center', va='center',
            transform=ax6.transAxes)

    plt.tight_layout()

    if save_path:
        plt.savefig(save_path, dpi=300, bbox_inches='tight')
        print(f"Comprehensive timing plots saved to: {save_path}")
    else:
        plt.show()

```

[FOLDER] User_Interface

Location: User_Interface

[FILE] tracking_app_v4.py

Path: User_Interface\tracking_app_v4.py

```
import tkinter as tk
from tkinter import ttk, filedialog, messagebox, scrolledtext
import threading
import os
import sys
from pathlib import Path
import cv2
from PIL import Image, ImageTk
import pandas as pd
import json
from datetime import datetime
import subprocess
import webbrowser

# Add parent directory to path for project imports
sys.path.append(os.path.dirname(os.path.dirname(os.path.abspath(__file__))))

# Add your project imports here
try:
    from Distance_measurement.main_court_tracker import *
    from Distance_measurement.corner_detection import *
    from Detect_and_Track.create_tracking_video_and_init_data import
    create_tracking_video_and_init_
    data
except ImportError as e:
    print(f"Warning: Could not import project modules: {e}")

class VideoPlayer:
    """Simple video player widget"""

    def __init__(self, parent, width=640, height=360):
        self.parent = parent
        self.width = width
        self.height = height

        # Video state
        self.video_path = None
        self.cap = None
        self.is_playing = False
        self.current_frame = 0
        self.total_frames = 0
        self.fps = 30
        self.frame_delay = int(1000 / self.fps)

        self.create_widgets()

    def create_widgets(self):
```

```

        """Create video player widgets"""
        # Main frame
        self.frame = ttk.Frame(self.parent)

        # Video display
        self.video_label = ttk.Label(self.frame, text="No video loaded",
                                     anchor='center', background='black',
foreground='white')
        self.video_label.grid(row=0, column=0, columnspan=4, pady=(0, 10),
sticky=(tk.W, tk.E))

        # Controls frame
        controls_frame = ttk.Frame(self.frame)
        controls_frame.grid(row=1, column=0, columnspan=4, sticky=(tk.W, tk.E))

        # Control buttons
        self.play_button = ttk.Button(controls_frame, text="■ Play",
command=self.toggle_play)
        self.play_button.grid(row=0, column=0, padx=2)

        ttk.Button(controls_frame, text="■ Stop", command=self.stop_video).grid(row=0,
column=1,
        padx=2)
        ttk.Button(controls_frame, text="■ Start", command=self.goto_start).grid(row=0,
column=2,
        padx=2)
        ttk.Button(controls_frame, text="■ End", command=self.goto_end).grid(row=0,
column=3,
        padx=2)

        # Progress bar
        self.progress_var = tk.DoubleVar()
        self.progress_bar = ttk.Scale(controls_frame, from_=0, to=100,
orient=tk.HORIZONTAL,
        variable=self.progress_var,
command=self.seek_video)
        self.progress_bar.grid(row=0, column=4, sticky=(tk.W, tk.E), padx=(10,
10))

        # Time labels
        self.time_label = ttk.Label(controls_frame, text="00:00 / 00:00")
        self.time_label.grid(row=0, column=5, padx=2)

        # Configure column weights
        controls_frame.columnconfigure(4, weight=1)

    def load_video(self, video_path):
        """Load a video file"""
        try:
            if self.cap:
                self.cap.release()

            self.video_path = video_path
            self.cap = cv2.VideoCapture(video_path)

            if not self.cap.isOpened():
                raise ValueError("Could not open video file")

            # Get video properties
            self.total_frames = int(self.cap.get(cv2.CAP_PROP_FRAME_COUNT))
            self.fps = self.cap.get(cv2.CAP_PROP_FPS) or 30

```



```

        self.frame_delay = int(1000 / self.fps)
        self.current_frame = 0

        # Load first frame
        ret, frame = self.cap.read()
        if ret:
            self.display_frame(frame)
            self.update_time_display()

        return True

    except Exception as e:
        messagebox.showerror("Error", f"Error loading video: {e}")
        return False

def display_frame(self, frame):
    """Display a frame in the video label"""
    try:
        # Resize frame to fit display
        height, width = frame.shape[:2]
        if width > self.width or height > self.height:
            scale = min(self.width/width, self.height/height)
            new_width = int(width * scale)
            new_height = int(height * scale)
            frame = cv2.resize(frame, (new_width, new_height))

        # Convert to RGB and then to PhotoImage
        frame_rgb = cv2.cvtColor(frame, cv2.COLOR_BGR2RGB)
        image = Image.fromarray(frame_rgb)
        photo = ImageTk.PhotoImage(image)

        self.video_label.configure(image=photo, text="")
        self.video_label.image = photo

    except Exception as e:
        print(f"Error displaying frame: {e}")

def toggle_play(self):
    """Toggle play/pause"""
    if not self.cap:
        return

    self.is_playing = not self.is_playing
    if self.is_playing:
        self.play_button.config(text="■ Pause")
        self.play_video()
    else:
        self.play_button.config(text="■ Play")

def play_video(self):
    """Play video frames"""
    if not self.is_playing or not self.cap:
        return

    ret, frame = self.cap.read()
    if ret:
        self.current_frame += 1
        self.display_frame(frame)
        self.update_progress()
        self.update_time_display()

```

```

        # Schedule next frame
        self.parent.after(self.frame_delay, self.play_video)
    else:
        # End of video
        self.is_playing = False
        self.play_button.config(text="■ Play")

def stop_video(self):
    """Stop video and go to beginning"""
    self.is_playing = False
    self.play_button.config(text="■ Play")
    self.goto_start()

def goto_start(self):
    """Go to start of video"""
    if not self.cap:
        return

    self.cap.set(cv2.CAP_PROP_POS_FRAMES, 0)
    self.current_frame = 0
    ret, frame = self.cap.read()
    if ret:
        self.display_frame(frame)
        self.cap.set(cv2.CAP_PROP_POS_FRAMES, 0) # Reset position
    self.update_progress()
    self.update_time_display()

def goto_end(self):
    """Go to end of video"""
    if not self.cap:
        return

    self.cap.set(cv2.CAP_PROP_POS_FRAMES, self.total_frames - 1)
    self.current_frame = self.total_frames - 1
    ret, frame = self.cap.read()
    if ret:
        self.display_frame(frame)
    self.update_progress()
    self.update_time_display()

def seek_video(self, value):
    """Seek to specific position"""
    if not self.cap:
        return

    # Calculate frame number from percentage
    frame_number = int((float(value) / 100) * self.total_frames)
    self.cap.set(cv2.CAP_PROP_POS_FRAMES, frame_number)
    self.current_frame = frame_number

    ret, frame = self.cap.read()
    if ret:
        self.display_frame(frame)
        self.cap.set(cv2.CAP_PROP_POS_FRAMES, frame_number) # Reset position
    self.update_time_display()

def update_progress(self):
    """Update progress bar"""
    if self.total_frames > 0:
        progress = (self.current_frame / self.total_frames) * 100

```

```

        self.progress_var.set(progress)

    def update_time_display(self):
        """Update time display"""
        if self.total_frames > 0 and self.fps > 0:
            current_seconds = self.current_frame / self.fps
            total_seconds = self.total_frames / self.fps

            current_time = f"{int(current_seconds//60):02d}:{int(current_seconds%60):02d}"
            total_time = f"{int(total_seconds//60):02d}:{int(total_seconds%60):02d}"

            self.time_label.config(text=f"{current_time} / {total_time}")

    def get_frame(self):
        """Get the video player frame widget"""
        return self.frame

    def destroy(self):
        """Clean up video player"""
        if self.cap:
            self.cap.release()

class FileViewerWindow:
    """Window for viewing files in-app"""

    def __init__(self, parent, file_path):
        self.window = tk.Toplevel(parent)
        self.window.title(f"File Viewer - {os.path.basename(file_path)}")
        self.window.geometry("900x600")
        self.window.transient(parent)

        self.file_path = file_path
        self.create_widgets()
        self.load_file()

    def create_widgets(self):
        """Create file viewer widgets"""
        # Toolbar
        toolbar = ttk.Frame(self.window)
        toolbar.grid(row=0, column=0, sticky=(tk.W, tk.E), padx=5, pady=5)

        ttk.Label(toolbar, text=f"File: {os.path.basename(self.file_path)}").grid(row=0,
column=0,
        padx=5)
        ttk.Button(toolbar, text="Open in External App",
command=self.open_external).grid(row=0,
        column=1, padx=5)
        ttk.Button(toolbar, text="Open Folder", command=self.open_folder).grid(row=0,
column=2,
        padx=5)

        # Content area
        self.content_frame = ttk.Frame(self.window, padding="10")
        self.content_frame.grid(row=1, column=0, sticky=(tk.W, tk.E, tk.N, tk.S))

        # Configure grid weights
        self.window.columnconfigure(0, weight=1)
        self.window.rowconfigure(1, weight=1)

```

```

self.content_frame.columnconfigure(0, weight=1)
self.content_frame.rowconfigure(0, weight=1)

def load_file(self):
    """Load and display file content"""
    file_ext = os.path.splitext(self.file_path)[1].lower()

    try:
        if file_ext == '.csv':
            self.load_csv()
        elif file_ext == '.json':
            self.load_json()
        elif file_ext in ['.txt', '.log']:
            self.load_text()
        elif file_ext in ['.mp4', '.avi', '.mov']:
            self.load_video()
        elif file_ext in ['.png', '.jpg', '.jpeg']:
            self.load_image()
        else:
            ttk.Label(self.content_frame, text=f"File type {file_ext} not supported for
                preview").pack(pady=20)

    except Exception as e:
        ttk.Label(self.content_frame, text=f"Error loading file:
            {e}").pack(pady=20)

def load_csv(self):
    """Load CSV file"""
    df = pd.read_csv(self.file_path)

    # Create treeview
    columns = list(df.columns)
    tree = ttk.Treeview(self.content_frame, columns=columns, show='headings',
        height=20)

    # Configure columns
    for col in columns:
        tree.heading(col, text=col)
        tree.column(col, width=120)

    # Insert data (first 1000 rows)
    display_df = df.head(1000)
    for _, row in display_df.iterrows():
        values = [f"{val:.3f}" if isinstance(val, float) else str(val) for
            val in row]
        tree.insert('', tk.END, values=values)

    # Add scrollbars
    v_scrollbar = ttk.Scrollbar(self.content_frame, orient=tk.VERTICAL,
        command=tree.yview)
    h_scrollbar = ttk.Scrollbar(self.content_frame, orient=tk.HORIZONTAL,
        command=tree.xview)
    tree.configure(yscrollcommand=v_scrollbar.set,
        xscrollcommand=h_scrollbar.set)

    # Info label
    info_label = ttk.Label(self.content_frame,
        text=f"Showing first 1000 of {len(df)} rows, {len(df.columns)}
            columns")
    info_label.grid(row=0, column=0, columnspan=2, pady=5)

```

```

# Grid layout
tree.grid(row=1, column=0, sticky=(tk.W, tk.E, tk.N, tk.S))
v_scrollbar.grid(row=1, column=1, sticky=(tk.N, tk.S))
h_scrollbar.grid(row=2, column=0, sticky=(tk.W, tk.E))

self.content_frame.columnconfigure(0, weight=1)
self.content_frame.rowconfigure(1, weight=1)

def load_json(self):
    """Load JSON file"""
    with open(self.file_path, 'r') as f:
        data = json.load(f)

    text_widget = scrolledtext.ScrolledText(self.content_frame, wrap=tk.WORD)
    text_widget.grid(row=0, column=0, sticky=(tk.W, tk.E, tk.N, tk.S))

    formatted_json = json.dumps(data, indent=2, ensure_ascii=False)
    text_widget.insert(1.0, formatted_json)
    text_widget.config(state='disabled')

def load_text(self):
    """Load text file"""
    with open(self.file_path, 'r', encoding='utf-8', errors='ignore') as f:
        content = f.read()

    text_widget = scrolledtext.ScrolledText(self.content_frame, wrap=tk.WORD)
    text_widget.grid(row=0, column=0, sticky=(tk.W, tk.E, tk.N, tk.S))

    text_widget.insert(1.0, content)
    text_widget.config(state='disabled')

def load_video(self):
    """Load video file"""
    # Create video player
    self.video_player = VideoPlayer(self.content_frame, width=800, height=450)
    self.video_player.get_frame().grid(row=0, column=0, sticky=(tk.W, tk.E,
tk.N, tk.S))

    # Load video
    self.video_player.load_video(self.file_path)

def load_image(self):
    """Load image file"""
    image = cv2.imread(self.file_path)
    if image is not None:
        # Convert to RGB
        image_rgb = cv2.cvtColor(image, cv2.COLOR_BGR2RGB)

        # Resize if too large
        height, width = image_rgb.shape[:2]
        max_width, max_height = 800, 600

        if width > max_width or height > max_height:
            scale = min(max_width/width, max_height/height)
            new_width = int(width * scale)
            new_height = int(height * scale)
            image_rgb = cv2.resize(image_rgb, (new_width, new_height))

        # Convert to PhotoImage
        pil_image = Image.fromarray(image_rgb)
        photo = ImageTk.PhotoImage(pil_image)

```

```

        image_label = ttk.Label(self.content_frame, image=photo)
        image_label.image = photo # Keep reference
        image_label.grid(row=0, column=0, sticky=(tk.W, tk.E, tk.N, tk.S))

def open_external(self):
    """Open file in external application"""
    try:
        if sys.platform == "win32":
            os.startfile(self.file_path)

        elif sys.platform == "darwin":
            subprocess.run(["open", self.file_path])
        else:
            subprocess.run(["xdg-open", self.file_path])
    except Exception as e:
        messagebox.showerror("Error", f"Could not open file: {e}")

def open_folder(self):
    """Open folder containing the file"""
    try:
        folder_path = os.path.dirname(os.path.abspath(self.file_path))
        if sys.platform == "win32":
            subprocess.run(f'explorer /select, "{os.path.abspath(self.file_path)}"')
        elif sys.platform == "darwin":
            subprocess.run(["open", "-R", os.path.abspath(self.file_path)])
        else:
            subprocess.run(["xdg-open", folder_path])
    except Exception as e:
        messagebox.showerror("Error", f"Could not open folder: {e}")

class TennisTrackingApp:
    def __init__(self, root):
        self.root = root
        self.root.title("Tennis Court Tracking System v1.0")
        self.root.geometry("1200x540")
        self.root.configure(bg='#f0f0f0')

        # Application state
        self.video_path = None
        self.output_name = ""
        self.init_df_path = ""
        self.init_df = None
        self.ziframes = None
        self.zitboxes = None
        self.results = None
        self.processing = False

        # Create main layout
        self.setup_styles()
        self.create_menu()
        self.create_main_layout()

    def setup_styles(self):
        """Configure custom styles for the application"""
        style = ttk.Style()
        style.theme_use('clam')

        # Configure custom styles
        style.configure('Title.TLabel', font=('Arial', 16, 'bold'),

```

```

foreground='#2c3e50')

        style.configure('Section.TLabel', font=('Arial', 12, 'bold'),
foreground='#34495e')
        style.configure('Success.TLabel', font=('Arial', 10),
foreground='#27ae60')
        style.configure('Error.TLabel', font=('Arial', 10), foreground='#e74c3c')
        style.configure('Process.TButton', font=('Arial', 10, 'bold'))

def create_menu(self):
    """Create application menu bar"""
    menubar = tk.Menu(self.root)
    self.root.config(menu=menubar)

    # File menu
    file_menu = tk.Menu(menubar, tearoff=0)
    menubar.add_cascade(label="File", menu=file_menu)
    file_menu.add_command(label="Open Video", command=self.select_video)
    file_menu.add_separator()
    file_menu.add_command(label="Exit", command=self.root.quit)

    # View menu
    view_menu = tk.Menu(menubar, tearoff=0)
    menubar.add_cascade(label="View", menu=view_menu)
    view_menu.add_command(label="Play Original Video",
command=self.play_original_video)
    view_menu.add_command(label="Play Clean Video",
command=self.play_clean_video)
    view_menu.add_command(label="Play Tracking Video",
command=self.play_tracking_video)
    view_menu.add_separator()
    view_menu.add_command(label="View Results", command=self.view_results)
    view_menu.add_command(label="Open Output Folder",
command=self.open_output_folder)

    # Tools menu
    tools_menu = tk.Menu(menubar, tearoff=0)
    menubar.add_cascade(label="Tools", menu=tools_menu)
    tools_menu.add_command(label="Corner Detection Test",
command=self.open_corner_test)

    # Help menu
    help_menu = tk.Menu(menubar, tearoff=0)
    menubar.add_cascade(label="Help", menu=help_menu)
    help_menu.add_command(label="About", command=self.show_about)

def create_main_layout(self):
    """Create the main application layout"""
    # Create main container with padding
    main_container = ttk.Frame(self.root, padding="3")
    main_container.grid(row=0, column=0, sticky=(tk.W, tk.E, tk.N, tk.S))

    # Configure grid weights
    self.root.columnconfigure(0, weight=1)
    self.root.rowconfigure(0, weight=1)
    main_container.columnconfigure(1, weight=1)
    main_container.rowconfigure(1, weight=2) # Give more weight to main
content
    main_container.rowconfigure(2, weight=0) # Fixed height for progress panel

```

```

        # Title
title_label = ttk.Label(main_container, text="Tennis Court Tracking System",
                        style='Title.TLabel')
title_label.grid(row=0, column=0, columnspan=2, pady=(0, 5))

        # Left panel - Controls
self.create_control_panel(main_container)

        # Right panel - Video Preview and Results
self.create_preview_panel(main_container)

        # Bottom panel - Progress and Logs
self.create_progress_panel(main_container)

    def create_control_panel(self, parent):
        """Create the left control panel"""
        control_frame = ttk.LabelFrame(parent, text="Control Panel", padding="5")
        control_frame.grid(row=1, column=0, sticky=(tk.W, tk.E, tk.N, tk.S),
padx=(0, 5))
        control_frame.configure(width=350)

        # Video Input Section
        ttk.Label(control_frame, text="Video Input", style='Section.TLabel').grid(row=0,
column=0,
            columnspan=2, sticky=tk.W, pady=(0, 10))

        # Video selection
        ttk.Button(control_frame, text="■ Select Video", command=self.select_video,
            width=20).grid(row=1, column=0, sticky=tk.W, pady=2)
        self.video_label = ttk.Label(control_frame, text="No video selected",
foreground='gray')
        self.video_label.grid(row=1, column=1, sticky=tk.W, padx=(10, 0))

        # Video playback buttons
        video_controls = ttk.Frame(control_frame)
        video_controls.grid(row=2, column=0, columnspan=2, sticky=tk.W, pady=5)
        ttk.Button(video_controls, text="■ Original",
command=self.play_original_video).grid(row=0,
            column=0, padx=(0, 3))
        ttk.Button(video_controls, text="■ Clean",
command=self.play_clean_video).grid(row=0,
            column=1, padx=3)
        ttk.Button(video_controls, text="■ Tracking",
command=self.play_tracking_video).grid(row=0,
            column=2, padx=3)

        # Output name
        ttk.Label(control_frame, text="Output Name:").grid(row=3, column=0, sticky=tk.W,
pady=(10,
5))
        self.output_name_var = tk.StringVar(value="tennis_analysis")
        ttk.Entry(control_frame, textvariable=self.output_name_var,
width=25).grid(row=3, column=1,
            sticky=tk.W, padx=(10, 0))

        # Team Colors Section
        ttk.Separator(control_frame, orient='horizontal').grid(row=4, column=0,
columnspan=2,
            sticky=(tk.W, tk.E), pady=20)

        ttk.Label(control_frame, text="Team Configuration",
style='Section.TLabel').grid(row=5,

```



```

        column=0, columnspan=2, sticky=tk.W, pady=(0, 10))

        # Team colors
        ttk.Label(control_frame, text="Team 1 Color:").grid(row=6, column=0,
sticky=tk.W, pady=2)
        self.team1_color = ttk.Combobox(control_frame, values=["red", "blue", "white", "
black",
            "green", "yellow"], width=12)
        self.team1_color.set("red")
        self.team1_color.grid(row=6, column=1, sticky=tk.W, padx=(10, 0))

        ttk.Label(control_frame, text="Team 2 Color:").grid(row=7, column=0,
sticky=tk.W, pady=2)
        self.team2_color = ttk.Combobox(control_frame, values=["red", "blue", "white", "
black",
            "green", "yellow"], width=12)
        self.team2_color.set("blue")
        self.team2_color.grid(row=7, column=1, sticky=tk.W, padx=(10, 0))

        # Ball only option
        self.ball_only_var = tk.BooleanVar(value=False)
        ttk.Checkbutton(control_frame, text="Track ball only",
variable=self.ball_only_var).grid(
            row=8, column=0, columnspan=2, sticky=tk.W, pady=10)

        # Processing Section
        ttk.Separator(control_frame, orient='horizontal').grid(row=9, column=0,
columnspan=2,
            sticky=(tk.W, tk.E), pady=20)
        ttk.Label(control_frame, text="Processing Pipeline",
style='Section.TLabel').grid(row=10,
            column=0, columnspan=2, sticky=tk.W, pady=(0, 10))

        # Step 1: Object Detection & Tracking (Combined)
        step1_frame = ttk.Frame(control_frame)
        step1_frame.grid(row=11, column=0, columnspan=2, sticky=(tk.W, tk.E),
pady=5)
        ttk.Label(step1_frame, text="1. Complete Object Tracking").grid(row=0, column=0,
            sticky=tk.W)
        self.tracking_button = ttk.Button(step1_frame, text="Start Processing",
            command=self.run_complete_tracking, style='Process.TButton')
        self.tracking_button.grid(row=0, column=1, padx=(10, 0))
        self.tracking_status = ttk.Label(step1_frame, text="■ Ready",
foreground='gray')
        self.tracking_status.grid(row=0, column=2, padx=(10, 0))

        # Step 2: Court Analysis
        step2_frame = ttk.Frame(control_frame)
        step2_frame.grid(row=12, column=0, columnspan=2, sticky=(tk.W, tk.E),
pady=5)
        ttk.Label(step2_frame, text="2. Court Analysis").grid(row=0, column=0,
sticky=tk.W)

        # Pipeline selection
        self.pipeline_var = tk.StringVar(value="standard")
        ttk.Radiobutton(step2_frame, text="Standard", variable=self.pipeline_var,
            value="standard").grid(row=1, column=0, sticky=tk.W, padx=(20, 0))
        ttk.Radiobutton(step2_frame, text="Dynamic", variable=self.pipeline_var,
            value="dynamic").grid(row=1, column=1, sticky=tk.W)

```

```

self.analysis_button = ttk.Button(step2_frame, text="Start Analysis",
    command=self.run_analysis, style='Process.TButton')
self.analysis_button.grid(row=0, column=1, padx=(10, 0))
self.analysis_status = ttk.Label(step2_frame, text="■ Ready",
foreground='gray')
self.analysis_status.grid(row=0, column=2, padx=(10, 0))

def create_preview_panel(self, parent):
    """Create the right preview panel"""
    preview_frame = ttk.LabelFrame(parent, text="Video Preview & Results",
padding="5")
    preview_frame.grid(row=1, column=1, sticky=(tk.W, tk.E, tk.N, tk.S))
    preview_frame.columnconfigure(0, weight=1)
    preview_frame.rowconfigure(1, weight=1) # Results area gets remaining
space

    # Video player
    self.video_player = VideoPlayer(preview_frame, width=480, height=180)
    self.video_player.get_frame().grid(row=0, column=0, sticky=(tk.W, tk.E),
pady=(0, 3))

    # Results notebook
    self.results_notebook = ttk.Notebook(preview_frame)
    self.results_notebook.grid(row=1, column=0, sticky=(tk.W, tk.E, tk.N,
tk.S))

    # Statistics tab
    self.stats_frame = ttk.Frame(self.results_notebook)
    self.results_notebook.add(self.stats_frame, text="Statistics")

    self.stats_text = scrolledtext.ScrolledText(self.stats_frame,
wrap=tk.WORD, height=3)
    self.stats_text.grid(row=0, column=0, sticky=(tk.W, tk.E, tk.N, tk.S),
padx=3, pady=3)
    self.stats_frame.columnconfigure(0, weight=1)
    self.stats_frame.rowconfigure(0, weight=1)

    # Output Files tab
    self.files_frame = ttk.Frame(self.results_notebook)
    self.results_notebook.add(self.files_frame, text="Output Files")

    # Files controls
    files_controls = ttk.Frame(self.files_frame)
    files_controls.grid(row=0, column=0, columnspan=2, sticky=(tk.W, tk.E),
padx=5, pady=5)

    ttk.Label(files_controls, text="Output folder contents:").grid(row=0,
column=0, sticky=tk.W)
    ttk.Button(files_controls, text="■ Refresh",
command=self.refresh_output_files).grid(row=0,
column=1, padx=(10, 0))
    ttk.Button(files_controls, text="■ Open Folder",
command=self.open_output_folder).grid(
row=0, column=2, padx=(5, 0))

    # Files tree
    self.files_tree = ttk.Treeview(self.files_frame, columns=('Size', 'Modified', '
Type'),
show='tree headings')
    self.files_tree.heading('#0', text='File Name')
    self.files_tree.heading('Size', text='Size')
    self.files_tree.heading('Modified', text='Modified')

```

```

        self.files_tree.heading('Type', text='Type')
        self.files_tree.grid(row=1, column=0, sticky=(tk.W, tk.E, tk.N, tk.S),
padx=3, pady=3)

        # Files scrollbar
files_scrollbar = ttk.Scrollbar(self.files_frame, orient=tk.VERTICAL,
        command=self.files_tree.yview)
        files_scrollbar.grid(row=1, column=1, sticky=(tk.N, tk.S))
        self.files_tree.configure(yscrollcommand=files_scrollbar.set)

        # Double-click to open file
        self.files_tree.bind('<Double-1>', self.on_file_double_click)

        # Right-click context menu
        self.create_file_context_menu()

        self.files_frame.columnconfigure(0, weight=1)
        self.files_frame.rowconfigure(1, weight=1)

        # Load initial file list
        self.refresh_output_files()

    def create_file_context_menu(self):
        """Create context menu for files tree"""
        self.file_context_menu = tk.Menu(self.root, tearoff=0)
        self.file_context_menu.add_command(label="Open in App",
command=self.open_file_in_app)
        self.file_context_menu.add_command(label="Open in External App",
        command=self.open_file_external)
        self.file_context_menu.add_separator()
        self.file_context_menu.add_command(label="Show in Folder",
command=self.show_file_in_folder)
        self.file_context_menu.add_separator()
        self.file_context_menu.add_command(label="Play Video (if video)",
        command=self.play_selected_video)

        self.files_tree.bind('<Button-3>', self.show_file_context_menu) #
Right-click

    def show_file_context_menu(self, event):
        """Show context menu for files"""
        item = self.files_tree.selection()[0] if self.files_tree.selection() else
None
        if item:
            self.file_context_menu.post(event.x_root, event.y_root)

    def create_progress_panel(self, parent):
        """Create the bottom progress panel"""
        progress_frame = ttk.LabelFrame(parent, text="Progress & Logs",
padding="3")
        progress_frame.grid(row=2, column=0, columnspan=2, sticky=(tk.W, tk.E),
pady=(3, 0))
        progress_frame.columnconfigure(0, weight=1)

        # Progress bar
        self.progress_var = tk.DoubleVar()
        self.progress_bar = ttk.Progressbar(progress_frame, variable=self.progress_var,

        mode='determinate')
        self.progress_bar.grid(row=0, column=0, sticky=(tk.W, tk.E), pady=(0, 5))

        self.progress_label = ttk.Label(progress_frame, text="Ready to start

```

```

processing...)
    self.progress_label.grid(row=0, column=1, padx=(10, 0))

    # Log output
    self.log_text = scrolledtext.ScrolledText(progress_frame, wrap=tk.WORD,
height=2)
    self.log_text.grid(row=1, column=0, columnspan=2, sticky=(tk.W, tk.E,
tk.N, tk.S))

    def select_video(self):
        """Handle video file selection"""
        file_path = filedialog.askopenfilename(
            title="Select Tennis Video",
            filetypes=[
                ("Video files", "*.mp4 *.avi *.mov *.mkv *.wmv"),
                ("All files", "*.*")
            ]
        )

        if file_path:
            self.video_path = file_path
            filename = os.path.basename(file_path)
            self.video_label.config(text=f"■ {filename}", foreground='black')

            # Auto-generate output name from filename
            base_name = os.path.splitext(filename)[0]
            self.output_name_var.set(base_name)

            # Load video in player
            self.video_player.load_video(file_path)

            self.log_message(f"Video selected: {filename}")

            # Check if there are existing analysis files for this video
            self.check_existing_analysis_files()

    def check_existing_analysis_files(self):
        """Check if analysis files exist for current video and enable statistics
if available"""
        if not self.video_path:
            return

        output_name = self.output_name_var.get()
        if not output_name:
            return

        # Check for key analysis files
        init_df_file = f"./Out/{output_name}_init_df.csv"
        analysis_files = [
            f"./Out/{output_name}_with_video_court_coords.csv",

            f"./Out/{output_name}_with_dynamic_court_coords.csv"
        ]

        # Update init_df_path if file exists
        if os.path.exists(init_df_file):
            self.init_df_path = init_df_file
            self.tracking_status.config(text="■ Complete", foreground='green')
            self.log_message(f"Found existing tracking data:
{os.path.basename(init_df_file)}")

            # Check for analysis results

```

```

        for analysis_file in analysis_files:
            if os.path.exists(analysis_file):
                self.analysis_status.config(text="■ Complete", foreground='green')
                self.log_message(f"Found existing analysis:
{os.path.basename(analysis_file)}")

                # Try to load and display existing results
                try:
                    self.load_existing_analysis_results(analysis_file)
                except Exception as e:
                    self.log_message(f"Could not load existing analysis: {e}", "
WARNING")
                    break

            # Refresh file list to show all available files
            self.refresh_output_files()

def load_existing_analysis_results(self, analysis_file):
    """Load existing analysis results for display"""
    try:
        # Load the analysis CSV
        df_mapped = pd.read_csv(analysis_file)

        # Try to generate basic statistics from the loaded data
        player_ids = df_mapped['ID'].unique()
        player_distances = pd.DataFrame()

        # Calculate basic player statistics
        for player_id in player_ids:
            if player_id == 'ball': # Skip ball for player stats
                continue

            player_data = df_mapped[df_mapped['ID'] == player_id]
            if len(player_data) < 2:
                continue

            # Calculate total distance (simplified)
            coords = player_data[['court_x', 'court_y']].dropna()
            if len(coords) > 1:
                distances = []
                for i in range(1, len(coords)):
                    prev_x, prev_y = coords.iloc[i-1]

                    curr_x, curr_y = coords.iloc[i]
                    dist = ((curr_x - prev_x)**2 + (curr_y - prev_y)**2)**0.5
                    distances.append(dist)

                total_distance = sum(distances)
                player_distances = pd.concat([player_distances, pd.DataFrame({
                    'ID': [player_id],
                    'TotalDistance': [total_distance],
                    'DataPoints': [len(player_data)]
                })], ignore_index=True)

            # Create simplified results tuple
            self.results = (df_mapped, player_distances, {}, None, None)
            self.display_results()

    except Exception as e:
        self.log_message(f"Error loading existing results: {e}", "ERROR")

def play_original_video(self):

```

```

        """Play original video in video player"""
        if self.video_path:
            self.video_player.load_video(self.video_path)
            self.log_message("Playing original video")
        else:
            messagebox.showwarning("Warning", "Please select a video file first.")

    def play_clean_video(self):
        """Play clean video if available"""
        output_name = self.output_name_var.get()
        if output_name:
            clean_video_path = f"./Out/{output_name}_out.mp4"

            if os.path.exists(clean_video_path):
                self.video_player.load_video(clean_video_path)
                self.log_message(f"Playing clean video: {os.path.basename(clean_video_path)}")
                return

            messagebox.showinfo("Info", "Clean video not available. Please run tracking first.")

    def play_tracking_video(self):
        """Play tracking video if available"""
        output_name = self.output_name_var.get()
        if output_name:
            tracking_video_path = f"./Out/{output_name}_out_tracked.mp4"

            if os.path.exists(tracking_video_path):
                self.video_player.load_video(tracking_video_path)
                self.log_message(f"Playing tracking video: {os.path.basename(tracking_video_path)}")
                return

            messagebox.showinfo("Info", "Tracking video not available. Please run tracking first.")

    def play_selected_video(self):
        """Play selected video from files tree"""
        selected_item = self.files_tree.selection()
        if selected_item:
            filename = self.files_tree.item(selected_item[0])['text']
            file_path = os.path.join("./Out", filename)

            # Check if it's a video file
            video_extensions = ['.mp4', '.avi', '.mov', '.mkv', '.wmv']
            if any(file_path.lower().endswith(ext) for ext in video_extensions):
                self.video_player.load_video(file_path)
                self.log_message(f"Playing video: {filename}")
            else:
                messagebox.showinfo("Info", "Selected file is not a video.")

    def on_file_double_click(self, event):
        """Handle double-click on file"""
        selected_item = self.files_tree.selection()
        if selected_item:
            filename = self.files_tree.item(selected_item[0])['text']
            file_path = os.path.join("./Out", filename)
            self.open_file_viewer(file_path)

    def open_file_in_app(self):

```

```

        """Open selected file in app viewer"""
        selected_item = self.files_tree.selection()
        if selected_item:
            filename = self.files_tree.item(selected_item[0])['text']
            file_path = os.path.join("./Out", filename)
            self.open_file_viewer(file_path)

def open_file_external(self):
    """Open selected file in external application"""
    selected_item = self.files_tree.selection()
    if selected_item:
        filename = self.files_tree.item(selected_item[0])['text']
        file_path = os.path.join("./Out", filename)
        try:
            if sys.platform == "win32":
                os.startfile(file_path)
            elif sys.platform == "darwin":
                subprocess.run(["open", file_path])
            else:
                subprocess.run(["xdg-open", file_path])
        except Exception as e:
            messagebox.showerror("Error", f"Could not open file: {e}")

def show_file_in_folder(self):

    """Show selected file in folder"""
    selected_item = self.files_tree.selection()
    if selected_item:
        filename = self.files_tree.item(selected_item[0])['text']
        file_path = os.path.abspath(os.path.join("./Out", filename))
        try:
            if sys.platform == "win32":
                subprocess.run(f'explorer /select,"{file_path}"', shell=True)
            elif sys.platform == "darwin":
                subprocess.run(["open", "-R", file_path])
            else:
                folder_path = os.path.dirname(file_path)
                subprocess.run(["xdg-open", folder_path])
        except Exception as e:
            messagebox.showerror("Error", f"Could not show file in folder:
{e}")

def open_file_viewer(self, file_path):
    """Open file in internal viewer"""
    if os.path.exists(file_path):
        FileViewerWindow(self.root, file_path)
    else:
        messagebox.showerror("Error", "File not found.")

def refresh_output_files(self):
    """Refresh the output files list"""
    try:
        # Clear existing items
        for item in self.files_tree.get_children():
            self.files_tree.delete(item)

        # Check output directory
        output_dir = Path("./Out")
        if not output_dir.exists():
            self.log_message("Output directory does not exist yet")
            return

```

```

        file_count = 0
        # Add all files from output directory
        for file_path in output_dir.iterdir():
            if file_path.is_file():
                try:
                    stat = os.stat(file_path)
                    size = stat.st_size
                    modified = datetime.fromtimestamp(stat.st_mtime).strftime("%Y-%m-%d %H:%M")
                    size_str = f"{size:,} bytes" if size < 1024*1024 else f"{size/(1024*1024):.1f} MB"

                    # Determine file type
                    ext = file_path.suffix.lower()
                    file_type = {
                        '.csv': 'Data',
                        '.mp4': 'Video', '.avi': 'Video', '.mov': 'Video',
                        '.png': 'Image', '.jpg': 'Image', '.jpeg': 'Image',
                        '.json': 'Config',
                        '.txt': 'Text', '.log': 'Log'
                    }.get(ext, 'Other')

                    filename = file_path.name
                    self.files_tree.insert('', tk.END, text=filename,
                                           values=(size_str, modified,
file_type))

                    file_count += 1
                except Exception:
                    continue

        # Log the number of files found
        self.log_message(f"Output files refreshed: {file_count} files found")

    except Exception as e:
        self.log_message(f"Error refreshing file list: {e}", "ERROR")

    def run_complete_tracking(self):
        """Run complete object detection, tracking, and video creation in a
        separate thread"""
        if not self.video_path:
            messagebox.showwarning("Warning", "Please select a video file first.")
            return

        if self.processing:
            messagebox.showinfo("Info", "Processing is already running.")
            return

        # Start tracking in a separate thread
        thread = threading.Thread(target=self._complete_tracking_worker)
        thread.daemon = True
        thread.start()

    def _complete_tracking_worker(self):
        """Worker thread for complete tracking process"""
        try:
            self.processing = True
            self.update_ui_state()

            self.output_name = self.output_name_var.get() or "tennis_analysis"
            teams_colors = [self.team1_color.get(), self.team2_color.get()]
            ball_only = self.ball_only_var.get()

```



```

        self.progress_var.set(0)
        self.update_progress("Starting complete tracking process...", 10)

        # Run the combined function
        self.log_message("Starting complete object detection, tracking, and
video creation...")

        self.init_df, self.ziframes, self.zitboxes =
create_tracking_video_and_init_data(
            self.video_path, self.output_name, teams_colors, ball_only
        )

        self.update_progress("Complete tracking process finished", 100)

        self.init_df_path = f"./Out/{self.output_name}_init_df.csv"

        self.log_message("Complete tracking process completed successfully!", "
SUCCESS")
        self.root.after(0, lambda: self.tracking_status.config(text="■ Complete",
            foreground='green'))
        self.root.after(0, self.refresh_output_files)

    except Exception as e:
        error_msg = f"Error in complete tracking process: {str(e)}"
        self.log_message(error_msg, "ERROR")
        self.root.after(0, lambda: self.tracking_status.config(text="■ Error",
            foreground='red'))
    finally:
        self.processing = False
        self.root.after(0, self.update_ui_state)

def run_analysis(self):
    """Run court analysis pipeline"""
    if not self.init_df_path or not os.path.exists(self.init_df_path):
        messagebox.showwarning("Warning", "Please run object tracking first.")
        return

    if self.processing:
        messagebox.showinfo("Info", "Processing is already running.")
        return

    # Start analysis in a separate thread
    thread = threading.Thread(target=self._analysis_worker)
    thread.daemon = True
    thread.start()

def _analysis_worker(self):
    """Worker thread for analysis process"""
    try:
        self.processing = True
        self.update_ui_state()

        self.progress_var.set(0)
        self.update_progress("Starting court analysis...", 10)

        if self.pipeline_var.get() == "dynamic":
            self.log_message("Running dynamic homography pipeline...")
            self.update_progress("Processing with dynamic homography...", 30)
            results = main_video_processing_pipeline_dynamic(

```

```

        video_path=self.video_path,
        data_csv_path=self.init_df_path
    )
else:
    self.log_message("Running standard homography pipeline...")
    self.update_progress("Processing with standard homography...", 30)
    results = main_video_processing_pipeline(
        video_path=self.video_path,
        data_csv_path=self.init_df_path
    )

self.update_progress("Analysis completed", 100)
self.results = results

self.log_message("Court analysis completed successfully!", "SUCCESS")
self.root.after(0, lambda: self.analysis_status.config(text="■ Complete",
    foreground='green'))
self.root.after(0, self.display_results)
self.root.after(0, self.refresh_output_files)

except Exception as e:
    error_msg = f"Error in analysis process: {str(e)}"
    self.log_message(error_msg, "ERROR")
self.root.after(0, lambda: self.analysis_status.config(text="■ Error",
    foreground='red'))
finally:
    self.processing = False
    self.root.after(0, self.update_ui_state)

def display_results(self):
    """Display analysis results in the statistics tab"""
    if not self.results:
        return

    try:
        df_mapped, player_distances, ball_results, average_corners,
frame_info = self.results

        stats_text = "=== TENNIS COURT ANALYSIS RESULTS ===\n\n"

        # Player Statistics
        stats_text += "PLAYER MOVEMENT ANALYSIS:\n"
        stats_text += "-" * 40 + "\n"

        if not player_distances.empty:
            for _, player in player_distances.iterrows():
                stats_text += f"\nPlayer {player['ID']}:\n"
                stats_text += f"Total Distance:
{player['TotalDistance']:.1f}m\n"
                if 'AverageSpeedKmh' in player:
                    stats_text += f"Average Speed:
{player['AverageSpeedKmh']:.1f} km/h\n"
                if 'MaxSpeedKmh' in player:

                    stats_text += f"Max Speed: {player['MaxSpeedKmh']:.1f}
km/h\n"

                if 'TimeActive' in player:
                    stats_text += f"Active Time:
{player['TimeActive']:.1f}s\n"
                if 'CourtCoverage' in player:
                    stats_text += f"Court Coverage:
{player['CourtCoverage']:.1f}m²\n"

```

```

        if 'DataPoints' in player:
            stats_text += f" Data Points: {player['DataPoints']}\n"
        else:
            stats_text += "No player movement data available.\n"

    # Ball Statistics
    stats_text += "\n\nBALL MOVEMENT ANALYSIS:\n"
    stats_text += "-" * 40 + "\n"
    if ball_results:
        stats_text += f"Total Distance:
{ball_results.get('total_distance', 0):.1f}m\n"
        stats_text += f"Average Speed: {ball_results.get('average_speed_kmh', 0):.1f}
km/h\n"
        stats_text += f"Max Speed: {ball_results.get('max_speed_kmh',
0):.1f} km/h\n"
        stats_text += f"Detection Gaps:
{ball_results.get('detection_gaps', 0)}\n"
        stats_text += f"Valid Points: {ball_results.get('valid_points',
0)}\n"
    else:
        stats_text += "No ball movement data available.\n"

    # Corner Detection Statistics
    if frame_info:
        stats_text += "\n\nCORNER DETECTION ANALYSIS:\n"
        stats_text += "-" * 40 + "\n"
        valid_frames = sum(1 for info in frame_info if info.get('valid',
False))

        total_frames = len(frame_info)
        stats_text += f"Frames Processed: {total_frames}\n"
        stats_text += f"Valid Corner Detections: {valid_frames}\n"
        stats_text += f"Success Rate:
{(valid_frames/total_frames*100):.1f}%\n"

        if average_corners:
            stats_text += f"\nAverage Corner Positions:\n"
            corner_labels = ["Bottom-Left", "Bottom-Right", "Top-Right", "
Top-Left"]
            for i, (corner, label) in enumerate(zip(average_corners,
corner_labels)):
                stats_text += f" {label}: ({corner[0]:.1f},
{corner[1]:.1f})\n"

    # Data Statistics
    stats_text += "\n\nDATA STATISTICS:\n"
    stats_text += "-" * 40 + "\n"
    stats_text += f"Total Tracking Points: {len(df_mapped)}\n"
    stats_text += f"Unique Objects: {df_mapped['ID'].nunique()}\n"
    stats_text += f"Frame Range: {df_mapped['frame'].min()} -
{df_mapped['frame'].max()}\n"

    # Update the text widget
    self.stats_text.delete(1.0, tk.END)
    self.stats_text.insert(1.0, stats_text)

except Exception as e:
    self.log_message(f"Error displaying results: {e}", "ERROR")

def open_output_folder(self):
    """Open the output folder in file explorer"""
    output_dir = os.path.abspath("./Out")

```

```

if os.path.exists(output_dir):
    try:
        if sys.platform == "win32":
            os.startfile(output_dir)
        elif sys.platform == "darwin":
            subprocess.run(["open", output_dir])
        else:
            subprocess.run(["xdg-open", output_dir])
    except Exception as e:
        self.log_message(f"Error opening output folder: {e}", "ERROR")
else:
    messagebox.showinfo("Info", "Output folder does not exist yet.")
    # Create the folder if it doesn't exist
    try:
        os.makedirs(output_dir, exist_ok=True)
        self.log_message("Created output folder: ./Out")
    except Exception as e:
        self.log_message(f"Error creating output folder: {e}", "ERROR")

def open_corner_test(self):
    """Open corner detection test window"""
    CornerTestWindow(self.root)

def view_results(self):
    """Open results viewer window"""
    if self.results:
        ResultsViewerWindow(self.root, self.results)
    else:
        messagebox.showinfo("Info", "No results available. Please run
analysis first.")

def show_about(self):
    """Show about dialog"""
    about_text = """Tennis Court Tracking System v1.0

A computer vision application for analyzing tennis gameplay using:
• YOLO object detection
• DeepSORT tracking algorithms
• Homography-based coordinate mapping
• Player and ball movement analysis

Features:
• Video playback with controls
• Output file management
• In-app file viewing

• Real-time progress tracking
• Combined processing pipeline

Developed for tennis performance analysis and statistics generation."""

    messagebox.showinfo("About", about_text)

def update_progress(self, message, value):
    """Update progress bar and label"""
    self.root.after(0, lambda: self.progress_var.set(value))
    self.root.after(0, lambda: self.progress_label.config(text=message))

def log_message(self, message, level="INFO"):
    """Add message to log output"""
    timestamp = datetime.now().strftime("%H:%M:%S")

```

```

def update_log():
    self.log_text.insert(tk.END, f"[{timestamp}] {level}: {message}\n")
    self.log_text.see(tk.END)

self.root.after(0, update_log)

def update_ui_state(self):
    """Update UI state based on processing status"""
    state = "disabled" if self.processing else "normal"
    self.tracking_button.config(state=state)
    self.analysis_button.config(state=state)

class CornerTestWindow:
    """Window for testing corner detection on single frames"""

    def __init__(self, parent):
        self.window = tk.Toplevel(parent)
        self.window.title("Corner Detection Test")
        self.window.geometry("800x600")
        self.window.transient(parent)

        self.image_path = None
        self.original_image = None
        self.processed_image = None

        self.create_widgets()

    def create_widgets(self):
        """Create widgets for corner test window"""
        # Control frame
        control_frame = ttk.Frame(self.window, padding="10")
        control_frame.grid(row=0, column=0, sticky=(tk.W, tk.E))

        ttk.Button(control_frame, text="Select Image",
command=self.select_image).grid(row=0,
        column=0, padx=5)
        ttk.Button(control_frame, text="Detect Corners",
command=self.detect_corners).grid(row=0,
        column=1, padx=5)
        ttk.Button(control_frame, text="Save Result",
command=self.save_result).grid(row=0,
        column=2, padx=5)

        self.status_label = ttk.Label(control_frame, text="Select an image to
start")
        self.status_label.grid(row=0, column=3, padx=(20, 0))

        # Image display frame
        image_frame = ttk.Frame(self.window, padding="10")
        image_frame.grid(row=1, column=0, sticky=(tk.W, tk.E, tk.N, tk.S))

        self.image_label = ttk.Label(image_frame, text="Image will appear here",
        anchor='center', background='#f0f0f0',
relief='sunken')
        self.image_label.grid(row=0, column=0, sticky=(tk.W, tk.E, tk.N, tk.S))

        # Configure grid weights
        self.window.columnconfigure(0, weight=1)
        self.window.rowconfigure(1, weight=1)
        image_frame.columnconfigure(0, weight=1)

```

```

        image_frame.rowconfigure(0, weight=1)

    def select_image(self):
        """Select image for corner detection"""
        file_path = filedialog.askopenfilename(
            title="Select Tennis Court Image",
            filetypes=[
                ("Image files", "*.png *.jpg *.jpeg *.bmp *.tiff"),
                ("All files", "*.*")
            ]
        )

        if file_path:
            self.image_path = file_path
            self.load_image()
            self.status_label.config(text="Image loaded - ready for corner
detection")

    def load_image(self):
        """Load and display the selected image"""
        try:
            # Load with OpenCV
            self.original_image = cv2.imread(self.image_path)

            # Convert to RGB for display
            image_rgb = cv2.cvtColor(self.original_image, cv2.COLOR_BGR2RGB)

            # Resize for display
            height, width = image_rgb.shape[:2]

            max_width, max_height = 700, 400

            if width > max_width or height > max_height:
                scale = min(max_width/width, max_height/height)
                new_width = int(width * scale)
                new_height = int(height * scale)
                image_rgb = cv2.resize(image_rgb, (new_width, new_height))

            # Convert to PhotoImage
            image = Image.fromarray(image_rgb)
            photo = ImageTk.PhotoImage(image)

            self.image_label.configure(image=photo, text="")
            self.image_label.image = photo

        except Exception as e:
            messagebox.showerror("Error", f"Error loading image: {e}")

    def detect_corners(self):
        """Detect and display corners on the image"""
        if self.original_image is None:
            messagebox.showwarning("Warning", "Please select an image first.")
            return

        try:
            # Import corner detector
            from Distance_measurement.corner_detection import CornerDetector

            cd = CornerDetector()
            corners = cd.detect_court_corners(self.original_image.copy(),
debug=False)

```

```

        # Draw corners on image
        result_image = self.original_image.copy()
        cd.draw_corners_on_frame(result_image, corners)

        # Convert to RGB for display
        image_rgb = cv2.cvtColor(result_image, cv2.COLOR_BGR2RGB)

        # Resize for display
        height, width = image_rgb.shape[:2]
        max_width, max_height = 700, 400

        if width > max_width or height > max_height:
            scale = min(max_width/width, max_height/height)
            new_width = int(width * scale)
            new_height = int(height * scale)
            image_rgb = cv2.resize(image_rgb, (new_width, new_height))

        # Convert to PhotoImage
        image = Image.fromarray(image_rgb)

        photo = ImageTk.PhotoImage(image)

        self.image_label.configure(image=photo)
        self.image_label.image = photo

        self.processed_image = result_image
        self.status_label.config(text=f"Found {len(corners)} corners")

    except Exception as e:
        messagebox.showerror("Error", f"Error detecting corners: {e}")

def save_result(self):
    """Save the processed image with corners"""
    if self.processed_image is None:
        messagebox.showwarning("Warning", "No processed image to save.")
        return

    file_path = filedialog.asksaveasfilename(
        title="Save Corner Detection Result",
        defaultextension=".png",
        filetypes=[
            ("PNG files", "*.png"),
            ("JPEG files", "*.jpg"),
            ("All files", "*.*")
        ]
    )

    if file_path:
        try:
            cv2.imwrite(file_path, self.processed_image)
            messagebox.showinfo("Success", "Image saved successfully!")
        except Exception as e:
            messagebox.showerror("Error", f"Error saving image: {e}")

class ResultsViewerWindow:
    """Window for viewing detailed analysis results"""

    def __init__(self, parent, results):
        self.window = tk.Toplevel(parent)
        self.window.title("Analysis Results Viewer")
        self.window.geometry("1000x700")

```

```

self.window.transient(parent)

self.results = results
self.create_widgets()
self.load_results()

def create_widgets(self):
    """Create widgets for results viewer"""

    # Create notebook for different result views
    self.notebook = ttk.Notebook(self.window, padding="10")
    self.notebook.grid(row=0, column=0, sticky=(tk.W, tk.E, tk.N, tk.S))

    # Player data tab
    self.player_frame = ttk.Frame(self.notebook)
    self.notebook.add(self.player_frame, text="Player Data")

    # Ball data tab
    self.ball_frame = ttk.Frame(self.notebook)
    self.notebook.add(self.ball_frame, text="Ball Data")

    # Corner data tab
    self.corner_frame = ttk.Frame(self.notebook)
    self.notebook.add(self.corner_frame, text="Corner Analysis")

    # Raw data tab
    self.raw_frame = ttk.Frame(self.notebook)
    self.notebook.add(self.raw_frame, text="Raw Data")

    # Configure grid weights
    self.window.columnconfigure(0, weight=1)
    self.window.rowconfigure(0, weight=1)

def load_results(self):
    """Load and display results in appropriate tabs"""
    try:
        df_mapped, player_distances, ball_results, average_corners,
frame_info = self.results

        # Player data
        self.create_player_view(player_distances)

        # Ball data
        self.create_ball_view(ball_results)

        # Corner data
        self.create_corner_view(average_corners, frame_info)

        # Raw data
        self.create_raw_view(df_mapped)

    except Exception as e:
        messagebox.showerror("Error", f"Error loading results: {e}")

def create_player_view(self, player_distances):
    """Create player data view"""
    if player_distances.empty:
        ttk.Label(self.player_frame, text="No player data
available").pack(pady=20)
    return

```



```

# Create treeview for player data
columns = list(player_distances.columns)
tree = ttk.Treeview(self.player_frame, columns=columns[1:], show='tree
headings')

# Configure columns
tree.heading('#0', text='Player ID')
tree.column('#0', width=100)

for col in columns[1:]:
    tree.heading(col, text=col)
    tree.column(col, width=120)

# Insert data
for _, row in player_distances.iterrows():
    values = [f"{val:.2f}" if isinstance(val, (int, float)) else str(val)
              for val in row.iloc[1:]]
    tree.insert('', tk.END, text=str(row.iloc[0]), values=values)

# Add scrollbars
v_scrollbar = ttk.Scrollbar(self.player_frame, orient=tk.VERTICAL,
command=tree.yview)
h_scrollbar = ttk.Scrollbar(self.player_frame, orient=tk.HORIZONTAL,
command=tree.xview)
tree.configure(yscrollcommand=v_scrollbar.set,
xscrollcommand=h_scrollbar.set)

# Grid layout
tree.grid(row=0, column=0, sticky=(tk.W, tk.E, tk.N, tk.S))
v_scrollbar.grid(row=0, column=1, sticky=(tk.N, tk.S))
h_scrollbar.grid(row=1, column=0, sticky=(tk.W, tk.E))

self.player_frame.columnconfigure(0, weight=1)
self.player_frame.rowconfigure(0, weight=1)

def create_ball_view(self, ball_results):
    """Create ball data view"""
    if not ball_results:
        ttk.Label(self.ball_frame, text="No ball data
available").pack(pady=20)
        return

    # Create text widget for ball data
    text_widget = scrolledtext.ScrolledText(self.ball_frame, wrap=tk.WORD)
    text_widget.grid(row=0, column=0, sticky=(tk.W, tk.E, tk.N, tk.S),
padx=10, pady=10)

    # Format ball data
    ball_text = "BALL MOVEMENT ANALYSIS\n" + "="*50 + "\n\n"
    for key, value in ball_results.items():
        if isinstance(value, (int, float)):
            ball_text += f"{key.replace('_', ' ').title(): {value:.2f}\n"
        else:
            ball_text += f"{key.replace('_', ' ').title(): {value}\n"

    text_widget.insert(1.0, ball_text)

    text_widget.config(state='disabled')

self.ball_frame.columnconfigure(0, weight=1)
self.ball_frame.rowconfigure(0, weight=1)

```

```

def create_corner_view(self, average_corners, frame_info):
    """Create corner analysis view"""
    # Create text widget for corner data
    text_widget = scrolledtext.ScrolledText(self.corner_frame, wrap=tk.WORD)
    text_widget.grid(row=0, column=0, sticky=(tk.W, tk.E, tk.N, tk.S),
padx=10, pady=10)

    corner_text = "CORNER DETECTION ANALYSIS\n" + "="*50 + "\n\n"

    # Average corners
    if average_corners:
        corner_text += "Average Corner Positions:\n"
        corner_labels = ["Bottom-Left", "Bottom-Right", "Top-Right", "
Top-Left"]
        for i, (corner, label) in enumerate(zip(average_corners,
corner_labels)):
            corner_text += f" {label}: ({corner[0]:.1f}, {corner[1]:.1f})\n"
            corner_text += "\n"

    # Frame analysis
    if frame_info:
        valid_frames = sum(1 for info in frame_info if info.get('valid',
False))
        total_frames = len(frame_info)
        corner_text += f"Frame Analysis:\n"
        corner_text += f" Total Frames Processed: {total_frames}\n"
        corner_text += f" Valid Corner Detections: {valid_frames}\n"
        corner_text += f" Success Rate:
{(valid_frames/total_frames*100):.1f}%\n\n"

        corner_text += "Frame-by-Frame Results:\n"
        for info in frame_info[:10]: # Show first 10 frames
            status = "■" if info.get('valid', False) else "□"
            corner_text += f" Frame {info.get('frame_number', 'N/A')}:
{status} "
            corner_text += f"({info.get('corners_found', 0)} corners found)\n"

        if len(frame_info) > 10:
            corner_text += f" ... and {len(frame_info) - 10} more frames\n"

    text_widget.insert(1.0, corner_text)
    text_widget.config(state='disabled')

    self.corner_frame.columnconfigure(0, weight=1)
    self.corner_frame.rowconfigure(0, weight=1)

def create_raw_view(self, df_mapped):
    """Create raw data view"""
    if df_mapped.empty:
        ttk.Label(self.raw_frame, text="No tracking data
available").pack(pady=20)
        return

    # Create treeview for raw data (show first 1000 rows)
    display_df = df_mapped.head(1000)
    columns = list(display_df.columns)

    tree = ttk.Treeview(self.raw_frame, columns=columns, show='headings',
height=20)

    # Configure columns

```

```

        for col in columns:
            tree.heading(col, text=col)
            tree.column(col, width=100)

        # Insert data
        for _, row in display_df.iterrows():
            values = [f"{val:.3f}" if isinstance(val, float) else str(val) for
val in row]
            tree.insert('', tk.END, values=values)

        # Add scrollbars
        v_scrollbar = ttk.Scrollbar(self.raw_frame, orient=tk.VERTICAL,
command=tree.yview)
        h_scrollbar = ttk.Scrollbar(self.raw_frame, orient=tk.HORIZONTAL,
command=tree.xview)
        tree.configure(yscrollcommand=v_scrollbar.set,
xscrollcommand=h_scrollbar.set)

        # Info label
        info_label = ttk.Label(self.raw_frame,
                               text=f"Showing first 1000 of {len(df_mapped)} total
tracking points")
        info_label.grid(row=0, column=0, columnspan=2, pady=5)

        # Grid layout
        tree.grid(row=1, column=0, sticky=(tk.W, tk.E, tk.N, tk.S))
        v_scrollbar.grid(row=1, column=1, sticky=(tk.N, tk.S))
        h_scrollbar.grid(row=2, column=0, sticky=(tk.W, tk.E))

        self.raw_frame.columnconfigure(0, weight=1)
        self.raw_frame.rowconfigure(1, weight=1)

def main():
    """Main application entry point"""
    # Create output directory if it doesn't exist
    os.makedirs("./Out", exist_ok=True)

    # Create and run the application
    root = tk.Tk()
    app = TennisTrackingApp(root)

    # Center the window
    root.update_idletasks()
    x = (root.winfo_screenwidth() // 2) - (root.winfo_width() // 2)
    y = (root.winfo_screenheight() // 2) - (root.winfo_height() // 2)
    root.geometry(f"{x}x{y}")

    root.mainloop()

if __name__ == "__main__":
    main()

```